

So the idea is to create a cloud sessions of our terminal like coding agents like basically terminal cli tools like Claude Code or codex cli, gemini cli, open code you know all these like grok, grok code or like grok build as well so like have a way to synchronize sessions or chats or conversations or like you know the thing that a new session creates like it's essentially a session with a thread of messages and you know responses and so on and so forth so we will have that stored on the cloud and the reason for that or why we want to do that is to sync these sessions within multiple devices. So the problem that I face which is like, I will build this eventually for scratching my own itch, the problem that I face is I have two devices and I use them both regularly like daily for most of my work. Sometimes when I lay on bed I use my laptop but apart from that I am mostly on my desktop which is Windows and my laptop is a MacBook. I use the same coding tools on both of them but again they are both configured differently locally right? Like they have a different set of MCP servers they have a different set of skills installed. So there is this mismatch which I created dev sync essentially for which was syncing it was mainly in for MCP configurations at the time but I think this idea is like a very good expansion or pivot for Dev sync now that I think about it because the idea for Dev sync was essentially to create a centralized way for configuring all the MCP servers for all the coding agents or you know tools that you use for example you can create a centralized MCP configuration file and you can sync them to sync that file to cursor Claude Code Codex Gemini anti-gravity grok all these you know open code all these agent and coding tools so that was essentially the idea of dev sync but it was limited to MCP servers but now I wouldn't want to make it universal and you know so a place for people to sync their entire MCP skills MCP servers or you know skills and anything else that's relevant in terms of like when we talk about coding agents or CLI tools or you know things like Claude Code essentially so yeah that's essentially what we are aiming for like that's essentially the idea to have a universal way to store use continue sessions across multiple devices so like a cloud based solution to development or like AI development. Like the problem is that let's suppose I worked 8 hours on my computer on my desktop and now let's suppose I created 20 sessions across 4 different projects when I was working and now that I am on the laptop like I'm on bed one way for me to you know get all the changes is to pull the you know latest commits and have an agent you know read through the entire things and like go through what has changed and then I can continue but sometimes you just want to continue working on the same session or you just want the full context right? You don't want to have an encoding agent go through all the code changes again and again. So yeah that's essentially the problem and

that's essentially the project idea. So the project idea is like very simple in the like on those core level like essentially we're just storing sessions on a like let's say S3 bucket or storing sessions on a in database records that's essentially all that we are doing and we are just communicating over from one device to another or to any device really so yeah that's the entire you know idea for the project. Let me know if this is a good idea and if I should build it or not. I mean I am going to build it anyways but yeah.

Please research thoroughly and present your thoughts.

Good news and hard news, and they're both worth hearing before you write a line of code.

The problem is real and loudly validated. There are open feature requests on both major repos asking for exactly this — a Codex issue requesting account-linked cross-device session sync so chat history and reasoning state persist in the cloud tied to your account rather than locally, and a Claude Code feature request describing your exact scenario: long sessions accumulating context, and having to start fresh on another device and rebuild context from project memory files. So you're not imagining the pain. Your "pull the commits and have an agent re-read everything" workaround is the workaround everyone hates.

The hard news: this space got crowded fast, from two directions.

First, the vendors are building it natively. Claude Code now has Remote Control and Teleport — you can run "claude --teleport" to continue a cloud session on your machine, or /remote-control to control a locally running session from your phone or web, and while Remote Control is connected, the session transcript is stored on Anthropic servers, keeping the conversation in sync across devices. Codex went even further: the CLI, Desktop app, IDE extension, and Cloud runtime share a unified backend, making sessions portable artefacts that can be started on one surface and continued on another. When the platform vendor treats cross-device sessions as core product, a third-party sync layer for that single tool is on borrowed time. Second, the open-source field is already busy. claude-sync syncs the entire ~/.claude directory — sessions, history, agents, skills, plugins, settings — across devices via encrypted cloud storage, supporting R2, S3, GCS, and WebDAV with end-to-end encryption. For Codex there's codex-sync, encrypted Git-backed sync of sessions, settings, skills, rules, and memories. On the config side — your original DevSync territory — there's sync-agents-settings, which syncs MCP configs from Claude Code to Gemini CLI, Codex, OpenCode, Kiro, and Cursor with one command, plus Plexus, sync-mcp, add-mcp, and others.

Now, where the actual opportunity still lives. Three gaps nobody has closed well:

- **Cross-vendor.** Every native solution is single-ecosystem (Anthropic syncs Claude Code, OpenAI syncs Codex). Nobody sane at either company will build "resume your Gemini session from your Codex history." A universal layer across Claude Code, Codex, Gemini CLI, opencode, and Grok is genuinely unowned. That's your differentiator, and it aligns perfectly with the DevSync pivot.
- **The native features have real limitations.** Teleport is one-way — you can pull a web session down to your terminal, but you can't push an existing local session up to the web, and it demands clean git state, the same repo, and a pushed branch. Remote Control requires the original machine to stay running. Your "desktop off, laptop in bed" case still isn't cleanly solved natively.
- **The technical trap everyone hits — and it's your trap specifically.** Claude Code indexes sessions by the project's filesystem path, so the same repo at different paths on two machines is treated as two separate projects — meaning even after syncing files, --resume won't pick up where you left off. Windows desktop (C:\Users\you\...) plus macOS laptop (/Users/you/...) is the worst version of this. Solving path remapping across OSes, per-tool, robustly, is unglamorous and hard — which is exactly why it's a moat. Most of the hobby projects punt on it or only handle macOS ↔ macOS.

My verdict: build it, but don't build "claude-sync with a database." Build the cross-vendor abstraction: a normalized session/config model that maps into each tool's native format (JSONL in ~/.claude/projects, ~/.codex/sessions, Gemini's store, etc.), with OS-aware path remapping as a first-class feature, and your existing MCP/skills sync folded in. Sessions, MCP servers, skills, and instruction files (CLAUDE.md ↔ AGENTS.md ↔ GEMINI.md) as one synced profile. That's a coherent product; single-tool session sync alone is a feature the vendors will eat. The risks to watch are format churn (these tools change their session schemas often) and the possibility that "resume mid-session cross-vendor" is impossible — you can't replay a Claude session into Codex — so cross-vendor value is config/skills/context, while same-tool value is true session resume. Keeping those two honest in your positioning will matter.

Since you're going to build it anyway: start with the Windows ↔ macOS path-remapping problem for Claude Code sessions specifically. If you solve that one thing well, you've already beaten most of the existing tools at your own use case, and everything else layers on top.

A deeper investigation could map the full competitive landscape across all six-plus agent CLIs — session file formats, what each vendor's roadmap already covers, and where the durable gaps are.

Your competitive and technical landscape report on cross-device sync for terminal AI coding agents is ready. It covers native vendor sync capabilities (Claude Code teleport/Remote Control, Codex App Server, Gemini CLI, opencode, Cursor, Grok, Copilot CLI, Amp), session storage formats and the critical Windows↔macOS path-remapping problem, a teardown of 25+ competing tools across session-sync, config/MCP-sync, and mobile-control categories, market demand signals from vendor issue trackers, and a staged strategic recommendation centered on a cross-vendor unified profile with OS-aware path remapping and E2E encryption as the defensible wedge.

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

```
# Cross-Device Sync for Terminal AI Coding Agents: Competitive &
Technical Landscape
```

```
## TL;DR
```

```
- **The durable opportunity is a cross-vendor abstraction layer
with OS-aware path remapping as its headline feature – NOT
another "sync my Claude sessions" tool.** Every major vendor is
now shipping single-ecosystem sync natively (Claude Code Remote
Control/teleport, Codex's unified App Server backend, Amp
threads, Copilot CLI cloud sync), so the defensible wedge is the
one thing no vendor will ever build: normalizing sessions + MCP +
skills + instruction files across *competing* tools and remapping
Windows↔macOS paths.
```

```
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex)
is effectively impossible and should be dropped from scope;**
config/skills/instruction portability and *same-vendor* cross-
device session continuity are the achievable, valuable core. The
single sharpest, currently-unsolved technical pain – which even
the leading competitor (claude-sync) explicitly punts on – is
Claude Code's absolute-path project indexing breaking on Windows
↔macOS sync.
```

```
- **Recommendation: build a "universal profile" that folds
DevSync's MCP-sync foundation in, adds E2E-encrypted session sync
with first-class path remapping for Claude Code first, then
Codex/Gemini/opencode.** Ship path remapping as the MVP
differentiator; treat E2E encryption as table stakes; do not try
to out-run vendors on same-vendor live handoff.
```

```
## Key Findings
```

```
1. **Native vendor sync is real but ecosystem-locked and gap-
```

riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.

2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...\` and `/Users/...\` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.

3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.

4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

Claude Code (Anthropic). Ships the richest consumer-facing multi-device story:

- **`/teleport`** (``claude --teleport``): pulls a *cloud* session (started at claude.ai/code or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with ``--teleport``, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires **clean git state** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo

checkout, the branch pushed to remote, and the same claude.ai account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."

- **Remote Control** (``claude remote-control` / `/remote-control`, --rc`): pushes a live local session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an undocumented ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `ANTHROPIC_BASE_URL` is redirected; unavailable under Zero Data Retention.`
- **Claude Code on the web** (claude.ai/code): runs on Anthropic cloud infra, no local filesystem.
- **Residual gap:** No true device-to-device *local* session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an *unexpected*, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

Codex CLI (OpenAI). The most architecturally mature cross-surface model:

- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared **App Server** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under ``~/ .codex/sessions/YYYY/MM/DD/rollout-*.jsonl``, with a ``session_meta`` first line.
- Rich lifecycle: ``codex resume`` (picker), ``--last``, ``--resume-session-id``, ``/fork``, archive/unarchive (v0.136), delete (v0.140). Imports histories from other agents via an ``external-agent-sessions`` crate.
- **Known gaps/bugs:** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing ``encrypted_content`` often can't be

continued after crossing provider/account
(`invalid\_encrypted\_content`); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI↔app-server live sync.

- **Residual gap:** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

Gemini CLI (Google). Auto-saves sessions to
`~/gemini/tmp/<project\_hash>/chats/` (project-scoped, hash of project root). `~/resume` session browser, `--resume`/`-r` flag, `~/chat save` checkpoints, and a shadow-git checkpointing system under `~/gemini/history/<project\_hash>`. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. **Residual gap:** everything local + path-hash-keyed; no sync.

opencode (sst). SQLite + file storage under
`~/local/share/opencode/` (`opencode.db`,
`storage/session|message|part`). Native **session sharing** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). **Residual gap:** sharing ≠ syncing; DB concurrency and cross-OS path issues remain.

Cursor CLI (cursor-agent). Chats live in local SQLite; CLI chats under `~/cursor/chats` and resume via `cursor-agent --resume`. **IDE chats and CLI chats do NOT share a session store** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend `&` to push to a Cloud Agent, pick up at cursor.com/agents), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). **Residual gap:** no local cross-device sync, IDE↔CLI split, path issues.

Grok / Grok Build (xAI). Sessions auto-saved to
`~/grok/sessions/`, keyed by working directory. `~/resume`, `--resume`, `-c`, `~/fork`; subscription-gated (SuperGrok/X Premium+). Config in `~/grok/config.toml`. No native cross-device sync. Still beta (v0.2.x as of July 2026). **Residual gap:** no sync at all.

GitHub Copilot CLI. Sessions in `~/.copilot/`(`session-state/<uuid>/events.jsonl`)`. Notably, Copilot CLI **syncs** sessions to your GitHub account by default and offers **remote control** from GitHub.com/GitHub Mobile (origin machine must stay online, `/keep-alive`)`. But **cross-device *resume*** is still not supported – manually copying a session dir and running `--resume`` overwrites `events.jsonl`` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session_id``. **Residual gap:** the sync is telemetry/history, not resumable working state across machines.

Amp (Sourcegraph). Threads sync to `ampcode.com` by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). **Residual gap:** vendor-locked, server-stored, no cross-vendor.

Cross-vendor summary: Every vendor is building *within* its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

Tool	Location	Format	Path keying	Cross-device resume native?
Claude Code	<code>~/.claude/projects/<encoded-path>/<session>.jsonl`</code>	JSONL, append-only event stream	Absolute path, non-alphanumerics→dashes	Via cloud only (teleport/RC)
Codex	<code>~/.codex/sessions/YYYY/MM/DD/rollout-*.jsonl`</code>	JSONL + <code>session_meta`</code> header	date-based; cwd in meta	Yes, via App Server
Gemini CLI	<code>~/.gemini/tmp/<project_hash>/chats/`</code>	JSON	Hash of project root	No
opencode	<code>~/.local/share/opencode/`</code>	SQLite + files	SQLite/JSON	project-keyed
Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)
Grok	<code>~/.grok/sessions/`</code>	JSON	working directory	No
Copilot CLI	<code>~/.copilot/session-state/<uuid>/events.jsonl`</code>			

JSONL events | uuid | Account history sync; no resume |

****The path-mapping problem (the crux).**** Claude Code derives the project folder name by escaping the absolute path: ``/Users/me/app`` → ``~/.claude/projects/-Users-me-app/``, while the same repo at ``C:\Users\me\app`` (or a different macOS username/location) produces a **different** folder. Consequently, after a naive file sync, ``claude --resume`` on the second machine cannot find the first machine's sessions. This is confirmed by the `claude-sync` author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker." Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via ``project_hash``; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. ****This is the single highest-value, currently-unmet technical feature.****

****Format churn risk.**** Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

****Mid-session cross-device resume feasibility (same vendor):**** Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present — **if** path keys are remapped. Codex's App Server makes it cloud-native already.

****Cross-vendor session replay (e.g., Claude Code session → Codex):**** ****Fundamentally impractical.**** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) ``encrypted_content`` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. ****Recommendation: treat cross-vendor as**

config/skills/instruction portability + context handoff, never as live session replay.**

3. Competitor teardown

****Session-continuity tools:****

- ****tawanorg/claude-sync**** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/.claude` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + ****age E2E encryption**** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.
- ****npow/session-sync**** (~1 star, very new): the only cross-vendor ***session*** translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- ****dantelex/aisync ("ss")****: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- ****codsync/codsync**** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- ****kroxiksut/codexSync****: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- ****waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)**** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- ****iHildy/opencode-synced**** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- ****Dinesh3184/claude-session-sync****: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- ****renefichtmueller/claude-sync****: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

****Config/MCP/skills sync (DevSync's neighborhood):****

- ****Leoyangl83/sync-agents-settings**** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.

- **nicepkg/vsync** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- **baranovxyz/agentsync** and **spxrogers/agentsync** (beta): canonical `.agents/`` config synced to 9–31 agents.
- **neon-solutions/add-mcp**: one-command MCP install across ~16 agents (Neon).
- **ztripez/mcp-sync**: MCP config sync across tools.
- **AgentSync (Rust)**: symlink-based config sync.
- **skills-sync**: skills + MCP workspace.
- **DevSynq** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

URL-addressable / mobile-control / viewer approaches:

- **TokenRip** (tokenrip.com): makes session state a URL-addressable artifact (`/a/<slug>`) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- **Omnara** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was **archived in February 2026** because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)
- **Syncode – AI Programming Tool**: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- **jazzyalex/agent-sessions** & **kenn-io/agentsview**: local-first session browsers/search across 20+ agents (read/resume-command, not sync).
- **LAN-workspace approaches** and Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

Takeaway: The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- **Claude Code:** #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- **Codex:** #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI ↔ app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
- **Copilot CLI:** #1947 (cloud-synced sessions), #1635 (cross-environment resume).
- **Cursor forum:** multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is. Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. **Cross-vendor unified profile** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.

2. **OS-aware path remapping as a first-class feature** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows ↔ macOS. This is the wedge no competitor has nailed.

3. **E2E encryption by default** – table stakes given session files contain code, secrets, and full transcripts.

Risks:

- **Format churn:** Claude Code's format "changes between

versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).

- ****Vendor lock-out / ToS:**** Storing session data is user-owned data (fine), but **auth tokens** are not transferable (codexSync explicitly refuses; Codex ``encrypted_content`` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.
- ****Vendors closing the gap:**** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This **validates** the demand while **confirming** that only the cross-vendor + cross-OS layer is defensible.
- ****Security/privacy expectations:**** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).
- ****Honest limits:**** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

****Stage 0 – Fold in DevSync (immediate).**** Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

****Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge).**** Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/ .claude/projects/`` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- ****First-class OS-aware path remapping:**** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the

target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.

- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.

- **Benchmark to advance:** ``claude --resume`` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode. Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid ``encrypted_content`/token` pitfalls. For Gemini, remap ``project_hash``. For opencode, use safe SQLite snapshotting (the ``sqlite3 .backup`` pattern the community scripts use).

- **Benchmark:** each new adapter must pass the same cross-OS resume test as Claude Code before GA.

Stage 3 – Unified profile + cross-vendor context handoff. One "profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not replay.

Monetization & thresholds. Free/open-source core (build trust, match claude-sync) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSynq/OpenSync validate willingness to pay). **Kill/pivot signal:** if Anthropic ships account-level *local* session sync with automatic path remapping across OSes (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

What NOT to do: Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and

volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.

- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

suggest good names for this project.

I kinda like the whole "open-<thing>" convention. like for example, opencode, openrouter, etc. so maybe we can name it something like that, which actually just makes sense and is simple and straight to the point and easy to understand.

give me both the "open-" names and normal regular names.

names I like for reference: Cursor, Linear, Slack, etc.

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape

TL;DR

- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows↔macOS paths.
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows↔macOS sync.
- **Recommendation:** build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode. Ship path remapping as the MVP differentiator; treat E2E encryption

as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...\` and `/Users/...\` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.
3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.
4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

- Claude Code (Anthropic).** Ships the richest consumer-facing multi-device story:
 - ***/teleport*** (`claude --teleport`): pulls a *cloud* session (started at claude.ai/code or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport`, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires **clean git state** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same claude.ai account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."
 - **Remote Control** (`claude remote-control` / `/remote-control`, `--rc`): pushes a *live local* session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an *undocumented* ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `ANTHROPIC_BASE_URL` is redirected; unavailable under Zero Data Retention.
 - **Claude Code on the web** (claude.ai/code): runs on Anthropic cloud infra, no local filesystem.
 - **Residual gap:** No true device-to-device *local* session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an *unexpected*, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

- Codex CLI (OpenAI).** The most architecturally mature cross-surface model:
 - CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared **App Server** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under `~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl`, with a

``session_meta`` first line.
 - Rich lifecycle: ``codex resume`` (picker), ``--last``, ``--resume-session-id``, ``/fork``,
 archive/unarchive (v0.136), delete (v0.140). Imports histories from other agents via an
``external-agent-sessions`` crate.
 - **Known gaps/bugs:** Desktop may not list CLI sessions if the app-server wasn't running at
 creation; ephemeral sessions aren't persisted; sessions containing ``encrypted_content`` often
 can't be continued after crossing provider/account (``invalid_encrypted_content``); Desktop's
 session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask
 for account-linked cross-device sync and CLI↔app-server live sync.
 - **Residual gap:** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local
 file portability still hits path issues.

Gemini CLI (Google). Auto-saves sessions to ``~/gemini/tmp/<project_hash>/chats/``
 (project-scoped, hash of project root). ``/resume`` session browser, ``--resume`/`-r`` flag,
``/chat save`` checkpoints, and a shadow-git checkpointing system under
``~/gemini/history/<project_hash>``. No native cross-device cloud sync. Note: Google announced
 Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. **Residual**
gap: everything local + path-hash-keyed; no sync.

opencode (sst). SQLite + file storage under ``~/local/share/opencode/`` (``opencode.db``,
``storage/session|message|part``). Native **session sharing** publishes to Cloudflare Durable
 Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-
 party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync,
 opencode-session-backup). **Residual gap:** sharing ≠ syncing; DB concurrency and cross-OS
 path issues remain.

Cursor CLI (cursor-agent). Chats live in local SQLite; CLI chats under ``~/cursor/chats``
 and resume via ``cursor-agent --resume``. **IDE chats and CLI chats do NOT share a session**
store (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend ``&`` to push to
 a Cloud Agent, pick up at cursor.com/agents), and "My Machines" lets cloud agents run on your
 registered machine. Strong user demand for account-level chat sync (forum). **Residual gap:**
 no local cross-device sync, IDE↔CLI split, path issues.

Grok / Grok Build (xAI). Sessions auto-saved to ``~/grok/sessions/``, keyed by working
 directory. ``/resume``, ``--resume``, ``-c``, ``/fork``; subscription-gated (SuperGrok/X Premium+).
 Config in ``~/grok/config.toml``. No native cross-device sync. Still beta (v0.2.x as of July
 2026). **Residual gap:** no sync at all.

GitHub Copilot CLI. Sessions in ``~/copilot/`` (``session-state/<uuid>/events.jsonl``).
 Notably, Copilot CLI **syncs sessions to your GitHub account by default** and offers **remote**
control from GitHub.com/GitHub Mobile (origin machine must stay online, ``/keep-alive``). But
cross-device resume is still not supported – manually copying a session dir and running
``--resume`` overwrites ``events.jsonl`` and destroys history (issue #1635); cloud-synced cross-
 device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via
 a user-supplied ``session_id``. **Residual gap:** the sync is telemetry/history, not resumable
 working state across machines.

Amp (Sourcegraph). Threads sync to ampcode.com by default and can be continued across
 devices; "orbs" (remote machines) keep running after you close your laptop. This is the
 closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph
 servers (a privacy consideration; MCP config is GUI-global, not file-based). **Residual gap:**
 vendor-locked, server-stored, no cross-vendor.

Cross-vendor summary: Every vendor is building *within* its ecosystem. The white space is
 precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

	Tool	Location	Format	Path keying	Cross-device resume native?
					--- --- --- --- ---
	Claude Code	<code>`~/claude/projects/<encoded-path>/<session>.jsonl`</code>	JSONL, append-only event		

stream		**Absolute path, non-alphanumerics→dashes**		Via cloud only (teleport/RC)
Codex		~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl		JSONL + `session_meta` header
				date-based; cwd in meta
				Yes, via App Server
Gemini CLI		~/gemini/tmp/<project_hash>/chats/		JSON
				Hash of project root
				No
opencode		~/local/share/opencode/		(SQLite + files)
				SQLite/JSON
				project-keyed
				Sharing only
Cursor		local SQLite		DB rows
				workspace/path metadata
				No (cloud handoff only)
				Grok
				~/grok/sessions/
				JSON
				working directory
				No
Copilot CLI		~/copilot/session-state/<uuid>/events.jsonl		JSONL events
				uuid
				Account
				history sync; no resume

****The path-mapping problem (the crux).**** Claude Code derives the project folder name by escaping the absolute path: `~/Users/me/app` → `~/claude/projects/-Users-me-app/`, while the same repo at `C:\Users\me\app` (or a different macOS username/location) produces a **different** folder. Consequently, after a naive file sync, `claude --resume` on the second machine cannot find the first machine's sessions. This is confirmed by the claude-sync author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker." Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via `project_hash`; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. ****This is the single highest-value, currently-unmet technical feature.****

****Format churn risk.**** Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

****Mid-session cross-device resume feasibility (same vendor):**** Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present – **if** path keys are remapped. Codex's App Server makes it cloud-native already.

****Cross-vendor session replay (e.g., Claude Code session → Codex):**** ****Fundamentally impractical.**** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) `encrypted_content` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. ****Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.****

3. Competitor teardown

****Session-continuity tools:****

- ****tawanorg/claude-sync**** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/claude` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + ****age E2E encryption**** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.
- ****npow/session-sync**** (~1 star, very new): the only cross-vendor **session** translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- ****dantelex/aisync ("ss")****: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- ****codsync/codsync**** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- ****kroxiksut/codexSync****: local-first "cold-sync" Codex state handoff, Windows-first;

explicitly does NOT transfer auth tokens (OpenAI licensing).

- **waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- **iHildy/opencode-synced** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- **Dinesh3184/claude-session-sync**: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- **renefichtmueller/claude-sync**: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

Config/MCP/skills sync (DevSync's neighborhood):

- **Leoyang183/sync-agents-settings** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- **nicepkg/vsync** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- **baranovxyz/agentsync** and **spxrogers/agentsync** (beta): canonical `.agents/` config synced to 9–31 agents.
- **neon-solutions/add-mcp**: one-command MCP install across ~16 agents (Neon).
- **ztripez/mcp-sync**: MCP config sync across tools.
- **AgentSync (Rust)**: symlink-based config sync.
- **skills-sync**: skills + MCP workspace.
- **DevSync** (devsync.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

URL-addressable / mobile-control / viewer approaches:

- **TokenRip** (tokenrip.com): makes session state a URL-addressable artifact (`/a/<slug>`) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- **Omnara** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was **archived** in February 2026 because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)
- **Syncode – AI Programming Tool**: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- **jazzyaalex/agent-sessions** & **kenn-io/agentsview**: local-first session browsers/search across 20+ agents (read/resume-command, not sync).
- **LAN-workspace approaches** and **Syncthing/NAS/dotfiles/chezmoi**+age: DIY approaches that work for config but hit the path problem for sessions.

Takeaway: The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- **Claude Code**: #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- **Codex**: #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI  app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
- **Copilot CLI**: #1947 (cloud-synced sessions), #1635 (cross-environment resume).
- **Cursor forum**: multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is. Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. **Cross-vendor unified profile** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.
2. **OS-aware path remapping as a first-class feature** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows↔macOS. This is the wedge no competitor has nailed.
3. **E2E encryption by default** – table stakes given session files contain code, secrets, and full transcripts.

Risks:

- **Format churn:** Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).
- **Vendor lock-out / ToS:** Storing session data is user-owned data (fine), but *auth tokens* are not transferable (codexSync explicitly refuses; Codex `encrypted\_content` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.
 - **Vendors closing the gap:** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This *validates* the demand while *confirming* that only the cross-vendor + cross-OS layer is defensible.
- **Security/privacy expectations:** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).
- **Honest limits:** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

Stage 0 – Fold in DevSync (immediate). Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge). Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/ .claude/projects/` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- **First-class OS-aware path remapping:** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
 - Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
 - **Benchmark to advance:** `claude --resume` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

****Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode.**** Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid `encrypted\_content`/token pitfalls. For Gemini, remap `project\_hash`. For opencode, use safe SQLite snapshotting (the `sqlite3 .backup` pattern the community scripts use).

- ****Benchmark:**** each new adapter must pass the same cross-OS resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One "profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not replay.

****Monetization & thresholds.**** Free/open-source core (build trust, match claude-sync) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSynq/OpenSync validate willingness to pay). ****Kill/pivot signal:**** if Anthropic ships account-level **local** session sync with automatic path remapping across OSes (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

Good instinct on both fronts — the **open-** convention signals "infrastructure, not a startup," and the Cursor/Linear/Slack style works because they're real words with a quiet second meaning.

open- names

- **openstate** — the most literally accurate. What you're syncing isn't files or chats, it's *agent state*. Clean, obvious, no baggage.
- **opencontext** — leads with the value rather than the mechanism. "Your context, on every machine."
- **opensession** — dead simple, does exactly what it says. Least clever, most searchable.
- **openhome** — you're syncing `~/.claude`, `~/.codex`, `~/.gemini`. Home directories. Nice double meaning, though it flirts with smart-home associations.
- **openroam** — the telecom metaphor: your setup roams with you between devices. Underrated.
- **openprofile** — matches the "unified profile" framing from the research:

sessions + MCP + skills + instructions as one portable thing.

- **openrig** — "rig" is how devs already talk about their setup. Casual, memorable.
- **opendots** — dotfiles for agents. Cute, instantly legible to the target audience.
- **openbridge** — between vendors, between OSes. Slightly generic but sturdy.

Regular names

- **Baton** — passing the baton from desktop to laptop. This is your product in one word. Short, real, zero explanation needed.
- **Splice** — joining two ends of tape or rope. You splice yesterday's desktop session onto tonight's laptop. Punchy.
- **Ferry** — carries things across a gap, on a schedule, reliably. Very Slack-like in feel.
- **Braid** — weaving separate threads (sessions, on separate machines) into one. Pretty, and "thread" is already the domain vocabulary.
- **Strand** — same thread metaphor, quieter. `strand.dev` feels like a real infra company.
- **Tandem** — two machines operating as one. Warm, human, easy to say.
- **Keel** — the backbone that keeps everything upright and aligned. Good for a config-truth product.
- **Skein** — a coiled bundle of thread. Beautiful and distinctive, but people will misspell and mispronounce it (SKAYN).
- **Syncline** — a real geological term that happens to contain "sync." Clever without being cute.
- **Parity** — keeping your machines at parity. Conceptually perfect, though there's a blockchain company with the name.

Conflicts worth avoiding: `openthread` (Google's IoT protocol), `openmemory` (mem0's product), `openrelay` ("open relay" is spam-server terminology), and on the regular side Loom, Roam, Warp, Nomad, Harbor, Tether, Recall, and Portage are all meaningfully taken — Warp especially, since it's a terminal and you'd be competing for the same mindshare.

If I had to pick one from each: **openstate** and **Baton**. Baton in particular passes the bar Linear and Slack pass — you hear it once, and the product explains itself.

Want me to check npm and domain availability on your shortlist?

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

Cross-Device Sync for Terminal AI Coding Agents: Competitive &

Technical Landscape

TL;DR

- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows ↔ macOS paths.
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows ↔ macOS sync.
- **Recommendation:** build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode. Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...\` and `/Users/...\` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche

tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.

3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.

4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

Claude Code (Anthropic). Ships the richest consumer-facing multi-device story:

- **/teleport** (``claude --teleport``): pulls a *cloud* session (started at `claude.ai/code` or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport``, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires **clean git state** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same `claude.ai` account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."
- **Remote Control** (``claude remote-control` / `/remote-control`, --rc`): pushes a live local session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an undocumented ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not`

communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `ANTHROPIC\_BASE\_URL` is redirected; unavailable under Zero Data Retention.

- **Claude Code on the web** (claude.ai/code): runs on Anthropic cloud infra, no local filesystem.
- **Residual gap:** No true device-to-device **local** session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an **unexpected**, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

Codex CLI (OpenAI). The most architecturally mature cross-surface model:

- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared **App Server** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under `~/.codex/sessions/YYYY/MM/DD/rollout-\*.jsonl`, with a `session\_meta` first line.
- Rich lifecycle: `codex resume` (picker), `--last`, `--resume-session-id`, `/fork`, archive/unarchive (v0.136), delete (v0.140). Imports histories from other agents via an `external-agent-sessions` crate.
- **Known gaps/bugs:** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing `encrypted\_content` often can't be continued after crossing provider/account (`invalid\_encrypted\_content`); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.
- **Residual gap:** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

Gemini CLI (Google). Auto-saves sessions to `~/.gemini/tmp/<project\_hash>/chats/` (project-scoped, hash of project root). `/resume` session browser, `--resume`/`-r` flag, `/chat save` checkpoints, and a shadow-git checkpointing system under `~/.gemini/history/<project\_hash>`. No native cross-device

cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. **Residual gap:** everything local + path-hash-keyed; no sync.

opencode (sst). SQLite + file storage under `~/local/share/opencode/`(`opencode.db`,
`storage/session|message|part`)`. Native **session sharing** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). **Residual gap:** sharing $\neq$ syncing; DB concurrency and cross-OS path issues remain.

Cursor CLI (cursor-agent). Chats live in local SQLite; CLI chats under `~/cursor/chats`` and resume via `cursor-agent --resume``. **IDE chats and CLI chats do NOT share a session store** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend `&`` to push to a Cloud Agent, pick up at `cursor.com/agents`), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). **Residual gap:** no local cross-device sync, IDE ↔ CLI split, path issues.

Grok / Grok Build (xAI). Sessions auto-saved to `~/grok/sessions/``, keyed by working directory. `/resume``, `--resume``, `-c``, `/fork``; subscription-gated (SuperGrok/X Premium+). Config in `~/grok/config.toml``. No native cross-device sync. Still beta (v0.2.x as of July 2026). **Residual gap:** no sync at all.

GitHub Copilot CLI. Sessions in `~/copilot/`(`session-state/<uuid>/events.jsonl`)`. Notably, Copilot CLI **syncs sessions to your GitHub account by default** and offers **remote control** from `GitHub.com/GitHub Mobile` (origin machine must stay online, `/keep-alive``). But **cross-device \*resume\*** is still not supported – manually copying a session dir and running `--resume`` overwrites `events.jsonl`` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session_id``. **Residual gap:** the sync is telemetry/history, not resumable working state across machines.

Amp (Sourcegraph). Threads sync to `ampcode.com` by default and

can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). ****Residual gap:**** vendor-locked, server-stored, no cross-vendor.

****Cross-vendor summary:**** Every vendor is building **within** its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

Tool	Location	Format	Path keying	Cross-device resume native?	
Claude Code	~/.claude/projects/<encoded-path>/<session>.jsonl`	JSONL, append-only event stream	**Absolute path, non-alphanumerics→dashes**	Via cloud only (teleport/RC)	
Codex	~/.codex/sessions/YYYY/MM/DD/rollout-*.jsonl`	JSONL + `session_meta` header	date-based; cwd in meta	Yes, via App Server	
Gemini CLI	~/.gemini/tmp/<project_hash>/chats/`	JSON	**Hash of project root**	No	
opencode	~/.local/share/opencode/`	(SQLite + files)	SQLite/JSON	project-keyed	Sharing only
Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)	
Grok	~/.grok/sessions/`	JSON	working directory	No	
Copilot CLI	~/.copilot/session-state/<uuid>/events.jsonl`	JSONL events	uuid	Account history sync	no resume

****The path-mapping problem (the crux).**** Claude Code derives the project folder name by escaping the absolute path:

`/Users/me/app` → `~/.claude/projects/-Users-me-app/`, while the same repo at `C:\Users\me\app` (or a different macOS username/location) produces a **different** folder. Consequently, after a naive file sync, `claude --resume` on the second machine cannot find the first machine's sessions. This is confirmed by the claude-sync author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across

different paths, the core use case feels significantly weaker." Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via `project\_hash`; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. ****This is the single highest-value, currently-unmet technical feature.****

****Format churn risk.**** Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

****Mid-session cross-device resume feasibility (same vendor):**** Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present — *if* path keys are remapped. Codex's App Server makes it cloud-native already.

****Cross-vendor session replay (e.g., Claude Code session → Codex):**** ****Fundamentally impractical.**** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) `encrypted\_content` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. ****Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.****

3. Competitor teardown

****Session-continuity tools:****

- ****tawanorg/claude-sync**** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/.claude` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + ****age E2E encryption**** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference

implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.

- ****npow/session-sync**** (~1 star, very new): the only cross-vendor **session** translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- ****dantelex/aisync ("ss")****: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- ****codsync/codsync**** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- ****kroxiksut/codexSync****: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- ****waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)**** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- ****iHildy/opencode-synced**** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- ****Dinesh3184/claude-session-sync****: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- ****renefichtmueller/claude-sync****: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

****Config/MCP/skills sync (DevSync's neighborhood):****

- ****Leoyang183/sync-agents-settings**** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- ****nicepkg/vsync**** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- ****baranovxyz/agentsync**** and ****spxrogers/agentsync**** (beta): canonical ``.agents/`` config synced to 9–31 agents.
- ****neon-solutions/add-mcp****: one-command MCP install across ~16 agents (Neon).
- ****ztripez/mcp-sync****: MCP config sync across tools.
- ****AgentSync (Rust)****: symlink-based config sync.
- ****skills-sync****: skills + MCP workspace.
- ****DevSynq**** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

****URL-addressable / mobile-control / viewer approaches:****

- ****TokenRip**** (tokenrip.com): makes session state a URL-addressable artifact (``/a/<slug>``) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- ****Omnara**** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was ****archived in February 2026**** because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)
- ****Syncode – AI Programming Tool****: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- ****jazzyalex/agent-sessions**** & ****kenn-io/agentsview****: local-first session browsers/search across 20+ agents (read/resume-command, not sync).
- ****LAN-workspace approaches**** and Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

****Takeaway:**** The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- ****Claude Code:**** #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- ****Codex:**** #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI  app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
- ****Copilot CLI:**** #1947 (cloud-synced sessions), #1635 (cross-

environment resume).

- **Cursor forum:** multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is. Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. **Cross-vendor unified profile** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.

2. **OS-aware path remapping as a first-class feature** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows↔macOS. This is the wedge no competitor has nailed.

3. **E2E encryption by default** – table stakes given session files contain code, secrets, and full transcripts.

Risks:

- **Format churn:** Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).

- **Vendor lock-out / ToS:** Storing session data is user-owned data (fine), but *auth tokens* are not transferable (codexSync explicitly refuses; Codex `encrypted\_content` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.

- **Vendors closing the gap:** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This *validates* the demand while

confirming that only the cross-vendor + cross-OS layer is defensible.

- ****Security/privacy expectations:**** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).
- ****Honest limits:**** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

****Stage 0 – Fold in DevSync (immediate).**** Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

****Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge).**** Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/.claude/projects/` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- ****First-class OS-aware path remapping:**** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
- ****Benchmark to advance:**** `claude --resume` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

****Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode.**** Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid

`encrypted\_content`/token pitfalls. For Gemini, remap
`project\_hash`. For opencode, use safe SQLite snapshotting (the
`sqlite3.backup` pattern the community scripts use).
- ****Benchmark:**** each new adapter must pass the same cross-OS
resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One
"profile" = sessions + MCP + skills + agents + instructions,
portable across machines and (for config) across vendors. Add
best-effort context handoff (summarize + re-inject) for cross-
vendor moves – explicitly labeled as context transfer, not
replay.

****Monetization & thresholds.**** Free/open-source core (build
trust, match claude-sync) with a paid tier for managed encrypted
storage, multi-machine dashboards, and team profiles
(DevSynq/OpenSync validate willingness to pay). ****Kill/pivot
signal:**** if Anthropic ships account-level **local** session sync
with automatic path remapping across OSes (not just history
mirroring, per #71794), the single-vendor wedge collapses – at
that point double down on cross-vendor config/skills/profile
portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport
(gated, one-way, origin-online, with documented ~10-min network
and undocumented ~15-min idle timeouts). Don't promise cross-
vendor session replay. Don't sync auth tokens. Don't store
plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment

based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

how about "openparity"? nah?

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape

TL;DR

- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows↔macOS paths.
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows↔macOS sync.
- **Recommendation: build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode.** Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...\` and `/Users/...\` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.
3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.
4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

- Claude Code (Anthropic).** Ships the richest consumer-facing multi-device story:
 - **`/teleport`** (``claude --teleport``): pulls a *cloud* session (started at claude.ai/code or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal

session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport`, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires ****clean git state**** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same claude.ai account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."

- ****Remote Control**** (`\`claude remote-control` / \`/remote-control`, --rc`): pushes a ***live local*** session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an ***undocumented*** ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `\`ANTHROPIC_BASE_URL`` is redirected; unavailable under Zero Data Retention.
- ****Claude Code on the web**** (claude.ai/code): runs on Anthropic cloud infra, no local filesystem.

- ****Residual gap:**** No true device-to-device ***local*** session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an ***unexpected***, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

- ****Codex CLI (OpenAI).**** The most architecturally mature cross-surface model: CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared ****App Server**** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under `\`~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl``, with a `\`session_meta`` first line.

- Rich lifecycle: `\`codex resume`` (picker), `\`--last``, `\`--resume-session-id``, `\`/fork``, `archive/unarchive` (v0.136), `delete` (v0.140). Imports histories from other agents via an `\`external-agent-sessions`` crate.

- ****Known gaps/bugs:**** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing `\`encrypted_content`` often can't be continued after crossing provider/account (`\`invalid_encrypted_content``); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.

- ****Residual gap:**** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

- ****Gemini CLI (Google).**** Auto-saves sessions to `\`~/gemini/tmp/<project_hash>/chats/`` (project-scoped, hash of project root). `\`/resume`` session browser, `\`--resume`` `\`-r`` flag, `\`/chat save`` checkpoints, and a shadow-git checkpointing system under `\`~/gemini/history/<project_hash>``. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. ****Residual gap:**** everything local + path-hash-keyed; no sync.

- ****opencode (sst).**** SQLite + file storage under `\`~/local/share/opencode/`` (`\`opencode.db``, `\`storage/session|message|part``). Native ****session sharing**** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). ****Residual gap:**** sharing ≠ syncing; DB concurrency and cross-OS path issues remain.

- ****Cursor CLI (cursor-agent).**** Chats live in local SQLite; CLI chats under `\`~/cursor/chats`` and resume via `\`cursor-agent --resume``. ****IDE chats and CLI chats do NOT share a session store**** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend `\`&`` to push to a Cloud Agent, pick up at cursor.com/agents), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). ****Residual gap:****

no local cross-device sync, IDE↔CLI split, path issues.

Grok / Grok Build (xAI). Sessions auto-saved to `~/grok/sessions/`, keyed by working directory. `~/resume`, `--resume`, `-c`, `~/fork`; subscription-gated (SuperGrok/X Premium+). Config in `~/grok/config.toml`. No native cross-device sync. Still beta (v0.2.x as of July 2026). **Residual gap:** no sync at all.

GitHub Copilot CLI. Sessions in `~/copilot/` (`session-state/<uuid>/events.jsonl`). Notably, Copilot CLI **syncs** sessions to your GitHub account by default and offers **remote control** from GitHub.com/GitHub Mobile (origin machine must stay online, `~/keep-alive`). But **cross-device resume** is still not supported – manually copying a session dir and running `--resume` overwrites `events.jsonl` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session\_id`. **Residual gap:** the sync is telemetry/history, not resumable working state across machines.

Amp (Sourcegraph). Threads sync to ampcode.com by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). **Residual gap:** vendor-locked, server-stored, no cross-vendor.

Cross-vendor summary: Every vendor is building *within* its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

Tool	Location	Format	Path keying	Cross-device resume native?
				--- --- --- --- ---
Claude Code	~/claude/projects/<encoded-path>/<session>.jsonl	JSONL, append-only event stream	Absolute path, non-alphanumerics→dashes	Via cloud only (teleport/RC)
Codex	~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl	JSONL + `session_meta` header	date-based; cwd in meta	Yes, via App Server
Gemini CLI	~/gemini/tmp/<project_hash>/chats/	JSON	Hash of project root	No
opencode	~/local/share/opencode/	(SQLite + files)	SQLite/JSON	project-keyed Sharing only
Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)
Grok	~/grok/sessions/	JSON	working directory	No
Copilot CLI	~/copilot/session-state/<uuid>/events.jsonl	JSONL events	uuid	Account history sync; no resume

The path-mapping problem (the crux). Claude Code derives the project folder name by escaping the absolute path: `/Users/me/app` → `~/claude/projects/-Users-me-app/`, while the same repo at `C:\Users\me\app` (or a different macOS username/location) produces a *different* folder. Consequently, after a naive file sync, `claude --resume` on the second machine cannot find the first machine's sessions. This is confirmed by the claude-sync author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker."

Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via `project\_hash`; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. **This is the single highest-value, currently-unmet technical feature.**

Format churn risk. Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

****Mid-session cross-device resume feasibility (same vendor):**** Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present – ***if*** path keys are remapped. Codex's App Server makes it cloud-native already.

****Cross-vendor session replay (e.g., Claude Code session → Codex):**** ****Fundamentally impractical.**** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) ``encrypted_content`` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. ****Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.****

3. Competitor teardown

****Session-continuity tools:****

- ****tawanorg/claude-sync**** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/ .claude`` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + ****age E2E encryption**** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.
- ****npow/session-sync**** (~1 star, very new): the only cross-vendor ***session*** translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- ****dantelex/aisync ("ss")****: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- ****codsync/codsync**** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- ****kroxiksut/codexSync****: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- ****waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)**** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- ****iHildy/opencode-synced**** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- ****Dinesh3184/claude-session-sync****: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- ****renefichtmueller/claude-sync****: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

****Config/MCP/skills sync (DevSync's neighborhood):****

- ****Leoyang183/sync-agents-settings**** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- ****nicepkg/vsync**** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- ****baranovxyz/agentsync**** and ****spxrogers/agentsync**** (beta): canonical ``.agents/`` config synced to 9–31 agents.
- ****neon-solutions/add-mcp****: one-command MCP install across ~16 agents (Neon).
- ****ztripez/mcp-sync****: MCP config sync across tools.
- ****AgentSync (Rust)****: symlink-based config sync.
- ****skills-sync****: skills + MCP workspace.
- ****DevSynq**** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

****URL-addressable / mobile-control / viewer approaches:****

- ****TokenRip**** (tokenrip.com): makes session state a URL-addressable artifact (``/a/<slug>``) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- ****Omnara**** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was ****archived in February 2026**** because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent

SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)

- **Syncode – AI Programming Tool**: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- **jazzyalex/agent-sessions** & **kenn-io/agentsview**: local-first session browsers/search across 20+ agents (read/resume-command, not sync).
- **LAN-workspace approaches** and Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

Takeaway: The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- **Claude Code**: #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- **Codex**: #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI ↔ app-server sessions), discussion #14067 (cross-device thread/contextsync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
- **Copilot CLI**: #1947 (cloud-synced sessions), #1635 (cross-environment resume).
- **Cursor forum**: multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is. Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. **Cross-vendor unified profile** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.
2. **OS-aware path remapping as a first-class feature** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows ↔ macOS. This is the wedge no competitor has nailed.
3. **E2E encryption by default** – table stakes given session files contain code, secrets, and full transcripts.

Risks:

- **Format churn**: Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).
- **Vendor lock-out / ToS**: Storing session data is user-owned data (fine), but *auth tokens* are not transferable (codexSync explicitly refuses; Codex `encrypted\_content` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.
- **Vendors closing the gap**: Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This *validates* the demand while *confirming* that only the cross-vendor + cross-OS layer is defensible.

- ****Security/privacy expectations:**** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).
- ****Honest limits:**** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

****Stage 0 – Fold in DevSync (immediate).**** Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

****Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge).****
 Target the founder's exact case (Windows desktop 🖥️ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/ .claude/projects/` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- ****First-class OS-aware path remapping:**** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
- ****Benchmark to advance:**** `claude --resume` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

****Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode.**** Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid `encrypted\_content`/token pitfalls. For Gemini, remap `project\_hash`. For opencode, use safe SQLite snapshotting (the `sqlite3 .backup` pattern the community scripts use).

- ****Benchmark:**** each new adapter must pass the same cross-OS resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One "profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not replay.

****Monetization & thresholds.**** Free/open-source core (build trust, match claude-sync) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSync/OpenSync validate willingness to pay). ****Kill/pivot signal:**** if Anthropic ships account-level *local* session sync with automatic path remapping across OSes (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync,

Gemini-Antigravity migration) are evolving fast; specifics may have changed since the cited docs.

- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

Honest answer: nah — but for a reason that's fixable.

The *concept* is right. Parity is genuinely what your product delivers: two machines held at equivalence. That's the pitch. The problem is the compound.

Three collisions, one of them serious:

- "Open-source parity" is already a heavily-used phrase in AI discourse — open-weight models closing the gap with proprietary APIs. That's the fatal one. Your name would be competing for search against an entire ongoing industry conversation that has nothing to do with you.
- Parity Technologies owns the name in dev-tool mindshare — and note that the Ethereum client literally lived at `openethereum/parity-ethereum`. "Open" and "parity" have already been adjacent in a well-known namespace.
- There's also Parity Trading, open-source trading technologies on GitHub, plus "open parity games" as an established term in theoretical CS.

The structural issue: the `open-` convention pairs with a *thing* — code, router, telemetry, search. The second half names the noun the project provides an open version of. "Parity" is a state of affairs, not an artifact, so `openparity` parses as a slogan rather than a product. Compare how naturally `opencontext` or `openstate` read: you immediately know what's in the box. There's also a mechanical thing — five syllables with mushy stress (o-pen-PAIR-i-ty). `encode` and `router` land hard on the second element. This one trails off.

If you want to keep the parity idea, better vessels for it:

- **Parry** — near-homophone of parity, but it's a real short word, one clean beat, and `parry.dev` is the kind of thing that sounds like infra. Loses the literal meaning, keeps the sound.
- **Even** — "keep your machines even." Dead simple, real word, quietly does the same job. Very Linear-ish.
- **Level** — same family. "Level your setup across devices." Slightly more generic but very sayable.
- **openpar** — if you're attached to the root, truncating kills the slogan-parse and the syllable problem. Reads like a real package name. Bit cryptic though.

My honest ranking against your earlier shortlist: **Baton** and **openstate** still

beat all of these, because they name the artifact instead of describing the outcome. Parity is what your landing page headline should say — "your machines, at parity" — not what the repo should be called.

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

```
# Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape
```

```
## TL;DR
```

```
- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows↔macOS paths.
```

```
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows↔macOS sync.
```

```
- **Recommendation: build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode.** Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.
```

```
## Key Findings
```

```
1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
```

2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between ``C:\Users\...\`` and ``/Users/...\`` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.

3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.

4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

Claude Code (Anthropic). Ships the richest consumer-facing multi-device story:

- **``/teleport``** (``claude --teleport``): pulls a *cloud* session (started at claude.ai/code or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with ``--teleport``, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires **clean git state** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same claude.ai account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."
- **Remote Control** (``claude remote-control`` / ``/remote-control``, ``--rc``): pushes a *live local* session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the

terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an **undocumented** ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `ANTHROPIC\_BASE\_URL` is redirected; unavailable under Zero Data Retention.

- **Claude Code on the web** (claude.ai/code): runs on Anthropic cloud infra, no local filesystem.
- **Residual gap:** No true device-to-device **local** session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an **unexpected**, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

Codex CLI (OpenAI). The most architecturally mature cross-surface model:

- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared **App Server** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under `~/.codex/sessions/YYYY/MM/DD/rollout-\*.jsonl`, with a `session\_meta` first line.

- Rich lifecycle: `codex resume` (picker), `--last`, `--resume-session-id`, `/fork`, archive/unarchive (v0.136), delete (v0.140). Imports histories from other agents via an `external-agent-sessions` crate.

- **Known gaps/bugs:** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing `encrypted\_content` often can't be continued after crossing provider/account (`invalid\_encrypted\_content`); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.

- **Residual gap:** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

****Gemini CLI (Google).**** Auto-saves sessions to `~/ .gemini/tmp/<project_hash>/chats/`` (project-scoped, hash of project root). `/resume`` session browser, `--resume`/`-r`` flag, `/chat save`` checkpoints, and a shadow-git checkpointing system under `~/ .gemini/history/<project_hash>``. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. ****Residual gap:**** everything local + path-hash-keyed; no sync.

****opencode (sst).**** SQLite + file storage under `~/ .local/share/opencode/`` (`opencode.db``, `storage/session|message|part``). Native ****session sharing**** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). ****Residual gap:**** sharing $\neq$ syncing; DB concurrency and cross-OS path issues remain.

****Cursor CLI (cursor-agent).**** Chats live in local SQLite; CLI chats under `~/ .cursor/chats`` and resume via `cursor-agent --resume``. ****IDE chats and CLI chats do NOT share a session store**** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend `&`` to push to a Cloud Agent, pick up at `cursor.com/agents`), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). ****Residual gap:**** no local cross-device sync, IDE ↔ CLI split, path issues.

****Grok / Grok Build (xAI).**** Sessions auto-saved to `~/ .grok/sessions/``, keyed by working directory. `/resume``, `--resume``, `-c``, `/fork``; subscription-gated (SuperGrok/X Premium+). Config in `~/ .grok/config.toml``. No native cross-device sync. Still beta (v0.2.x as of July 2026). ****Residual gap:**** no sync at all.

****GitHub Copilot CLI.**** Sessions in `~/ .copilot/`` (`session-state/<uuid>/events.jsonl``). Notably, Copilot CLI ****syncs sessions to your GitHub account by default**** and offers ****remote control**** from `GitHub.com/GitHub Mobile` (origin machine must stay online, `/keep-alive``). But **\*\*cross-device \*resume\* is still not supported\*\*** – manually copying a session dir and running `--resume`` overwrites `events.jsonl`` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request

(#1947). The Copilot SDK supports resumable sessions via a user-supplied `session\_id`. **Residual gap:** the sync is telemetry/history, not resumable working state across machines.

Amp (Sourcegraph). Threads sync to ampcode.com by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). **Residual gap:** vendor-locked, server-stored, no cross-vendor.

Cross-vendor summary: Every vendor is building *within* its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

Tool	Location	Format	Path keying	Cross-device resume native?	
Claude Code	~/.claude/projects/<encoded-path>/<session>.jsonl	JSONL, append-only event stream	Absolute path, non-alphanumerics→dashes	Via cloud only (teleport/RC)	
Codex	~/.codex/sessions/YYYY/MM/DD/rollout-*.jsonl	JSONL + `session_meta` header	date-based; cwd in meta	Yes, via App Server	
Gemini CLI	~/.gemini/tmp/<project_hash>/chats/	JSON	Hash of project root	No	
opencode	~/.local/share/opencode/	(SQLite + files)	SQLite/JSON	project-keyed	Sharing only
Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)	
Grok	~/.grok/sessions/	JSON	working directory	No	
Copilot CLI	~/.copilot/session-state/<uuid>/events.jsonl	JSONL events	uuid	Account history sync; no resume	

The path-mapping problem (the crux). Claude Code derives the project folder name by escaping the absolute path:

`/Users/me/app` → `~/.claude/projects/-Users-me-app/`, while the same repo at `C:\Users\me\app` (or a different macOS username/location) produces a *different* folder. Consequently, after a naive file sync, `claude --resume` on the second machine

cannot find the first machine's sessions. This is confirmed by the claude-sync author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker." Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via `project\_hash`; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. ****This is the single highest-value, currently-unmet technical feature.****

****Format churn risk.**** Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

****Mid-session cross-device resume feasibility (same vendor):**** Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present — **if** path keys are remapped. Codex's App Server makes it cloud-native already.

****Cross-vendor session replay (e.g., Claude Code session → Codex):**** ****Fundamentally impractical.**** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) `encrypted\_content` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. ****Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.****

3. Competitor teardown

****Session-continuity tools:****

- ****tawanorg/claude-sync**** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/.claude`

(sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + **age E2E encryption** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. **Punts on path remapping** (its explicit, acknowledged weakness). Open-source, no business model.

- **npow/session-sync** (~1 star, very new): the only cross-vendor *session* translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- **dantelex/aisync ("ss")** : multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- **codsync/codsync** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- **kroxiksut/codexSync** : local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- **waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- **iHildy/opencode-synced** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- **Dinesh3184/claude-session-sync** : iCloud Drive sync GUI for Claude Code (macOS + Windows).
- **renefichtmueller/claude-sync** : multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

****Config/MCP/skills sync (DevSync's neighborhood):****

- **Leoyang183/sync-agents-settings** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- **nicepkg/vsync** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- **baranovxyz/agentsync** and **spxrogers/agentsync** (beta): canonical ``.agents/`` config synced to 9–31 agents.
- **neon-solutions/add-mcp** : one-command MCP install across ~16 agents (Neon).
- **ztripez/mcp-sync** : MCP config sync across tools.

- **AgentSync (Rust)**: symlink-based config sync.
- **skills-sync**: skills + MCP workspace.
- **DevSynq** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

URL-addressable / mobile-control / viewer approaches:

- **TokenRip** (tokenrip.com): makes session state a URL-addressable artifact (`/a/<slug>`) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- **Omnara** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was **archived in February 2026** because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)
- **Syncode – AI Programming Tool**: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- **jazzyalex/agent-sessions** & **kenn-io/agentsview**: local-first session browsers/search across 20+ agents (read/resume-command, not sync).

LAN-workspace approaches and Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

Takeaway: The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- **Claude Code**: #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.

- **Codex:** #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI ↔ app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
- **Copilot CLI:** #1947 (cloud-synced sessions), #1635 (cross-environment resume).
- **Cursor forum:** multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is. Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. **Cross-vendor unified profile** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.
2. **OS-aware path remapping as a first-class feature** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows ↔ macOS. This is the wedge no competitor has nailed.
3. **E2E encryption by default** – table stakes given session files contain code, secrets, and full transcripts.

Risks:

- **Format churn:** Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).
- **Vendor lock-out / ToS:** Storing session data is user-owned data (fine), but *auth tokens* are not transferable (codexSync explicitly refuses; Codex `encrypted\_content` breaks across accounts). Never sync credentials; never depend on undocumented

cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.

- ****Vendors closing the gap:**** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This **validates** the demand while **confirming** that only the cross-vendor + cross-OS layer is defensible.

- ****Security/privacy expectations:**** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).

- ****Honest limits:**** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

****Stage 0 – Fold in DevSync (immediate).**** Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

****Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge).**** Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/.claude/projects/` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- ****First-class OS-aware path remapping:**** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
- ****Benchmark to advance:**** `claude --resume` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption,

across 3 consecutive Claude Code releases.

****Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode.**** Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid `encrypted\_content`/token pitfalls. For Gemini, remap `project\_hash`. For opencode, use safe SQLite snapshotting (the `sqlite3 .backup` pattern the community scripts use).

- ****Benchmark:**** each new adapter must pass the same cross-OS resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One "profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not replay.

****Monetization & thresholds.**** Free/open-source core (build trust, match claude-sync) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSynq/OpenSync validate willingness to pay). ****Kill/pivot signal:**** if Anthropic ships account-level **local** session sync with automatic path remapping across OSes (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing

approach carries ongoing maintenance risk.

- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

new names

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape

TL;DR

- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows↔macOS paths.
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows↔macOS sync.
- **Recommendation: build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode.** Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...\` and `/Users/...\` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.
3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.
4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

1. Native vendor capability matrix (as of July 2026)

- Claude Code (Anthropic).** Ships the richest consumer-facing multi-device story:
 - **``/teleport``** (``claude --teleport``): pulls a *cloud* session (started at `claude.ai/code` or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport``, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires **clean git state** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same `claude.ai` account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."
 - **Remote Control** (``claude remote-control` / `/remote-control`, --rc`): pushes a live local session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an undocumented ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when ANTHROPIC_BASE_URL` is redirected; unavailable under Zero Data Retention.`
 - **Claude Code on the web** (`claude.ai/code`): runs on Anthropic cloud infra, no local filesystem.
 - **Residual gap**: No true device-to-device *local* session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an *unexpected*, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.
- Codex CLI (OpenAI).** The most architecturally mature cross-surface model:
 - CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared **App Server** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under `~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl``, with a `session_meta`` first line.
 - Rich lifecycle: `codex resume`` (picker), `--last``, `--resume-session-id``, `/fork``, `archive/unarchive (v0.136)`, `delete (v0.140)`. Imports histories from other agents via an `external-agent-sessions`` crate.
 - **Known gaps/bugs**: Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing `encrypted_content`` often can't be continued after crossing provider/account (`invalid_encrypted_content``); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.
 - **Residual gap**: Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.
- Gemini CLI (Google).** Auto-saves sessions to `~/gemini/tmp/<project_hash>/chats/`` (project-scoped, hash of project root). `/resume`` session browser, `--resume`/`-r`` flag, `/chat save`` checkpoints, and a shadow-git checkpointing system under `~/gemini/history/<project_hash>``. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers.
 - **Residual gap**: everything local + path-hash-keyed; no sync.
- opencode (sst).** SQLite + file storage under `~/local/share/opencode/`` (`opencode.db``, `storage/session|message|part``). Native **session sharing** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup).
 - **Residual gap**: sharing ≠ syncing; DB concurrency and cross-OS

path issues remain.

Cursor CLI (cursor-agent). Chats live in local SQLite; CLI chats under `~/cursor/chats`` and resume via `cursor-agent --resume``. **IDE chats and CLI chats do NOT share a session store** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend `&`` to push to a Cloud Agent, pick up at `cursor.com/agents`), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). **Residual gap:** no local cross-device sync, IDE↔CLI split, path issues.

Grok / Grok Build (xAI). Sessions auto-saved to `~/grok/sessions/``, keyed by working directory. `~/resume``, `--resume``, `-c``, `/fork``; subscription-gated (SuperGrok/X Premium+). Config in `~/grok/config.toml``. No native cross-device sync. Still beta (v0.2.x as of July 2026). **Residual gap:** no sync at all.

GitHub Copilot CLI. Sessions in `~/copilot/`` (`session-state/<uuid>/events.jsonl``). Notably, Copilot CLI **syncs sessions to your GitHub account by default** and offers **remote control** from `GitHub.com/GitHub Mobile` (origin machine must stay online, `/keep-alive``). But **cross-device resume** is still not supported – manually copying a session dir and running `--resume`` overwrites `events.jsonl`` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session_id``. **Residual gap:** the sync is telemetry/history, not resumable working state across machines.

Amp (Sourcegraph). Threads sync to `ampcode.com` by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). **Residual gap:** vendor-locked, server-stored, no cross-vendor.

Cross-vendor summary: Every vendor is building *within* its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

	Tool	Location	Format	Path keying	Cross-device resume native?
					--- --- --- --- ---
	Claude Code	<code>~/claude/projects/<encoded-path>/<session>.jsonl`</code>	JSONL, append-only event stream	Absolute path, non-alphanumerics→dashes	Via cloud only (teleport/RC)
	Codex	<code>~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl`</code>	JSONL + <code>session_meta`</code> header	date-based; cwd in meta	Yes, via App Server
	Gemini CLI	<code>~/gemini/tmp/<project_hash>/chats/`</code>	JSON	Hash of project root	No
	opencode	<code>~/local/share/opencode/`</code> (SQLite + files)	SQLite/JSON	project-keyed	Sharing only
	Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)
	Grok	<code>~/grok/sessions/`</code>	JSON	working directory	No
	Copilot CLI	<code>~/copilot/session-state/<uuid>/events.jsonl`</code>	JSONL events	uuid	Account history sync; no resume

The path-mapping problem (the crux). Claude Code derives the project folder name by escaping the absolute path: `/Users/me/app`` → `~/claude/projects/-Users-me-app/``, while the same repo at `C:\Users\me\app`` (or a different macOS username/location) produces a *different* folder. Consequently, after a naive file sync, `claude --resume`` on the second machine cannot find the first machine's sessions. This is confirmed by the `claude-sync` author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker."

Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via `project_hash``; Cursor stores path metadata in chat rows (`cursaves` rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. **This is the single highest-value, currently-unmet technical**

feature.**

Format churn risk. Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

Mid-session cross-device resume feasibility (same vendor): Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present – *if* path keys are remapped. Codex's App Server makes it cloud-native already.

Cross-vendor session replay (e.g., Claude Code session → Codex): **Fundamentally impractical.** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) `encrypted_content` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. **Recommendation:** treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.

3. Competitor teardown

Session-continuity tools:

- **tawanorg/claude-sync** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/.claude` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + **age** E2E encryption ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. **Punts on path remapping** (its explicit, acknowledged weakness). Open-source, no business model.
- **npow/session-sync** (~1 star, very new): the only cross-vendor *session* translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- **dantelex/aisync ("ss")**: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- **codsync/codsync** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- **kroxiksut/codexSync**: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- **waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- **iHildy/opencode-synced** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- **Dinesh3184/claude-session-sync**: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- **renefichtmueller/claude-sync**: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

Config/MCP/skills sync (DevSync's neighborhood):

- **Leoyang183/sync-agents-settings** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- **nicepkg/vsync** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- **baranovxyz/agentsync** and **spxrogers/agentsync** (beta): canonical `~/.agents/` config synced to 9–31 agents.
- **neon-solutions/add-mcp**: one-command MCP install across ~16 agents (Neon).
- **ztripez/mcp-sync**: MCP config sync across tools.
- **AgentSync (Rust)**: symlink-based config sync.
- **skills-sync**: skills + MCP workspace.
- **DevSynq** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

URL-addressable / mobile-control / viewer approaches:

- **TokenRip** (tokenrip.com): makes session state a URL-addressable artifact (`/a/<slug>`) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- **Omnara** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was **archived** in February 2026 because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)
- **Syncode – AI Programming Tool**: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- **jazzyalex/agent-sessions** & **kenn-io/agentsview**: local-first session browsers/search across 20+ agents (read/resume-command, not sync).
- **LAN-workspace approaches** and **Syncthing/NAS/dotfiles/chezmoi+age**: DIY approaches that work for config but hit the path problem for sessions.

Takeaway: The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- **Claude Code**: #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- **Codex**: #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI ↔ app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
- **Copilot CLI**: #1947 (cloud-synced sessions), #1635 (cross-environment resume).
- **Cursor forum**: multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is. Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. **Cross-vendor unified profile** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.
2. **OS-aware path remapping as a first-class feature** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows ↔ macOS. This is the wedge no competitor has nailed.
3. **E2E encryption by default** – table stakes given session files contain code, secrets, and full transcripts.

Risks:

- **Format churn**: Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).

- ****Vendor lock-out / ToS:**** Storing session data is user-owned data (fine), but **auth tokens** are not transferable (codexSync explicitly refuses; Codex ``encrypted_content`` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.
- ****Vendors closing the gap:**** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This **validates** the demand while **confirming** that only the cross-vendor + cross-OS layer is defensible.
- ****Security/privacy expectations:**** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).
- ****Honest limits:**** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

****Stage 0 – Fold in DevSync (immediate).**** Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

****Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge).****
 Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/ .claude/projects/`` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- ****First-class OS-aware path remapping:**** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
- ****Benchmark to advance:**** ``claude --resume`` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

****Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode.**** Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid ``encrypted_content`/token` pitfalls. For Gemini, remap ``project_hash``. For opencode, use safe SQLite snapshotting (the ``sqlite3 .backup`` pattern the community scripts use).

- ****Benchmark:**** each new adapter must pass the same cross-OS resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One "profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not replay.

****Monetization & thresholds.**** Free/open-source core (build trust, match claude-sync) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSync/OpenSync validate willingness to pay). ****Kill/pivot signal:**** if Anthropic ships account-level **local** session sync with automatic path remapping across OSes (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

open- names

- **openpair** — device pairing is the literal mental model. Two machines, paired. Shortest path to "I get it."
- **openattach** — `tmux attach` is exactly what your product does to a session. Terminal people will feel this one instantly.
- **openstash** — git-native vocabulary. Stash your session here, pop it there. Reads like a command you'd already know.
- **openhandoff** — names the moment rather than the mechanism. Desktop → bed. Slight Apple shadow, but it's the right word.
- **openspool** — a spool holds thread; a spooler holds work in flight. Both meanings land.
- **openmirror** — your second machine reflects the first. Plain, obvious, hard to misread.
- **opencairn** — cairns mark the trail so you can pick it back up. Distinctive, a little poetic for infra.
- **openwake** — the wake a ship leaves behind; also waking a session. Pretty, slightly abstract.
- **openresume** — it's the actual flag (`--resume`) in every one of these CLIs. Perfectly literal, but fights CVs in search.

Regular names

- **Cinch** — binds two things tight *and* means "trivially easy." Best double meaning on this list. Short, warm, real.
- **Weft** — the crosswise thread that ties the whole weave together. Four letters, unclaimed, quietly perfect for a threads-and-sessions product.
- **Shuttle** — carries the weft across the loom, and shuttles between two points. Two metaphors, one word.
- **Hitch** — hitching your machines together. Friendly, mechanical, easy to say out loud.
- **Berth** — where a vessel docks and rests until you come back for it. Nautical without being cute.

- **Skiff** — the small boat that runs between ship and shore. Sounds fast and light.
- **Junction** — where separate lines meet and continue. Sturdy, infrastructural.
- **Convey** — conveyance and conveyor belt. Slightly formal, very clear.
- **Tenon** — the joinery piece that slots into its mate. Obscure enough to own outright.
- **Satchel** — you carry your setup in it. Warmest option here; least "infra."

Conflicts to check before you fall in love: Shuttle (shuttle.rs, Rust deploys), Skiff (Notion's dead email product still has residue), Stow as a standalone (GNU Stow is a dotfile manager — too close to your space), Anchor, Relay, Latch, Clasp, Dovetail, Seam, and Rivet are all meaningfully claimed in dev tooling.

Strongest three across both lists, if you want a shortlist to sit with: **openpair**, **Cinch**, **Weft**.

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

```
# Cross-Device Sync for Terminal AI Coding Agents: Competitive &
Technical Landscape
```

```
## TL;DR
```

```
- **The durable opportunity is a cross-vendor abstraction layer
with OS-aware path remapping as its headline feature – NOT
another "sync my Claude sessions" tool.** Every major vendor is
now shipping single-ecosystem sync natively (Claude Code Remote
Control/teleport, Codex's unified App Server backend, Amp
threads, Copilot CLI cloud sync), so the defensible wedge is the
one thing no vendor will ever build: normalizing sessions + MCP +
skills + instruction files across *competing* tools and remapping
Windows↔macOS paths.
```

```
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex)
is effectively impossible and should be dropped from scope;**
config/skills/instruction portability and *same-vendor* cross-
device session continuity are the achievable, valuable core. The
single sharpest, currently-unsolved technical pain – which even
the leading competitor (claude-sync) explicitly punts on – is
Claude Code's absolute-path project indexing breaking on Windows
↔macOS sync.
```

```
- **Recommendation: build a "universal profile" that folds
DevSync's MCP-sync foundation in, adds E2E-encrypted session sync
with first-class path remapping for Claude Code first, then
Codex/Gemini/opencode.** Ship path remapping as the MVP
```

differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...\` and `/Users/...\` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.
3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.
4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

Claude Code (Anthropic). Ships the richest consumer-facing multi-device story:

- **`/teleport`** (`claude --teleport`): pulls a *cloud* session (started at `claude.ai/code` or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI,

session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport`, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires **clean git state** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same claude.ai account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."

- **Remote Control** (`claude remote-control` / `/remote-control`, --rc`): pushes a *live local* session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an *undocumented* ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `ANTHROPIC_BASE_URL` is redirected; unavailable under Zero Data Retention.

- **Claude Code on the web** (claude.ai/code): runs on Anthropic cloud infra, no local filesystem.

- **Residual gap:** No true device-to-device *local* session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an *unexpected*, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

Codex CLI (OpenAI). The most architecturally mature cross-surface model:

- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared **App Server** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under `~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl`, with a `session_meta` first line.
- Rich lifecycle: `codex resume` (picker), --last`, --resume-`

session-id`, `/fork`, archive/unarchive (v0.136), delete (v0.140). Imports histories from other agents via an `external-agent-sessions` crate.

- **Known gaps/bugs:** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing `encrypted\_content` often can't be continued after crossing provider/account (`invalid\_encrypted\_content`); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.

- **Residual gap:** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

Gemini CLI (Google). Auto-saves sessions to `~/.gemini/tmp/<project\_hash>/chats/` (project-scoped, hash of project root). `/resume` session browser, `--resume`/`-r` flag, `/chat save` checkpoints, and a shadow-git checkpointing system under `~/.gemini/history/<project\_hash>`. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. **Residual gap:** everything local + path-hash-keyed; no sync.

opencode (sst). SQLite + file storage under `~/.local/share/opencode/` (`opencode.db`, `storage/session|message|part`). Native **session sharing** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). **Residual gap:** sharing ≠ syncing; DB concurrency and cross-OS path issues remain.

Cursor CLI (cursor-agent). Chats live in local SQLite; CLI chats under `~/.cursor/chats` and resume via `cursor-agent --resume`. **IDE chats and CLI chats do NOT share a session store** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend `&` to push to a Cloud Agent, pick up at cursor.com/agents), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). **Residual gap:** no local cross-device sync, IDE ↔ CLI split, path issues.

Grok / Grok Build (xAI). Sessions auto-saved to `~/grok/sessions/``, keyed by working directory. `/resume``, `--resume``, `-c``, `/fork``; subscription-gated (SuperGrok/X Premium+). Config in `~/grok/config.toml``. No native cross-device sync. Still beta (v0.2.x as of July 2026). **Residual gap:** no sync at all.

GitHub Copilot CLI. Sessions in `~/copilot/`` (`session-state/<uuid>/events.jsonl``). Notably, Copilot CLI **syncs** sessions to your GitHub account by default and offers **remote control** from GitHub.com/GitHub Mobile (origin machine must stay online, `/keep-alive``). But **cross-device resume** is still not supported – manually copying a session dir and running `--resume`` overwrites `events.jsonl`` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session_id``. **Residual gap:** the sync is telemetry/history, not resumable working state across machines.

Amp (Sourcegraph). Threads sync to `ampcode.com` by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). **Residual gap:** vendor-locked, server-stored, no cross-vendor.

Cross-vendor summary: Every vendor is building *within* its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

Tool	Location	Format	Path keying	Cross-device resume native?
Claude Code	<code>~/claude/projects/<encoded-path>/<session>.jsonl`</code>	JSONL, append-only event stream	Absolute path, non-alphanumerics→dashes	Via cloud only (teleport/RC)
Codex	<code>~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl`</code>	JSONL + <code>session_meta`</code> header	date-based; cwd in meta	Yes, via App Server
Gemini CLI	<code>~/gemini/tmp/<project_hash>/chats/`</code>	JSON		

```
**Hash of project root** | No |  
| opencode | `~/.local/share/opencode/` (SQLite + files) |  
SQLite/JSON | project-keyed | Sharing only |  
| Cursor | local SQLite | DB rows | workspace/path metadata | No  
(cloud handoff only) |  
| Grok | `~/.grok/sessions/` | JSON | working directory | No |  
| Copilot CLI | `~/.copilot/session-state/<uuid>/events.jsonl` |  
JSONL events | uuid | Account history sync; no resume |
```

****The path-mapping problem (the crux).**** Claude Code derives the project folder name by escaping the absolute path: ``/Users/me/app`` → ``~/.claude/projects/-Users-me-app/``, while the same repo at ``C:\Users\me\app`` (or a different macOS username/location) produces a *different* folder. Consequently, after a naive file sync, ``claude --resume`` on the second machine cannot find the first machine's sessions. This is confirmed by the `claude-sync` author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker." Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via ``project_hash``; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. ****This is the single highest-value, currently-unmet technical feature.****

****Format churn risk.**** Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

****Mid-session cross-device resume feasibility (same vendor):**** Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present — *if* path keys are remapped. Codex's App Server makes it cloud-native already.

****Cross-vendor session replay (e.g., Claude Code session →**

Codex):** ****Fundamentally impractical.**** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) `encrypted\_content` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. ****Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.****

3. Competitor teardown

****Session-continuity tools:****

- ****tawanorg/claude-sync**** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/.claude` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + ****age E2E encryption**** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.
- ****npow/session-sync**** (~1 star, very new): the only cross-vendor *session* translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- ****dantelex/aisync ("ss")****: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- ****codsync/codsync**** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- ****kroxiksut/codexSync****: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- ****waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)**** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- ****iHildy/opencode-synced**** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- ****Dinesh3184/claude-session-sync****: iCloud Drive sync GUI for Claude Code (macOS + Windows).

- **renefichtmueller/claude-sync**: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

Config/MCP/skills sync (DevSync's neighborhood):

- **Leoyangl83/sync-agents-settings** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.

- **nicepkg/vsync** (~46 stars, active):

MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.

- **baranovxyz/agentsync** and **spxrogers/agentsync** (beta): canonical ``.agents/`` config synced to 9–31 agents.

- **neon-solutions/add-mcp**: one-command MCP install across ~16 agents (Neon).

- **ztripez/mcp-sync**: MCP config sync across tools.

- **AgentSync (Rust)**: symlink-based config sync.

- **skills-sync**: skills + MCP workspace.

- **DevSynq** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

URL-addressable / mobile-control / viewer approaches:

- **TokenRip** (tokenrip.com): makes session state a URL-addressable artifact (``/a/<slug>``) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").

- **Omnara** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was **archived in February 2026** because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)

- **Syncode – AI Programming Tool**: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).

- **jazzyalex/agent-sessions** & **kenn-io/agentsview**: local-first session browsers/search across 20+ agents (read/resume-command, not sync).

- **LAN-workspace approaches** and

Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

Takeaway: The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader

(claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- **Claude Code:** #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- **Codex:** #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI ↔ app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
- **Copilot CLI:** #1947 (cloud-synced sessions), #1635 (cross-environment resume).
- **Cursor forum:** multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is. Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. **Cross-vendor unified profile** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.

2. **OS-aware path remapping as a first-class feature** – the concrete, unsolved pain that makes same-vendor session sync

actually work across Windows↔macOS. This is the wedge no competitor has nailed.

3. **E2E encryption by default** – table stakes given session files contain code, secrets, and full transcripts.

Risks:

- **Format churn:** Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).
- **Vendor lock-out / ToS:** Storing session data is user-owned data (fine), but *auth tokens* are not transferable (codexSync explicitly refuses; Codex `encrypted_content` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.
- **Vendors closing the gap:** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This *validates* the demand while *confirming* that only the cross-vendor + cross-OS layer is defensible.
- **Security/privacy expectations:** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).
- **Honest limits:** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

Stage 0 – Fold in DevSync (immediate). Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge). Target the founder's exact case

(Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of
`~/.claude/projects/` and global config to user-owned storage
(S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- **First-class OS-aware path remapping:** on pull, rewrite the
project folder key AND in-file absolute path references (cwd, git
snapshots, file refs) from the source machine's path to the
target machine's path, using the git remote URL as the project
identity (the technique cursaves uses for Cursor). This is the
feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic
writes, conflict preservation.
- **Benchmark to advance:** `claude --resume` on the MacBook
reliably finds and resumes a session created on the Windows
desktop (different absolute paths) in <10s, with no corruption,
across 3 consecutive Claude Code releases.

**Stage 2 – Second and third vendors: Codex + Gemini CLI +
opencode.** Add adapters. For Codex, cooperate with (don't fight)
the App Server – sync what's local and useful, avoid
`encrypted\_content`/token pitfalls. For Gemini, remap
`project\_hash`. For opencode, use safe SQLite snapshotting (the
`sqlite3 .backup` pattern the community scripts use).

- **Benchmark:** each new adapter must pass the same cross-OS
resume test as Claude Code before GA.

Stage 3 – Unified profile + cross-vendor context handoff. One
"profile" = sessions + MCP + skills + agents + instructions,
portable across machines and (for config) across vendors. Add
best-effort context handoff (summarize + re-inject) for cross-
vendor moves – explicitly labeled as context transfer, not
replay.

Monetization & thresholds. Free/open-source core (build
trust, match claude-sync) with a paid tier for managed encrypted
storage, multi-machine dashboards, and team profiles
(DevSynq/OpenSync validate willingness to pay). **Kill/pivot
signal:** if Anthropic ships account-level *local* session sync
with automatic path remapping across OSes (not just history
mirroring, per #71794), the single-vendor wedge collapses – at
that point double down on cross-vendor config/skills/profile
portability, which no vendor will build.

What NOT to do: Don't build on Remote Control/teleport

(gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

new unique, never seen before, creative names

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape

TL;DR

- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows↔macOS paths.
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows↔macOS sync.
- **Recommendation: build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode.** Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. ****Native vendor sync is real but ecosystem-locked and gap-riddled.**** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
2. ****The path-mapping problem is the crux and is largely unsolved.**** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...\` and `/Users/...\` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.
3. ****Demand is loud and repeatedly expressed**** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.
4. ****The competitor field is fragmented, early, and mostly config-only or single-vendor.**** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

- **Claude Code (Anthropic).**** Ships the richest consumer-facing multi-device story:
 - ****`/teleport`**** (`claude --teleport`): pulls a **cloud** session (started at claude.ai/code or the mobile app) down into the local terminal. It is ****one-way**** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport`, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires ****clean git state**** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same claude.ai account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."
 - ****Remote Control**** (`claude remote-control` / `/remote-control`, `--rc`): pushes a **live local** session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an **undocumented** ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `ANTHROPIC\_BASE\_URL` is redirected; unavailable under Zero Data Retention.
 - ****Claude Code on the web**** (claude.ai/code): runs on Anthropic cloud infra, no local filesystem.
 - ****Residual gap:**** No true device-to-device **local** session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an **unexpected**, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.
- **Codex CLI (OpenAI).**** The most architecturally mature cross-surface model:
 - CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared ****App Server**** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under `~/.codex/sessions/YYYY/MM/DD/rollout-\*.jsonl`, with a `session_meta` first line.
 - Rich lifecycle: ``codex resume`` (picker), ``--last``, ``--resume-session-id``, ``/fork``, `archive/unarchive` (v0.136), `delete` (v0.140). Imports histories from other agents via an `external-agent-sessions`` crate.

- **Known gaps/bugs:** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing `encrypted\_content` often can't be continued after crossing provider/account (`invalid\_encrypted\_content`); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI↔app-server live sync.

- **Residual gap:** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

Gemini CLI (Google). Auto-saves sessions to `~/gemini/tmp/<project\_hash>/chats/` (project-scoped, hash of project root). `./resume` session browser, `--resume`/`-r` flag, `./chat save` checkpoints, and a shadow-git checkpointing system under `~/gemini/history/<project\_hash>`. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. **Residual gap:** everything local + path-hash-keyed; no sync.

opencode (sst). SQLite + file storage under `~/local/share/opencode/` (`opencode.db`, `storage/session|message|part`). Native **session sharing** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). **Residual gap:** sharing ≠ syncing; DB concurrency and cross-OS path issues remain.

Cursor CLI (cursor-agent). Chats live in local SQLite; CLI chats under `~/cursor/chats` and resume via `cursor-agent --resume`. **IDE chats and CLI chats do NOT share a session store** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend `&` to push to a Cloud Agent, pick up at cursor.com/agents), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). **Residual gap:** no local cross-device sync, IDE↔CLI split, path issues.

Grok / Grok Build (xAI). Sessions auto-saved to `~/grok/sessions/`, keyed by working directory. `./resume`, `--resume`, `-c`, `./fork`; subscription-gated (SuperGrok/X Premium+). Config in `~/grok/config.toml`. No native cross-device sync. Still beta (v0.2.x as of July 2026). **Residual gap:** no sync at all.

GitHub Copilot CLI. Sessions in `~/copilot/` (`session-state/<uuid>/events.jsonl`). Notably, Copilot CLI **syncs sessions to your GitHub account by default** and offers **remote control** from GitHub.com/GitHub Mobile (origin machine must stay online, `./keep-alive`). But **cross-device resume** is still not supported – manually copying a session dir and running `--resume` overwrites `events.jsonl` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session\_id`. **Residual gap:** the sync is telemetry/history, not resumable working state across machines.

Amp (Sourcegraph). Threads sync to ampcode.com by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). **Residual gap:** vendor-locked, server-stored, no cross-vendor.

Cross-vendor summary: Every vendor is building *within* its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

	Tool	Location	Format	Path keying	Cross-device resume native?
					--- --- --- --- ---
	Claude Code	~/claude/projects/<encoded-path>/<session>.jsonl	JSONL, append-only event stream	Absolute path, non-alphanumerics→dashes	Via cloud only (teleport/RC)
	Codex	~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl	JSONL + `session_meta` header	date-based; cwd in meta	Yes, via App Server
	Gemini CLI	~/gemini/tmp/<project_hash>/chats/	JSON	Hash of project root	No

opencode	~/ .local/share/opencode/`	(SQLite + files)	SQLite/JSON	project-keyed	Sharing only
Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)	
		Grok	~/ .grok/sessions/`	JSON	working directory
Copilot CLI	~/ .copilot/session-state/<uuid>/events.jsonl`		JSONL events	uuid	Account history sync; no resume

****The path-mapping problem (the crux).**** Claude Code derives the project folder name by escaping the absolute path: ``/Users/me/app`` → ``~/ .claude/projects/-Users-me-app/``, while the same repo at ``C:\Users\me\app`` (or a different macOS username/location) produces a *different* folder. Consequently, after a naive file sync, ``claude --resume`` on the second machine cannot find the first machine's sessions. This is confirmed by the claude-sync author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker."

Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via ``project_hash``; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. ****This is the single highest-value, currently-unmet technical feature.****

****Format churn risk.**** Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

****Mid-session cross-device resume feasibility (same vendor):**** Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present – *if* path keys are remapped. Codex's App Server makes it cloud-native already.

****Cross-vendor session replay (e.g., Claude Code session → Codex):**** ****Fundamentally impractical.**** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) ``encrypted_content`` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. ****Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.****

3. Competitor teardown

****Session-continuity tools:****

- ****tawanorg/claude-sync**** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs ``~/ .claude`` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + ****age E2E encryption**** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.
- ****npow/session-sync**** (~1 star, very new): the only cross-vendor *session* translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- ****dantelex/aisync ("ss")****: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- ****codsync/codsync**** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- ****kroxiksut/codexSync****: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- ****waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)**** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- ****iHildy/opencode-synced**** (~40 stars, active): opencode config + optional session sync via

- **Git or Turso** (concurrency-safe); opt-in sessions.
- ****Dinesh3184/claude-session-sync****: iCloud Drive sync GUI for Claude Code (macOS + Windows).
 - ****renefichtmueller/claude-sync****: multi-backend (Git/iCloud/Dropbox/Syncething/rsync) with snapshots/rollback and encryption.
- ****Config/MCP/skills sync (DevSync's neighborhood)****
- ****Leoyang183/sync-agents-settings**** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
 - ****nicepkg/vsync**** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
 - ****baranovxyz/agentsync**** and ****spxrogers/agentsync**** (beta): canonical ``.agents/`` config synced to 9–31 agents.
 - ****neon-solutions/add-mcp****: one-command MCP install across ~16 agents (Neon).
 - ****ztripez/mcp-sync****: MCP config sync across tools.
 - ****AgentSync (Rust)****: symlink-based config sync.
 - ****skills-sync****: skills + MCP workspace.
 - ****DevSynq**** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.
 - ****URL-addressable / mobile-control / viewer approaches****
 - ****TokenRip**** (tokenrip.com): makes session state a URL-addressable artifact (``/a/<slug>``) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
 - ****Omnara**** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was ****archived in February 2026**** because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)
 - ****Syncode – AI Programming Tool****: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
 - ****jazzyalex/agent-sessions**** & ****kenn-io/agentsview****: local-first session browsers/search across 20+ agents (read/resume-command, not sync).
 - ****LAN-workspace approaches**** and **Syncthing/NAS/dotfiles/chezmoi+age**: DIY approaches that work for config but hit the path problem for sessions.

****Takeaway:**** The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- ****Claude Code:**** #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- ****Codex:**** #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI  app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
 - ****Copilot CLI:**** #1947 (cloud-synced sessions), #1635 (cross-environment resume).
- ****Cursor forum:**** multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is. Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. **Cross-vendor unified profile** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.
2. **OS-aware path remapping as a first-class feature** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows↔macOS. This is the wedge no competitor has nailed.
3. **E2E encryption by default** – table stakes given session files contain code, secrets, and full transcripts.

Risks:

- **Format churn:** Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).
- **Vendor lock-out / ToS:** Storing session data is user-owned data (fine), but *auth tokens* are not transferable (codexSync explicitly refuses; Codex ``encrypted_content`` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.
 - **Vendors closing the gap:** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This *validates* the demand while *confirming* that only the cross-vendor + cross-OS layer is defensible.
- **Security/privacy expectations:** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).
- **Honest limits:** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

Stage 0 – Fold in DevSync (immediate). Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge). Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/\.claude/projects/`` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- **First-class OS-aware path remapping:** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
 - Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
 - **Benchmark to advance:** ``claude --resume`` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode. Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid ``encrypted_content`/token` pitfalls. For Gemini, remap ``project_hash``. For opencode, use safe SQLite snapshotting (the ``sqlite3 .backup`` pattern the community scripts use).

- **Benchmark:** each new adapter must pass the same cross-OS resume test as Claude Code

before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One "profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not replay.

****Monetization & thresholds.**** Free/open-source core (build trust, match claude-sync) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSynq/OpenSync validate willingness to pay). ****Kill/pivot signal:**** if Anthropic ships account-level *local* session sync with automatic path remapping across OSes (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini-Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

Fair warning up front: I checked a couple of these and **Filum** — my favorite coinage of the batch, Latin for "thread" — is dead. Filum.ai is a real, funded CX software company founded in 2020, with an active GitHub org doing LLM tooling. Killing my own darling so you don't have to. Everything below is coined rather than borrowed, so collision risk is lower — but verify before you buy anything.

open- coinages

- **opensyne** (*rhymes with "sign"*) — from *auld lang syne*, "the time since." Carries continuity across time, and quietly echoes sync without saying it.
- **openstatelet** — coined: the small portable unit of agent state. Ugly-cute in the way `applet` and `servlet` are, and instantly legible to any dev.
- **openperdure** — from *perdurance*, persisting unbroken through time. Rare enough that you'd own the whole search page on day one.
- **opencontigua** — from *contiguous*: unbroken, end touching end. Your work as one continuous span across two machines.
- **openmnemo** — Mnemosyne's root, memory itself. Short, mythic, and the `-mnemo-` stem is unclaimed in dev tooling.
- **openravelin** — a *ravelin* is an outwork that protects the main fort; *ravel*

is also thread. Double root, one word.

- **openvessa** — coined from *vessel*. The thing that carries your state between places.
- **openkairo** — from *kairos*, the opportune moment to resume. **Flag:** Kairos is a known face-recognition company; the stem isn't clean.

Regular coinages

- **Ferrata** — from *via ferrata*, the cabled mountain route you clip into so you can stop and resume safely. Gorgeous metaphor, no software claim on it that I know of.
- **Threadle** — *thread* + the *-le* of needle/handle. Coined, warm, and it tells you the domain in one syllable and a half.
- **Perdura** — from *perdurare*, to last all the way through. Sounds like real *infra*, means exactly your value prop.
- **Contigua** — the standalone version. "Your work, contiguous."
- **Statelet** — coined and blunt. The most engineer-brained name here; it'd fit right in next to *worktree* and *daemon*.
- **Syne** — four letters, unclaimed as a dev tool, and pleasingly ambiguous between "since" and "sync."
- **Vessa** — carries things, sounds like it's been around, hasn't been.
- **Mnemo** — memory, compressed. Short enough for a CLI verb: *mnemo push*.
- **Skeinly** — from *skein*, a coiled bundle of thread. More playful than the rest; probably too cute for *infra*, but it's here.

Near-misses I'd steer around: Tessel (JS hardware platform, dev-adjacent), Corda (R3 blockchain), Sillage (real French word, unpronounceable to English speakers), and anything on *-sync* (saturated).

If I'm ranking on how they'd look on a README header: **Ferrata**, **opensyne**, **Statelet**. Ferrata is the one with a story you can tell in a sentence — clip in, keep climbing, don't lose the route.

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

```
# Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape
```

```
## TL;DR
```

```
- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote
```

Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across **competing** tools and remapping Windows ↔ macOS paths.

- ****Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;**** config/skills/instruction portability and **same-vendor** cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows ↔ macOS sync.

- ****Recommendation: build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode.**** Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. ****Native vendor sync is real but ecosystem-locked and gap-riddled.**** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.

2. ****The path-mapping problem is the crux and is largely unsolved.**** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...\` and `/Users/...\` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.

3. ****Demand is loud and repeatedly expressed**** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their

own walls.

4. ****The competitor field is fragmented, early, and mostly config-only or single-vendor.**** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

****Claude Code (Anthropic).**** Ships the richest consumer-facing multi-device story:

- ****`/teleport`**** (``claude --teleport``): pulls a **cloud** session (started at `claude.ai/code` or the mobile app) down into the local terminal. It is ****one-way**** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport``, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires ****clean git state**** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same `claude.ai` account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."
- ****Remote Control**** (``claude remote-control` / `/remote-control`, --rc`): pushes a *live local* session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an *undocumented* ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `ANTHROPIC_BASE_URL` is redirected; unavailable under Zero Data Retention.`
- ****Claude Code on the web**** (`claude.ai/code`): runs on Anthropic cloud infra, no local filesystem.
- ****Residual gap:**** No true device-to-device **local** session

migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an **unexpected**, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

Codex CLI (OpenAI). The most architecturally mature cross-surface model:

- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared **App Server** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under `~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl``, with a ``session_meta`` first line.
- Rich lifecycle: ``codex resume`` (picker), ``--last``, ``--resume-session-id``, ``/fork``, archive/unarchive (v0.136), delete (v0.140). Imports histories from other agents via an ``external-agent-sessions`` crate.
- **Known gaps/bugs:** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing ``encrypted_content`` often can't be continued after crossing provider/account (``invalid_encrypted_content``); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.
- **Residual gap:** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

Gemini CLI (Google). Auto-saves sessions to `~/gemini/tmp/<project_hash>/chats/`` (project-scoped, hash of project root). ``/resume`` session browser, ``--resume`/`-r`` flag, ``/chat save`` checkpoints, and a shadow-git checkpointing system under `~/gemini/history/<project_hash>``. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. **Residual gap:** everything local + path-hash-keyed; no sync.

opencode (sst). SQLite + file storage under `~/local/share/opencode/`` (``opencode.db``, ``storage/session|message|part``). Native **session sharing**

publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup).
****Residual gap:**** sharing $\neq$ syncing; DB concurrency and cross-OS path issues remain.

****Cursor CLI (cursor-agent).**** Chats live in local SQLite; CLI chats under `~/ .cursor/chats`` and resume via ``cursor-agent --resume``. **\*\*IDE chats and CLI chats do NOT share a session store\*\*** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend ``&`` to push to a Cloud Agent, pick up at `cursor.com/agents`), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). ****Residual gap:**** no local cross-device sync, IDE↔CLI split, path issues.

****Grok / Grok Build (xAI).**** Sessions auto-saved to `~/ .grok/sessions/``, keyed by working directory. ``/resume``, ``--resume``, ``-c``, ``/fork``; subscription-gated (SuperGrok/X Premium+). Config in `~/ .grok/config.toml``. No native cross-device sync. Still beta (v0.2.x as of July 2026). ****Residual gap:**** no sync at all.

****GitHub Copilot CLI.**** Sessions in `~/ .copilot/`` (``session-state/<uuid>/events.jsonl``). Notably, Copilot CLI **\*\*syncs sessions to your GitHub account by default\*\*** and offers **\*\*remote control\*\*** from GitHub.com/GitHub Mobile (origin machine must stay online, ``/keep-alive``). But **\*\*cross-device \*resume\* is still not supported\*\*** – manually copying a session dir and running ``--resume`` overwrites ``events.jsonl`` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied ``session_id``. ****Residual gap:**** the sync is telemetry/history, not resumable working state across machines.

****Amp (Sourcegraph).**** Threads sync to `ampcode.com` by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). ****Residual gap:**** vendor-locked, server-stored, no cross-vendor.

****Cross-vendor summary:**** Every vendor is building **within** its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

Tool	Location	Format	Path keying	Cross-device resume native?
Claude Code	`\${HOME}/.claude/projects/<encoded-path>/<session>.jsonl`	JSONL, append-only event stream	**Absolute path, non-alphanumerics→dashes**	Via cloud only (teleport/RC)
Codex	`\${HOME}/.codex/sessions/YYYY/MM/DD/rollout-*.jsonl`	JSONL + `session_meta` header	date-based; cwd in meta	Yes, via App Server
Gemini CLI	`\${HOME}/.gemini/tmp/<project_hash>/chats/`	JSON	**Hash of project root**	No
opencode	`\${HOME}/.local/share/opencode/`	(SQLite + files)	SQLite/JSON	project-keyed Sharing only
Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)
Grok	`\${HOME}/.grok/sessions/`	JSON	working directory	No
Copilot CLI	`\${HOME}/.copilot/session-state/<uuid>/events.jsonl`	JSONL events	uuid	Account history sync; no resume

****The path-mapping problem (the crux).**** Claude Code derives the project folder name by escaping the absolute path: ``/Users/me/app`` → ``~/.claude/projects/-Users-me-app/``, while the same repo at ``C:\Users\me\app`` (or a different macOS username/location) produces a **different** folder. Consequently, after a naive file sync, ``claude --resume`` on the second machine cannot find the first machine's sessions. This is confirmed by the `claude-sync` author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker." Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via ``project_hash``; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path

references on pull. ****This is the single highest-value, currently-unmet technical feature.****

****Format churn risk.**** Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

****Mid-session cross-device resume feasibility (same vendor):**** Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present – *if* path keys are remapped. Codex's App Server makes it cloud-native already.

****Cross-vendor session replay (e.g., Claude Code session → Codex):**** ****Fundamentally impractical.**** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) `encrypted_content` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. ****Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.****

3. Competitor teardown

****Session-continuity tools:****

- ****tawanorg/claude-sync**** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/ .claude` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + ****age E2E encryption**** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.
- ****npow/session-sync**** (~1 star, very new): the only cross-vendor *session* translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- ****dantelex/aisync ("ss")****: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder

(iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)

- ****codsync/codsync**** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- ****kroxiksut/codexSync****: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- ****waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)**** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- ****iHildy/opencode-synced**** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- ****Dinesh3184/claude-session-sync****: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- ****renefichtmueller/claude-sync****: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

****Config/MCP/skills sync (DevSync's neighborhood):****

- ****Leoyang183/sync-agents-settings**** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- ****nicepkg/vsync**** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- ****baranovxyz/agentsync**** and ****spxrogers/agentsync**** (beta): canonical ``.agents/`` config synced to 9–31 agents.
- ****neon-solutions/add-mcp****: one-command MCP install across ~16 agents (Neon).
- ****ztripez/mcp-sync****: MCP config sync across tools.
- ****AgentSync (Rust)****: symlink-based config sync.
- ****skills-sync****: skills + MCP workspace.
- ****DevSynq**** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

****URL-addressable / mobile-control / viewer approaches:****

- ****TokenRip**** (tokenrip.com): makes session state a URL-addressable artifact (``/a/<slug>``) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- ****Omnara**** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo

(~2.6k stars) was **archived** in February 2026 because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)

- **Syncode – AI Programming Tool**: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- **jazzyalex/agent-sessions** & **kenn-io/agentsview**: local-first session browsers/search across 20+ agents (read/resume-command, not sync).
- **LAN-workspace approaches** and Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

Takeaway: The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- **Claude Code**: #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- **Codex**: #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI  app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
- **Copilot CLI**: #1947 (cloud-synced sessions), #1635 (cross-environment resume).
- **Cursor forum**: multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature

requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

****Where the durable moat is.**** Vendors will keep building ***intra*-ecosystem sync** (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. ****Cross-vendor unified profile**** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.

2. ****OS-aware path remapping as a first-class feature**** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows↔macOS. This is the wedge no competitor has nailed.

3. ****E2E encryption by default**** – table stakes given session files contain code, secrets, and full transcripts.

****Risks:****

- ****Format churn:**** Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).

- ****Vendor lock-out / ToS:**** Storing session data is user-owned data (fine), but ***auth tokens*** are not transferable (codexSync explicitly refuses; Codex `encrypted\_content` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.

- ****Vendors closing the gap:**** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This ***validates*** the demand while ***confirming*** that only the cross-vendor + cross-OS layer is defensible.

- ****Security/privacy expectations:**** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).

- ****Honest limits:**** Cross-vendor mid-session replay is

impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

****Stage 0 – Fold in DevSync (immediate).**** Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

****Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge).**** Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/.claude/projects/` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- ****First-class OS-aware path remapping:**** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
- ****Benchmark to advance:**** `claude --resume` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

****Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode.**** Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid `encrypted\_content`/token pitfalls. For Gemini, remap `project\_hash`. For opencode, use safe SQLite snapshotting (the `sqlite3 .backup` pattern the community scripts use).

- ****Benchmark:**** each new adapter must pass the same cross-OS resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One

"profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not replay.

****Monetization & thresholds.**** Free/open-source core (build trust, match claude-sync) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSynq/OpenSync validate willingness to pay). ****Kill/pivot signal:**** if Anthropic ships account-level **local** session sync with automatic path remapping across OSes (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

more, but this time, simpler

Cross-Device Sync for Terminal AI Coding Agents: Competitive and

Technical Landscape

Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape

TL;DR

- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows↔macOS paths.
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows↔macOS sync.
- **Recommendation:** build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode. Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...\` and `/Users/...\` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.
3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.
4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

- Claude Code (Anthropic).** Ships the richest consumer-facing multi-device story:
 - **`/teleport`** (`claude --teleport`): pulls a *cloud* session (started at claude.ai/code or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport`, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires **clean git state** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same claude.ai account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."
 - **Remote Control** (`claude remote-control` / `/remote-control`, `--rc`): pushes a *live local* session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local

process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an **undocumented** ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when ``ANTHROPIC_BASE_URL`` is redirected; unavailable under Zero Data Retention.

- **Claude Code on the web** (claude.ai/code): runs on Anthropic cloud infra, no local filesystem.

- **Residual gap:** No true device-to-device **local** session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an **unexpected**, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

Codex CLI (OpenAI). The most architecturally mature cross-surface model:

- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared **App Server** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under `~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl``, with a ``session_meta`` first line.

- Rich lifecycle: ``codex resume`` (picker), ``--last``, ``--resume-session-id``, ``/fork``, `archive/unarchive` (v0.136), `delete` (v0.140). Imports histories from other agents via an ``external-agent-sessions`` crate.

- **Known gaps/bugs:** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing ``encrypted_content`` often can't be continued after crossing provider/account (``invalid_encrypted_content``); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.

- **Residual gap:** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

Gemini CLI (Google). Auto-saves sessions to `~/gemini/tmp/<project_hash>/chats/`` (project-scoped, hash of project root). ``/resume`` session browser, ``--resume`` ``-r`` flag, ``/chat save`` checkpoints, and a shadow-git checkpointing system under `~/gemini/history/<project_hash>``. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. **Residual gap:** everything local + path-hash-keyed; no sync.

opencode (sst). SQLite + file storage under `~/local/share/opencode/`` (``opencode.db``, ``storage/session|message|part``). Native **session sharing** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). **Residual gap:** sharing ≠ syncing; DB concurrency and cross-OS path issues remain.

Cursor CLI (cursor-agent). Chats live in local SQLite; CLI chats under `~/cursor/chats`` and resume via ``cursor-agent --resume``. **IDE chats and CLI chats do NOT share a session store** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend ``&`` to push to a Cloud Agent, pick up at cursor.com/agents), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). **Residual gap:** no local cross-device sync, IDE ↔ CLI split, path issues.

Grok / Grok Build (xAI). Sessions auto-saved to `~/grok/sessions/``, keyed by working directory. ``/resume``, ``--resume``, ``-c``, ``/fork``; subscription-gated (SuperGrok/X Premium+). Config in `~/grok/config.toml``. No native cross-device sync. Still beta (v0.2.x as of July 2026). **Residual gap:** no sync at all.

GitHub Copilot CLI. Sessions in `~/copilot/`` (``session-state/<uuid>/events.jsonl``). Notably, Copilot CLI **syncs** sessions to your GitHub account by default and offers **remote control** from GitHub.com/GitHub Mobile (origin machine must stay online, ``/keep-alive``). But

****cross-device *resume* is still not supported**** – manually copying a session dir and running `--resume` overwrites `events.jsonl` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session_id`. ****Residual gap:**** the sync is telemetry/history, not resumable working state across machines.

****Amp (Sourcegraph).**** Threads sync to `ampcode.com` by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). ****Residual gap:**** vendor-locked, server-stored, no cross-vendor.

****Cross-vendor summary:**** Every vendor is building *within* its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

	Tool	Location	Format	Path keying	Cross-device resume native?
					--- --- --- --- ---
Claude Code		<code>~/ .claude/projects/<encoded-path>/<session>.jsonl</code>	JSONL, append-only event stream	**Absolute path, non-alphanumerics→dashes**	Via cloud only (teleport/RC)
Codex		<code>~/ .codex/sessions/YYYY/MM/DD/rollout-*.jsonl</code>	JSONL + <code>session_meta</code> header	date-based; cwd in meta	Yes, via App Server
Gemini CLI		<code>~/ .gemini/tmp/<project_hash>/chats/</code>	JSON	**Hash of project root**	No
opencode		<code>~/ .local/share/opencode/</code>	(SQLite + files)	SQLite/JSON	project-keyed
					Sharing only
Cursor		local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)
				Grok	<code>~/ .grok/sessions/</code>
Copilot CLI		<code>~/ .copilot/session-state/<uuid>/events.jsonl</code>	JSONL events	uuid	Account history sync; no resume

****The path-mapping problem (the crux).**** Claude Code derives the project folder name by escaping the absolute path: `/Users/me/app` → `~/ .claude/projects/-Users-me-app/`, while the same repo at `C:\Users\me\app` (or a different macOS username/location) produces a *different* folder. Consequently, after a naive file sync, `claude --resume` on the second machine cannot find the first machine's sessions. This is confirmed by the `claude-sync` author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker."

Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via `project_hash`; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. ****This is the single highest-value, currently-unmet technical feature.****

****Format churn risk.**** Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

****Mid-session cross-device resume feasibility (same vendor):**** Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present – *if* path keys are remapped. Codex's App Server makes it cloud-native already.

****Cross-vendor session replay (e.g., Claude Code session → Codex):**** ****Fundamentally impractical.**** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) `encrypted_content` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. ****Recommendation:** treat cross-vendor as config/skills/instruction portability +

context handoff, never as live session replay.**

3. Competitor teardown

Session-continuity tools:

- **tawanorg/claude-sync** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/.claude` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + **age E2E encryption** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. **Punts on path remapping** (its explicit, acknowledged weakness). Open-source, no business model.
 - **npow/session-sync** (~1 star, very new): the only cross-vendor **session** translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
 - **dantelex/aisync** ("ss"): multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
 - **codsync/codsync** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
 - **kroxiksut/codexSync**: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
 - **waynesutton/opencode-sync-plugin** & **codex-sync-plugin** (OpenSync) (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- **iHildy/opencode-synced** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- **Dinesh3184/claude-session-sync**: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- **renefichtmueller/claude-sync**: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

Config/MCP/skills sync (DevSync's neighborhood):

- **Leoyang183/sync-agents-settings** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
 - **nicepkg/vsync** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- **baranovxyz/agentsync** and **spxrogers/agentsync** (beta): canonical `~/.agents/` config synced to 9–31 agents.
 - **neon-solutions/add-mcp**: one-command MCP install across ~16 agents (Neon).
 - **ztripez/mcp-sync**: MCP config sync across tools.
 - **AgentSync** (Rust): symlink-based config sync.
 - **skills-sync**: skills + MCP workspace.
 - **DevSynq** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

URL-addressable / mobile-control / viewer approaches:

- **TokenRip** (tokenrip.com): makes session state a URL-addressable artifact (`/a/<slug>`) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- **Omnara** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was **archived** in February 2026 because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)
- **Syncode – AI Programming Tool**: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- **jazzyalex/agent-sessions** & **kenn-io/agentsview**: local-first session browsers/search across 20+ agents (read/resume-command, not sync).
- **LAN-workspace approaches** and Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

Takeaway: The config-sync lane (where DevSync sits) is crowded but shallow; the session-

sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- **Claude Code:** #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- **Codex:** #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI ↔ app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
- **Copilot CLI:** #1947 (cloud-synced sessions), #1635 (cross-environment resume).
- **Cursor forum:** multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is. Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. **Cross-vendor unified profile** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.
2. **OS-aware path remapping as a first-class feature** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows ↔ macOS. This is the wedge no competitor has nailed.
3. **E2E encryption by default** – table stakes given session files contain code, secrets, and full transcripts.

Risks:

- **Format churn:** Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).
- **Vendor lock-out / ToS:** Storing session data is user-owned data (fine), but *auth tokens* are not transferable (codexSync explicitly refuses; Codex `encrypted\_content` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.
- **Vendors closing the gap:** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This *validates* the demand while *confirming* that only the cross-vendor + cross-OS layer is defensible.
- **Security/privacy expectations:** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).
- **Honest limits:** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

Stage 0 – Fold in DevSync (immediate). Keep and harden the existing MCP-config sync as the

"config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

****Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge).****

Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/ .claude/projects/` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- ****First-class OS-aware path remapping:**** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
- ****Benchmark to advance:**** `claude --resume` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

****Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode.**** Add adapters. For

Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid `encrypted\_content`/token pitfalls. For Gemini, remap `project\_hash`. For opencode, use safe SQLite snapshotting (the `sqlite3 .backup` pattern the community scripts use).

- ****Benchmark:**** each new adapter must pass the same cross-OS resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One "profile" = sessions + MCP +

skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not replay.

****Monetization & thresholds.**** Free/open-source core (build trust, match claude-sync) with a

paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSynq/OpenSync validate willingness to pay). ****Kill/pivot signal:**** if Anthropic ships account-level *local* session sync with automatic path remapping across OSe (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.

- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

open- names

- **openhop** — hop from one machine to the other. Short, obvious, a little

fun.

- **openpass** — pass your session along. Reads like the verb it is.
- **openpack** — pack it up here, unpack it there.
- **openhold** — holds your state while you're away from the desk.
- **opendock** — dock and undock a session. Slight Docker shadow, but the metaphor is right.
- **openmove** — as plain as it gets. Move your work.
- **openlift** — lift a session off one machine, set it down on another.
- **opentote** — carries things. Warmer than the rest.
- **openpath** — the path-remapping problem, named. **Flag:** Openpath is a Motorola access-control product.

Regular names

- **Hop** — one syllable, one idea. Desktop to laptop, hop.
- **Carry** — what the product does, stated plainly. Warm and unpretentious.
- **Tote** — same idea, shorter, a bit more distinctive as a mark.
- **Span** — it spans two machines. Clean, infrastructural, one beat.
- **Twin** — your laptop is your desktop's twin. Instantly understood.
- **Swap** — swap machines mid-task. Very sayable.
- **Perch** — where you land and pick things back up. Light, unclaimed.
- **Knot** — ties the two ends of your day together. Threads vocabulary, minus the cleverness.
- **Cart** — hauls your setup around. Plain to the point of charming.

Taken, skip: Beam (Erlang's BEAM VM — devs will misread it instantly), Keep (Google), Trunk (trunk.io, dev tooling), Nest (Google), Crate (Rust).

Simplest three that still feel like a product: **Hop, Carry, openhop**. Hop is the one you can say out loud in a sentence without explaining — "just hop it over to the laptop."

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

```
# Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape
```

```
## TL;DR
```

```
- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature — NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp
```

threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows↔macOS paths.

- **Mid-session cross-vendor replay** (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope; **config/skills/instruction portability** and ***same-vendor* cross-device session continuity** are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows↔macOS sync.

- **Recommendation:** build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opcode. Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.

2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...\` and `/Users/...\` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.

3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.

4. ****The competitor field is fragmented, early, and mostly config-only or single-vendor.**** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

****Claude Code (Anthropic).**** Ships the richest consumer-facing multi-device story:

- ****`/teleport`**** (``claude --teleport``): pulls a **cloud** session (started at `claude.ai/code` or the mobile app) down into the local terminal. It is ****one-way**** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with ``--teleport``, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires ****clean git state**** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same `claude.ai` account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."
- ****Remote Control**** (``claude remote-control`` / ``/remote-control``, ``--rc``): pushes a **live local** session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an **undocumented** ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when ``ANTHROPIC_BASE_URL`` is redirected; unavailable under Zero Data Retention.
- ****Claude Code on the web**** (`claude.ai/code`): runs on Anthropic cloud infra, no local filesystem.
- ****Residual gap:**** No true device-to-device **local** session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun

syncing conversation history across devices on the same account (reported in issue #71794 as an **unexpected**, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

****Codex CLI (OpenAI).**** The most architecturally mature cross-surface model:

- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared ****App Server**** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under `~/.codex/sessions/YYYY/MM/DD/rollout-*.jsonl``, with a ``session_meta`` first line.
- Rich lifecycle: ``codex resume`` (picker), ``--last``, ``--resume-session-id``, ``/fork``, archive/unarchive (v0.136), delete (v0.140). Imports histories from other agents via an ``external-agent-sessions`` crate.
- ****Known gaps/bugs:**** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing ``encrypted_content`` often can't be continued after crossing provider/account (``invalid_encrypted_content``); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.
- ****Residual gap:**** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

****Gemini CLI (Google).**** Auto-saves sessions to `~/.gemini/tmp/<project_hash>/chats/`` (project-scoped, hash of project root). ``/resume`` session browser, ``--resume`/`-r`` flag, ``/chat save`` checkpoints, and a shadow-git checkpointing system under `~/.gemini/history/<project_hash>``. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. ****Residual gap:**** everything local + path-hash-keyed; no sync.

****opencode (sst).**** SQLite + file storage under `~/.local/share/opencode/`` (``opencode.db``, ``storage/session|message|part``). Native ****session sharing**** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party

ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup).
Residual gap: sharing $\neq$ syncing; DB concurrency and cross-OS path issues remain.

Cursor CLI (cursor-agent). Chats live in local SQLite; CLI chats under `~/cursor/chats`` and resume via `cursor-agent --resume``. **IDE chats and CLI chats do NOT share a session store** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend `&`` to push to a Cloud Agent, pick up at cursor.com/agents), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). **Residual gap:** no local cross-device sync, IDE ↔ CLI split, path issues.

Grok / Grok Build (xAI). Sessions auto-saved to `~/grok/sessions/``, keyed by working directory. `/resume``, `--resume``, `-c``, `/fork``; subscription-gated (SuperGrok/X Premium+). Config in `~/grok/config.toml``. No native cross-device sync. Still beta (v0.2.x as of July 2026). **Residual gap:** no sync at all.

GitHub Copilot CLI. Sessions in `~/copilot/`` (`session-state/<uuid>/events.jsonl``). Notably, Copilot CLI **syncs sessions to your GitHub account by default** and offers **remote control** from [GitHub.com/GitHub Mobile](https://github.com/GitHubMobile) (origin machine must stay online, `/keep-alive``). But **cross-device resume** is still not supported – manually copying a session dir and running `--resume`` overwrites `events.jsonl`` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session_id``. **Residual gap:** the sync is telemetry/history, not resumable working state across machines.

Amp (Sourcegraph). Threads sync to ampcode.com by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). **Residual gap:** vendor-locked, server-stored, no cross-vendor.

Cross-vendor summary: Every vendor is building *within* its ecosystem. The white space is precisely between ecosystems and

across OSes.

2. Session storage formats & technical feasibility

Tool	Location	Format	Path keying	Cross-device resume native?
Claude Code	`\${HOME}/.claude/projects/<encoded-path>/<session>.jsonl`	JSONL, append-only event stream	**Absolute path, non-alphanumerics→dashes**	Via cloud only (teleport/RC)
Codex	`\${HOME}/.codex/sessions/YYYY/MM/DD/rollout-*.jsonl`	JSONL + `session_meta` header	date-based; cwd in meta	Yes, via App Server
Gemini CLI	`\${HOME}/.gemini/tmp/<project_hash>/chats/`	JSON	**Hash of project root**	No
opencode	`\${HOME}/.local/share/opencode/`	(SQLite + files)	SQLite/JSON	project-keyed
Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)
Grok	`\${HOME}/.grok/sessions/`	JSON	working directory	No
Copilot CLI	`\${HOME}/.copilot/session-state/<uuid>/events.jsonl`	JSONL events	uuid	Account history sync; no resume

****The path-mapping problem (the crux).**** Claude Code derives the project folder name by escaping the absolute path: ``/Users/me/app`` → ``~/.claude/projects/-Users-me-app/``, while the same repo at ``C:\Users\me\app`` (or a different macOS username/location) produces a *different* folder. Consequently, after a naive file sync, ``claude --resume`` on the second machine cannot find the first machine's sessions. This is confirmed by the `claude-sync` author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker." Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via ``project_hash``; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. ****This is the single highest-value, currently-unmet technical feature.****

****Format churn risk.**** Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

****Mid-session cross-device resume feasibility (same vendor):**** Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present – **if** path keys are remapped. Codex's App Server makes it cloud-native already.

****Cross-vendor session replay (e.g., Claude Code session → Codex):**** ****Fundamentally impractical.**** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) ``encrypted_content`` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. ****Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.****

3. Competitor teardown

****Session-continuity tools:****

- ****tawanorg/claude-sync**** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/ .claude`` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + ****age E2E encryption**** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.
- ****npow/session-sync**** (~1 star, very new): the only cross-vendor **session** translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- ****dantelex/aisync ("ss")****: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)

- ****codsync/codsync**** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- ****kroxiksut/codexSync****: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- ****waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)**** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- ****iHildy/opencode-synced**** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- ****Dinesh3184/claude-session-sync****: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- ****renefichtmueller/claude-sync****: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

****Config/MCP/skills sync (DevSync's neighborhood):****

- ****Leoyangl83/sync-agents-settings**** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- ****nicepkg/vsync**** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- ****baranovxyz/agentsync**** and ****spxrogers/agentsync**** (beta): canonical ``.agents/`` config synced to 9–31 agents.
- ****neon-solutions/add-mcp****: one-command MCP install across ~16 agents (Neon).
- ****ztripez/mcp-sync****: MCP config sync across tools.
- ****AgentSync (Rust)****: symlink-based config sync.
- ****skills-sync****: skills + MCP workspace.
- ****DevSynq**** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

****URL-addressable / mobile-control / viewer approaches:****

- ****TokenRip**** (tokenrip.com): makes session state a URL-addressable artifact (``/a/<slug>``) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- ****Omnara**** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was ****archived in February 2026**** because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the

company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)

- ****Syncode – AI Programming Tool****: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).

- ****jazzyalex/agent-sessions**** & ****kenn-io/agentsview****: local-first session browsers/search across 20+ agents (read/resume-command, not sync).

- ****LAN-workspace approaches**** and

Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

****Takeaway:**** The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- ****Claude Code:**** #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.

- ****Codex:**** #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI  app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).

- ****Copilot CLI:**** #1947 (cloud-synced sessions), #1635 (cross-environment resume).

- ****Cursor forum:**** multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

****Where the durable moat is.**** Vendors will keep building **intra**-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. ****Cross-vendor unified profile**** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.

2. ****OS-aware path remapping as a first-class feature**** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows↔macOS. This is the wedge no competitor has nailed.

3. ****E2E encryption by default**** – table stakes given session files contain code, secrets, and full transcripts.

****Risks:****

- ****Format churn:**** Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).

- ****Vendor lock-out / ToS:**** Storing session data is user-owned data (fine), but **auth tokens** are not transferable (codexSync explicitly refuses; Codex `encrypted\_content` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.

- ****Vendors closing the gap:**** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This **validates** the demand while **confirming** that only the cross-vendor + cross-OS layer is defensible.

- ****Security/privacy expectations:**** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).

- ****Honest limits:**** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

****Stage 0 – Fold in DevSync (immediate).**** Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

****Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge).**** Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/.claude/projects/` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- ****First-class OS-aware path remapping:**** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
- ****Benchmark to advance:**** `claude --resume` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

****Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode.**** Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid `encrypted\_content`/token pitfalls. For Gemini, remap `project\_hash`. For opencode, use safe SQLite snapshotting (the `sqlite3 .backup` pattern the community scripts use).

- ****Benchmark:**** each new adapter must pass the same cross-OS resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One "profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not

replay.

****Monetization & thresholds.**** Free/open-source core (build trust, match claude-sync) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSynq/OpenSync validate willingness to pay). ****Kill/pivot signal:**** if Anthropic ships account-level **local** session sync with automatic path remapping across OSes (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

nice but all names are very short, i prefer 6-9 letters/chars per word

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape

TL;DR

- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows↔macOS paths.
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows↔macOS sync.
- **Recommendation:** build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode. Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...` and `/Users/...` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.
3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.
4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

- **Claude Code (Anthropic).** Ships the richest consumer-facing multi-device story:
 - **`/teleport`** (`claude --teleport`): pulls a *cloud* session (started at claude.ai/code or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport`, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires **clean git state** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same claude.ai account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."
 - **Remote Control** (`claude remote-control` / `/remote-control`, `--rc`): pushes a *live local* session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an *undocumented* ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research

preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `ANTHROPIC\_BASE\_URL` is redirected; unavailable under Zero Data Retention.

- **Claude Code on the web** (claude.ai/code): runs on Anthropic cloud infra, no local filesystem.

- **Residual gap:** No true device-to-device **local** session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an **unexpected**, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

Codex CLI (OpenAI). The most architecturally mature cross-surface model:

- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared **App Server** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under `~/.codex/sessions/YYYY/MM/DD/rollout-\*.jsonl`, with a `session_meta` first line.

- Rich lifecycle: `codex resume` (picker), `--last`, `--resume-session-id`, `/fork`, `archive/unarchive` (v0.136), `delete` (v0.140). Imports histories from other agents via an `external-agent-sessions` crate.

- **Known gaps/bugs:** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing `encrypted_content` often can't be continued after crossing provider/account (`invalid_encrypted_content`); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.

- **Residual gap:** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

Gemini CLI (Google). Auto-saves sessions to `~/.gemini/tmp/<project_hash>/chats/`` (project-scoped, hash of project root). `/resume` session browser, `--resume`-r`` flag, `/chat save` checkpoints, and a shadow-git checkpointing system under `~/.gemini/history/<project_hash>``. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. **Residual gap:** everything local + path-hash-keyed; no sync.

opencode (sst). SQLite + file storage under `~/.local/share/opencode/`` (`opencode.db``, `storage/session|message|part``). Native **session sharing** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). **Residual gap:** sharing ≠ syncing; DB concurrency and cross-OS path issues remain.

Cursor CLI (cursor-agent). Chats live in local SQLite; CLI chats under `~/.cursor/chats`` and resume via `cursor-agent --resume``. **IDE chats and CLI chats do NOT share a session store** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend `&`` to push to a Cloud Agent, pick up at cursor.com/agents), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). **Residual gap:** no local cross-device sync, IDE ↔ CLI split, path issues.

Grok / Grok Build (xAI). Sessions auto-saved to `~/.grok/sessions/``, keyed by working directory. `/resume``, `--resume``, `-c``, `/fork``; subscription-gated (SuperGrok/X Premium+). Config in `~/.grok/config.toml``. No native cross-device sync. Still beta (v0.2.x as of July 2026). **Residual gap:** no sync at all.

GitHub Copilot CLI. Sessions in `~/.copilot/`` (`session-state/<uuid>/events.jsonl``). Notably, Copilot CLI **syncs** sessions to your GitHub account by default and offers **remote control** from GitHub.com/GitHub Mobile (origin machine must stay online, `/keep-alive``). But **cross-device resume** is still not supported – manually copying a session dir and running `--resume`` overwrites `events.jsonl`` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session_id``. **Residual gap:** the sync is telemetry/history, not resumable working state across machines.

Go binary; syncs `~/.claude` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + **age** E2E encryption ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. **Punts on path remapping** (its explicit, acknowledged weakness). Open-source, no business model.

- **npow/session-sync** (~1 star, very new): the only cross-vendor **session** translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- **dantelex/aisync** ("ss"): multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- **codsync/codsync** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- **kroxiksut/codexSync**: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- **waynesutton/opencode-sync-plugin** & **codex-sync-plugin** (OpenSync) (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- **iHildy/opencode-synced** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- **Dinesh3184/claude-session-sync**: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- **renefichtmueller/claude-sync**: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

Config/MCP/skills sync (DevSync's neighborhood):

- **Leoyang183/sync-agents-settings** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- **nicepkg/vsync** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- **baranovxyz/agentsync** and **spxrogers/agentsync** (beta): canonical `~/.agents/` config synced to 9–31 agents.
- **neon-solutions/add-mcp**: one-command MCP install across ~16 agents (Neon).
- **ztripez/mcp-sync**: MCP config sync across tools.
- **AgentSync** (Rust): symlink-based config sync.
- **skills-sync**: skills + MCP workspace.
- **DevSynq** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

URL-addressable / mobile-control / viewer approaches:

- **TokenRip** (tokenrip.com): makes session state a URL-addressable artifact (`~/a/<slug>`) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- **Omnara** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was **archived** in February 2026 because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)
- **Syncode – AI Programming Tool**: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- **jazzyalex/agent-sessions** & **kenn-io/agentsview**: local-first session browsers/search across 20+ agents (read/resume-command, not sync).
- **LAN-workspace approaches** and **Syncthing/NAS/dotfiles/chezmoi+age**: DIY approaches that work for config but hit the path problem for sessions.

Takeaway: The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- **Claude Code:** #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- **Codex:** #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI ↔ app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
 - **Copilot CLI:** #1947 (cloud-synced sessions), #1635 (cross-environment resume).
- **Cursor forum:** multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is. Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. **Cross-vendor unified profile** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.
2. **OS-aware path remapping as a first-class feature** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows ↔ macOS. This is the wedge no competitor has nailed.
3. **E2E encryption by default** – table stakes given session files contain code, secrets, and full transcripts.

Risks:

- **Format churn:** Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).
- **Vendor lock-out / ToS:** Storing session data is user-owned data (fine), but *auth tokens* are not transferable (codexSync explicitly refuses; Codex `encrypted\_content` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.
 - **Vendors closing the gap:** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This *validates* the demand while *confirming* that only the cross-vendor + cross-OS layer is defensible.
- **Security/privacy expectations:** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).
 - **Honest limits:** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

Stage 0 – Fold in DevSync (immediate). Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

****Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge).****
 Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/ .claude/projects/` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- ****First-class OS-aware path remapping:**** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
- ****Benchmark to advance:**** `claude --resume` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

****Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode.**** Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid `encrypted\_content`/token pitfalls. For Gemini, remap `project\_hash`. For opencode, use safe SQLite snapshotting (the `sqlite3 .backup` pattern the community scripts use).

- ****Benchmark:**** each new adapter must pass the same cross-OS resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One "profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not replay.

****Monetization & thresholds.**** Free/open-source core (build trust, match claude-sync) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSynq/OpenSync validate willingness to pay). ****Kill/pivot signal:**** if Anthropic ships account-level *local* session sync with automatic path remapping across OSe (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

Reading the constraint as applying to the meaningful word — "open" adds four characters on its own, so the suffixes below are 6–7 and the standalone names are 6–9.

open- names

- **openparcel** — a parcel is a thing packed up and sent somewhere. Plain and exact.

- **openbundle** — bundle up your setup, unbundle it on the other machine.
- **openpocket** — everything you'd carry in a pocket, available anywhere.
- **openholdall** — a holdall literally means "holds all your stuff." Nice fit for a unified profile.
- **openpassage** — the crossing between one machine and the next.
- **openrejoin** — you rejoin the session you left. Simple verb, does the work.
- **openlayover** — the pause between two legs of the same trip. Your session, mid-journey.
- **openroaming** — telecom roaming: your setup follows you. **Flag:** OpenRoaming is an existing WBA Wi-Fi standard.
- **opentransit** — plain and infrastructural. Mild overlap with Vault's transit engine.

Regular names

- **Carryall** — a bag built to hold everything you need. Probably the single most on-the-nose name I've given you, in a good way.
- **Holdall** — the British version of the same word. Slightly more distinctive as a mark.
- **Layover** — where you stop, then continue the same journey. Travel-warm, not techy.
- **Foothold** — the place you get your grip back. Good for "resume where you left off."
- **Knapsack** — you carry your environment on your back. Familiar, concrete, unclaimed in dev tooling.
- **Mooring** — where a boat is tied up waiting for you to return to it.
- **Stowage** — where things are kept safely in transit.
- **Overlap** — the shared state between two machines. Quietly technical.
- **Rejoin** — the action, named directly. Reads well as a command: `rejoin desktop`.

Taken, skip: Waypoint (HashiCorp), Sidecar (Apple's feature *and* the k8s pattern), Handoff (Apple Continuity — thematically perfect, legally unwise), Courier (courier.com is a real dev-tool company), Duplex (Google), Passport (Passport.js).

Best three by feel: **Carryall**, **Holdall**, **openparcel**. Carryall and Holdall both say "one bag, everything in it, goes where you go," which is your product described without a single metaphor to explain.

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape

TL;DR

- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows ↔ macOS paths.
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows ↔ macOS sync.
- **Recommendation:** build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode. Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...` and `/Users/...` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly

admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.

3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.

4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

Claude Code (Anthropic). Ships the richest consumer-facing multi-device story:

- **`/teleport`** (``claude --teleport``): pulls a *cloud* session (started at `claude.ai/code` or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport``, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires **clean git state** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same `claude.ai` account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."
- **Remote Control** (``claude remote-control`` / ``/remote-control``, `--rc``): pushes a *live local* session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an *undocumented* ~15-minute idle timeout on the Desktop/Code

surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `ANTHROPIC\_BASE\_URL` is redirected; unavailable under Zero Data Retention.

- **Claude Code on the web** (claude.ai/code): runs on Anthropic cloud infra, no local filesystem.

- **Residual gap:** No true device-to-device **local** session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an *unexpected*, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

Codex CLI (OpenAI). The most architecturally mature cross-surface model:

- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared **App Server** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under

`~/.codex/sessions/YYYY/MM/DD/rollout-\*.jsonl`, with a `session\_meta` first line.

- Rich lifecycle: `codex resume` (picker), `--last`, `--resume-session-id`, `/fork`, archive/unarchive (v0.136), delete (v0.140). Imports histories from other agents via an `external-agent-sessions` crate.

- **Known gaps/bugs:** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing `encrypted\_content` often can't be continued after crossing provider/account (`invalid\_encrypted\_content`); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.

- **Residual gap:** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

Gemini CLI (Google). Auto-saves sessions to

`~/.gemini/tmp/<project\_hash>/chats/` (project-scoped, hash of project root). `/resume` session browser, `--resume`/`-r` flag, `/chat save` checkpoints, and a shadow-git checkpointing system

under `~/.gemini/history/<project\_hash>`. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. **Residual gap:** everything local + path-hash-keyed; no sync.

opencode (sst). SQLite + file storage under `~/.local/share/opencode/` (`opencode.db`, `storage/session|message|part`). Native **session sharing** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). **Residual gap:** sharing ≠ syncing; DB concurrency and cross-OS path issues remain.

Cursor CLI (cursor-agent). Chats live in local SQLite; CLI chats under `~/.cursor/chats` and resume via `cursor-agent --resume`. **IDE chats and CLI chats do NOT share a session store** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend `&` to push to a Cloud Agent, pick up at cursor.com/agents), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). **Residual gap:** no local cross-device sync, IDE ↔ CLI split, path issues.

Grok / Grok Build (xAI). Sessions auto-saved to `~/.grok/sessions/`, keyed by working directory. `/resume`, `--resume`, `-c`, `/fork`; subscription-gated (SuperGrok/X Premium+). Config in `~/.grok/config.toml`. No native cross-device sync. Still beta (v0.2.x as of July 2026). **Residual gap:** no sync at all.

GitHub Copilot CLI. Sessions in `~/.copilot/` (`session-state/<uuid>/events.jsonl`). Notably, Copilot CLI **syncs sessions to your GitHub account by default** and offers **remote control** from [GitHub.com/GitHub Mobile](https://github.com/GitHubMobile) (origin machine must stay online, `/keep-alive`). But **cross-device resume** is still not supported – manually copying a session dir and running `--resume` overwrites `events.jsonl` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session\_id`. **Residual gap:** the sync is telemetry/history, not resumable working state across machines.

****Amp (Sourcegraph).**** Threads sync to ampcode.com by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). ****Residual gap:**** vendor-locked, server-stored, no cross-vendor.

****Cross-vendor summary:**** Every vendor is building **within** its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

Tool	Location	Format	Path keying	Cross-device resume native?
Claude Code	`\${HOME}/.claude/projects/<encoded-path>/<session>.jsonl`	JSONL, append-only event stream	**Absolute path, non-alphanumerics→dashes**	Via cloud only (teleport/RC)
Codex	`\${HOME}/.codex/sessions/YYYY/MM/DD/rollout-*.jsonl`	JSONL + `session_meta` header	date-based; cwd in meta	Yes, via App Server
Gemini CLI	`\${HOME}/.gemini/tmp/<project_hash>/chats/`	JSON	**Hash of project root**	No
opencode	`\${HOME}/.local/share/opencode/`	(SQLite + files)	SQLite/JSON	project-keyed
Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)
Grok	`\${HOME}/.grok/sessions/`	JSON	working directory	No
Copilot CLI	`\${HOME}/.copilot/session-state/<uuid>/events.jsonl`	JSONL events	uuid	Account history sync; no resume

****The path-mapping problem (the crux).**** Claude Code derives the project folder name by escaping the absolute path: ``/Users/me/app`` → ``~/.claude/projects/-Users-me-app/``, while the same repo at ``C:\Users\me\app`` (or a different macOS username/location) produces a **different** folder. Consequently, after a naive file sync, ``claude --resume`` on the second machine cannot find the first machine's sessions. This is confirmed by the claude-sync author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation?"

Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker." Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via `project\_hash`; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. **This is the single highest-value, currently-unmet technical feature.**

Format churn risk. Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

Mid-session cross-device resume feasibility (same vendor): Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present – *if* path keys are remapped. Codex's App Server makes it cloud-native already.

Cross-vendor session replay (e.g., Claude Code session → Codex): **Fundamentally impractical.** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) `encrypted\_content` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. **Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.**

3. Competitor teardown

Session-continuity tools:

- **tawanorg/claude-sync** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/.claude` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + **age E2E encryption** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-

derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.

- ****npow/session-sync**** (~1 star, very new): the only cross-vendor *session* translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- ****dantelex/aisync ("ss")****: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- ****codsync/codsync**** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- ****kroxiksut/codexSync****: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- ****waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)**** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- ****iHildy/opencode-synced**** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- ****Dinesh3184/claude-session-sync****: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- ****renefichtmueller/claude-sync****: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

****Config/MCP/skills sync (DevSync's neighborhood):****

- ****Leoyangl83/sync-agents-settings**** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- ****nicepkg/vsync**** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- ****baranovxyz/agentsync**** and ****spxrogers/agentsync**** (beta): canonical `` .agents/`` config synced to 9–31 agents.
- ****neon-solutions/add-mcp****: one-command MCP install across ~16 agents (Neon).
- ****ztripez/mcp-sync****: MCP config sync across tools.
- ****AgentSync (Rust)****: symlink-based config sync.
- ****skills-sync****: skills + MCP workspace.
- ****DevSynq**** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

****URL-addressable / mobile-control / viewer approaches:****

- ****TokenRip**** (tokenrip.com): makes session state a URL-addressable artifact (``/a/<slug>``) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- ****Omnara**** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was ****archived in February 2026**** because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)
- ****Syncode – AI Programming Tool****: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- ****jazzyalex/agent-sessions**** & ****kenn-io/agentsview****: local-first session browsers/search across 20+ agents (read/resume-command, not sync).
- ****LAN-workspace approaches**** and Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

****Takeaway:**** The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- ****Claude Code:**** #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- ****Codex:**** #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI  app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
- ****Copilot CLI:**** #1947 (cloud-synced sessions), #1635 (cross-

environment resume).

- **Cursor forum:** multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is. Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. **Cross-vendor unified profile** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.

2. **OS-aware path remapping as a first-class feature** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows↔macOS. This is the wedge no competitor has nailed.

3. **E2E encryption by default** – table stakes given session files contain code, secrets, and full transcripts.

Risks:

- **Format churn:** Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).

- **Vendor lock-out / ToS:** Storing session data is user-owned data (fine), but *auth tokens* are not transferable (codexSync explicitly refuses; Codex `encrypted\_content` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.

- **Vendors closing the gap:** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This *validates* the demand while

confirming that only the cross-vendor + cross-OS layer is defensible.

- ****Security/privacy expectations:**** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).
- ****Honest limits:**** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

****Stage 0 – Fold in DevSync (immediate).**** Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

****Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge).**** Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/.claude/projects/` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- ****First-class OS-aware path remapping:**** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
- ****Benchmark to advance:**** `claude --resume` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

****Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode.**** Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid

`encrypted\_content`/token pitfalls. For Gemini, remap
`project\_hash`. For opencode, use safe SQLite snapshotting (the
`sqlite3.backup` pattern the community scripts use).
- ****Benchmark:**** each new adapter must pass the same cross-OS
resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One
"profile" = sessions + MCP + skills + agents + instructions,
portable across machines and (for config) across vendors. Add
best-effort context handoff (summarize + re-inject) for cross-
vendor moves – explicitly labeled as context transfer, not
replay.

****Monetization & thresholds.**** Free/open-source core (build
trust, match claude-sync) with a paid tier for managed encrypted
storage, multi-machine dashboards, and team profiles
(DevSynq/OpenSync validate willingness to pay). ****Kill/pivot
signal:**** if Anthropic ships account-level **local** session sync
with automatic path remapping across OSes (not just history
mirroring, per #71794), the single-vendor wedge collapses – at
that point double down on cross-vendor config/skills/profile
portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport
(gated, one-way, origin-online, with documented ~10-min network
and undocumented ~15-min idle timeouts). Don't promise cross-
vendor session replay. Don't sync auth tokens. Don't store
plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment

based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

recontext?

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape

TL;DR

- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows↔macOS paths.
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows↔macOS sync.
- **Recommendation: build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode.** Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...` and `/Users/...` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.
3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.
4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

- Claude Code (Anthropic).** Ships the richest consumer-facing multi-device story:
 - **`/teleport`** (`claude --teleport`): pulls a *cloud* session (started at claude.ai/code or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal

session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport`, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires ****clean git state**** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same claude.ai account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."

- ****Remote Control**** (`\`claude remote-control` / \`/remote-control`, --rc`): pushes a ***live local*** session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an ***undocumented*** ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `\`ANTHROPIC_BASE_URL`` is redirected; unavailable under Zero Data Retention.
- ****Claude Code on the web**** (claude.ai/code): runs on Anthropic cloud infra, no local filesystem.

- ****Residual gap:**** No true device-to-device ***local*** session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an ***unexpected***, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

****Codex CLI (OpenAI).**** The most architecturally mature cross-surface model:

- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared ****App Server**** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under `\`~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl``, with a `\`session_meta`` first line.

- Rich lifecycle: `\`codex resume`` (picker), `\`--last``, `\`--resume-session-id``, `\`/fork``, `archive/unarchive` (v0.136), `delete` (v0.140). Imports histories from other agents via an `\`external-agent-sessions`` crate.

- ****Known gaps/bugs:**** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing `\`encrypted_content`` often can't be continued after crossing provider/account (`\`invalid_encrypted_content``); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.

- ****Residual gap:**** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

****Gemini CLI (Google).**** Auto-saves sessions to `\`~/gemini/tmp/<project_hash>/chats/`` (project-scoped, hash of project root). `\`/resume`` session browser, `\`--resume`` `\`-r`` flag, `\`/chat save`` checkpoints, and a shadow-git checkpointing system under `\`~/gemini/history/<project_hash>``. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. ****Residual gap:**** everything local + path-hash-keyed; no sync.

****opencode (sst).**** SQLite + file storage under `\`~/local/share/opencode/`` (`\`opencode.db``, `\`storage/session|message|part``). Native ****session sharing**** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). ****Residual gap:**** sharing ≠ syncing; DB concurrency and cross-OS path issues remain.

****Cursor CLI (cursor-agent).**** Chats live in local SQLite; CLI chats under `\`~/cursor/chats`` and resume via `\`cursor-agent --resume``. ****IDE chats and CLI chats do NOT share a session store**** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend `\`&`` to push to a Cloud Agent, pick up at cursor.com/agents), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). ****Residual gap:****

no local cross-device sync, IDE↔CLI split, path issues.

Grok / Grok Build (xAI). Sessions auto-saved to `~/grok/sessions/`, keyed by working directory. `~/resume`, `--resume`, `-c`, `~/fork`; subscription-gated (SuperGrok/X Premium+). Config in `~/grok/config.toml`. No native cross-device sync. Still beta (v0.2.x as of July 2026). **Residual gap:** no sync at all.

GitHub Copilot CLI. Sessions in `~/copilot/` (`session-state/<uuid>/events.jsonl`). Notably, Copilot CLI **syncs** sessions to your GitHub account by default and offers **remote control** from GitHub.com/GitHub Mobile (origin machine must stay online, `~/keep-alive`). But **cross-device resume** is still not supported – manually copying a session dir and running `--resume` overwrites `events.jsonl` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session\_id`. **Residual gap:** the sync is telemetry/history, not resumable working state across machines.

Amp (Sourcegraph). Threads sync to ampcode.com by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). **Residual gap:** vendor-locked, server-stored, no cross-vendor.

Cross-vendor summary: Every vendor is building *within* its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

Tool	Location	Format	Path keying	Cross-device resume native?
				--- --- --- --- ---
Claude Code	~/claude/projects/<encoded-path>/<session>.jsonl	JSONL, append-only event stream	Absolute path, non-alphanumerics→dashes	Via cloud only (teleport/RC)
Codex	~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl	JSONL + `session_meta` header	date-based; cwd in meta	Yes, via App Server
Gemini CLI	~/gemini/tmp/<project_hash>/chats/	JSON	Hash of project root	No
opencode	~/local/share/opencode/	(SQLite + files)	SQLite/JSON	project-keyed Sharing only
Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)
Grok	~/grok/sessions/	JSON	working directory	No
Copilot CLI	~/copilot/session-state/<uuid>/events.jsonl	JSONL events	uuid	Account history sync; no resume

The path-mapping problem (the crux). Claude Code derives the project folder name by escaping the absolute path: `/Users/me/app` → `~/claude/projects/-Users-me-app/`, while the same repo at `C:\Users\me\app` (or a different macOS username/location) produces a *different* folder. Consequently, after a naive file sync, `claude --resume` on the second machine cannot find the first machine's sessions. This is confirmed by the claude-sync author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker."

Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via `project\_hash`; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. **This is the single highest-value, currently-unmet technical feature.**

Format churn risk. Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

****Mid-session cross-device resume feasibility (same vendor):**** Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present – ***if*** path keys are remapped. Codex's App Server makes it cloud-native already.

****Cross-vendor session replay (e.g., Claude Code session → Codex):**** ****Fundamentally impractical.**** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) ``encrypted_content`` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. ****Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.****

3. Competitor teardown

****Session-continuity tools:****

- ****tawanorg/claude-sync**** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/ .claude`` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + ****age E2E encryption**** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.
- ****npow/session-sync**** (~1 star, very new): the only cross-vendor ***session*** translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- ****dantelex/aisync ("ss")****: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- ****codsync/codsync**** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- ****kroxiksut/codexSync****: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- ****waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)**** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- ****iHildy/opencode-synced**** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- ****Dinesh3184/claude-session-sync****: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- ****renefichtmueller/claude-sync****: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

****Config/MCP/skills sync (DevSync's neighborhood):****

- ****Leoyang183/sync-agents-settings**** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- ****nicepkg/vsync**** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- ****baranovxyz/agentsync**** and ****spxrogers/agentsync**** (beta): canonical ``.agents/`` config synced to 9–31 agents.
- ****neon-solutions/add-mcp****: one-command MCP install across ~16 agents (Neon).
- ****ztripez/mcp-sync****: MCP config sync across tools.
- ****AgentSync (Rust)****: symlink-based config sync.
- ****skills-sync****: skills + MCP workspace.
- ****DevSynq**** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

****URL-addressable / mobile-control / viewer approaches:****

- ****TokenRip**** (tokenrip.com): makes session state a URL-addressable artifact (``/a/<slug>``) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- ****Omnara**** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was ****archived in February 2026**** because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent

SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)

- **Syncode – AI Programming Tool**: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- **jazzyalex/agent-sessions** & **kenn-io/agentsview**: local-first session browsers/search across 20+ agents (read/resume-command, not sync).
- **LAN-workspace approaches** and Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

Takeaway: The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- **Claude Code**: #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- **Codex**: #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI ↔ app-server sessions), discussion #14067 (cross-device thread/contextsync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
- **Copilot CLI**: #1947 (cloud-synced sessions), #1635 (cross-environment resume).
- **Cursor forum**: multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is. Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. **Cross-vendor unified profile** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.
2. **OS-aware path remapping as a first-class feature** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows ↔ macOS. This is the wedge no competitor has nailed.
3. **E2E encryption by default** – table stakes given session files contain code, secrets, and full transcripts.

Risks:

- **Format churn**: Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).
- **Vendor lock-out / ToS**: Storing session data is user-owned data (fine), but *auth tokens* are not transferable (codexSync explicitly refuses; Codex `encrypted\_content` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.
- **Vendors closing the gap**: Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This *validates* the demand while *confirming* that only the cross-vendor + cross-OS layer is defensible.

- ****Security/privacy expectations:**** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).
- ****Honest limits:**** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

****Stage 0 – Fold in DevSync (immediate).**** Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

****Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge).****
 Target the founder's exact case (Windows desktop 🖥️ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/ .claude/projects/` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- ****First-class OS-aware path remapping:**** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
- ****Benchmark to advance:**** `claude --resume` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

****Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode.**** Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid `encrypted\_content`/token pitfalls. For Gemini, remap `project\_hash`. For opencode, use safe SQLite snapshotting (the `sqlite3 .backup` pattern the community scripts use).

- ****Benchmark:**** each new adapter must pass the same cross-OS resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One "profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not replay.

****Monetization & thresholds.**** Free/open-source core (build trust, match claude-sync) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSync/OpenSync validate willingness to pay). ****Kill/pivot signal:**** if Anthropic ships account-level *local* session sync with automatic path remapping across OSes (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync,

Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.

- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

Length-wise it fits (9 characters), and it's the most *accurate* name you've floated. But there's one flaw worth sitting with before you commit.

The problem: it names the thing you're eliminating. Your own pitch from the top of this thread was that you *don't* want an agent re-reading all the code changes to rebuild what it already knew. That's re-contextualizing — and it's the pain, not the cure. A name that reads as "do the context again" points at the wrong half of the story. Compare Carryall or Baton, which describe the relief rather than the chore.

There's a defensible counter-reading, though: you're not re-contexting *yourself*, you're re-contexting the **laptop**. The machine gets handed the context it lacks. If your marketing consistently frames the object as the device — "recontext your MacBook in two seconds" — the name works and the ambiguity mostly disappears. That's a real framing, just one you'd have to actively maintain.

Collision check. Two small companies use it (a marketing-analytics reContext, unfunded and essentially dormant, and a Vermont edtech LLC) — neither is a threat in dev tooling. The more relevant one: Google Cloud's Imagen "product recontext" API on Vertex AI, which edits product images into different scenes. Different domain, but it puts "recontext" in genAI vocabulary as an image-editing operation. Note that the preview model was slated for removal in March 2026, so the term may be fading — worth re-checking whether Google still uses that naming.

Other marks against: "context" is the single most saturated word in AI product naming right now, and descriptive names are the hardest to own or trademark. You'd be one of many.

Marks for: instantly legible to your exact audience, no explanation needed, reads well as a CLI (`recontext pull`, `recontext push`), and the `re-` prefix has strong dev-tool precedent (Retool, Replit).

Verdict: solidly above average — better than openparity, more legible than Ferrata, less distinctive than Carryall or Baton. If you want the semantics without the inversion, the nearest neighbors are **Regain**, **Reclaim**, or **Rejoin** — same prefix and rhythm, but they describe getting something back rather than redoing it. **Rejoin** in particular is close to perfect for "rejoin the session you left on the other machine."

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

```
# Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape
```

TL;DR

- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows ↔ macOS paths.
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows ↔ macOS sync.
- **Recommendation:** build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode. Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...\` and `/Users/...\` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.

3. ****Demand is loud and repeatedly expressed**** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.

4. ****The competitor field is fragmented, early, and mostly config-only or single-vendor.**** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

****Claude Code (Anthropic).**** Ships the richest consumer-facing multi-device story:

- ****`/teleport`**** (``claude --teleport``): pulls a **cloud** session (started at `claude.ai/code` or the mobile app) down into the local terminal. It is ****one-way**** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with ``--teleport``, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires ****clean git state**** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same `claude.ai` account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."
- ****Remote Control**** (``claude remote-control`` / ``/remote-control``, ``--rc``): pushes a **live local** session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an **undocumented** ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when ``ANTHROPIC_BASE_URL`` is

redirected; unavailable under Zero Data Retention.

- **Claude Code on the web** (claude.ai/code): runs on Anthropic cloud infra, no local filesystem.

- **Residual gap:** No true device-to-device *local* session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an *unexpected*, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

Codex CLI (OpenAI). The most architecturally mature cross-surface model:

- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared **App Server** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under

`~/.codex/sessions/YYYY/MM/DD/rollout-\*.jsonl`, with a `session\_meta` first line.

- Rich lifecycle: `codex resume` (picker), `--last`, `--resume-session-id`, `/fork`, archive/unarchive (v0.136), delete (v0.140). Imports histories from other agents via an `external-agent-sessions` crate.

- **Known gaps/bugs:** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing `encrypted\_content` often can't be continued after crossing provider/account (`invalid\_encrypted\_content`); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.

- **Residual gap:** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

Gemini CLI (Google). Auto-saves sessions to

`~/.gemini/tmp/<project\_hash>/chats/` (project-scoped, hash of project root). `/resume` session browser, `--resume`/`-r` flag, `/chat save` checkpoints, and a shadow-git checkpointing system under `~/.gemini/history/<project\_hash>`. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. **Residual gap:** everything local + path-hash-keyed; no sync.

opencode (sst). SQLite + file storage under `~/.local/share/opencode/`(`opencode.db`,
`storage/session|message|part`)`. Native **session sharing** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). **Residual gap:** sharing $\neq$ syncing; DB concurrency and cross-OS path issues remain.

Cursor CLI (cursor-agent). Chats live in local SQLite; CLI chats under `~/.cursor/chats`` and resume via `cursor-agent --resume``. **IDE chats and CLI chats do NOT share a session store** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend `&`` to push to a Cloud Agent, pick up at cursor.com/agents), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). **Residual gap:** no local cross-device sync, IDE ↔ CLI split, path issues.

Grok / Grok Build (xAI). Sessions auto-saved to `~/.grok/sessions/``, keyed by working directory. `/resume``, `--resume``, `-c``, `/fork``; subscription-gated (SuperGrok/X Premium+). Config in `~/.grok/config.toml``. No native cross-device sync. Still beta (v0.2.x as of July 2026). **Residual gap:** no sync at all.

GitHub Copilot CLI. Sessions in `~/.copilot/`(`session-state/<uuid>/events.jsonl`)`. Notably, Copilot CLI **syncs sessions to your GitHub account by default** and offers **remote control** from [GitHub.com/GitHub Mobile](https://github.com/GitHubMobile) (origin machine must stay online, `/keep-alive``). But **cross-device \*resume\*** is still not supported – manually copying a session dir and running `--resume`` overwrites `events.jsonl`` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session_id``. **Residual gap:** the sync is telemetry/history, not resumable working state across machines.

Amp (Sourcegraph). Threads sync to ampcode.com by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is

GUI-global, not file-based). ****Residual gap:**** vendor-locked, server-stored, no cross-vendor.

****Cross-vendor summary:**** Every vendor is building **within** its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

Tool	Location	Format	Path keying	Cross-device resume native?
Claude Code	`\${~}/.claude/projects/<encoded-path>/<session>.jsonl`	JSONL, append-only event stream	**Absolute path, non-alphanumerics→dashes**	Via cloud only (teleport/RC)
Codex	`\${~}/.codex/sessions/YYYY/MM/DD/rollout-*.jsonl`	JSONL + `session_meta` header	date-based; cwd in meta	Yes, via App Server
Gemini CLI	`\${~}/.gemini/tmp/<project_hash>/chats/`	JSON	**Hash of project root**	No
opencode	`\${~}/.local/share/opencode/`	(SQLite + files)	SQLite/JSON	project-keyed
Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)
Grok	`\${~}/.grok/sessions/`	JSON	working directory	No
Copilot CLI	`\${~}/.copilot/session-state/<uuid>/events.jsonl`	JSONL events	uuid	Account history sync; no resume

****The path-mapping problem (the crux).**** Claude Code derives the project folder name by escaping the absolute path: ``/Users/me/app`` → ``~/.claude/projects/-Users-me-app/``, while the same repo at ``C:\Users\me\app`` (or a different macOS username/location) produces a **different** folder. Consequently, after a naive file sync, ``claude --resume`` on the second machine cannot find the first machine's sessions. This is confirmed by the `claude-sync` author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker." Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via ``project_hash``; Cursor stores path metadata in chat rows

(cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. ****This is the single highest-value, currently-unmet technical feature.****

****Format churn risk.**** Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

****Mid-session cross-device resume feasibility (same vendor):**** Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present – *if* path keys are remapped. Codex's App Server makes it cloud-native already.

****Cross-vendor session replay (e.g., Claude Code session → Codex):**** ****Fundamentally impractical.**** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) `encrypted\_content` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. ****Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.****

3. Competitor teardown

****Session-continuity tools:****

- ****tawanorg/claude-sync**** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/.claude` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + ****age E2E encryption**** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.
- ****npow/session-sync**** (~1 star, very new): the only cross-vendor *session* translator – Claude Code → Codex/Amp/Gemini via

PostToolUse/Stop hooks. Proof the idea is wanted; nascent.

- **dantelex/aisync ("ss")**: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- **codsync/codsync** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- **kroxiksut/codexSync**: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- **waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- **iHildy/opencode-synced** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- **Dinesh3184/claude-session-sync**: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- **renefichtmueller/claude-sync**: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

Config/MCP/skills sync (DevSync's neighborhood):

- **Leoyangl83/sync-agents-settings** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- **nicepkg/vsync** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- **baranovxyz/agentsync** and **spxrogers/agentsync** (beta): canonical ``.agents/`` config synced to 9–31 agents.
- **neon-solutions/add-mcp**: one-command MCP install across ~16 agents (Neon).
- **ztripez/mcp-sync**: MCP config sync across tools.
- **AgentSync (Rust)**: symlink-based config sync.
- **skills-sync**: skills + MCP workspace.
- **DevSynq** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

URL-addressable / mobile-control / viewer approaches:

- **TokenRip** (tokenrip.com): makes session state a URL-addressable artifact (``/a/<slug>``) any agent can fetch – a genuinely different architectural bet ("don't sync files; give

state a permanent URL").

- **Omnara** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was **archived in February 2026** because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)
- **Syncode – AI Programming Tool**: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- **jazzyalex/agent-sessions** & **kenn-io/agentsview**: local-first session browsers/search across 20+ agents (read/resume-command, not sync).
- **LAN-workspace approaches** and Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

Takeaway: The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- **Claude Code:** #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- **Codex:** #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI  app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
- **Copilot CLI:** #1947 (cloud-synced sessions), #1635 (cross-environment resume).
- **Cursor forum:** multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

****Where the durable moat is.**** Vendors will keep building ***intra*-ecosystem sync** (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. ****Cross-vendor unified profile**** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.

2. ****OS-aware path remapping as a first-class feature**** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows↔macOS. This is the wedge no competitor has nailed.

3. ****E2E encryption by default**** – table stakes given session files contain code, secrets, and full transcripts.

****Risks:****

- ****Format churn:**** Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).

- ****Vendor lock-out / ToS:**** Storing session data is user-owned data (fine), but ***auth tokens*** are not transferable (codexSync explicitly refuses; Codex `encrypted\_content` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.

- ****Vendors closing the gap:**** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This ***validates*** the demand while ***confirming*** that only the cross-vendor + cross-OS layer is defensible.

- ****Security/privacy expectations:**** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no

plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).

- **Honest limits:** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

Stage 0 – Fold in DevSync (immediate). Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge). Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/.claude/projects/` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- **First-class OS-aware path remapping:** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
- **Benchmark to advance:** `claude --resume` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode. Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid `encrypted\_content`/token pitfalls. For Gemini, remap `project\_hash`. For opencode, use safe SQLite snapshotting (the `sqlite3 .backup` pattern the community scripts use).

- **Benchmark:** each new adapter must pass the same cross-OS

resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One "profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not replay.

****Monetization & thresholds.**** Free/open-source core (build trust, match claude-sync) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSynq/OpenSync validate willingness to pay). ****Kill/pivot signal:**** if Anthropic ships account-level *local* session sync with automatic path remapping across OSes (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.



rethread? rethreads? hmmm

resession? resessions? eh

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape

TL;DR

- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows↔macOS paths.
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows↔macOS sync.
- **Recommendation:** build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode. Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...` and `/Users/...` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.
3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.
4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

- **Claude Code (Anthropic).** Ships the richest consumer-facing multi-device story:
- **`/teleport`** (`claude --teleport`): pulls a *cloud* session (started at claude.ai/code or

the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport`, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires **clean git state** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same claude.ai account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."

- **Remote Control** (`claude remote-control` / `remote-control`, --rc`): pushes a **live local** session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an **undocumented** ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `ANTHROPIC_BASE_URL` is redirected; unavailable under Zero Data Retention.

- **Claude Code on the web** (claude.ai/code): runs on Anthropic cloud infra, no local filesystem.

- **Residual gap:** No true device-to-device **local** session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an **unexpected**, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

Codex CLI (OpenAI). The most architecturally mature cross-surface model:

- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared **App Server** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under `~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl`, with a `session_meta` first line.

- Rich lifecycle: `codex resume` (picker), --last`, --resume-session-id`, /fork`, archive/unarchive` (v0.136), delete` (v0.140). Imports histories from other agents via an external-agent-sessions` crate.`

- **Known gaps/bugs:** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing `encrypted_content` often can't be continued after crossing provider/account (`invalid_encrypted_content`); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.

- **Residual gap:** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

Gemini CLI (Google). Auto-saves sessions to `~/gemini/tmp/<project_hash>/chats/` (project-scoped, hash of project root). /resume` session browser, --resume`/`-r` flag, /chat save` checkpoints, and a shadow-git checkpointing system under ~/gemini/history/<project_hash>`. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. Residual gap: everything local + path-hash-keyed; no sync.`

opencode (sst). SQLite + file storage under `~/local/share/opencode/` (opencode.db`, storage/session|message|part`). Native session sharing publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). Residual gap: sharing ≠ syncing; DB concurrency and cross-OS path issues remain.`

Cursor CLI (cursor-agent). Chats live in local SQLite; CLI chats under `~/cursor/chats` and resume via cursor-agent --resume`. IDE chats and CLI chats do NOT share a session store (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend &` to push to a Cloud Agent, pick up at cursor.com/agents), and "My Machines" lets cloud agents run on your`

registered machine. Strong user demand for account-level chat sync (forum). **Residual gap:** no local cross-device sync, IDE↔CLI split, path issues.

Grok / Grok Build (xAI). Sessions auto-saved to `~/grok/sessions/`, keyed by working directory. `./resume`, `--resume`, `-c`, `./fork`; subscription-gated (SuperGrok/X Premium+). Config in `~/grok/config.toml`. No native cross-device sync. Still beta (v0.2.x as of July 2026). **Residual gap:** no sync at all.

GitHub Copilot CLI. Sessions in `~/copilot/` (`session-state/<uuid>/events.jsonl`). Notably, Copilot CLI **syncs sessions to your GitHub account by default** and offers **remote control** from GitHub.com/GitHub Mobile (origin machine must stay online, `./keep-alive`). But **cross-device resume** is still not supported – manually copying a session dir and running `--resume` overwrites `events.jsonl` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session\_id`. **Residual gap:** the sync is telemetry/history, not resumable working state across machines.

Amp (Sourcegraph). Threads sync to ampcode.com by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). **Residual gap:** vendor-locked, server-stored, no cross-vendor.

Cross-vendor summary: Every vendor is building *within* its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

	Tool	Location	Format	Path keying	Cross-device resume native?
					--- --- --- --- ---
	Claude Code	~/claude/projects/<encoded-path>/<session>.jsonl	JSONL, append-only event stream	Absolute path, non-alphanumerics→dashes	Via cloud only (teleport/RC)
	Codex	~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl	JSONL + `session_meta` header	date-based; cwd in meta	Yes, via App Server
	Gemini CLI	~/gemini/tmp/<project_hash>/chats/	JSON	Hash of project root	No
	opencode	~/local/share/opencode/	(SQLite + files)	SQLite/JSON	project-keyed
					Sharing only
	Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)
				Grok	~/grok/sessions/
					JSON
					working directory
					No
	Copilot CLI	~/copilot/session-state/<uuid>/events.jsonl	JSONL events	uuid	Account history sync; no resume

The path-mapping problem (the crux). Claude Code derives the project folder name by escaping the absolute path: `/Users/me/app` → `~/claude/projects/-Users-me-app/`, while the same repo at `C:\Users\me\app` (or a different macOS username/location) produces a *different* folder. Consequently, after a naive file sync, `claude --resume` on the second machine cannot find the first machine's sessions. This is confirmed by the claude-sync author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker."

Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via `project\_hash`; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. **This is the single highest-value, currently-unmet technical feature.**

Format churn risk. Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

****Mid-session cross-device resume feasibility (same vendor):**** Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present – ***if*** path keys are remapped. Codex's App Server makes it cloud-native already.

****Cross-vendor session replay (e.g., Claude Code session → Codex):**** ****Fundamentally impractical.**** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) ``encrypted_content`` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. ****Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.****

3. Competitor teardown

****Session-continuity tools:****

- ****tawanorg/claude-sync**** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/ .claude`` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + ****age E2E encryption**** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.
- ****npow/session-sync**** (~1 star, very new): the only cross-vendor ***session*** translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- ****dantelex/aisync ("ss")****: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- ****codsync/codsync**** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- ****kroxiksut/codexSync****: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- ****waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)**** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- ****iHildy/opencode-synced**** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- ****Dinesh3184/claude-session-sync****: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- ****renefichtmueller/claude-sync****: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

****Config/MCP/skills sync (DevSync's neighborhood):****

- ****Leoyang183/sync-agents-settings**** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- ****nicepkg/vsync**** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- ****baranovxyz/agentsync**** and ****spxrogers/agentsync**** (beta): canonical ``.agents/`` config synced to 9–31 agents.
- ****neon-solutions/add-mcp****: one-command MCP install across ~16 agents (Neon).
- ****ztripez/mcp-sync****: MCP config sync across tools.
- ****AgentSync (Rust)****: symlink-based config sync.
- ****skills-sync****: skills + MCP workspace.
- ****DevSynq**** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

****URL-addressable / mobile-control / viewer approaches:****

- ****TokenRip**** (tokenrip.com): makes session state a URL-addressable artifact (``/a/<slug>``) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- ****Omnara**** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was ****archived in February 2026**** because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent

SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)

- **Syncode – AI Programming Tool**: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- **jazzyalex/agent-sessions** & **kenn-io/agentsview**: local-first session browsers/search across 20+ agents (read/resume-command, not sync).
- **LAN-workspace approaches** and Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

Takeaway: The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- **Claude Code**: #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- **Codex**: #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI ↔ app-server sessions), discussion #14067 (cross-device thread/contextsync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
- **Copilot CLI**: #1947 (cloud-synced sessions), #1635 (cross-environment resume).
- **Cursor forum**: multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is. Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. **Cross-vendor unified profile** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.
2. **OS-aware path remapping as a first-class feature** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows↔macOS. This is the wedge no competitor has nailed.
3. **E2E encryption by default** – table stakes given session files contain code, secrets, and full transcripts.

Risks:

- **Format churn**: Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).
- **Vendor lock-out / ToS**: Storing session data is user-owned data (fine), but *auth tokens* are not transferable (codexSync explicitly refuses; Codex `encrypted\_content` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.
- **Vendors closing the gap**: Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This *validates* the demand while *confirming* that only the cross-vendor + cross-OS layer is defensible.

- ****Security/privacy expectations:**** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).
- ****Honest limits:**** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

****Stage 0 – Fold in DevSync (immediate).**** Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

****Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge).****
 Target the founder's exact case (Windows desktop 🖥️ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/ .claude/projects/` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- ****First-class OS-aware path remapping:**** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
- ****Benchmark to advance:**** `claude --resume` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

****Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode.**** Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid `encrypted\_content`/token pitfalls. For Gemini, remap `project\_hash`. For opencode, use safe SQLite snapshotting (the `sqlite3 .backup` pattern the community scripts use).

- ****Benchmark:**** each new adapter must pass the same cross-OS resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One "profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not replay.

****Monetization & thresholds.**** Free/open-source core (build trust, match claude-sync) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSync/OpenSync validate willingness to pay). ****Kill/pivot signal:**** if Anthropic ships account-level *local* session sync with automatic path remapping across OSes (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync,

Gemini-Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

rethread — the strongest thing you've landed on in this family. "Thread" is already the native noun for what you're syncing (Amp literally calls them threads), and rethreading a needle is exactly the gesture: find the loose end, pass it through again, keep going. Eight characters, clean two-beat rhythm. Its one real weakness is that **thread** means concurrency to every programmer alive. **rethread** will get a half-second misread as something about pthreads before the right meaning lands. Survivable — context resolves it fast — but it's there.

rethreads — no. Phonetically it collapses into "retreads," which is used tires. Never pluralize this one.

resession — kill it. It's one phoneme from **recession**, and you will lose every search, every autocorrect, and every verbal introduction to the economy. "resessions" is worse. This one's not fixable.

Better options in the same **re-** family, keeping your 6–9 range:

- **Reattach** — **tmux attach** is the exact mental model, and terminal people already have the muscle memory. Probably the most immediately understood name in this whole conversation for your specific audience. Minor note: **reattach-to-user-namespace** is a known macOS tmux helper, so the word isn't virgin territory in that world.
- **Respool** — a spool holds thread in flight; respooling is winding it back onto the reel. Same metaphor as rethread, none of the concurrency baggage, and far more distinctive.
- **Reseat** — hardware people reseat a card or connector; humans re-take their seat. Two clean meanings, both right.
- **Reboard** — board the same journey again after stepping off. Warm, travel-flavored, easy to say.
- **Restring** — restringing an instrument so it plays again. Pretty, and unclaimed as far as I know.
- **Rejoin** — still my pick from last round. Plainest of the lot, zero ambiguity, reads perfectly as a command.

If I ranked your instinct against mine: **rethread** is genuinely good and I wouldn't talk you out of it. But **respool** does the same job without the pthread flicker, and **reattach** is the one your users will understand before they finish reading it.

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape

TL;DR

- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows ↔ macOS paths.
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows ↔ macOS sync.
- **Recommendation:** build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode. Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...` and `/Users/...` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly

admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.

3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.

4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

Claude Code (Anthropic). Ships the richest consumer-facing multi-device story:

- **`/teleport`** (``claude --teleport``): pulls a *cloud* session (started at `claude.ai/code` or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport``, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires **clean git state** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same `claude.ai` account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."
- **Remote Control** (``claude remote-control`` / ``/remote-control``, `--rc``): pushes a *live local* session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an *undocumented* ~15-minute idle timeout on the Desktop/Code

surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `ANTHROPIC\_BASE\_URL` is redirected; unavailable under Zero Data Retention.

- **Claude Code on the web** (claude.ai/code): runs on Anthropic cloud infra, no local filesystem.

- **Residual gap:** No true device-to-device **local** session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an **unexpected**, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

Codex CLI (OpenAI). The most architecturally mature cross-surface model:

- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared **App Server** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under `~/.codex/sessions/YYYY/MM/DD/rollout-\*.jsonl`, with a `session\_meta` first line.

- Rich lifecycle: `codex resume` (picker), `--last`, `--resume-session-id`, `/fork`, archive/unarchive (v0.136), delete (v0.140). Imports histories from other agents via an `external-agent-sessions` crate.

- **Known gaps/bugs:** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing `encrypted\_content` often can't be continued after crossing provider/account (`invalid\_encrypted\_content`); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.

- **Residual gap:** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

Gemini CLI (Google). Auto-saves sessions to

`~/.gemini/tmp/<project\_hash>/chats/` (project-scoped, hash of project root). `/resume` session browser, `--resume`/`-r` flag, `/chat save` checkpoints, and a shadow-git checkpointing system

under `~/.gemini/history/<project\_hash>`. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. **Residual gap:** everything local + path-hash-keyed; no sync.

opencode (sst). SQLite + file storage under `~/.local/share/opencode/` (`opencode.db`, `storage/session|message|part`). Native **session sharing** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). **Residual gap:** sharing ≠ syncing; DB concurrency and cross-OS path issues remain.

Cursor CLI (cursor-agent). Chats live in local SQLite; CLI chats under `~/.cursor/chats` and resume via `cursor-agent --resume`. **IDE chats and CLI chats do NOT share a session store** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend `&` to push to a Cloud Agent, pick up at cursor.com/agents), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). **Residual gap:** no local cross-device sync, IDE ↔ CLI split, path issues.

Grok / Grok Build (xAI). Sessions auto-saved to `~/.grok/sessions/`, keyed by working directory. `/resume`, `--resume`, `-c`, `/fork`; subscription-gated (SuperGrok/X Premium+). Config in `~/.grok/config.toml`. No native cross-device sync. Still beta (v0.2.x as of July 2026). **Residual gap:** no sync at all.

GitHub Copilot CLI. Sessions in `~/.copilot/` (`session-state/<uuid>/events.jsonl`). Notably, Copilot CLI **syncs sessions to your GitHub account by default** and offers **remote control** from [GitHub.com](https://github.com)/GitHub Mobile (origin machine must stay online, `/keep-alive`). But **cross-device resume** is still not supported – manually copying a session dir and running `--resume` overwrites `events.jsonl` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session\_id`. **Residual gap:** the sync is telemetry/history, not resumable working state across machines.

****Amp (Sourcegraph).**** Threads sync to ampcode.com by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). ****Residual gap:**** vendor-locked, server-stored, no cross-vendor.

****Cross-vendor summary:**** Every vendor is building **within** its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

Tool	Location	Format	Path keying	Cross-device resume native?
Claude Code	`\${HOME}/.claude/projects/<encoded-path>/<session>.jsonl`	JSONL, append-only event stream	**Absolute path, non-alphanumerics→dashes**	Via cloud only (teleport/RC)
Codex	`\${HOME}/.codex/sessions/YYYY/MM/DD/rollout-*.jsonl`	JSONL + `session_meta` header	date-based; cwd in meta	Yes, via App Server
Gemini CLI	`\${HOME}/.gemini/tmp/<project_hash>/chats/`	JSON	**Hash of project root**	No
opencode	`\${HOME}/.local/share/opencode/`	(SQLite + files)	SQLite/JSON	project-keyed
Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)
Grok	`\${HOME}/.grok/sessions/`	JSON	working directory	No
Copilot CLI	`\${HOME}/.copilot/session-state/<uuid>/events.jsonl`	JSONL events	uuid	Account history sync; no resume

****The path-mapping problem (the crux).**** Claude Code derives the project folder name by escaping the absolute path: ``/Users/me/app`` → ``~/.claude/projects/-Users-me-app/``, while the same repo at ``C:\Users\me\app`` (or a different macOS username/location) produces a **different** folder. Consequently, after a naive file sync, ``claude --resume`` on the second machine cannot find the first machine's sessions. This is confirmed by the claude-sync author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation?"

Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker." Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via `project\_hash`; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. **This is the single highest-value, currently-unmet technical feature.**

Format churn risk. Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

Mid-session cross-device resume feasibility (same vendor): Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present – *if* path keys are remapped. Codex's App Server makes it cloud-native already.

Cross-vendor session replay (e.g., Claude Code session → Codex): **Fundamentally impractical.** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) `encrypted\_content` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. **Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.**

3. Competitor teardown

Session-continuity tools:

- **tawanorg/claude-sync** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/.claude` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + **age E2E encryption** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-

derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.

- ****npow/session-sync**** (~1 star, very new): the only cross-vendor *session* translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- ****dantelex/aisync ("ss")****: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- ****codsync/codsync**** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- ****kroxiksut/codexSync****: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- ****waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)**** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- ****iHildy/opencode-synced**** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- ****Dinesh3184/claude-session-sync****: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- ****renefichtmueller/claude-sync****: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

****Config/MCP/skills sync (DevSync's neighborhood):****

- ****Leoyangl83/sync-agents-settings**** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- ****nicepkg/vsync**** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- ****baranovxyz/agentsync**** and ****spxrogers/agentsync**** (beta): canonical ``.agents/`` config synced to 9–31 agents.
- ****neon-solutions/add-mcp****: one-command MCP install across ~16 agents (Neon).
- ****ztripez/mcp-sync****: MCP config sync across tools.
- ****AgentSync (Rust)****: symlink-based config sync.
- ****skills-sync****: skills + MCP workspace.
- ****DevSynq**** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

****URL-addressable / mobile-control / viewer approaches:****

- ****TokenRip**** (tokenrip.com): makes session state a URL-addressable artifact (``/a/<slug>``) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- ****Omnara**** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was ****archived in February 2026**** because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)
- ****Syncode – AI Programming Tool****: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- ****jazzyalex/agent-sessions**** & ****kenn-io/agentsview****: local-first session browsers/search across 20+ agents (read/resume-command, not sync).
- ****LAN-workspace approaches**** and Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

****Takeaway:**** The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- ****Claude Code:**** #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- ****Codex:**** #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI  app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
- ****Copilot CLI:**** #1947 (cloud-synced sessions), #1635 (cross-

environment resume).

- **Cursor forum:** multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is. Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. **Cross-vendor unified profile** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.

2. **OS-aware path remapping as a first-class feature** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows↔macOS. This is the wedge no competitor has nailed.

3. **E2E encryption by default** – table stakes given session files contain code, secrets, and full transcripts.

Risks:

- **Format churn:** Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).

- **Vendor lock-out / ToS:** Storing session data is user-owned data (fine), but *auth tokens* are not transferable (codexSync explicitly refuses; Codex `encrypted\_content` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.

- **Vendors closing the gap:** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This *validates* the demand while

confirming that only the cross-vendor + cross-OS layer is defensible.

- ****Security/privacy expectations:**** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).
- ****Honest limits:**** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

****Stage 0 – Fold in DevSync (immediate).**** Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

****Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge).**** Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/.claude/projects/` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- ****First-class OS-aware path remapping:**** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
- ****Benchmark to advance:**** `claude --resume` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

****Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode.**** Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid

`encrypted\_content`/token pitfalls. For Gemini, remap
`project\_hash`. For opencode, use safe SQLite snapshotting (the
`sqlite3.backup` pattern the community scripts use).
- ****Benchmark:**** each new adapter must pass the same cross-OS
resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One
"profile" = sessions + MCP + skills + agents + instructions,
portable across machines and (for config) across vendors. Add
best-effort context handoff (summarize + re-inject) for cross-
vendor moves – explicitly labeled as context transfer, not
replay.

****Monetization & thresholds.**** Free/open-source core (build
trust, match claude-sync) with a paid tier for managed encrypted
storage, multi-machine dashboards, and team profiles
(DevSynq/OpenSync validate willingness to pay). ****Kill/pivot
signal:**** if Anthropic ships account-level **local** session sync
with automatic path remapping across OSes (not just history
mirroring, per #71794), the single-vendor wedge collapses – at
that point double down on cross-vendor config/skills/profile
portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport
(gated, one-way, origin-online, with documented ~10-min network
and undocumented ~15-min idle timeouts). Don't promise cross-
vendor session replay. Don't sync auth tokens. Don't store
plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment

based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

Reboard is good

Rethread is good too

which one?

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape

TL;DR

- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows↔macOS paths.
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows↔macOS sync.
- **Recommendation:** build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode. Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...` and `/Users/...` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.
3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.
4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

- Claude Code (Anthropic).** Ships the richest consumer-facing multi-device story:
 - teleport** (`\`claude --teleport\``): pulls a *cloud* session (started at `claude.ai/code` or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport`, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires **clean git state** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same `claude.ai` account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."
 - Remote Control** (`\`claude remote-control\` / \`/remote-control\`, --rc`): pushes a *live local* session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an *undocumented* ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `\`ANTHROPIC_BASE_URL\`` is redirected; unavailable under Zero Data Retention.
 - Claude Code on the web** (`claude.ai/code`): runs on Anthropic cloud infra, no local filesystem.
 - Residual gap:** No true device-to-device *local* session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an *unexpected*, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.
 - Codex CLI (OpenAI).** The most architecturally mature cross-surface model:
 - CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared **App Server** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under `\`~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl\`,` with a `\`session_meta\`` first line.
 - Rich lifecycle: `\`codex resume\`` (picker), `\`--last\`, \`--resume-session-id\`, \`/fork\`,` `archive/unarchive` (v0.136), `delete` (v0.140). Imports histories from other agents via an `\`external-agent-sessions\`` crate.
 - Known gaps/bugs:** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing `\`encrypted_content\`` often can't be continued after crossing provider/account (`\`invalid_encrypted_content\``); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.
 - Residual gap:** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.
 - Gemini CLI (Google).** Auto-saves sessions to `\`~/geminl/tmp/<project_hash>/chats/\`` (project-scoped, hash of project root). `\`/resume\`` session browser, `\`--resume\`/\`-r\`` flag, `\`/chat save\`` checkpoints, and a shadow-git checkpointing system under `\`~/geminl/history/<project_hash>\``. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. **Residual gap:** everything local + path-hash-keyed; no sync.
 - opencode (sst).** SQLite + file storage under `\`~/local/share/opencode/\`` (`\`opencode.db\`, \`storage/session|message|part\``). Native **session sharing** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). **Residual gap:** sharing ≠ syncing; DB concurrency and cross-OS path issues remain.

Cursor CLI (cursor-agent). Chats live in local SQLite; CLI chats under `~/cursor/chats` and resume via `cursor-agent --resume`. **IDE chats and CLI chats do NOT share a session store** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend `&` to push to a Cloud Agent, pick up at `cursor.com/agents`), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). **Residual gap:** no local cross-device sync, IDE↔CLI split, path issues.

Grok / Grok Build (xAI). Sessions auto-saved to `~/grok/sessions/`, keyed by working directory. `/resume`, `--resume`, `-c`, `/fork`; subscription-gated (SuperGrok/X Premium+). Config in `~/grok/config.toml`. No native cross-device sync. Still beta (v0.2.x as of July 2026). **Residual gap:** no sync at all.

GitHub Copilot CLI. Sessions in `~/copilot/` (`session-state/<uuid>/events.jsonl`). Notably, Copilot CLI **syncs sessions to your GitHub account by default** and offers **remote control** from `GitHub.com/GitHub Mobile` (origin machine must stay online, `/keep-alive`). But **cross-device resume** is still not supported – manually copying a session dir and running `--resume` overwrites `events.jsonl` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session_id`. **Residual gap:** the sync is telemetry/history, not resumable working state across machines.

Amp (Sourcegraph). Threads sync to `ampcode.com` by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). **Residual gap:** vendor-locked, server-stored, no cross-vendor.

Cross-vendor summary: Every vendor is building *within* its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

	Tool	Location	Format	Path keying	Cross-device resume native?
					--- --- --- --- ---
	Claude Code	<code>~/claude/projects/<encoded-path>/<session>.jsonl</code>	JSONL, append-only event stream	Absolute path, non-alphanumerics→dashes	Via cloud only (teleport/RC)
	Codex	<code>~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl</code>	JSONL + <code>session_meta</code> header	date-based; cwd in meta	Yes, via App Server
	Gemini CLI	<code>~/gemini/tmp/<project_hash>/chats/</code>	JSON	Hash of project root	No
	opencode	<code>~/local/share/opencode/</code>	(SQLite + files)	SQLite/JSON	project-keyed
					Sharing only
	Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)
				Grok	<code>~/grok/sessions/</code>
					JSON
					working directory
					No
	Copilot CLI	<code>~/copilot/session-state/<uuid>/events.jsonl</code>	JSONL events	uuid	Account history sync; no resume

The path-mapping problem (the crux). Claude Code derives the project folder name by escaping the absolute path: `/Users/me/app` → `~/claude/projects/-Users-me-app/`, while the same repo at `C:\Users\me\app` (or a different macOS username/location) produces a *different* folder. Consequently, after a naive file sync, `claude --resume` on the second machine cannot find the first machine's sessions. This is confirmed by the `claude-sync` author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker."

Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via `project_hash`; Cursor stores path metadata in chat rows (`cursaves` rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. **This is the single highest-value, currently-unmet technical feature.**

****Format churn risk.**** Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

****Mid-session cross-device resume feasibility (same vendor):**** Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present – ***if*** path keys are remapped. Codex's App Server makes it cloud-native already.

****Cross-vendor session replay (e.g., Claude Code session → Codex):**** ****Fundamentally impractical.**** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) ``encrypted_content`` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. ****Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.****

3. Competitor teardown

****Session-continuity tools:****

- ****tawanorg/claude-sync**** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/ .claude`` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + ****age E2E encryption**** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.
- ****npow/session-sync**** (~1 star, very new): the only cross-vendor ***session*** translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- ****dantelex/aisync ("ss")****: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- ****codsync/codsync**** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- ****kroxiksut/codexSync****: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- ****waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)**** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- ****iHildy/opencode-synced**** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- ****Dinesh3184/claude-session-sync****: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- ****renefichtmueller/claude-sync****: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

****Config/MCP/skills sync (DevSync's neighborhood):****

- ****Leoyang183/sync-agents-settings**** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- ****nicepkg/vsync**** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- ****baranovxyz/agentsync**** and ****spxrogers/agentsync**** (beta): canonical ``.agents/`` config synced to 9–31 agents.
- ****neon-solutions/add-mcp****: one-command MCP install across ~16 agents (Neon).
- ****ztripez/mcp-sync****: MCP config sync across tools.
- ****AgentSync (Rust)****: symlink-based config sync.
- ****skills-sync****: skills + MCP workspace.
- ****DevSynq**** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

****URL-addressable / mobile-control / viewer approaches:****

- ****TokenRip**** (tokenrip.com): makes session state a URL-addressable artifact (``/a/<slug>``) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a

permanent URL").

- **Omnara** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was **archived in February 2026** because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)
- **Syncode – AI Programming Tool**: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- **jazzya/agent-sessions** & **kenn-io/agentsview**: local-first session browsers/search across 20+ agents (read/resume-command, not sync).
- **LAN-workspace approaches** and Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

Takeaway: The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- **Claude Code**: #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- **Codex**: #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI ↔ app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
- **Copilot CLI**: #1947 (cloud-synced sessions), #1635 (cross-environment resume).
- **Cursor forum**: multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is. Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. **Cross-vendor unified profile** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.
2. **OS-aware path remapping as a first-class feature** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows ↔ macOS. This is the wedge no competitor has nailed.
3. **E2E encryption by default** – table stakes given session files contain code, secrets, and full transcripts.

Risks:

- **Format churn:** Claude Code's format "changes between versions" by

Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write — the pattern codexSync/cursaves use).

- **Vendor lock-out / ToS:** Storing session data is user-owned data (fine), but **auth tokens** are not transferable (codexSync explicitly refuses; Codex ``encrypted_content`` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR — don't build on them.
- **Vendors closing the gap:** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This **validates** the demand while **confirming** that only the cross-vendor + cross-OS layer is defensible.
- **Security/privacy expectations:** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote — all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).
- **Honest limits:** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

Stage 0 — Fold in DevSync (immediate). Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

Stage 1 — MVP: Claude Code session sync with path remapping + E2E encryption (the wedge). Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/ .claude/projects/`` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- **First-class OS-aware path remapping:** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks — lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict

preservation.

- **Benchmark to advance:** ``claude --resume`` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

Stage 2 — Second and third vendors: Codex + Gemini CLI + opencode.

Add adapters. For Codex, cooperate with (don't fight) the App Server — sync what's local and useful, avoid ``encrypted_content``/token pitfalls. For Gemini, remap ``project_hash``. For opencode, use safe SQLite snapshotting (the ``sqlite3 .backup`` pattern the community scripts use).

- **Benchmark:** each new adapter must pass the same cross-OS resume test as Claude Code before GA.

Stage 3 — Unified profile + cross-vendor context handoff. One "profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves — explicitly labeled as context transfer, not replay.

Monetization & thresholds. Free/open-source core (build trust, match claude-sync) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSynq/OpenSync validate willingness to pay). **Kill/pivot signal:** if Anthropic ships account-level *local* session sync with automatic path remapping across OSes (not just history mirroring, per #71794), the single-vendor wedge collapses — at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

What NOT to do: Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources — treat as ~40-116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats — especially Claude Code's — are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.

- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

Rethread. And the search settled it.

"Reboarding" isn't an open word — it's established HR jargon with a whole content industry behind it. Workable defines reboarding as reintegrating employees returning after an absence or transitioning within the company, and it cemented itself in HR vocabulary during the pandemic return-to-office wave. SHRM, Workable, and a dozen HR SaaS blogs all own that keyword. You'd be launching a dev tool into a search space fully colonized by human-resources content — and worse, by a term that means "onboarding, but again," which is adjacent enough to developer tooling (onboarding a new machine!) to create genuine confusion rather than clean separation.

Ironically, one of those HR pages describes reboarding as closing the gap between where someone left off and where things currently are. That's *precisely* your product's job — which is exactly why the collision is dangerous rather than distant. Same meaning, wrong industry, established first.

Rethread wins on merits too, not just by default:

- "Thread" is already the native noun for what you're syncing. Amp calls sessions threads; every one of these tools has message threads. The name sits inside the domain vocabulary instead of importing a metaphor from outside it.
- Rethreading a needle is a *precise* physical action — find the loose end, pass it through, continue the same seam. Reboarding is vaguer; you board lots of things.
- It scales into your whole product surface naturally: threads, spools, the seam of work you're keeping continuous. Reboard gives you gates and terminals, which don't map onto sessions.
- Reads well as a CLI: `rethread pull`, `rethread push`. Reboard reads slightly like a device operation.

The one cost you're accepting: the momentary pthread flicker for systems programmers. It resolves in half a second once they see the product, and it's a far smaller tax than competing with SHRM for a search term.

If the concurrency thing ever nags at you, **respool** is the escape hatch — same metaphor family, zero collision. But I'd ship Rethread.

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape

TL;DR

- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows ↔ macOS paths.
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows ↔ macOS sync.
- **Recommendation:** build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode. Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...` and `/Users/...` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly

admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.

3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.

4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

Claude Code (Anthropic). Ships the richest consumer-facing multi-device story:

- **`/teleport`** (``claude --teleport``): pulls a *cloud* session (started at `claude.ai/code` or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport``, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires **clean git state** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same `claude.ai` account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."
- **Remote Control** (``claude remote-control`` / ``/remote-control``, `--rc``): pushes a *live local* session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an *undocumented* ~15-minute idle timeout on the Desktop/Code

surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `ANTHROPIC\_BASE\_URL` is redirected; unavailable under Zero Data Retention.

- **Claude Code on the web** (claude.ai/code): runs on Anthropic cloud infra, no local filesystem.
- **Residual gap:** No true device-to-device **local** session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an **unexpected**, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

Codex CLI (OpenAI). The most architecturally mature cross-surface model:

- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared **App Server** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under `~/.codex/sessions/YYYY/MM/DD/rollout-\*.jsonl`, with a `session\_meta` first line.
- Rich lifecycle: `codex resume` (picker), `--last`, `--resume-session-id`, `/fork`, archive/unarchive (v0.136), delete (v0.140). Imports histories from other agents via an `external-agent-sessions` crate.
- **Known gaps/bugs:** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing `encrypted\_content` often can't be continued after crossing provider/account (`invalid\_encrypted\_content`); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.
- **Residual gap:** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

Gemini CLI (Google). Auto-saves sessions to `~/.gemini/tmp/<project\_hash>/chats/` (project-scoped, hash of project root). `/resume` session browser, `--resume`/`-r` flag, `/chat save` checkpoints, and a shadow-git checkpointing system

under `~/ .gemini/history/<project\_hash>`. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. **Residual gap:** everything local + path-hash-keyed; no sync.

opencode (sst). SQLite + file storage under `~/ .local/share/opencode/` (`opencode.db`, `storage/session|message|part`). Native **session sharing** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). **Residual gap:** sharing ≠ syncing; DB concurrency and cross-OS path issues remain.

Cursor CLI (cursor-agent). Chats live in local SQLite; CLI chats under `~/ .cursor/chats` and resume via `cursor-agent --resume`. **IDE chats and CLI chats do NOT share a session store** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend `&` to push to a Cloud Agent, pick up at cursor.com/agents), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). **Residual gap:** no local cross-device sync, IDE ↔ CLI split, path issues.

Grok / Grok Build (xAI). Sessions auto-saved to `~/ .grok/sessions/`, keyed by working directory. `/resume`, `--resume`, `-c`, `/fork`; subscription-gated (SuperGrok/X Premium+). Config in `~/ .grok/config.toml`. No native cross-device sync. Still beta (v0.2.x as of July 2026). **Residual gap:** no sync at all.

GitHub Copilot CLI. Sessions in `~/ .copilot/` (`session-state/<uuid>/events.jsonl`). Notably, Copilot CLI **syncs sessions to your GitHub account by default** and offers **remote control** from [GitHub.com](https://github.com)/GitHub Mobile (origin machine must stay online, `/keep-alive`). But **cross-device resume** is still not supported – manually copying a session dir and running `--resume` overwrites `events.jsonl` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session\_id`. **Residual gap:** the sync is telemetry/history, not resumable working state across machines.

****Amp (Sourcegraph).**** Threads sync to ampcode.com by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). ****Residual gap:**** vendor-locked, server-stored, no cross-vendor.

****Cross-vendor summary:**** Every vendor is building **within** its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

Tool	Location	Format	Path keying	Cross-device resume native?
Claude Code	`\${HOME}/.claude/projects/<encoded-path>/<session>.jsonl`	JSONL, append-only event stream	**Absolute path, non-alphanumerics→dashes**	Via cloud only (teleport/RC)
Codex	`\${HOME}/.codex/sessions/YYYY/MM/DD/rollout-*.jsonl`	JSONL + `session_meta` header	date-based; cwd in meta	Yes, via App Server
Gemini CLI	`\${HOME}/.gemini/tmp/<project_hash>/chats/`	JSON	**Hash of project root**	No
opencode	`\${HOME}/.local/share/opencode/`	(SQLite + files)	SQLite/JSON	project-keyed
Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)
Grok	`\${HOME}/.grok/sessions/`	JSON	working directory	No
Copilot CLI	`\${HOME}/.copilot/session-state/<uuid>/events.jsonl`	JSONL events	uuid	Account history sync; no resume

****The path-mapping problem (the crux).**** Claude Code derives the project folder name by escaping the absolute path: ``/Users/me/app`` → ``~/.claude/projects/-Users-me-app/``, while the same repo at ``C:\Users\me\app`` (or a different macOS username/location) produces a **different** folder. Consequently, after a naive file sync, ``claude --resume`` on the second machine cannot find the first machine's sessions. This is confirmed by the claude-sync author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation?"

Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker." Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via `project\_hash`; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. **This is the single highest-value, currently-unmet technical feature.**

Format churn risk. Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

Mid-session cross-device resume feasibility (same vendor): Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present — *if* path keys are remapped. Codex's App Server makes it cloud-native already.

Cross-vendor session replay (e.g., Claude Code session → Codex): **Fundamentally impractical.** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) `encrypted\_content` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. **Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.**

3. Competitor teardown

Session-continuity tools:

- **tawanorg/claude-sync** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/.claude` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + **age E2E encryption** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-

derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.

- ****npow/session-sync**** (~1 star, very new): the only cross-vendor *session* translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- ****dantelex/aisync ("ss")****: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- ****codsync/codsync**** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- ****kroxiksut/codexSync****: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- ****waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)**** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- ****iHildy/opencode-synced**** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- ****Dinesh3184/claude-session-sync****: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- ****renefichtmueller/claude-sync****: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

****Config/MCP/skills sync (DevSync's neighborhood):****

- ****Leoyangl83/sync-agents-settings**** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- ****nicepkg/vsync**** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- ****baranovxyz/agentsync**** and ****spxrogers/agentsync**** (beta): canonical ``.agents/`` config synced to 9–31 agents.
- ****neon-solutions/add-mcp****: one-command MCP install across ~16 agents (Neon).
- ****ztripez/mcp-sync****: MCP config sync across tools.
- ****AgentSync (Rust)****: symlink-based config sync.
- ****skills-sync****: skills + MCP workspace.
- ****DevSynq**** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

****URL-addressable / mobile-control / viewer approaches:****

- ****TokenRip**** (tokenrip.com): makes session state a URL-addressable artifact (``/a/<slug>``) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- ****Omnara**** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was ****archived in February 2026**** because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)
- ****Syncode – AI Programming Tool****: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- ****jazzyalex/agent-sessions**** & ****kenn-io/agentsview****: local-first session browsers/search across 20+ agents (read/resume-command, not sync).
- ****LAN-workspace approaches**** and Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

****Takeaway:**** The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- ****Claude Code:**** #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- ****Codex:**** #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI  app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
- ****Copilot CLI:**** #1947 (cloud-synced sessions), #1635 (cross-

environment resume).

- **Cursor forum:** multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is. Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. **Cross-vendor unified profile** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.

2. **OS-aware path remapping as a first-class feature** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows↔macOS. This is the wedge no competitor has nailed.

3. **E2E encryption by default** – table stakes given session files contain code, secrets, and full transcripts.

Risks:

- **Format churn:** Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).

- **Vendor lock-out / ToS:** Storing session data is user-owned data (fine), but *auth tokens* are not transferable (codexSync explicitly refuses; Codex `encrypted\_content` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.

- **Vendors closing the gap:** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This *validates* the demand while

confirming that only the cross-vendor + cross-OS layer is defensible.

- ****Security/privacy expectations:**** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).
- ****Honest limits:**** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

****Stage 0 – Fold in DevSync (immediate).**** Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

****Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge).**** Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/.claude/projects/` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- ****First-class OS-aware path remapping:**** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
- ****Benchmark to advance:**** `claude --resume` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

****Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode.**** Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid

`encrypted\_content`/token pitfalls. For Gemini, remap
`project\_hash`. For opencode, use safe SQLite snapshotting (the
`sqlite3.backup` pattern the community scripts use).
- ****Benchmark:**** each new adapter must pass the same cross-OS
resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One
"profile" = sessions + MCP + skills + agents + instructions,
portable across machines and (for config) across vendors. Add
best-effort context handoff (summarize + re-inject) for cross-
vendor moves – explicitly labeled as context transfer, not
replay.

****Monetization & thresholds.**** Free/open-source core (build
trust, match claude-sync) with a paid tier for managed encrypted
storage, multi-machine dashboards, and team profiles
(DevSynq/OpenSync validate willingness to pay). ****Kill/pivot
signal:**** if Anthropic ships account-level **local** session sync
with automatic path remapping across OSes (not just history
mirroring, per #71794), the single-vendor wedge collapses – at
that point double down on cross-vendor config/skills/profile
portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport
(gated, one-way, origin-online, with documented ~10-min network
and undocumented ~15-min idle timeouts). Don't promise cross-
vendor session replay. Don't sync auth tokens. Don't store
plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment

based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

how about any of these?

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape

TL;DR

- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows↔macOS paths.
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows↔macOS sync.
- **Recommendation: build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode.** Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...\` and `/Users/...\` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.
3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.
4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

- Claude Code (Anthropic).** Ships the richest consumer-facing multi-device story:
 - **`/teleport`** (`claude --teleport`): pulls a *cloud* session (started at claude.ai/code or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal

session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport`, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires ****clean git state**** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same claude.ai account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."

- ****Remote Control**** (`\`claude remote-control` / \`/remote-control`, --rc`): pushes a ***live local*** session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an ***undocumented*** ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `\`ANTHROPIC_BASE_URL`` is redirected; unavailable under Zero Data Retention.
- ****Claude Code on the web**** (claude.ai/code): runs on Anthropic cloud infra, no local filesystem.

- ****Residual gap:**** No true device-to-device ***local*** session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an ***unexpected***, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

****Codex CLI (OpenAI).**** The most architecturally mature cross-surface model:

- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared ****App Server**** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under `\`~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl``, with a `\`session_meta`` first line.

- Rich lifecycle: `\`codex resume`` (picker), `\`--last``, `\`--resume-session-id``, `\`/fork``, `archive/unarchive` (v0.136), `delete` (v0.140). Imports histories from other agents via an `\`external-agent-sessions`` crate.

- ****Known gaps/bugs:**** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing `\`encrypted_content`` often can't be continued after crossing provider/account (`\`invalid_encrypted_content``); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.

- ****Residual gap:**** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

****Gemini CLI (Google).**** Auto-saves sessions to `\`~/gemini/tmp/<project_hash>/chats/`` (project-scoped, hash of project root). `\`/resume`` session browser, `\`--resume`` `\`-r`` flag, `\`/chat save`` checkpoints, and a shadow-git checkpointing system under `\`~/gemini/history/<project_hash>``. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. ****Residual gap:**** everything local + path-hash-keyed; no sync.

****opencode (sst).**** SQLite + file storage under `\`~/local/share/opencode/`` (`\`opencode.db``, `\`storage/session|message|part``). Native ****session sharing**** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). ****Residual gap:**** sharing ≠ syncing; DB concurrency and cross-OS path issues remain.

****Cursor CLI (cursor-agent).**** Chats live in local SQLite; CLI chats under `\`~/cursor/chats`` and resume via `\`cursor-agent --resume``. ****IDE chats and CLI chats do NOT share a session store**** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend `\`&`` to push to a Cloud Agent, pick up at cursor.com/agents), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). ****Residual gap:****

no local cross-device sync, IDE↔CLI split, path issues.

Grok / Grok Build (xAI). Sessions auto-saved to `~/grok/sessions/`, keyed by working directory. `~/resume`, `--resume`, `-c`, `~/fork`; subscription-gated (SuperGrok/X Premium+). Config in `~/grok/config.toml`. No native cross-device sync. Still beta (v0.2.x as of July 2026). **Residual gap:** no sync at all.

GitHub Copilot CLI. Sessions in `~/copilot/` (`session-state/<uuid>/events.jsonl`). Notably, Copilot CLI **syncs** sessions to your GitHub account by default and offers **remote control** from GitHub.com/GitHub Mobile (origin machine must stay online, `~/keep-alive`). But **cross-device resume** is still not supported – manually copying a session dir and running `--resume` overwrites `events.jsonl` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session\_id`. **Residual gap:** the sync is telemetry/history, not resumable working state across machines.

Amp (Sourcegraph). Threads sync to ampcode.com by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). **Residual gap:** vendor-locked, server-stored, no cross-vendor.

Cross-vendor summary: Every vendor is building *within* its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

Tool	Location	Format	Path keying	Cross-device resume native?
				--- --- --- --- ---
Claude Code	~//.claude/projects/<encoded-path>/<session>.jsonl	JSONL, append-only event stream	Absolute path, non-alphanumerics→dashes	Via cloud only (teleport/RC)
Codex	~//.codex/sessions/YYYY/MM/DD/rollout-*.jsonl	JSONL + `session_meta` header	date-based; cwd in meta	Yes, via App Server
Gemini CLI	~//.gemini/tmp/<project_hash>/chats/	JSON	Hash of project root	No
opencode	~//.local/share/opencode/	(SQLite + files)	SQLite/JSON	project-keyed Sharing only
Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)
Grok	~//.grok/sessions/	JSON	working directory	No
Copilot CLI	~//.copilot/session-state/<uuid>/events.jsonl	JSONL events	uuid	Account history sync; no resume

The path-mapping problem (the crux). Claude Code derives the project folder name by escaping the absolute path: `/Users/me/app` → `~//.claude/projects/-Users-me-app/`, while the same repo at `C:\Users\me\app` (or a different macOS username/location) produces a *different* folder. Consequently, after a naive file sync, `claude --resume` on the second machine cannot find the first machine's sessions. This is confirmed by the claude-sync author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker."

Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via `project\_hash`; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. **This is the single highest-value, currently-unmet technical feature.**

Format churn risk. Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

****Mid-session cross-device resume feasibility (same vendor):**** Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present – ***if*** path keys are remapped. Codex's App Server makes it cloud-native already.

****Cross-vendor session replay (e.g., Claude Code session → Codex):**** ****Fundamentally impractical.**** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) ``encrypted_content`` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. ****Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.****

3. Competitor teardown

****Session-continuity tools:****

- ****tawanorg/claude-sync**** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/.claude`` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + ****age E2E encryption**** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.
- ****npow/session-sync**** (~1 star, very new): the only cross-vendor ***session*** translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- ****dantelex/aisync ("ss")****: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- ****codsync/codsync**** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- ****kroxiksut/codexSync****: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- ****waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)**** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- ****iHildy/opencode-synced**** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- ****Dinesh3184/claude-session-sync****: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- ****renefichtmueller/claude-sync****: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

****Config/MCP/skills sync (DevSync's neighborhood):****

- ****Leoyang183/sync-agents-settings**** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- ****nicepkg/vsync**** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- ****baranovxyz/agentsync**** and ****spxrogers/agentsync**** (beta): canonical ``.agents/`` config synced to 9–31 agents.
- ****neon-solutions/add-mcp****: one-command MCP install across ~16 agents (Neon).
- ****ztripez/mcp-sync****: MCP config sync across tools.
- ****AgentSync (Rust)****: symlink-based config sync.
- ****skills-sync****: skills + MCP workspace.
- ****DevSynq**** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

****URL-addressable / mobile-control / viewer approaches:****

- ****TokenRip**** (tokenrip.com): makes session state a URL-addressable artifact (`~/a/<slug>``) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- ****Omnara**** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was ****archived in February 2026**** because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent

SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)

- **Syncode – AI Programming Tool**: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- **jazzyalex/agent-sessions** & **kenn-io/agentsview**: local-first session browsers/search across 20+ agents (read/resume-command, not sync).
- **LAN-workspace approaches** and Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

Takeaway: The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- **Claude Code**: #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- **Codex**: #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI ↔ app-server sessions), discussion #14067 (cross-device thread/contextsync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
- **Copilot CLI**: #1947 (cloud-synced sessions), #1635 (cross-environment resume).
- **Cursor forum**: multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is. Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. **Cross-vendor unified profile** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.
2. **OS-aware path remapping as a first-class feature** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows↔macOS. This is the wedge no competitor has nailed.
3. **E2E encryption by default** – table stakes given session files contain code, secrets, and full transcripts.

Risks:

- **Format churn**: Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).
- **Vendor lock-out / ToS**: Storing session data is user-owned data (fine), but *auth tokens* are not transferable (codexSync explicitly refuses; Codex `encrypted\_content` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.
- **Vendors closing the gap**: Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This *validates* the demand while *confirming* that only the cross-vendor + cross-OS layer is defensible.

- ****Security/privacy expectations:**** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).
- ****Honest limits:**** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

****Stage 0 – Fold in DevSync (immediate).**** Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

****Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge).****
 Target the founder's exact case (Windows desktop 🖥️ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/ .claude/projects/` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- ****First-class OS-aware path remapping:**** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
- ****Benchmark to advance:**** `claude --resume` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

****Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode.**** Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid `encrypted\_content`/token pitfalls. For Gemini, remap `project\_hash`. For opencode, use safe SQLite snapshotting (the `sqlite3 .backup` pattern the community scripts use).

- ****Benchmark:**** each new adapter must pass the same cross-OS resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One "profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not replay.

****Monetization & thresholds.**** Free/open-source core (build trust, match claude-sync) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSync/OpenSync validate willingness to pay). ****Kill/pivot signal:**** if Anthropic ships account-level *local* session sync with automatic path remapping across OSes (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync,

Gemini-Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

Honest read: this list is weaker than where you already were, and the #1 has a problem the scoring missed.

Rethra — two issues. First, say it out loud a few times: RETH-ra. It sits one phoneme from *urethra*, and that's the kind of thing someone points out once in a Discord and it never dies. Second, the pronunciation is genuinely ambiguous — REE-thra? RETH-ra? RAY-thra? A name people can't confidently say is a name they won't say. And it isn't coined: Rethra was the main town and political center of the Slavic Redarians in the 10th–12th centuries, a major worship site in present-day Mecklenburg. It has a Wikipedia page and a whole Slavic-mythology content ecosystem around it. Not a blocker, but it's not the blank slate a coinage is supposed to give you. The bigger cost: truncating "rethread" to "Rethra" throws away the entire reason the name worked — the thread. You're left with a sound.

Knotra — the silent K is disqualifying for a CLI. People will type **notra**, search "notra", and mis-hear it every time you say it aloud. You cannot afford a spelling trap in a tool whose primary interface is typing its name.

Rebranch — the worst semantic fit on the list, and it's subtle. In git, *branching is divergence*. Your product is the opposite: convergence, continuity, one line of work across two machines. You'd be borrowing a word from your users' daily vocabulary and inverting its meaning. Also sits a letter away from "rebrand."

Synknot — try saying it. The **nk-n** cluster stalls the mouth. Written, it reads like a typo.

Statebraid / Statepath / CarryState — these are the honest ones. They're descriptions rather than names, which caps how distinctive they can be, but none of them will embarrass you. **Statepath** is quietly the smartest of the three, because "path" points directly at the actual hard problem in your product — the Windows↔macOS path remapping that the research flagged as your one real moat. A name that gestures at your differentiator is doing more work than a name that gestures at a metaphor.

On the scoring itself: 8.8 versus 8.3 on a brand name is false precision. There's no measurable quantity there, and rubrics systematically overrate "distinctive coinage" while missing exactly the things that kill names in practice — phonetic neighbors, spelling traps, pronunciation ambiguity. Rethra scoring highest is that failure mode in action.

My call: none of these beat **Rethread**. It's pronounceable on first sight, spellable on first hearing, sits inside your domain's native vocabulary, and describes the actual gesture. If you want something off this list, take **Statepath** — and only because it names your moat.

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape

TL;DR

- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows ↔ macOS paths.

- **Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows ↔ macOS sync.

- **Recommendation:** build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode. Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.

2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files

between ``C:\Users\...\`` and ``/Users/...\`` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.

3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.

4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

Claude Code (Anthropic). Ships the richest consumer-facing multi-device story:

- **``/teleport``** (``claude --teleport``): pulls a *cloud* session (started at claude.ai/code or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with ``--teleport``, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires **clean git state** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same claude.ai account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."
- **Remote Control** (``claude remote-control`` / ``/remote-control``, ``--rc``): pushes a *live local* session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times

out and the process exits." (Separately, issue #39970 reports an **undocumented** ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `ANTHROPIC\_BASE\_URL` is redirected; unavailable under Zero Data Retention.

- **Claude Code on the web** (claude.ai/code): runs on Anthropic cloud infra, no local filesystem.

- **Residual gap:** No true device-to-device **local** session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an **unexpected**, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

Codex CLI (OpenAI). The most architecturally mature cross-surface model:

- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared **App Server** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under

`~/codex/sessions/YYYY/MM/DD/rollout-\*.jsonl`, with a `session\_meta` first line.

- Rich lifecycle: `codex resume` (picker), `--last`, `--resume-session-id`, `/fork`, archive/unarchive (v0.136), delete (v0.140). Imports histories from other agents via an `external-agent-sessions` crate.

- **Known gaps/bugs:** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing `encrypted\_content` often can't be continued after crossing provider/account (`invalid\_encrypted\_content`); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.

- **Residual gap:** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

Gemini CLI (Google). Auto-saves sessions to

`~/gemini/tmp/<project\_hash>/chats/` (project-scoped, hash of

project root). ``/resume`` session browser, ``--resume`/`-r`` flag, ``/chat save`` checkpoints, and a shadow-git checkpointing system under ``~/.gemini/history/<project_hash>``. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. ****Residual gap:**** everything local + path-hash-keyed; no sync.

****opencode (sst).**** SQLite + file storage under ``~/.local/share/opencode/`` (``opencode.db``, ``storage/session|message|part``). Native ****session sharing**** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). ****Residual gap:**** sharing $\neq$ syncing; DB concurrency and cross-OS path issues remain.

****Cursor CLI (cursor-agent).**** Chats live in local SQLite; CLI chats under ``~/.cursor/chats`` and resume via ``cursor-agent --resume``. ****IDE chats and CLI chats do NOT share a session store**** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend ``&`` to push to a Cloud Agent, pick up at `cursor.com/agents`), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). ****Residual gap:**** no local cross-device sync, IDE ↔ CLI split, path issues.

****Grok / Grok Build (xAI).**** Sessions auto-saved to ``~/.grok/sessions/``, keyed by working directory. ``/resume``, ``--resume``, ``-c``, ``/fork``; subscription-gated (SuperGrok/X Premium+). Config in ``~/.grok/config.toml``. No native cross-device sync. Still beta (v0.2.x as of July 2026). ****Residual gap:**** no sync at all.

****GitHub Copilot CLI.**** Sessions in ``~/.copilot/`` (``session-state/<uuid>/events.jsonl``). Notably, Copilot CLI ****syncs sessions to your GitHub account by default**** and offers ****remote control**** from GitHub.com/GitHub Mobile (origin machine must stay online, ``/keep-alive``). But ****cross-device *resume* is still not supported**** – manually copying a session dir and running ``--resume`` overwrites ``events.jsonl`` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied ``session_id``. ****Residual gap:**** the sync is

telemetry/history, not resumable working state across machines.

****Amp (Sourcegraph).**** Threads sync to ampcode.com by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). ****Residual gap:**** vendor-locked, server-stored, no cross-vendor.

****Cross-vendor summary:**** Every vendor is building **within** its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

Tool	Location	Format	Path keying	Cross-device resume native?
Claude Code	~/.claude/projects/<encoded-path>/<session>.jsonl`	JSONL, append-only event stream	**Absolute path, non-alphanumerics→dashes**	Via cloud only (teleport/RC)
Codex	~/.codex/sessions/YYYY/MM/DD/rollout-*.jsonl`	JSONL + `session_meta` header	date-based; cwd in meta	Yes, via App Server
Gemini CLI	~/.gemini/tmp/<project_hash>/chats/`	JSON	**Hash of project root**	No
opencode	~/.local/share/opencode/`	(SQLite + files)	SQLite/JSON	project-keyed
Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)
Grok	~/.grok/sessions/`	JSON	working directory	No
Copilot CLI	~/.copilot/session-state/<uuid>/events.jsonl`	JSONL events	uuid	Account history sync; no resume

****The path-mapping problem (the crux).**** Claude Code derives the project folder name by escaping the absolute path: ``/Users/me/app`` → `~/.claude/projects/-Users-me-app/``, while the same repo at `C:\Users\me\app`` (or a different macOS username/location) produces a **different** folder. Consequently, after a naive file sync, ``claude --resume`` on the second machine cannot find the first machine's sessions. This is confirmed by the `claude-sync` author's own documentation and raised pointedly

by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker." Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via `project\_hash`; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. ****This is the single highest-value, currently-unmet technical feature.****

****Format churn risk.**** Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

****Mid-session cross-device resume feasibility (same vendor):**** Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present – *if* path keys are remapped. Codex's App Server makes it cloud-native already.

****Cross-vendor session replay (e.g., Claude Code session → Codex):**** ****Fundamentally impractical.**** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) `encrypted\_content` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. ****Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.****

3. Competitor teardown

****Session-continuity tools:****

- ****tawanorg/claude-sync**** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/.claude` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + ****age E2E encryption****

("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.

- ****npow/session-sync**** (~1 star, very new): the only cross-vendor *session* translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- ****dantelex/aisync ("ss")****: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- ****codsync/codsync**** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- ****kroxiksut/codexSync****: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- ****waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)**** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- ****iHildy/opencode-synced**** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- ****Dinesh3184/claude-session-sync****: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- ****renefichtmueller/claude-sync****: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

****Config/MCP/skills sync (DevSync's neighborhood):****

- ****Leoyangl83/sync-agents-settings**** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- ****nicepkg/vsync**** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- ****baranovxyz/agentsync**** and ****spxrogers/agentsync**** (beta): canonical ``.agents/`` config synced to 9–31 agents.
- ****neon-solutions/add-mcp****: one-command MCP install across ~16 agents (Neon).
- ****ztripez/mcp-sync****: MCP config sync across tools.
- ****AgentSync (Rust)****: symlink-based config sync.
- ****skills-sync****: skills + MCP workspace.

- **DevSynq** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

URL-addressable / mobile-control / viewer approaches:

- **TokenRip** (tokenrip.com): makes session state a URL-

addressable artifact (`/a/<slug>`) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").

- **Omnara** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was **archived** in February 2026 because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)

- **Syncode – AI Programming Tool**: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).

- **jazzyalex/agent-sessions** & **kenn-io/agentsview**: local-first session browsers/search across 20+ agents (read/resume-command, not sync).

- **LAN-workspace approaches** and

Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

Takeaway: The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- **Claude Code**: #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of

transcripts/plan/memory), #26045 (cross-device memory sync),

#45358 (cross-machine session sync, Mac+Windows+Linux), #51816

(Mac+Windows instance-to-instance), #60058 (connect from any

client), #22648 (account-level settings sync, with a long list of

duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows

Anthropic quietly began syncing history across devices.

- **Codex**: #12593 (VS Code account-linked cross-device sync),

#14722 (sync CLI  app-server sessions), discussion #14067

(cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).

- **Copilot CLI:** #1947 (cloud-synced sessions), #1635 (cross-environment resume).

- **Cursor forum:** multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is. Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. **Cross-vendor unified profile** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.

2. **OS-aware path remapping as a first-class feature** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows↔macOS. This is the wedge no competitor has nailed.

3. **E2E encryption by default** – table stakes given session files contain code, secrets, and full transcripts.

Risks:

- **Format churn:** Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).

- **Vendor lock-out / ToS:** Storing session data is user-owned data (fine), but *auth tokens* are not transferable (codexSync explicitly refuses; Codex `encrypted\_content` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.

- ****Vendors closing the gap:**** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This **validates** the demand while **confirming** that only the cross-vendor + cross-OS layer is defensible.
- ****Security/privacy expectations:**** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).
- ****Honest limits:**** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

****Stage 0 – Fold in DevSync (immediate).**** Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

****Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge).**** Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/.claude/projects/` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- ****First-class OS-aware path remapping:**** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
- ****Benchmark to advance:**** `claude --resume` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

****Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode.**** Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid `encrypted\_content`/token pitfalls. For Gemini, remap `project\_hash`. For opencode, use safe SQLite snapshotting (the `sqlite3.backup` pattern the community scripts use).

- ****Benchmark:**** each new adapter must pass the same cross-OS resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One "profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not replay.

****Monetization & thresholds.**** Free/open-source core (build trust, match claude-sync) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSynq/OpenSync validate willingness to pay). ****Kill/pivot signal:**** if Anthropic ships account-level **local** session sync with automatic path remapping across OSes (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.

- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.

- Vendor capabilities (teleport, Remote Control, Codex App

Server, Copilot sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.

- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

rethread is apparantly already an ai company (ai memory ig)

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape

TL;DR

- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows↔macOS paths.

- **Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows↔macOS sync.

- **Recommendation:** build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode. Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...` and `/Users/...` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.
3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.
4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

Claude Code (Anthropic). Ships the richest consumer-facing multi-device story:

- **``/teleport``** (`claude --teleport`): pulls a *cloud* session (started at `claude.ai/code` or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport`, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires **clean git state** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same `claude.ai` account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."
 - **Remote Control** (`claude remote-control` / `/remote-control`, `--rc`): pushes a *live local* session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an *undocumented* ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `ANTHROPIC_BASE_URL` is redirected; unavailable under Zero Data Retention.
 - **Claude Code on the web** (`claude.ai/code`): runs on Anthropic cloud infra, no local filesystem.
 - **Residual gap:** No true device-to-device *local* session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an *unexpected*, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

Codex CLI (OpenAI). The most architecturally mature cross-surface model:

- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared **App Server** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under `~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl`, with a `session_meta` first line.
 - Rich lifecycle: `codex resume` (picker), `--last`, `--resume-session-id`, `/fork`, `archive/unarchive` (v0.136), `delete` (v0.140). Imports histories from other agents via an `external-agent-sessions` crate.
- **Known gaps/bugs:** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing `encrypted_content` often can't be continued after crossing provider/account (`invalid_encrypted_content`); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.
- **Residual gap:** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

Gemini CLI (Google). Auto-saves sessions to `~/gemini/tmp/<project_hash>/chats/` (project-scoped, hash of project root). `/resume` session browser, `--resume` `/-r` flag, `/chat save` checkpoints, and a shadow-git checkpointing system under `~/gemini/history/<project_hash>`. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. **Residual gap:** everything local + path-hash-keyed; no sync.

opencode (sst). SQLite + file storage under `~/local/share/opencode/` (`opencode.db`, `storage/session|message|part`). Native **session sharing** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). **Residual gap:** sharing ≠ syncing; DB concurrency and cross-OS path issues remain.

Cursor CLI (cursor-agent). Chats live in local SQLite; CLI chats under `~/cursor/chats` and resume via `cursor-agent --resume`. **IDE chats and CLI chats do NOT share a session**

store** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend `&` to push to a Cloud Agent, pick up at cursor.com/agents), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). **Residual gap:** no local cross-device sync, IDE↔CLI split, path issues.

Grok / Grok Build (xAI). Sessions auto-saved to `~/grok/sessions/`, keyed by working directory. `./resume`, `--resume`, `-c`, `./fork`; subscription-gated (SuperGrok/X Premium+). Config in `~/grok/config.toml`. No native cross-device sync. Still beta (v0.2.x as of July 2026). **Residual gap:** no sync at all.

GitHub Copilot CLI. Sessions in `~/copilot/` (`session-state/<uuid>/events.jsonl`). Notably, Copilot CLI **syncs sessions to your GitHub account by default** and offers **remote control** from [GitHub.com](https://github.com)/GitHub Mobile (origin machine must stay online, `./keep-alive`). But **cross-device resume** is still not supported – manually copying a session dir and running `--resume` overwrites `events.jsonl` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session\_id`. **Residual gap:** the sync is telemetry/history, not resumable working state across machines.

Amp (Sourcegraph). Threads sync to ampcode.com by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). **Residual gap:** vendor-locked, server-stored, no cross-vendor.

Cross-vendor summary: Every vendor is building *within* its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

	Tool	Location	Format	Path keying	Cross-device resume native?
					--- --- --- --- ---
	Claude Code	~/claude/projects/<encoded-path>/<session>.jsonl`	JSONL, append-only event stream	**Absolute path, non-alphanumerics→dashes**	Via cloud only (teleport/RC)
	Codex	~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl`	JSONL + `session_meta` header	date-based; cwd in meta	Yes, via App Server
	Gemini CLI	~/gemini/tmp/<project_hash>/chats/`	JSON	**Hash of project root**	No
	opencode	~/local/share/opencode/`	(SQLite + files)	SQLite/JSON	project-keyed
					Sharing only
	Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)
				Grok	~/grok/sessions/`
					JSON
					working directory
					No
	Copilot CLI	~/copilot/session-state/<uuid>/events.jsonl`	JSONL events	uuid	Account history sync; no resume

The path-mapping problem (the crux). Claude Code derives the project folder name by escaping the absolute path: `~/Users/me/app` → `~/claude/projects/-Users-me-app/`, while the same repo at `C:\Users\me\app` (or a different macOS username/location) produces a *different* folder. Consequently, after a naive file sync, `claude --resume` on the second machine cannot find the first machine's sessions. This is confirmed by the `claude-sync` author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker."

Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via `project\_hash`; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. **This is the single highest-value, currently-unmet technical feature.**

Format churn risk. Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any

release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

Mid-session cross-device resume feasibility (same vendor): Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present – *if* path keys are remapped. Codex's App Server makes it cloud-native already.

Cross-vendor session replay (e.g., Claude Code session → Codex): **Fundamentally impractical.** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) `encrypted_content` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. **Recommendation:** treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.

3. Competitor teardown

Session-continuity tools:

- **tawanorg/claude-sync** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/.claude` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + **age** E2E encryption ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. **Punts on path remapping** (its explicit, acknowledged weakness). Open-source, no business model.
- **npow/session-sync** (~1 star, very new): the only cross-vendor *session* translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- **dantelex/aisync** ("ss"): multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- **codsyc/codsyc** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- **kroxiksut/codexSync**: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- **waynesutton/opencode-sync-plugin & codex-sync-plugin** (OpenSync) (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- **iHildy/opencode-synced** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- **Dinesh3184/claude-session-sync**: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- **renefichtmueller/claude-sync**: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

Config/MCP/skills sync (DevSync's neighborhood):

- **Leoyang183/sync-agents-settings** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- **nicepkg/vsync** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- **baranovxyz/agentsync** and **spxrogers/agentsync** (beta): canonical `~/.agents/` config synced to 9–31 agents.
- **neon-solutions/add-mcp**: one-command MCP install across ~16 agents (Neon).
- **ztripez/mcp-sync**: MCP config sync across tools.
- **AgentSync** (Rust): symlink-based config sync.
- **skills-sync**: skills + MCP workspace.
- **DevSynq** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

URL-addressable / mobile-control / viewer approaches:

- **TokenRip** (tokenrip.com): makes session state a URL-addressable artifact (`/a/<slug>`) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- **Omnara** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff;

the original open-source wrapper repo (~2.6k stars) was **archived in February 2026** because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)

- **Syncode – AI Programming Tool**: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- **jazzyalex/agent-sessions** & **kenn-io/agentsview**: local-first session browsers/search across 20+ agents (read/resume-command, not sync).
- **LAN-workspace approaches** and Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

Takeaway: The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- **Claude Code**: #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- **Codex**: #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI ↔ app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
- **Copilot CLI**: #1947 (cloud-synced sessions), #1635 (cross-environment resume).
- **Cursor forum**: multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is. Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. **Cross-vendor unified profile** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.
2. **OS-aware path remapping as a first-class feature** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows↔macOS. This is the wedge no competitor has nailed.
3. **E2E encryption by default** – table stakes given session files contain code, secrets, and full transcripts.

Risks:

- **Format churn**: Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).
- **Vendor lock-out / ToS**: Storing session data is user-owned data (fine), but *auth tokens* are not transferable (codexSync explicitly refuses; Codex `encrypted\_content` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.

- ****Vendors closing the gap:**** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This **validates** the demand while **confirming** that only the cross-vendor + cross-OS layer is defensible.
- ****Security/privacy expectations:**** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).
- ****Honest limits:**** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

****Stage 0 – Fold in DevSync (immediate).**** Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

****Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge).****
 Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/.claude/projects/` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- ****First-class OS-aware path remapping:**** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
- ****Benchmark to advance:**** `claude --resume` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

****Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode.**** Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid `encrypted\_content`/token pitfalls. For Gemini, remap `project\_hash`. For opencode, use safe SQLite snapshotting (the `sqlite3 .backup` pattern the community scripts use).

- ****Benchmark:**** each new adapter must pass the same cross-OS resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One "profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not replay.

****Monetization & thresholds.**** Free/open-source core (build trust, match claude-sync) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSync/OpenSync validate willingness to pay). ****Kill/pivot signal:**** if Anthropic ships account-level **local** session sync with automatic path remapping across OSes (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.

- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

Confirmed, and it's worse than a name clash — it's a positioning clash.

Rethread.dev captures preferences, decisions, and context from AI conversations and brings it back when you need it, across six platforms, end-to-end encrypted across every device you own. Read that against your own product's pitch and the overlap is uncomfortable: cross-platform, cross-device, E2E encrypted, context continuity. Different surface — theirs is a Chrome extension for chat UIs, yours is CLI coding agents — but the words are the same words. They also hold `rethread.dev`, and they're already showing up in roundups: one comparison recommends OpenMemory or Rethread for developers switching between Claude and ChatGPT.

So: dead. Not "risky" — dead. Two products in adjacent lanes with identical positioning vocabulary and the same name is the one scenario worth avoiding outright.

The more useful lesson here. You've now lost names to: an AI memory extension, a genAI image API, an HR training category, a CX analytics company. That's not bad luck. It's that you keep building names out of *thread*, *context*, and *memory* — the three most saturated nouns in AI product naming right now. Anything assembled from that vocabulary will collide, because everyone is drawing from the same small bag.

The escape is to name the **gesture** or the **object**, not the AI concept. Nobody in the AI space is naming things after luggage, rope, or handoffs — which is exactly why those are still open.

Where I'd go now:

- **Respool** — my flagged escape hatch from two turns ago, and it holds up. Same thread family, different noun, sits outside AI vocabulary entirely. Spooling is also real infra vocabulary (print spoolers, job spools), so it reads native to systems people.
- **Carryall** — names the gesture. One bag, everything in it, goes where you go. Zero AI-vocabulary overlap.
- **Baton** — still the cleanest one-word version of your entire product.
- **Statepath** — from that ranked list, the only one worth keeping, because it points at your actual moat.

Practical suggestion before you get attached again: pick three finalists and check all of them in one pass — plain web search, npm, GitHub org, and the `.dev/.com`. You've now burned four names post-attachment, and the

checking is cheap compared to the re-deciding.

Want me to run that check on Respool, Carryall, and Baton together?

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

```
# Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape
```

```
## TL;DR
```

```
- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows↔macOS paths.
```

```
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows↔macOS sync.
```

```
- **Recommendation: build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode.** Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.
```

```
## Key Findings
```

```
1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
```

2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between ``C:\Users\...\`` and ``/Users/...\`` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.

3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.

4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

Claude Code (Anthropic). Ships the richest consumer-facing multi-device story:

- **``/teleport``** (``claude --teleport``): pulls a *cloud* session (started at claude.ai/code or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with ``--teleport``, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires **clean git state** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same claude.ai account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."
- **Remote Control** (``claude remote-control`` / ``/remote-control``, ``--rc``): pushes a *live local* session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the

terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an **undocumented** ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `ANTHROPIC\_BASE\_URL` is redirected; unavailable under Zero Data Retention.

- **Claude Code on the web** (claude.ai/code): runs on Anthropic cloud infra, no local filesystem.

- **Residual gap:** No true device-to-device **local** session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an **unexpected**, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

Codex CLI (OpenAI). The most architecturally mature cross-surface model:

- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared **App Server** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under `~/.codex/sessions/YYYY/MM/DD/rollout-\*.jsonl`, with a `session\_meta` first line.

- Rich lifecycle: `codex resume` (picker), `--last`, `--resume-session-id`, `/fork`, archive/unarchive (v0.136), delete (v0.140). Imports histories from other agents via an `external-agent-sessions` crate.

- **Known gaps/bugs:** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing `encrypted\_content` often can't be continued after crossing provider/account (`invalid\_encrypted\_content`); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.

- **Residual gap:** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

****Gemini CLI (Google).**** Auto-saves sessions to `~/ .gemini/tmp/<project_hash>/chats/`` (project-scoped, hash of project root). `/resume`` session browser, `--resume`/`-r`` flag, `/chat save`` checkpoints, and a shadow-git checkpointing system under `~/ .gemini/history/<project_hash>``. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. ****Residual gap:**** everything local + path-hash-keyed; no sync.

****opencode (sst).**** SQLite + file storage under `~/ .local/share/opencode/`` (`opencode.db``, `storage/session|message|part``). Native ****session sharing**** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). ****Residual gap:**** sharing $\neq$ syncing; DB concurrency and cross-OS path issues remain.

****Cursor CLI (cursor-agent).**** Chats live in local SQLite; CLI chats under `~/ .cursor/chats`` and resume via `cursor-agent --resume``. ****IDE chats and CLI chats do NOT share a session store**** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend `&`` to push to a Cloud Agent, pick up at `cursor.com/agents`), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). ****Residual gap:**** no local cross-device sync, IDE ↔ CLI split, path issues.

****Grok / Grok Build (xAI).**** Sessions auto-saved to `~/ .grok/sessions/``, keyed by working directory. `/resume``, `--resume``, `-c``, `/fork``; subscription-gated (SuperGrok/X Premium+). Config in `~/ .grok/config.toml``. No native cross-device sync. Still beta (v0.2.x as of July 2026). ****Residual gap:**** no sync at all.

****GitHub Copilot CLI.**** Sessions in `~/ .copilot/`` (`session-state/<uuid>/events.jsonl``). Notably, Copilot CLI ****syncs sessions to your GitHub account by default**** and offers ****remote control**** from `GitHub.com/GitHub Mobile` (origin machine must stay online, `/keep-alive``). But **\*\*cross-device \*resume\* is still not supported\*\*** – manually copying a session dir and running `--resume`` overwrites `events.jsonl`` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request

(#1947). The Copilot SDK supports resumable sessions via a user-supplied `session\_id`. **Residual gap:** the sync is telemetry/history, not resumable working state across machines.

Amp (Sourcegraph). Threads sync to ampcode.com by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). **Residual gap:** vendor-locked, server-stored, no cross-vendor.

Cross-vendor summary: Every vendor is building *within* its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

Tool	Location	Format	Path keying	Cross-device resume native?	
Claude Code	~/.claude/projects/<encoded-path>/<session>.jsonl	JSONL, append-only event stream	Absolute path, non-alphanumerics→dashes	Via cloud only (teleport/RC)	
Codex	~/.codex/sessions/YYYY/MM/DD/rollout-*.jsonl	JSONL + `session_meta` header	date-based; cwd in meta	Yes, via App Server	
Gemini CLI	~/.gemini/tmp/<project_hash>/chats/	JSON	Hash of project root	No	
opencode	~/.local/share/opencode/	(SQLite + files)	SQLite/JSON	project-keyed	Sharing only
Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)	
Grok	~/.grok/sessions/	JSON	working directory	No	
Copilot CLI	~/.copilot/session-state/<uuid>/events.jsonl	JSONL events	uuid	Account history sync; no resume	

The path-mapping problem (the crux). Claude Code derives the project folder name by escaping the absolute path:

`/Users/me/app` → `~/.claude/projects/-Users-me-app/`, while the same repo at `C:\Users\me\app` (or a different macOS username/location) produces a *different* folder. Consequently, after a naive file sync, `claude --resume` on the second machine

cannot find the first machine's sessions. This is confirmed by the claude-sync author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker." Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via `project\_hash`; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. ****This is the single highest-value, currently-unmet technical feature.****

****Format churn risk.**** Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

****Mid-session cross-device resume feasibility (same vendor):**** Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present — *if* path keys are remapped. Codex's App Server makes it cloud-native already.

****Cross-vendor session replay (e.g., Claude Code session → Codex):**** ****Fundamentally impractical.**** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) `encrypted\_content` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. ****Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.****

3. Competitor teardown

****Session-continuity tools:****

- ****tawanorg/claude-sync**** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/.claude`

(sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + **age** E2E encryption

("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. **Punts on path remapping** (its explicit, acknowledged weakness). Open-source, no business model.

- **npow/session-sync** (~1 star, very new): the only cross-vendor **session** translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- **dantelex/aisync ("ss")**: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- **codsync/codsync** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- **kroxiksut/codexSync**: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- **waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- **iHildy/opencode-synced** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- **Dinesh3184/claude-session-sync**: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- **renefichtmueller/claude-sync**: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

Config/MCP/skills sync (DevSync's neighborhood):

- **Leoyang183/sync-agents-settings** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- **nicepkg/vsync** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- **baranovxyz/agentsync** and **spxrogers/agentsync** (beta): canonical ``.agents/`` config synced to 9–31 agents.
- **neon-solutions/add-mcp**: one-command MCP install across ~16 agents (Neon).
- **ztripez/mcp-sync**: MCP config sync across tools.

- **AgentSync (Rust)**: symlink-based config sync.
- **skills-sync**: skills + MCP workspace.
- **DevSynq** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

URL-addressable / mobile-control / viewer approaches:

- **TokenRip** (tokenrip.com): makes session state a URL-addressable artifact (`/a/<slug>`) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- **Omnara** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was **archived in February 2026** because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)
- **Syncode – AI Programming Tool**: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- **jazzyalex/agent-sessions** & **kenn-io/agentsview**: local-first session browsers/search across 20+ agents (read/resume-command, not sync).

- **LAN-workspace approaches** and

Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

Takeaway: The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- **Claude Code**: #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.

- **Codex:** #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI ↔ app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
- **Copilot CLI:** #1947 (cloud-synced sessions), #1635 (cross-environment resume).
- **Cursor forum:** multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is. Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. **Cross-vendor unified profile** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.
2. **OS-aware path remapping as a first-class feature** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows ↔ macOS. This is the wedge no competitor has nailed.
3. **E2E encryption by default** – table stakes given session files contain code, secrets, and full transcripts.

Risks:

- **Format churn:** Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).
- **Vendor lock-out / ToS:** Storing session data is user-owned data (fine), but *auth tokens* are not transferable (codexSync explicitly refuses; Codex `encrypted\_content` breaks across accounts). Never sync credentials; never depend on undocumented

cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.

- ****Vendors closing the gap:**** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This **validates** the demand while **confirming** that only the cross-vendor + cross-OS layer is defensible.

- ****Security/privacy expectations:**** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).

- ****Honest limits:**** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

****Stage 0 – Fold in DevSync (immediate).**** Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

****Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge).**** Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/.claude/projects/` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- ****First-class OS-aware path remapping:**** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
- ****Benchmark to advance:**** `claude --resume` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption,

across 3 consecutive Claude Code releases.

****Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode.**** Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid `encrypted\_content`/token pitfalls. For Gemini, remap `project\_hash`. For opencode, use safe SQLite snapshotting (the `sqlite3 .backup` pattern the community scripts use).
- ****Benchmark:**** each new adapter must pass the same cross-OS resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One "profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not replay.

****Monetization & thresholds.**** Free/open-source core (build trust, match claude-sync) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSynq/OpenSync validate willingness to pay). ****Kill/pivot signal:**** if Anthropic ships account-level **local** session sync with automatic path remapping across OSes (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing

approach carries ongoing maintenance risk.

- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

how about some wordplay names, for example (totally out of context here just as an example), there's this food brand called "Chaayos" that serves chai/tea. it is a wordplay for "Chaos". sounds fun. suggest names like that.

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape

TL;DR

- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows↔macOS paths.
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows↔macOS sync.
- **Recommendation: build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode.** Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...` and `/Users/...` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.
3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.
4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No

tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

- Claude Code (Anthropic).** Ships the richest consumer-facing multi-device story:
 - teleport** (`claude --teleport`): pulls a *cloud* session (started at `claude.ai/code` or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport`, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires **clean git state** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same `claude.ai` account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."
 - Remote Control** (`claude remote-control` / `/remote-control`, `--rc`): pushes a *live local* session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an *undocumented* ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `ANTHROPIC_BASE_URL` is redirected; unavailable under Zero Data Retention.
 - Claude Code on the web** (`claude.ai/code`): runs on Anthropic cloud infra, no local filesystem.
 - Residual gap:** No true device-to-device *local* session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an *unexpected*, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.
 - Codex CLI (OpenAI).** The most architecturally mature cross-surface model:
 - CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared **App Server** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under `~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl`, with a `session_meta` first line.
 - Rich lifecycle: `codex resume` (picker), `--last`, `--resume-session-id`, `/fork`, `archive/unarchive` (v0.136), `delete` (v0.140). Imports histories from other agents via an `external-agent-sessions` crate.
 - Known gaps/bugs:** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing `encrypted_content` often can't be continued after crossing provider/account (`invalid_encrypted_content`); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.
 - Residual gap:** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.
 - Gemini CLI (Google).** Auto-saves sessions to `~/gemini/tmp/<project_hash>/chats/` (project-scoped, hash of project root). `/resume` session browser, `--resume` `/-r` flag, `/chat save` checkpoints, and a shadow-git checkpointing system under `~/gemini/history/<project_hash>`. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. **Residual gap:** everything local + path-hash-keyed; no sync.
 - opencode (sst).** SQLite + file storage under `~/local/share/opencode/` (`opencode.db`, `storage/session|message|part`). Native **session sharing** publishes to Cloudflare Durable

Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). **Residual gap:** sharing ≠ syncing; DB concurrency and cross-OS path issues remain.

Cursor CLI (cursor-agent). Chats live in local SQLite; CLI chats under `~/cursor/chats` and resume via `cursor-agent --resume`. **IDE chats and CLI chats do NOT share a session store** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend `&` to push to a Cloud Agent, pick up at cursor.com/agents), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). **Residual gap:** no local cross-device sync, IDE↔CLI split, path issues.

Grok / Grok Build (xAI). Sessions auto-saved to `~/grok/sessions/`, keyed by working directory. `resume`, `--resume`, `-c`, `/fork`; subscription-gated (SuperGrok/X Premium+). Config in `~/grok/config.toml`. No native cross-device sync. Still beta (v0.2.x as of July 2026). **Residual gap:** no sync at all.

GitHub Copilot CLI. Sessions in `~/copilot/` (`session-state/<uuid>/events.jsonl`). Notably, Copilot CLI **syncs sessions to your GitHub account by default** and offers **remote control** from GitHub.com/GitHub Mobile (origin machine must stay online, `/keep-alive`). But **cross-device resume** is still not supported – manually copying a session dir and running `--resume` overwrites `events.jsonl` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session\_id`. **Residual gap:** the sync is telemetry/history, not resumable working state across machines.

Amp (Sourcegraph). Threads sync to ampcode.com by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). **Residual gap:** vendor-locked, server-stored, no cross-vendor.

Cross-vendor summary: Every vendor is building *within* its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

	Tool	Location	Format	Path keying	Cross-device resume native?
					--- --- --- --- ---
	Claude Code	~/claude/projects/<encoded-path>/<session>.jsonl	JSONL, append-only event stream	Absolute path, non-alphanumerics→dashes	Via cloud only (teleport/RC)
	Codex	~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl	JSONL + `session_meta` header	date-based; cwd in meta	Yes, via App Server
	Gemini CLI	~/gemini/tmp/<project_hash>/chats/	JSON	Hash of project root	No
	opencode	~/local/share/opencode/	(SQLite + files)	SQLite/JSON	project-keyed Sharing only
	Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)
	Grok	~/grok/sessions/	JSON	working directory	No
	Copilot CLI	~/copilot/session-state/<uuid>/events.jsonl	JSONL events	uuid	Account history sync; no resume

The path-mapping problem (the crux). Claude Code derives the project folder name by escaping the absolute path: `/Users/me/app` → `~/claude/projects/-Users-me-app/`, while the same repo at `C:\Users\me\app` (or a different macOS username/location) produces a *different* folder. Consequently, after a naive file sync, `claude --resume` on the second machine cannot find the first machine's sessions. This is confirmed by the claude-sync author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker." Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via `project\_hash`; Cursor stores path metadata in

chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. ****This is the single highest-value, currently-unmet technical feature.****

****Format churn risk.**** Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

****Mid-session cross-device resume feasibility (same vendor):**** Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present – ***if*** path keys are remapped. Codex's App Server makes it cloud-native already.

****Cross-vendor session replay (e.g., Claude Code session → Codex):**** ****Fundamentally impractical.**** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) ``encrypted_content`` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. ****Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.****

3. Competitor teardown

****Session-continuity tools:****

- ****tawanorg/claude-sync**** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/.claude`` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + ****age E2E encryption**** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.
- ****npow/session-sync**** (~1 star, very new): the only cross-vendor ***session*** translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- ****dantelex/aisync ("ss")****: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- ****codsync/codsync**** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- ****kroxiksut/codexSync****: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- ****waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)**** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- ****iHildy/opencode-synced**** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- ****Dinesh3184/claude-session-sync****: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- ****renefichtmueller/claude-sync****: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

****Config/MCP/skills sync (DevSync's neighborhood):****

- ****Leoyang183/sync-agents-settings**** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- ****nicepkg/vsync**** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- ****baranovxyz/agentsync**** and ****spxrogers/agentsync**** (beta): canonical ``.agents/`` config synced to 9–31 agents.
- ****neon-solutions/add-mcp****: one-command MCP install across ~16 agents (Neon).
- ****ztripez/mcp-sync****: MCP config sync across tools.
- ****AgentSync (Rust)****: symlink-based config sync.
- ****skills-sync****: skills + MCP workspace.
- ****DevSynq**** (devsynq.app): commercial-looking MCP/API-key sync across IDEs +

launcher/marketplace.

****URL-addressable / mobile-control / viewer approaches:****

- ****TokenRip**** (tokenrip.com): makes session state a URL-addressable artifact (`/a/<slug>`) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- ****Omnara**** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was ****archived in February 2026**** because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)
- ****Syncode – AI Programming Tool****: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- ****jazzyalex/agent-sessions**** & ****kenn-io/agentsview****: local-first session browsers/search across 20+ agents (read/resume-command, not sync).
- ****LAN-workspace approaches**** and Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

****Takeaway:**** The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- ****Claude Code:**** #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- ****Codex:**** #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI  app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
- ****Copilot CLI:**** #1947 (cloud-synced sessions), #1635 (cross-environment resume).
- ****Cursor forum:**** multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

****Where the durable moat is.**** Vendors will keep building **intra**-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. ****Cross-vendor unified profile**** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.
2. ****OS-aware path remapping as a first-class feature**** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows  macOS. This is the wedge no competitor has nailed.
3. ****E2E encryption by default**** – table stakes given session files contain code, secrets, and full transcripts.

****Risks:****

- ****Format churn:**** Claude Code's format "changes between versions" by Anthropic's own

admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).

- ****Vendor lock-out / ToS:**** Storing session data is user-owned data (fine), but **auth tokens** are not transferable (codexSync explicitly refuses; Codex ``encrypted_content`` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.
 - ****Vendors closing the gap:**** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This **validates** the demand while **confirming** that only the cross-vendor + cross-OS layer is defensible.
- ****Security/privacy expectations:**** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).
- ****Honest limits:**** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

****Stage 0 – Fold in DevSync (immediate).**** Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

****Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge).**** Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/\.claude/projects/`` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- ****First-class OS-aware path remapping:**** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
- ****Benchmark to advance:**** ``claude --resume`` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

****Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode.**** Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid ``encrypted_content`/token` pitfalls. For Gemini, remap ``project_hash``. For opencode, use safe SQLite snapshotting (the ``sqlite3 .backup`` pattern the community scripts use).

- ****Benchmark:**** each new adapter must pass the same cross-OS resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One "profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not replay.

****Monetization & thresholds.**** Free/open-source core (build trust, match claude-sync) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSynq/OpenSync validate willingness to pay). ****Kill/pivot signal:**** if Anthropic ships account-level **local** session sync with automatic path remapping across OSe (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-

vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

Great reference point — Chaayos works because the pun is *audible*, the host word is familiar, and the new meaning actually fits. Here's that pattern applied to yours.

The strongest one first:

Reinstate — look at what's hiding in it: **re-in-STATE**. It's literally the word "state" (what you're syncing) inside a real word that means *restore something to its previous condition*. The pun and the product definition are the same thing. Nine characters, everyone knows how to say it, and it reads great as a command: **reinstate** on the laptop and you're back.

More "state" plays:

- **Interstate** — between states. You're syncing state *between* machines. The highway imagery even fits the desktop→laptop journey.
- **Instate** — the shorter cut. "Instate your MacBook." Sounds like a real verb because it is one.
- **Estate** — your dev estate: everything you own, all in one place. Quieter pun, still there.

"Session" plays:

- **Cession** — sounds *identical* to "session" spoken aloud, but means the formal transfer of something to another party. Best sound-alike on this list. Downside: people will assume you misspelled session.
- **Obsesh** — from obsession, for people obsessive about their sessions. Fun, probably too meme-y to age well.

"Sync" plays:

- **Syncopate** — sync sitting inside a real musical word about a beat that shifts but never stops. Nice meaning, slightly long.
- **Synergy** — synergy with the sync swapped in. Purest Chaayos-style construction here, but "synergy" carries corporate-cringe baggage.
- **Syncretic** — syncretism means fusing separate traditions into one.

Smartest fit conceptually; probably too obscure for a landing page.

Thread/weaving plays:

- **Threadmill** — treadmill + thread, and a *thread mill* is a genuine textile thing, so the pun lands twice. Very fun to say. Slight risk: treadmill connotes going nowhere.
- **Heirloom** — heir + **loom**. Something passed from one to the next, plus the weaving root. Prettiest name in this whole conversation, honestly.
- **Handloom** — same family, and it'd have some nice resonance in India specifically.

Others:

- **Devise** — **dev** inside a word meaning to plan or invent.
- **Rapport** — **port** inside a word meaning the connection between two parties. Your two machines have rapport.
- **Opensesame** — for your **open-** instinct: it's the magic phrase that *opens* things, and "ses" winks at session. Genuinely the wittiest option.
Flag: OpenSesame is an established corporate e-learning marketplace.

Known collisions to skip: Succession (thematically perfect — passing to the next — but the HBO show owns it), Loom on its own (taken), Recession (for obvious reasons).

My picks: **Reinstate** if you want the pun to do real work, **Threadmill** if you want it to be fun, **Heirloom** if you want it to be beautiful. Reinstate is the one where nobody has to have the joke explained — they just quietly notice "state" sitting in the middle and think it's clever.

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

```
# Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape
```

```
## TL;DR
```

```
- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows↔macOS paths.
```

```
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex)
```

is effectively impossible and should be dropped from scope;** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows ↔ macOS sync.

- **Recommendation: build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode.** Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...\` and `/Users/...\` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.
3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.
4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

Claude Code (Anthropic). Ships the richest consumer-facing multi-device story:

- **`/teleport`** (``claude --teleport``): pulls a *cloud* session (started at `claude.ai/code` or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport``, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires **clean git state** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same `claude.ai` account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."
- **Remote Control** (``claude remote-control`` / ``/remote-control``, `--rc``): pushes a *live local* session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an *undocumented* ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when ``ANTHROPIC_BASE_URL`` is redirected; unavailable under Zero Data Retention.
- **Claude Code on the web** (`claude.ai/code`): runs on Anthropic cloud infra, no local filesystem.
- **Residual gap:** No true device-to-device *local* session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an *unexpected*, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

****Codex CLI (OpenAI).**** The most architecturally mature cross-surface model:

- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared ****App Server**** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under `~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl``, with a ``session_meta`` first line.
- Rich lifecycle: ``codex resume`` (picker), ``--last``, ``--resume-session-id``, ``/fork``, archive/unarchive (v0.136), delete (v0.140). Imports histories from other agents via an ``external-agent-sessions`` crate.
- ****Known gaps/bugs:**** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing ``encrypted_content`` often can't be continued after crossing provider/account (``invalid_encrypted_content``); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.
- ****Residual gap:**** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

****Gemini CLI (Google).**** Auto-saves sessions to `~/gemini/tmp/<project_hash>/chats/`` (project-scoped, hash of project root). ``/resume`` session browser, ``--resume`/`-r`` flag, ``/chat save`` checkpoints, and a shadow-git checkpointing system under `~/gemini/history/<project_hash>``. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. ****Residual gap:**** everything local + path-hash-keyed; no sync.

****opencode (sst).**** SQLite + file storage under `~/local/share/opencode/`` (``opencode.db``, ``storage/session|message|part``). Native ****session sharing**** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). ****Residual gap:**** sharing ≠ syncing; DB concurrency and cross-OS path issues remain.

Cursor CLI (cursor-agent). Chats live in local SQLite; CLI chats under `~/cursor/chats` and resume via `cursor-agent --resume`. **IDE chats and CLI chats do NOT share a session store** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend `&` to push to a Cloud Agent, pick up at cursor.com/agents), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). **Residual gap:** no local cross-device sync, IDE ↔ CLI split, path issues.

Grok / Grok Build (xAI). Sessions auto-saved to `~/grok/sessions/`, keyed by working directory. `/resume`, `--resume`, `-c`, `/fork`; subscription-gated (SuperGrok/X Premium+). Config in `~/grok/config.toml`. No native cross-device sync. Still beta (v0.2.x as of July 2026). **Residual gap:** no sync at all.

GitHub Copilot CLI. Sessions in `~/copilot/` (`session-state/<uuid>/events.jsonl`). Notably, Copilot CLI **syncs sessions to your GitHub account by default** and offers **remote control** from [GitHub.com/GitHub Mobile](https://github.com/GitHubMobile) (origin machine must stay online, `/keep-alive`). But **cross-device resume** is still not supported – manually copying a session dir and running `--resume` overwrites `events.jsonl` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session_id`. **Residual gap:** the sync is telemetry/history, not resumable working state across machines.

Amp (Sourcegraph). Threads sync to ampcode.com by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). **Residual gap:** vendor-locked, server-stored, no cross-vendor.

Cross-vendor summary: Every vendor is building *within* its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

Tool	Location	Format	Path keying	Cross-device resume
------	----------	--------	-------------	---------------------

```

native? |
|---|---|---|---|---|
| Claude Code | `~/.claude/projects/<encoded-
path>/<session>.jsonl` | JSONL, append-only event stream |
**Absolute path, non-alphanumerics→dashes** | Via cloud only
(teleport/RC) |
| Codex | `~/.codex/sessions/YYYY/MM/DD/rollout-*.jsonl` | JSONL
+ `session_meta` header | date-based; cwd in meta | Yes, via App
Server |
| Gemini CLI | `~/.gemini/tmp/<project_hash>/chats/` | JSON |
**Hash of project root** | No |
| opencode | `~/.local/share/opencode/` (SQLite + files) |
SQLite/JSON | project-keyed | Sharing only |
| Cursor | local SQLite | DB rows | workspace/path metadata | No
(cloud handoff only) |
| Grok | `~/.grok/sessions/` | JSON | working directory | No |
| Copilot CLI | `~/.copilot/session-state/<uuid>/events.jsonl` |
JSONL events | uuid | Account history sync; no resume |

```

****The path-mapping problem (the crux).**** Claude Code derives the project folder name by escaping the absolute path: ``/Users/me/app`` → ``~/.claude/projects/-Users-me-app/``, while the same repo at ``C:\Users\me\app`` (or a different macOS username/location) produces a *different* folder. Consequently, after a naive file sync, ``claude --resume`` on the second machine cannot find the first machine's sessions. This is confirmed by the `claude-sync` author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker." Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via ``project_hash``; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. ****This is the single highest-value, currently-unmet technical feature.****

****Format churn risk.**** Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-

parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

****Mid-session cross-device resume feasibility (same vendor):****
Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present – *if* path keys are remapped. Codex's App Server makes it cloud-native already.

****Cross-vendor session replay (e.g., Claude Code session → Codex):**** ****Fundamentally impractical.**** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) `encrypted\_content` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. ****Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.****

3. Competitor teardown

****Session-continuity tools:****

- ****tawanorg/claude-sync**** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/.claude` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + ****age E2E encryption**** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.
- ****npow/session-sync**** (~1 star, very new): the only cross-vendor *session* translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- ****dantelex/aisync ("ss")****: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- ****codsync/codsync**** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- ****kroxiksut/codexSync****: local-first "cold-sync" Codex state

handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).

- **waynesutton/opencode-sync-plugin & codex-sync-plugin** (OpenSync) (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- **iHildy/opencode-synced** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- **Dinesh3184/claude-session-sync**: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- **renefichtmueller/claude-sync**: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

Config/MCP/skills sync (DevSync's neighborhood):

- **Leoyangl83/sync-agents-settings** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- **nicepkg/vsync** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- **baranovxyz/agentsync** and **spxrogers/agentsync** (beta): canonical `.agents/` config synced to 9–31 agents.
- **neon-solutions/add-mcp**: one-command MCP install across ~16 agents (Neon).
- **ztripez/mcp-sync**: MCP config sync across tools.
- **AgentSync (Rust)**: symlink-based config sync.
- **skills-sync**: skills + MCP workspace.
- **DevSynq** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

URL-addressable / mobile-control / viewer approaches:

- **TokenRip** (tokenrip.com): makes session state a URL-addressable artifact (`/a/<slug>`) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- **Omnara** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was **archived in February 2026** because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)
- **Syncode – AI Programming Tool**: iOS + macOS companion, LAN

pairing, cross-device (Mac execute → iPhone follow-up).

- ****jazzyalex/agent-sessions**** & ****kenn-io/agentsview****: local-first session browsers/search across 20+ agents (read/resume-command, not sync).

- ****LAN-workspace approaches**** and

Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

****Takeaway:**** The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- ****Claude Code:**** #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.

- ****Codex:**** #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI ↔ app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).

- ****Copilot CLI:**** #1947 (cloud-synced sessions), #1635 (cross-environment resume).

- ****Cursor forum:**** multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

****Where the durable moat is.**** Vendors will keep building

***intra*-ecosystem sync** (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. ****Cross-vendor unified profile**** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.

2. ****OS-aware path remapping as a first-class feature**** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows↔macOS. This is the wedge no competitor has nailed.

3. ****E2E encryption by default**** – table stakes given session files contain code, secrets, and full transcripts.

****Risks:****

- ****Format churn:**** Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).

- ****Vendor lock-out / ToS:**** Storing session data is user-owned data (fine), but **auth tokens** are not transferable (codexSync explicitly refuses; Codex ``encrypted_content`` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.

- ****Vendors closing the gap:**** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This **validates** the demand while **confirming** that only the cross-vendor + cross-OS layer is defensible.

- ****Security/privacy expectations:**** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).

- ****Honest limits:**** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

****Stage 0 – Fold in DevSync (immediate).**** Keep and harden the

existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

****Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge).**** Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/.claude/projects/` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- ****First-class OS-aware path remapping:**** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
- ****Benchmark to advance:**** `claude --resume` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

****Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode.**** Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid `encrypted\_content`/token pitfalls. For Gemini, remap `project\_hash`. For opencode, use safe SQLite snapshotting (the `sqlite3 .backup` pattern the community scripts use).

- ****Benchmark:**** each new adapter must pass the same cross-OS resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One "profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not replay.

****Monetization & thresholds.**** Free/open-source core (build

trust, match claude-sync) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSynq/OpenSync validate willingness to pay). ****Kill/pivot signal:**** if Anthropic ships account-level **local** session sync with automatic path remapping across OSes (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

Okay, I really like reinstate but and like there is no competition for reinstate like I just search Google for reinstate and the only results were you know dictionary results like what is the meaning of the word reinstate and so on. There is no company named reinstate as far as I googled. So I really liked it but it's a bit like too complex like. It's not something that would click off or you know stick with users that hear it for the first time you know like they want something that is clicky and something that sticks. And something that is preferably shorter and like you know just simpler but reinstate is a really nice name so that is like a fallback if we don't find a good name let's use that.

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape

TL;DR

- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows↔macOS paths.
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows↔macOS sync.
- **Recommendation:** build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode. Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...\` and `/Users/...\` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.
3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.
4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

- Claude Code (Anthropic).** Ships the richest consumer-facing multi-device story:
- **"/teleport"** (`claude --teleport`): pulls a *cloud* session (started at claude.ai/code or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport`, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires **clean git state** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same claude.ai account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."
 - **Remote Control** (`claude remote-control` / `/remote-control`, `--rc`): pushes a *live local* session to phone/web. The session runs locally the whole time; web/mobile are "a window

into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an **undocumented** ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when ``ANTHROPIC_BASE_URL`` is redirected; unavailable under Zero Data Retention.

- **Claude Code on the web** (claude.ai/code): runs on Anthropic cloud infra, no local filesystem.
- **Residual gap:** No true device-to-device **local** session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an **unexpected**, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

Codex CLI (OpenAI). The most architecturally mature cross-surface model:

- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared **App Server** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under `~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl``, with a ``session_meta`` first line.
- Rich lifecycle: ``codex resume`` (picker), ``--last``, ``--resume-session-id``, ``/fork``, `archive/unarchive` (v0.136), `delete` (v0.140). Imports histories from other agents via an ``external-agent-sessions`` crate.
- **Known gaps/bugs:** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing ``encrypted_content`` often can't be continued after crossing provider/account (``invalid_encrypted_content``); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.
- **Residual gap:** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

Gemini CLI (Google). Auto-saves sessions to `~/gemini/tmp/<project_hash>/chats/`` (project-scoped, hash of project root). ``/resume`` session browser, ``--resume`/`-r`` flag, ``/chat save`` checkpoints, and a shadow-git checkpointing system under `~/gemini/history/<project_hash>``. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. **Residual gap:** everything local + path-hash-keyed; no sync.

opencode (sst). SQLite + file storage under `~/local/share/opencode/`` (``opencode.db``, ``storage/session|message|part``). Native **session sharing** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). **Residual gap:** sharing ≠ syncing; DB concurrency and cross-OS path issues remain.

Cursor CLI (cursor-agent). Chats live in local SQLite; CLI chats under `~/cursor/chats`` and resume via ``cursor-agent --resume``. **IDE chats and CLI chats do NOT share a session store** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend ``&`` to push to a Cloud Agent, pick up at cursor.com/agents), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). **Residual gap:** no local cross-device sync, IDE ↔ CLI split, path issues.

Grok / Grok Build (xAI). Sessions auto-saved to `~/grok/sessions/``, keyed by working directory. ``/resume``, ``--resume``, ``-c``, ``/fork``; subscription-gated (SuperGrok/X Premium+). Config in `~/grok/config.toml``. No native cross-device sync. Still beta (v0.2.x as of July 2026). **Residual gap:** no sync at all.

GitHub Copilot CLI. Sessions in `~/copilot/`` (``session-state/<uuid>/events.jsonl``). Notably, Copilot CLI **syncs sessions to your GitHub account by default** and offers **remote**

control** from GitHub.com/GitHub Mobile (origin machine must stay online, `/keep-alive`). But **cross-device *resume* is still not supported** – manually copying a session dir and running `--resume` overwrites `events.jsonl` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session\_id`. **Residual gap:** the sync is telemetry/history, not resumable working state across machines.

Amp (Sourcegraph). Threads sync to ampcode.com by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). **Residual gap:** vendor-locked, server-stored, no cross-vendor.

Cross-vendor summary: Every vendor is building *within* its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

	Tool	Location	Format	Path keying	Cross-device resume native?
					--- --- --- --- ---
	Claude Code	`\${HOME}/.claude/projects/<encoded-path>/<session>.jsonl`	JSONL, append-only event stream	**Absolute path, non-alphanumerics→dashes**	Via cloud only (teleport/RC)
	Codex	`\${HOME}/.codex/sessions/YYYY/MM/DD/rollout-*.jsonl`	JSONL + `session_meta` header	date-based; cwd in meta	Yes, via App Server
	Gemini CLI	`\${HOME}/.gemini/tmp/<project_hash>/chats/`	JSON	**Hash of project root**	No
	opencode	`\${HOME}/.local/share/opencode/`	(SQLite + files)	SQLite/JSON	project-keyed
					Sharing only
	Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)
				Grok	`\${HOME}/.grok/sessions/`
					JSON
					working directory
					No
	Copilot CLI	`\${HOME}/.copilot/session-state/<uuid>/events.jsonl`	JSONL events	uuid	Account history sync; no resume

The path-mapping problem (the crux). Claude Code derives the project folder name by escaping the absolute path: `/Users/me/app` → `\${HOME}/.claude/projects/-Users-me-app/`, while the same repo at `C:\Users\me\app` (or a different macOS username/location) produces a *different* folder. Consequently, after a naive file sync, `claude --resume` on the second machine cannot find the first machine's sessions. This is confirmed by the claude-sync author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker." Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via `project\_hash`; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. **This is the single highest-value, currently-unmet technical feature.**

Format churn risk. Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

Mid-session cross-device resume feasibility (same vendor): Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present – *if* path keys are remapped. Codex's App Server makes it cloud-native already.

Cross-vendor session replay (e.g., Claude Code session → Codex): **Fundamentally impractical.** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) `encrypted\_content` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state

replay. ****Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.****

3. Competitor teardown

****Session-continuity tools:****

- ****tawanorg/claude-sync**** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/.claude` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + ****age E2E encryption**** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.
- ****npow/session-sync**** (~1 star, very new): the only cross-vendor ***session*** translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- ****dantelex/aisync ("ss")****: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- ****codsync/codsync**** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- ****kroxiksut/codexSync****: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- ****waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)**** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- ****iHildy/opencode-synced**** (~40 stars, active): opencodeconfig + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- ****Dinesh3184/claude-session-sync****: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- ****renefichtmueller/claude-sync****: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

****Config/MCP/skills sync (DevSync's neighborhood):****

- ****Leoyang183/sync-agents-settings**** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- ****nicepkg/vsync**** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- ****baranovxyz/agentsync**** and ****spxrogers/agentsync**** (beta): canonical `~/.agents/` config synced to 9–31 agents.
- ****neon-solutions/add-mcp****: one-command MCP install across ~16 agents (Neon).
- ****ztripez/mcp-sync****: MCP config sync across tools.
- ****AgentSync (Rust)****: symlink-based config sync.
- ****skills-sync****: skills + MCP workspace.
- ****DevSynq**** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

****URL-addressable / mobile-control / viewer approaches:****

- ****TokenRip**** (tokenrip.com): makes session state a URL-addressable artifact (`/a/<slug>`) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- ****Omnara**** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was ****archived in February 2026**** because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)
- ****Syncode – AI Programming Tool****: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- ****jazzyalex/agent-sessions**** & ****kenn-io/agentsview****: local-first session browsers/search across 20+ agents (read/resume-command, not sync).
- ****LAN-workspace approaches**** and Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

****Takeaway:**** The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- ****Claude Code:**** #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- ****Codex:**** #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI ↔ app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
 - ****Copilot CLI:**** #1947 (cloud-synced sessions), #1635 (cross-environment resume).
- ****Cursor forum:**** multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

****Where the durable moat is.**** Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. ****Cross-vendor unified profile**** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.
2. ****OS-aware path remapping as a first-class feature**** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows ↔ macOS. This is the wedge no competitor has nailed.
3. ****E2E encryption by default**** – table stakes given session files contain code, secrets, and full transcripts.

****Risks:****

- ****Format churn:**** Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).
- ****Vendor lock-out / ToS:**** Storing session data is user-owned data (fine), but *auth tokens* are not transferable (codexSync explicitly refuses; Codex `encrypted\_content` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.
 - ****Vendors closing the gap:**** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This *validates* the demand while *confirming* that only the cross-vendor + cross-OS layer is defensible.
- ****Security/privacy expectations:**** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).
- ****Honest limits:**** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

****Stage 0 – Fold in DevSync (immediate).**** Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

****Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge).****
Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/ .claude/projects/` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- ****First-class OS-aware path remapping:**** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
- ****Benchmark to advance:**** `claude --resume` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

****Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode.**** Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid `encrypted\_content`/token pitfalls. For Gemini, remap `project\_hash`. For opencode, use safe SQLite snapshotting (the `sqlite3 .backup` pattern the community scripts use).

- ****Benchmark:**** each new adapter must pass the same cross-OS resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One "profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not replay.

****Monetization & thresholds.**** Free/open-source core (build trust, match claude-sync) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSync/OpenSync validate willingness to pay). ****Kill/pivot signal:**** if Anthropic ships account-level *local* session sync with automatic path remapping across OSe (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

Good news: your favorite has shorter forms that keep the exact same pun.

Keep the pun, cut the length:

- **Instate** — same word, minus the "re." Seven characters, one fewer syllable, and "state" is still sitting right there in plain sight. `instate` on the laptop and you're running. The only cost is that "instate" is a less familiar word than "reinstate," so the pun is slightly quieter.
- **Restate** — the shortest version where "state" is still visible. Very familiar word, easy to say, clicks fast. Minor wrinkle: "restate" means *say it again*, which brushes against the same problem recontext had — it names redoing rather than restoring.
- **Estate** — six characters. Your dev estate: everything you own, held in one place. Softer pun, but it sounds like a real, confident brand.

The one I'd actually put above Reinstate:

Carryon. Both meanings are your product, exactly:

- A carry-on is the bag you take with you between places — everything you need, nothing left behind.
- "Carry on" means *continue what you were doing*.

That's not a pun that needs explaining; it's two obvious meanings that both happen to be your pitch. Seven characters, universally familiar, warm, and it says itself out loud the first time someone sees it. `carryon pull` reads like a real command. Worth checking against luggage brands before committing, but in dev tooling I'd expect it to be open.

Other short ones:

- **Sesh** — four characters, and it's already how developers casually say "session." Maximum stickiness, zero explanation needed. Risk: reads young and slangy, which may or may not be your brand.
- **Upstate** — upload state. Real word, fun to notice, though the New York association is strong.
- **Cession** — sounds identical to "session" but means the formal transfer of something to another party. Perfect meaning; people will think you typo'd.
- **Rehome** — six characters, and you're literally syncing home directories (`~/ .claude`, `~/ .codex`). Slight pet-adoption connotation.
- **Totem** — "tote" hiding inside a short, memorable word. Thin pun, strong brand feel.

My call: Carryon beats Reinstate on every axis you named — shorter, simpler, clickier, sticks on first hearing — and it loses nothing in meaning. If you want to stay in the state family, **Instate** is the direct fix to your complaint.

Keep Reinstate as the fallback. It's a genuinely good name and, as you found,

wide open.

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape

TL;DR

- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows ↔ macOS paths.
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows ↔ macOS sync.
- **Recommendation:** build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode. Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.

2. **The path-mapping problem is the crux and is largely**

unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...\` and `/Users/...\` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.

3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.

4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

Claude Code (Anthropic). Ships the richest consumer-facing multi-device story:

- **`/teleport`** (``claude --teleport``): pulls a *cloud* session (started at `claude.ai/code` or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport``, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires **clean git state** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same `claude.ai` account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."
- **Remote Control** (``claude remote-control` / `/remote-control`, --rc`): pushes a live local session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the`

session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an **undocumented** ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `ANTHROPIC\_BASE\_URL` is redirected; unavailable under Zero Data Retention.

- **Claude Code on the web** (claude.ai/code): runs on Anthropic cloud infra, no local filesystem.

- **Residual gap:** No true device-to-device **local** session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an **unexpected**, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

Codex CLI (OpenAI). The most architecturally mature cross-surface model:

- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared **App Server** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under

`~/.codex/sessions/YYYY/MM/DD/rollout-\*.jsonl`, with a `session\_meta` first line.

- Rich lifecycle: `codex resume` (picker), `--last`, `--resume-session-id`, `/fork`, archive/unarchive (v0.136), delete (v0.140). Imports histories from other agents via an `external-agent-sessions` crate.

- **Known gaps/bugs:** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing `encrypted\_content` often can't be continued after crossing provider/account (`invalid\_encrypted\_content`); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.

- **Residual gap:** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

****Gemini CLI (Google).**** Auto-saves sessions to `~/ .gemini/tmp/<project_hash>/chats/`` (project-scoped, hash of project root). `/resume`` session browser, `--resume`/`-r`` flag, `/chat save`` checkpoints, and a shadow-git checkpointing system under `~/ .gemini/history/<project_hash>``. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. ****Residual gap:**** everything local + path-hash-keyed; no sync.

****opencode (sst).**** SQLite + file storage under `~/ .local/share/opencode/`` (`opencode.db``, `storage/session|message|part``). Native ****session sharing**** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). ****Residual gap:**** sharing $\neq$ syncing; DB concurrency and cross-OS path issues remain.

****Cursor CLI (cursor-agent).**** Chats live in local SQLite; CLI chats under `~/ .cursor/chats`` and resume via `cursor-agent --resume``. ****IDE chats and CLI chats do NOT share a session store**** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend `&`` to push to a Cloud Agent, pick up at `cursor.com/agents`), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). ****Residual gap:**** no local cross-device sync, IDE ↔ CLI split, path issues.

****Grok / Grok Build (xAI).**** Sessions auto-saved to `~/ .grok/sessions/``, keyed by working directory. `/resume``, `--resume``, `-c``, `/fork``; subscription-gated (SuperGrok/X Premium+). Config in `~/ .grok/config.toml``. No native cross-device sync. Still beta (v0.2.x as of July 2026). ****Residual gap:**** no sync at all.

****GitHub Copilot CLI.**** Sessions in `~/ .copilot/`` (`session-state/<uuid>/events.jsonl``). Notably, Copilot CLI ****syncs sessions to your GitHub account by default**** and offers ****remote control**** from `GitHub.com/GitHub Mobile` (origin machine must stay online, `/keep-alive``). But **\*\*cross-device \*resume\* is still not supported\*\*** – manually copying a session dir and running `--resume`` overwrites `events.jsonl`` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request

(#1947). The Copilot SDK supports resumable sessions via a user-supplied `session\_id`. **Residual gap:** the sync is telemetry/history, not resumable working state across machines.

Amp (Sourcegraph). Threads sync to ampcode.com by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). **Residual gap:** vendor-locked, server-stored, no cross-vendor.

Cross-vendor summary: Every vendor is building *within* its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

Tool	Location	Format	Path keying	Cross-device resume native?	
Claude Code	~/.claude/projects/<encoded-path>/<session>.jsonl	JSONL, append-only event stream	Absolute path, non-alphanumerics→dashes	Via cloud only (teleport/RC)	
Codex	~/.codex/sessions/YYYY/MM/DD/rollout-*.jsonl	JSONL + `session_meta` header	date-based; cwd in meta	Yes, via App Server	
Gemini CLI	~/.gemini/tmp/<project_hash>/chats/	JSON	Hash of project root	No	
opencode	~/.local/share/opencode/	(SQLite + files)	SQLite/JSON	project-keyed	Sharing only
Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)	
Grok	~/.grok/sessions/	JSON	working directory	No	
Copilot CLI	~/.copilot/session-state/<uuid>/events.jsonl	JSONL events	uuid	Account history sync; no resume	

The path-mapping problem (the crux). Claude Code derives the project folder name by escaping the absolute path:

`/Users/me/app` → `~/.claude/projects/-Users-me-app/`, while the same repo at `C:\Users\me\app` (or a different macOS username/location) produces a *different* folder. Consequently, after a naive file sync, `claude --resume` on the second machine

cannot find the first machine's sessions. This is confirmed by the claude-sync author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker." Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via `project\_hash`; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. ****This is the single highest-value, currently-unmet technical feature.****

****Format churn risk.**** Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

****Mid-session cross-device resume feasibility (same vendor):**** Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present — *if* path keys are remapped. Codex's App Server makes it cloud-native already.

****Cross-vendor session replay (e.g., Claude Code session → Codex):**** ****Fundamentally impractical.**** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) `encrypted\_content` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. ****Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.****

3. Competitor teardown

****Session-continuity tools:****

- ****tawanorg/claude-sync**** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/.claude`

(sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + **age** E2E encryption

("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. **Punts on path remapping** (its explicit, acknowledged weakness). Open-source, no business model.

- **npow/session-sync** (~1 star, very new): the only cross-vendor **session** translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- **dantelex/aisync ("ss")**: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- **codsync/codsync** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- **kroxiksut/codexSync**: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- **waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- **iHildy/opencode-synced** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- **Dinesh3184/claude-session-sync**: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- **renefichtmueller/claude-sync**: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

Config/MCP/skills sync (DevSync's neighborhood):

- **Leoyang183/sync-agents-settings** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- **nicepkg/vsync** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- **baranovxyz/agentsync** and **spxrogers/agentsync** (beta): canonical ``.agents/`` config synced to 9–31 agents.
- **neon-solutions/add-mcp**: one-command MCP install across ~16 agents (Neon).
- **ztripez/mcp-sync**: MCP config sync across tools.

- **AgentSync (Rust)**: symlink-based config sync.
- **skills-sync**: skills + MCP workspace.
- **DevSynq** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

URL-addressable / mobile-control / viewer approaches:

- **TokenRip** (tokenrip.com): makes session state a URL-addressable artifact (`/a/<slug>`) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- **Omnara** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was **archived in February 2026** because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)
- **Syncode – AI Programming Tool**: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- **jazzyalex/agent-sessions** & **kenn-io/agentsview**: local-first session browsers/search across 20+ agents (read/resume-command, not sync).

- **LAN-workspace approaches** and

Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

Takeaway: The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- **Claude Code**: #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.

- **Codex:** #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI ↔ app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
- **Copilot CLI:** #1947 (cloud-synced sessions), #1635 (cross-environment resume).
- **Cursor forum:** multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is. Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. **Cross-vendor unified profile** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.
2. **OS-aware path remapping as a first-class feature** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows ↔ macOS. This is the wedge no competitor has nailed.
3. **E2E encryption by default** – table stakes given session files contain code, secrets, and full transcripts.

Risks:

- **Format churn:** Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).
- **Vendor lock-out / ToS:** Storing session data is user-owned data (fine), but *auth tokens* are not transferable (codexSync explicitly refuses; Codex `encrypted\_content` breaks across accounts). Never sync credentials; never depend on undocumented

cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.

- ****Vendors closing the gap:**** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This **validates** the demand while **confirming** that only the cross-vendor + cross-OS layer is defensible.

- ****Security/privacy expectations:**** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).

- ****Honest limits:**** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

****Stage 0 – Fold in DevSync (immediate).**** Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

****Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge).**** Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/.claude/projects/` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- ****First-class OS-aware path remapping:**** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
- ****Benchmark to advance:**** `claude --resume` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption,

across 3 consecutive Claude Code releases.

****Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode.**** Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid `encrypted\_content`/token pitfalls. For Gemini, remap `project\_hash`. For opencode, use safe SQLite snapshotting (the `sqlite3 .backup` pattern the community scripts use).

- ****Benchmark:**** each new adapter must pass the same cross-OS resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One "profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not replay.

****Monetization & thresholds.**** Free/open-source core (build trust, match claude-sync) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSynq/OpenSync validate willingness to pay). ****Kill/pivot signal:**** if Anthropic ships account-level **local** session sync with automatic path remapping across OSes (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing

approach carries ongoing maintenance risk.

- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

I really like carry-on but the issue that I see with it is that people won't be able to like you know tell that it is carry-on when they read it they would you know maybe think it's like a singular word not two words like a single word like crayon something you know And I really don't want to do "CarryOn" Because it just looks like a generic you know a very generic it's like software as a service company name or like any other tech company you know like it's very generic and weird like this style of having two words or like without the space and making its initial capital i don't really like it

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape

TL;DR

- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows↔macOS paths.
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows↔macOS sync.
- **Recommendation:** build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode. Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...` and `/Users/...` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of

niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.

3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.

4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

- Claude Code (Anthropic).** Ships the richest consumer-facing multi-device story:
- **teleport** (`claude --teleport`): pulls a *cloud* session (started at `claude.ai/code` or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport`, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires **clean git state** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same `claude.ai` account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."
 - **Remote Control** (`claude remote-control` / `/remote-control`, `--rc`): pushes a *live local* session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an *undocumented* ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `ANTHROPIC_BASE_URL` is redirected; unavailable under Zero Data Retention.
 - **Claude Code on the web** (`claude.ai/code`): runs on Anthropic cloud infra, no local filesystem.
 - **Residual gap:** No true device-to-device *local* session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an *unexpected*, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.
- Codex CLI (OpenAI).** The most architecturally mature cross-surface model:
- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared **App Server** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under `~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl`, with a `session_meta` first line.
 - Rich lifecycle: `codex resume` (picker), `--last`, `--resume-session-id`, `/fork`, `archive/unarchive` (v0.136), `delete` (v0.140). Imports histories from other agents via an `external-agent-sessions` crate.
 - **Known gaps/bugs:** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing `encrypted_content` often can't be continued after crossing provider/account (`invalid_encrypted_content`); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.
 - **Residual gap:** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

Gemini CLI (Google). Auto-saves sessions to `~/gemini/tmp/<project_hash>/chats/` (project-scoped, hash of project root). `/resume` session browser, `--resume` `/-r` flag,

`~/chat save` checkpoints, and a shadow-git checkpointing system under
`~/gemini/history/<project\_hash>`. No native cross-device cloud sync. Note: Google announced
Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. ****Residual
gap:**** everything local + path-hash-keyed; no sync.

****opencode (sst).**** SQLite + file storage under `~/local/share/opencode/` (`opencode.db`,
`storage/session|message|part`). Native ****session sharing**** publishes to Cloudflare Durable
Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-
party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync,
opencode-session-backup). ****Residual gap:**** sharing ≠ syncing; DB concurrency and cross-OS
path issues remain.

****Cursor CLI (cursor-agent).**** Chats live in local SQLite; CLI chats under `~/cursor/chats`
and resume via `cursor-agent --resume`. ****IDE chats and CLI chats do NOT share a session
store**** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend `&` to push to
a Cloud Agent, pick up at cursor.com/agents), and "My Machines" lets cloud agents run on your
registered machine. Strong user demand for account-level chat sync (forum). ****Residual gap:****
no local cross-device sync, IDE↔CLI split, path issues.

****Grok / Grok Build (xAI).**** Sessions auto-saved to `~/grok/sessions/`, keyed by working
directory. `~/resume`, `--resume`, `-c`, `~/fork`; subscription-gated (SuperGrok/X Premium+).
Config in `~/grok/config.toml`. No native cross-device sync. Still beta (v0.2.x as of July
2026). ****Residual gap:**** no sync at all.

****GitHub Copilot CLI.**** Sessions in `~/copilot/` (`session-state/<uuid>/events.jsonl`).
Notably, Copilot CLI ****syncs sessions to your GitHub account by default**** and offers ****remote
control**** from GitHub.com/GitHub Mobile (origin machine must stay online, `~/keep-alive`). But
****cross-device *resume* is still not supported**** – manually copying a session dir and running
`--resume` overwrites `events.jsonl` and destroys history (issue #1635); cloud-synced cross-
device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via
a user-supplied `session\_id`. ****Residual gap:**** the sync is telemetry/history, not resumable
working state across machines.

****Amp (Sourcegraph).**** Threads sync to ampcode.com by default and can be continued across
devices; "orbs" (remote machines) keep running after you close your laptop. This is the
closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph
servers (a privacy consideration; MCP config is GUI-global, not file-based). ****Residual gap:****
vendor-locked, server-stored, no cross-vendor.

****Cross-vendor summary:**** Every vendor is building **within** its ecosystem. The white space is
precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

	Tool	Location	Format	Path keying	Cross-device resume native?
					--- --- --- --- ---
	Claude Code	~/claude/projects/<encoded-path>/<session>.jsonl`	JSONL, append-only event stream	**Absolute path, non-alphanumerics→dashes**	Via cloud only (teleport/RC)
	Codex	~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl`	JSONL + `session_meta` header	date-based; cwd in meta	Yes, via App Server
	Gemini CLI	~/gemini/tmp/<project_hash>/chats/`	JSON	**Hash of project root**	No
	opencode	~/local/share/opencode/`	(SQLite + files)	SQLite/JSON	project-keyed
					Sharing only
	Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)
	Grok	~/grok/sessions/`	JSON	working directory	No
	Copilot CLI	~/copilot/session-state/<uuid>/events.jsonl`	JSONL events	uuid	Account history sync; no resume

****The path-mapping problem (the crux).**** Claude Code derives the project folder name by
escaping the absolute path: `~/Users/me/app` → `~/claude/projects/-Users-me-app/`, while the
same repo at `C:\Users\me\app` (or a different macOS username/location) produces a **different**

folder. Consequently, after a naive file sync, `claude --resume` on the second machine cannot find the first machine's sessions. This is confirmed by the claude-sync author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker."

Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via `project\_hash`; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. ****This is the single highest-value, currently-unmet technical feature.****

****Format churn risk.**** Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

****Mid-session cross-device resume feasibility (same vendor):**** Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present – ***if*** path keys are remapped. Codex's App Server makes it cloud-native already.

****Cross-vendor session replay (e.g., Claude Code session → Codex):**** ****Fundamentally impractical.**** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) `encrypted\_content` that can't even survive an account change. npow/session-sync attempts this by translating ClaudeCode tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. ****Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.****

3. Competitor teardown

****Session-continuity tools:****

- ****tawanorg/claude-sync**** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/.claude` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + ****age E2E encryption**** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.
- ****npow/session-sync**** (~1 star, very new): the only cross-vendor ***session*** translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- ****dantelex/aisync ("ss")****: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- ****codsync/codsync**** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- ****kroxiksut/codexSync****: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- ****waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)**** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- ****iHildy/opencode-synced**** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- ****Dinesh3184/claude-session-sync****: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- ****renefichtmueller/claude-sync****: multi-backend (Git/iCloud/Dropbox/SyncThing/rsync) with snapshots/rollback and encryption.

****Config/MCP/skills sync (DevSync's neighborhood):****

- ****Leoyang183/sync-agents-settings**** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- ****nicepkg/vsync**** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.

- **baranovxyz/agentsync** and **spxrogers/agentsync** (beta): canonical ``.agents/`` config synced to 9–31 agents.
 - **neon-solutions/add-mcp**: one-command MCP install across ~16 agents (Neon).
 - **ztripez/mcp-sync**: MCP config sync across tools.
 - **AgentSync (Rust)**: symlink-based config sync.
 - **skills-sync**: skills + MCP workspace.
 - **DevSync** (devsyncq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.
- URL-addressable / mobile-control / viewer approaches:**
- **TokenRip** (tokenrip.com): makes session state a URL-addressable artifact (``/a/<slug>``) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
 - **Omnara** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was **archived** in February 2026 because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)
 - **Syncode – AI Programming Tool**: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
 - **jazzyalex/agent-sessions** & **kenn-io/agentsview**: local-first session browsers/search across 20+ agents (read/resume-command, not sync).
 - **LAN-workspace approaches** and **Syncthing/NAS/dotfiles/chezmoi+age**: DIY approaches that work for config but hit the path problem for sessions.

Takeaway: The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- **Claude Code**: #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- **Codex**: #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI ↔ app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
 - **Copilot CLI**: #1947 (cloud-synced sessions), #1635 (cross-environment resume).
- **Cursor forum**: multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is. Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. **Cross-vendor unified profile** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.
2. **OS-aware path remapping as a first-class feature** – the concrete, unsolved pain that

makes same-vendor session sync actually work across Windows↔macOS. This is the wedge no competitor has nailed.

3. **E2E encryption by default** – table stakes given session files contain code, secrets, and full transcripts.

Risks:

- **Format churn:** Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).
- **Vendor lock-out / ToS:** Storing session data is user-owned data (fine), but *auth tokens* are not transferable (codexSync explicitly refuses; Codex `encrypted_content` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.
 - **Vendors closing the gap:** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This *validates* the demand while *confirming* that only the cross-vendor + cross-OS layer is defensible.
- **Security/privacy expectations:** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).
 - **Honest limits:** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

Stage 0 – Fold in DevSync (immediate). Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge). Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/ .claude/projects/` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- **First-class OS-aware path remapping:** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
 - Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
 - **Benchmark to advance:** `claude --resume` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode. Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid `encrypted_content`/token pitfalls. For Gemini, remap `project_hash`. For opencode, use safe SQLite snapshotting (the `sqlite3 .backup` pattern the community scripts use).

- **Benchmark:** each new adapter must pass the same cross-OS resume test as Claude Code before GA.

Stage 3 – Unified profile + cross-vendor context handoff. One "profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not replay.

Monetization & thresholds. Free/open-source core (build trust, match claude-sync) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles

(DevSynq/OpenSync validate willingness to pay). ****Kill/pivot signal:**** if Anthropic ships account-level **local** session sync with automatic path remapping across OSes (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.

- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.

- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini-Antigravity migration) are evolving fast; specifics may have changed since the cited docs.

- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

Two paths here, and I think the second one is where you land.

First, the hyphen is more legitimate than you're giving it credit for.

carry-on is the actual dictionary spelling of the noun, and hyphenated lowercase is completely native to dev tooling — *create-react-app*, *add-mcp*, *claude-sync*, *sync-agents-settings*. Nobody reads those as awkward. A hyphen isn't CamelCase; it's the opposite of CamelCase, and it kills the crayon problem outright. *npm i -g carry-on* reads perfectly.

But you clearly want a single word that needs no punctuation to parse correctly. So:

Layover. This is the one. Nobody has ever read "layover" as two words — it's fully fused in English, unlike *carryall* or *holdall* which visibly split. Seven characters, universally understood, zero explanation needed. And the meaning is *exactly* your product: a layover isn't the end of the trip, it's a pause in the middle of one journey before you continue on the next leg. Your desktop session isn't over — it's on a layover until you open the MacBook. It's warm, it's memorable, and it has none of the tech-generic feel you're reacting against.

If you want the bag idea in one clean word:

- **Duffel** — six characters, reads as one word instantly, distinctive, and it's the bag you throw everything into and go. Strong consonants, easy to say.
- **Cabin** — five characters. "Cabin bag" *is* carry-on, and Cabin reads as one simple word with no ambiguity at all. Softest and shortest option.
- **Satchel** — seven, unmistakably singular, a bit more literary.

Other true single words in the same spirit:

- **Transit** — you're in transit between machines. Clean, infrastructural, no split-reading.
- **Stopover** — same idea as Layover, slightly longer, fractionally more likely to split visually.

My ranking for you specifically: Layover, then Duffel, then Cabin.

Layover gives you everything carry-on gave you — the journey, the pause, the continuation — as one unsplittable word that a person hears once and keeps.

Worth a quick availability check before you commit, since Layover is a common enough word that a travel startup may have grabbed the `.com`. Want me to check Layover, Duffel, and Cabin?

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

```
# Cross-Device Sync for Terminal AI Coding Agents: Competitive &
Technical Landscape
```

```
## TL;DR
```

```
- **The durable opportunity is a cross-vendor abstraction layer
with OS-aware path remapping as its headline feature – NOT
another "sync my Claude sessions" tool.** Every major vendor is
now shipping single-ecosystem sync natively (Claude Code Remote
Control/teleport, Codex's unified App Server backend, Amp
threads, Copilot CLI cloud sync), so the defensible wedge is the
one thing no vendor will ever build: normalizing sessions + MCP +
skills + instruction files across *competing* tools and remapping
Windows↔macOS paths.
```

```
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex)
is effectively impossible and should be dropped from scope;**
config/skills/instruction portability and *same-vendor* cross-
device session continuity are the achievable, valuable core. The
single sharpest, currently-unsolved technical pain – which even
the leading competitor (claude-sync) explicitly punts on – is
Claude Code's absolute-path project indexing breaking on Windows
↔macOS sync.
```

```
- **Recommendation: build a "universal profile" that folds
DevSync's MCP-sync foundation in, adds E2E-encrypted session sync
with first-class path remapping for Claude Code first, then
Codex/Gemini/opencode.** Ship path remapping as the MVP
differentiator; treat E2E encryption as table stakes; do not try
to out-run vendors on same-vendor live handoff.
```

```
## Key Findings
```


1. ****Native vendor sync is real but ecosystem-locked and gap-riddled.**** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.

2. ****The path-mapping problem is the crux and is largely unsolved.**** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...\` and `/Users/...\` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.

3. ****Demand is loud and repeatedly expressed**** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.

4. ****The competitor field is fragmented, early, and mostly config-only or single-vendor.**** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

****Claude Code (Anthropic).**** Ships the richest consumer-facing multi-device story:

- ****`/teleport`**** (`claude --teleport`): pulls a **cloud** session (started at claude.ai/code or the mobile app) down into the local terminal. It is ****one-way**** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport`, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires ****clean git state**** ("Your working directory must have no uncommitted changes.

Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same claude.ai account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."

- **Remote Control** (``claude remote-control` / `/remote-control`, --rc`): pushes a *live local* session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an *undocumented* ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `ANTHROPIC_BASE_URL` is redirected; unavailable under Zero Data Retention.`
- **Claude Code on the web** (`claude.ai/code`): runs on Anthropic cloud infra, no local filesystem.
- **Residual gap:** No true device-to-device **local** session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an **unexpected**, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

Codex CLI (OpenAI). The most architecturally mature cross-surface model:

- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared **App Server** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under ``~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl``, with a ``session_meta`` first line.
- Rich lifecycle: ``codex resume`` (picker), ``--last``, ``--resume-session-id``, ``/fork``, archive/unarchive (v0.136), delete (v0.140). Imports histories from other agents via an ``external-agent-sessions`` crate.
- **Known gaps/bugs:** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't

persisted; sessions containing `encrypted\_content` often can't be continued after crossing provider/account (`invalid\_encrypted\_content`); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.

- **Residual gap:** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

Gemini CLI (Google). Auto-saves sessions to `~/.gemini/tmp/<project\_hash>/chats/` (project-scoped, hash of project root). `/resume` session browser, `--resume`/`-r` flag, `/chat save` checkpoints, and a shadow-git checkpointing system under `~/.gemini/history/<project\_hash>`. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. **Residual gap:** everything local + path-hash-keyed; no sync.

opencode (sst). SQLite + file storage under `~/.local/share/opencode/` (`opencode.db`, `storage/session|message|part`). Native **session sharing** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). **Residual gap:** sharing ≠ syncing; DB concurrency and cross-OS path issues remain.

Cursor CLI (cursor-agent). Chats live in local SQLite; CLI chats under `~/.cursor/chats` and resume via `cursor-agent --resume`. **IDE chats and CLI chats do NOT share a session store** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend `&` to push to a Cloud Agent, pick up at cursor.com/agents), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). **Residual gap:** no local cross-device sync, IDE ↔ CLI split, path issues.

Grok / Grok Build (xAI). Sessions auto-saved to `~/.grok/sessions/`, keyed by working directory. `/resume`, `--resume`, `-c`, `/fork`; subscription-gated (SuperGrok/X Premium+). Config in `~/.grok/config.toml`. No native cross-device sync. Still beta (v0.2.x as of July 2026). **Residual**

gap:** no sync at all.

GitHub Copilot CLI. Sessions in `~/.copilot/` (`session-state/<uuid>/events.jsonl`). Notably, Copilot CLI **syncs** sessions to your GitHub account by default and offers **remote control** from GitHub.com/GitHub Mobile (origin machine must stay online, `/keep-alive`). But **cross-device *resume*** is still not supported – manually copying a session dir and running `--resume` overwrites `events.jsonl` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session\_id`. **Residual gap:** the sync is telemetry/history, not resumable working state across machines.

Amp (Sourcegraph). Threads sync to ampcode.com by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). **Residual gap:** vendor-locked, server-stored, no cross-vendor.

Cross-vendor summary: Every vendor is building *within* its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

Tool	Location	Format	Path keying	Cross-device resume native?
Claude Code	~/.claude/projects/<encoded-path>/<session>.jsonl	JSONL, append-only event stream	Absolute path, non-alphanumerics→dashes	Via cloud only (teleport/RC)
Codex	~/.codex/sessions/YYYY/MM/DD/rollout-*.jsonl	JSONL + `session_meta` header	date-based; cwd in meta	Yes, via App Server
Gemini CLI	~/.gemini/tmp/<project_hash>/chats/	JSON	Hash of project root	No
opencode	~/.local/share/opencode/	(SQLite + files)	SQLite/JSON	project-keyed
Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)

```
| Grok | `~/.grok/sessions/` | JSON | working directory | No |  
| Copilot CLI | `~/.copilot/session-state/<uuid>/events.jsonl` |  
JSONL events | uuid | Account history sync; no resume |
```

****The path-mapping problem (the crux).**** Claude Code derives the project folder name by escaping the absolute path: ``/Users/me/app`` → ``~/.claude/projects/-Users-me-app/``, while the same repo at ``C:\Users\me\app`` (or a different macOS username/location) produces a *different* folder. Consequently, after a naive file sync, ``claude --resume`` on the second machine cannot find the first machine's sessions. This is confirmed by the `claude-sync` author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker." Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via ``project_hash``; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. ****This is the single highest-value, currently-unmet technical feature.****

****Format churn risk.**** Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

****Mid-session cross-device resume feasibility (same vendor):**** Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present — *if* path keys are remapped. Codex's App Server makes it cloud-native already.

****Cross-vendor session replay (e.g., Claude Code session → Codex):**** ****Fundamentally impractical.**** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) ``encrypted_content`` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via

hooks, but this is best-effort context transfer, not faithful state replay. **Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.**

3. Competitor teardown

Session-continuity tools:

- **tawanorg/claude-sync** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/.claude` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + **age E2E encryption** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. **Punts on path remapping** (its explicit, acknowledged weakness). Open-source, no business model.
- **npow/session-sync** (~1 star, very new): the only cross-vendor *session* translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- **dantelex/aisync ("ss")**: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- **codsync/codsync** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- **kroxiksut/codexSync**: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- **waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- **iHildy/opencode-synced** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- **Dinesh3184/claude-session-sync**: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- **renefichtmueller/claude-sync**: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

Config/MCP/skills sync (DevSync's neighborhood):

- ****Leoyang183/sync-agents-settings**** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- ****nicepkg/vsync**** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- ****baranovxyz/agentsync**** and ****spxrogers/agentsync**** (beta): canonical ``.agents/`` config synced to 9–31 agents.
- ****neon-solutions/add-mcp****: one-command MCP install across ~16 agents (Neon).
- ****ztripez/mcp-sync****: MCP config sync across tools.
- ****AgentSync (Rust)****: symlink-based config sync.
- ****skills-sync****: skills + MCP workspace.
- ****DevSynq**** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

****URL-addressable / mobile-control / viewer approaches:****

- ****TokenRip**** (tokenrip.com): makes session state a URL-addressable artifact (``/a/<slug>``) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- ****Omnara**** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was ****archived in February 2026**** because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)
- ****Syncode – AI Programming Tool****: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- ****jazzyalex/agent-sessions**** & ****kenn-io/agentsview****: local-first session browsers/search across 20+ agents (read/resume-command, not sync).
- ****LAN-workspace approaches**** and Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

****Takeaway:**** The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- **Claude Code:** #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- **Codex:** #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI ↔ app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
- **Copilot CLI:** #1947 (cloud-synced sessions), #1635 (cross-environment resume).
- **Cursor forum:** multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is. Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. **Cross-vendor unified profile** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.
2. **OS-aware path remapping as a first-class feature** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows ↔ macOS. This is the wedge no competitor has nailed.
3. **E2E encryption by default** – table stakes given session files contain code, secrets, and full transcripts.

Risks:

- **Format churn:** Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).
- **Vendor lock-out / ToS:** Storing session data is user-owned data (fine), but *auth tokens* are not transferable (codexSync explicitly refuses; Codex `encrypted_content` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.
- **Vendors closing the gap:** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This *validates* the demand while *confirming* that only the cross-vendor + cross-OS layer is defensible.
- **Security/privacy expectations:** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).
- **Honest limits:** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

Stage 0 – Fold in DevSync (immediate). Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge). Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/ .claude/projects/`` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- **First-class OS-aware path remapping:** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git

snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.

- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.

- **Benchmark to advance:** ``claude --resume`` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode. Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid ``encrypted_content`/token` pitfalls. For Gemini, remap ``project_hash``. For opencode, use safe SQLite snapshotting (the ``sqlite3 .backup`` pattern the community scripts use).

- **Benchmark:** each new adapter must pass the same cross-OS resume test as Claude Code before GA.

Stage 3 – Unified profile + cross-vendor context handoff. One "profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not replay.

Monetization & thresholds. Free/open-source core (build trust, match claude-sync) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSynq/OpenSync validate willingness to pay). **Kill/pivot signal:** if Anthropic ships account-level **local** session sync with automatic path remapping across OSes (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

What NOT to do: Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opcode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

How about Open Duffel or Open Cabin?

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape

TL;DR

- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows↔macOS paths.
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows↔macOS sync.
- **Recommendation:** build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opcode. Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files

between `C:\Users\...\` and `/Users/...\` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.

3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.

4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

- Claude Code (Anthropic).** Ships the richest consumer-facing multi-device story:
 - teleport** (`claude --teleport`): pulls a *cloud* session (started at `claude.ai/code` or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport`, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires **clean git state** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same `claude.ai` account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."
 - Remote Control** (`claude remote-control` / `/remote-control`, `--rc`): pushes a *live local* session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an *undocumented* ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `ANTHROPIC_BASE_URL` is redirected; unavailable under Zero Data Retention.
 - Claude Code on the web** (`claude.ai/code`): runs on Anthropic cloud infra, no local filesystem.
 - Residual gap:** No true device-to-device *local* session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an *unexpected*, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.
- Codex CLI (OpenAI).** The most architecturally mature cross-surface model:
 - CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared **App Server** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under `~/.codex/sessions/YYYY/MM/DD/rollout-*.jsonl`, with a `session_meta` first line.
 - Rich lifecycle: `codex resume` (picker), `--last`, `--resume-session-id`, `/fork`, `archive/unarchive` (v0.136), `delete` (v0.140). Imports histories from other agents via an `external-agent-sessions` crate.
 - Known gaps/bugs:** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing `encrypted_content` often can't be continued after crossing provider/account (`invalid_encrypted_content`); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.
 - Residual gap:** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

****Gemini CLI (Google).**** Auto-saves sessions to `~/gemini/tmp/<project\_hash>/chats/` (project-scoped, hash of project root). `~/resume` session browser, `--resume`/`-r` flag, `~/chat save` checkpoints, and a shadow-git checkpointing system under `~/gemini/history/<project\_hash>`. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. ****Residual gap:**** everything local + path-hash-keyed; no sync.

****opencode (sst).**** SQLite + file storage under `~/local/share/opencode/` (`opencode.db`, `storage/session|message|part`). Native ****session sharing**** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). ****Residual gap:**** sharing ≠ syncing; DB concurrency and cross-OS path issues remain.

****Cursor CLI (cursor-agent).**** Chats live in local SQLite; CLI chats under `~/cursor/chats` and resume via `cursor-agent --resume`. ****IDE chats and CLI chats do NOT share a session store**** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend `&` to push to a Cloud Agent, pick up at cursor.com/agents), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). ****Residual gap:**** no local cross-device sync, IDE↔CLI split, path issues.

****Grok / Grok Build (xAI).**** Sessions auto-saved to `~/grok/sessions/`, keyed by working directory. `~/resume`, `--resume`, `-c`, `~/fork`; subscription-gated (SuperGrok/X Premium+). Config in `~/grok/config.toml`. No native cross-device sync. Still beta (v0.2.x as of July 2026). ****Residual gap:**** no sync at all.

****GitHub Copilot CLI.**** Sessions in `~/copilot/` (`session-state/<uuid>/events.jsonl`). Notably, Copilot CLI ****syncs sessions to your GitHub account by default**** and offers ****remote control**** from GitHub.com/GitHub Mobile (origin machine must stay online, `~/keep-alive`). But ****cross-device *resume* is still not supported**** – manually copying a session dir and running `--resume` overwrites `events.jsonl` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session\_id`. ****Residual gap:**** the sync is telemetry/history, not resumable working state across machines.

****Amp (Sourcegraph).**** Threads sync to ampcode.com by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). ****Residual gap:**** vendor-locked, server-stored, no cross-vendor.

****Cross-vendor summary:**** Every vendor is building **within** its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

	Tool	Location	Format	Path keying	Cross-device resume native?
					--- --- --- --- ---
	Claude Code	~/claude/projects/<encoded-path>/<session>.jsonl`	JSONL, append-only event stream	**Absolute path, non-alphanumerics→dashes**	Via cloud only (teleport/RC)
	Codex	~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl`	JSONL + `session_meta` header	date-based; cwd in meta	Yes, via App Server
	Gemini CLI	~/gemini/tmp/<project_hash>/chats/`	JSON	**Hash of project root**	No
	opencode	~/local/share/opencode/`	(SQLite + files)	SQLite/JSON	project-keyed
					Sharing only
	Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)
					Grok
	Grok	~/grok/sessions/`	JSON	working directory	No
	Copilot CLI	~/copilot/session-state/<uuid>/events.jsonl`	JSONL events	uuid	Account history sync; no resume

****The path-mapping problem (the crux).**** Claude Code derives the project folder name by

escaping the absolute path: ``/Users/me/app`` → ``~/ .claude/projects/-Users-me-app/``, while the same repo at ``C:\Users\me\app`` (or a different macOS username/location) produces a **different** folder. Consequently, after a naive file sync, ``claude --resume`` on the second machine cannot find the first machine's sessions. This is confirmed by the claude-sync author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker."

Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via ``project_hash``; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. ****This is the single highest-value, currently-unmet technical feature.****

****Format churn risk.**** Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

****Mid-session cross-device resume feasibility (same vendor):**** Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present — **if** path keys are remapped. Codex's App Server makes it cloud-native already.

****Cross-vendor session replay (e.g., Claude Code session → Codex):**** ****Fundamentally impractical.**** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) ``encrypted_content`` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. ****Recommendation:** treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.

3. Competitor teardown

****Session-continuity tools:****

- ****tawanorg/claude-sync**** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs ``~/ .claude`` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + ****age E2E encryption**** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.
- ****npow/session-sync**** (~1 star, very new): the only cross-vendor **session** translator — Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- ****dantelex/aisync ("ss")****: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing — treat traction as unverified.)
- ****codsync/codsync**** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- ****kroxiksut/codexSync****: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- ****waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)**** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- ****iHildy/opencode-synced**** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- ****Dinesh3184/claude-session-sync****: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- ****renefichtmueller/claude-sync****: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

****Config/MCP/skills sync (DevSync's neighborhood):****

- ****Leoyang183/sync-agents-settings**** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.

- **nicepkg/vsync** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- **baranovxyz/agentsync** and **spxrogers/agentsync** (beta): canonical ``.agents/`` config synced to 9–31 agents.
- **neon-solutions/add-mcp**: one-command MCP install across ~16 agents (Neon).
 - **ztripez/mcp-sync**: MCP config sync across tools.
 - **AgentSync (Rust)**: symlink-based config sync.
 - **skills-sync**: skills + MCP workspace.
- **DevSync** (devsync.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

URL-addressable / mobile-control / viewer approaches:

- **TokenRip** (tokenrip.com): makes session state a URL-addressable artifact (``/a/<slug>``) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- **Omnara** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was **archived** in February 2026 because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)
- **Syncode – AI Programming Tool**: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- **jazzyalex/agent-sessions** & **kenn-io/agentsview**: local-first session browsers/search across 20+ agents (read/resume-command, not sync).
- **LAN-workspace approaches** and **Syncthing/NAS/dotfiles/chezmoi+age**: DIY approaches that work for config but hit the path problem for sessions.

Takeaway: The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- **Claude Code**: #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- **Codex**: #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI ↔ app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
 - **Copilot CLI**: #1947 (cloud-synced sessions), #1635 (cross-environment resume).
- **Cursor forum**: multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is. Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. **Cross-vendor unified profile** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's

- native format. This is DevSync's existing strength, extended.
2. ****OS-aware path remapping as a first-class feature**** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows↔macOS. This is the wedge no competitor has nailed.
 3. ****E2E encryption by default**** – table stakes given session files contain code, secrets, and full transcripts.

****Risks:****

- ****Format churn:**** Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).
- ****Vendor lock-out / ToS:**** Storing session data is user-owned data (fine), but **auth tokens** are not transferable (codexSync explicitly refuses; Codex `encrypted\_content` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.
 - ****Vendors closing the gap:**** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This **validates** the demand while **confirming** that only the cross-vendor + cross-OS layer is defensible.
- ****Security/privacy expectations:**** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).
- ****Honest limits:**** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

****Stage 0 – Fold in DevSync (immediate).**** Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

****Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge).**** Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/ .claude/projects/` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- ****First-class OS-aware path remapping:**** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
- ****Benchmark to advance:**** `claude --resume` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

****Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode.**** Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid `encrypted\_content`/token pitfalls. For Gemini, remap `project\_hash`. For opencode, use safe SQLite snapshotting (the `sqlite3 .backup` pattern the community scripts use).

- ****Benchmark:**** each new adapter must pass the same cross-OS resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One "profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not replay.

****Monetization & thresholds.**** Free/open-source core (build trust, match claude-sync) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSynq/OpenSync validate willingness to pay). ****Kill/pivot signal:**** if Anthropic ships account-level **local** session sync with automatic path remapping across OSeS (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini-Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

Duffel is dead, and I should have caught it before suggesting it — that one's my miss. It's not a luggage brand, it's a **developer API company**: London-based, founded 2017, 51–200 employees, backed by Benchmark, Blossom, Index Ventures, Kima, and Y Combinator — with "API, Developer tools" listed as its own specialties. It's raised \$37M and serves 500+ companies including TravelPerk and Ramp. They own duffel.com and the `duffelhq` handles. Naming a dev tool Duffel is the worst collision in this entire thread.

Cabin is cleaner, but the bigger problem is structural.

Look at how the `open-` convention actually works: `opencode` → *code*. `openrouter` → *router*. `opentelemetry` → *telemetry*. `openssl` → *ssl*. The second half is always the **literal thing** being provided openly. That's what makes the prefix parse.

Now run your two through it. "Openduffel" = an open duffel bag. "Opencabin" = an open cabin. The prefix takes your metaphor and drags it back to literal — so instead of evoking travel and continuity, it reads like a luggage company or a cabin rental that happens to be open-source. The prefix and the metaphor are fighting for the same slot.

There's a second, sharper reason to drop it: **`open-` is infrastructure positioning, and you've spent the last several turns telling me you want a brand.** You wanted clicky. You wanted sticky. You wanted something that isn't generic-SaaS-shaped. Cursor isn't OpenCursor. Linear isn't OpenLinear. The `open-` prefix signals "commodity layer, adopt me as plumbing" — which is a fine strategy, but it's the opposite of memorable.

So pick a lane rather than blending them:

- **Infra lane** → `open-` + a literal noun: `opensession`, `openstate`. Honest, forgettable, protocol-flavored.
- **Brand lane** → Layover, Reinstale, Instale. What you actually said you want.

And you can have both eventually. Ship **Layover** as the product, and if you later publish your normalized session format as a spec, *that* becomes `openstate` or `opensession`. Company gets a brand, protocol gets the prefix. Very common pattern.

One piece of good news: I checked Layover, and dev tooling is essentially clear. There's a Layover Studio — an unfunded two-person web design shop in Cheyenne, founded 2023 — and a micro-hotel company called Layover Inc. Neither is a threat, neither is in your category. `layover.com` will belong to travel, but `layover.dev` is likely open.

Layover stands as the front-runner. Reinstale is your fallback, and it's a strong one.

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

```
# Cross-Device Sync for Terminal AI Coding Agents: Competitive &
Technical Landscape
```

```
## TL;DR
```

```
- **The durable opportunity is a cross-vendor abstraction layer
with OS-aware path remapping as its headline feature – NOT
another "sync my Claude sessions" tool.** Every major vendor is
now shipping single-ecosystem sync natively (Claude Code Remote
Control/teleport, Codex's unified App Server backend, Amp
threads, Copilot CLI cloud sync), so the defensible wedge is the
one thing no vendor will ever build: normalizing sessions + MCP +
skills + instruction files across *competing* tools and remapping
Windows↔macOS paths.
```

```
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex)
is effectively impossible and should be dropped from scope;**
config/skills/instruction portability and *same-vendor* cross-
device session continuity are the achievable, valuable core. The
single sharpest, currently-unsolved technical pain – which even
the leading competitor (claude-sync) explicitly punts on – is
Claude Code's absolute-path project indexing breaking on Windows
↔macOS sync.
```

```
- **Recommendation: build a "universal profile" that folds
DevSync's MCP-sync foundation in, adds E2E-encrypted session sync
with first-class path remapping for Claude Code first, then
Codex/Gemini/opencode.** Ship path remapping as the MVP
```


differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...\` and `/Users/...\` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.
3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.
4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

Claude Code (Anthropic). Ships the richest consumer-facing multi-device story:

- **`/teleport`** (`claude --teleport`): pulls a *cloud* session (started at `claude.ai/code` or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI,

session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport`, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires **clean git state** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same claude.ai account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."

- **Remote Control** (`claude remote-control` / `/remote-control`, --rc`): pushes a *live local* session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an *undocumented* ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `ANTHROPIC_BASE_URL` is redirected; unavailable under Zero Data Retention.

- **Claude Code on the web** (claude.ai/code): runs on Anthropic cloud infra, no local filesystem.

- **Residual gap:** No true device-to-device *local* session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an *unexpected*, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

Codex CLI (OpenAI). The most architecturally mature cross-surface model:

- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared **App Server** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under `~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl`, with a `session_meta` first line.
- Rich lifecycle: `codex resume` (picker), --last`, --resume-`

session-id`, `/fork`, archive/unarchive (v0.136), delete (v0.140). Imports histories from other agents via an `external-agent-sessions` crate.

- **Known gaps/bugs:** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing `encrypted\_content` often can't be continued after crossing provider/account (`invalid\_encrypted\_content`); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.

- **Residual gap:** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

Gemini CLI (Google). Auto-saves sessions to `~/.gemini/tmp/<project\_hash>/chats/` (project-scoped, hash of project root). `/resume` session browser, `--resume`/`-r` flag, `/chat save` checkpoints, and a shadow-git checkpointing system under `~/.gemini/history/<project\_hash>`. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. **Residual gap:** everything local + path-hash-keyed; no sync.

opencode (sst). SQLite + file storage under `~/.local/share/opencode/` (`opencode.db`, `storage/session|message|part`). Native **session sharing** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). **Residual gap:** sharing ≠ syncing; DB concurrency and cross-OS path issues remain.

Cursor CLI (cursor-agent). Chats live in local SQLite; CLI chats under `~/.cursor/chats` and resume via `cursor-agent --resume`. **IDE chats and CLI chats do NOT share a session store** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend `&` to push to a Cloud Agent, pick up at cursor.com/agents), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). **Residual gap:** no local cross-device sync, IDE ↔ CLI split, path issues.

Grok / Grok Build (xAI). Sessions auto-saved to `~/grok/sessions/``, keyed by working directory. `/resume``, `--resume``, `-c``, `/fork``; subscription-gated (SuperGrok/X Premium+). Config in `~/grok/config.toml``. No native cross-device sync. Still beta (v0.2.x as of July 2026). **Residual gap:** no sync at all.

GitHub Copilot CLI. Sessions in `~/copilot/`` (`session-state/<uuid>/events.jsonl``). Notably, Copilot CLI **syncs** sessions to your GitHub account by default and offers **remote control** from GitHub.com/GitHub Mobile (origin machine must stay online, `/keep-alive``). But **cross-device resume** is still not supported – manually copying a session dir and running `--resume`` overwrites `events.jsonl`` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session_id``. **Residual gap:** the sync is telemetry/history, not resumable working state across machines.

Amp (Sourcegraph). Threads sync to `ampcode.com` by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). **Residual gap:** vendor-locked, server-stored, no cross-vendor.

Cross-vendor summary: Every vendor is building *within* its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

Tool	Location	Format	Path keying	Cross-device resume native?
Claude Code	<code>~/claude/projects/<encoded-path>/<session>.jsonl`</code>	JSONL, append-only event stream	Absolute path, non-alphanumerics→dashes	Via cloud only (teleport/RC)
Codex	<code>~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl`</code>	JSONL + <code>session_meta`</code> header	date-based; cwd in meta	Yes, via App Server
Gemini CLI	<code>~/gemini/tmp/<project_hash>/chats/`</code>	JSON		

```
**Hash of project root** | No |  
| opencode | `~/.local/share/opencode/` (SQLite + files) |  
SQLite/JSON | project-keyed | Sharing only |  
| Cursor | local SQLite | DB rows | workspace/path metadata | No  
(cloud handoff only) |  
| Grok | `~/.grok/sessions/` | JSON | working directory | No |  
| Copilot CLI | `~/.copilot/session-state/<uuid>/events.jsonl` |  
JSONL events | uuid | Account history sync; no resume |
```

****The path-mapping problem (the crux).**** Claude Code derives the project folder name by escaping the absolute path: ``/Users/me/app`` → ``~/.claude/projects/-Users-me-app/``, while the same repo at ``C:\Users\me\app`` (or a different macOS username/location) produces a *different* folder. Consequently, after a naive file sync, ``claude --resume`` on the second machine cannot find the first machine's sessions. This is confirmed by the `claude-sync` author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker." Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via ``project_hash``; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. ****This is the single highest-value, currently-unmet technical feature.****

****Format churn risk.**** Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

****Mid-session cross-device resume feasibility (same vendor):**** Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present — *if* path keys are remapped. Codex's App Server makes it cloud-native already.

****Cross-vendor session replay (e.g., Claude Code session →**

Codex):** ****Fundamentally impractical.**** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) `encrypted\_content` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. ****Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.****

3. Competitor teardown

****Session-continuity tools:****

- ****tawanorg/claude-sync**** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/.claude` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + ****age E2E encryption**** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.
- ****npow/session-sync**** (~1 star, very new): the only cross-vendor *session* translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- ****dantelex/aisync ("ss")****: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- ****codsync/codsync**** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- ****kroxiksut/codexSync****: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- ****waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)**** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- ****iHildy/opencode-synced**** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- ****Dinesh3184/claude-session-sync****: iCloud Drive sync GUI for Claude Code (macOS + Windows).

- **renefichtmueller/claude-sync**: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

Config/MCP/skills sync (DevSync's neighborhood):

- **Leoyangl83/sync-agents-settings** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.

- **nicepkg/vsync** (~46 stars, active):

MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.

- **baranovxyz/agentsync** and **spxrogers/agentsync** (beta): canonical ``.agents/`` config synced to 9–31 agents.

- **neon-solutions/add-mcp**: one-command MCP install across ~16 agents (Neon).

- **ztripez/mcp-sync**: MCP config sync across tools.

- **AgentSync (Rust)**: symlink-based config sync.

- **skills-sync**: skills + MCP workspace.

- **DevSynq** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

URL-addressable / mobile-control / viewer approaches:

- **TokenRip** (tokenrip.com): makes session state a URL-addressable artifact (``/a/<slug>``) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").

- **Omnara** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was **archived in February 2026** because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)

- **Syncode – AI Programming Tool**: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).

- **jazzyalex/agent-sessions** & **kenn-io/agentsview**: local-first session browsers/search across 20+ agents (read/resume-command, not sync).

- **LAN-workspace approaches** and

Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

Takeaway: The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader

(claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- **Claude Code:** #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- **Codex:** #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI ↔ app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
- **Copilot CLI:** #1947 (cloud-synced sessions), #1635 (cross-environment resume).
- **Cursor forum:** multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is. Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. **Cross-vendor unified profile** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.

2. **OS-aware path remapping as a first-class feature** – the concrete, unsolved pain that makes same-vendor session sync

actually work across Windows↔macOS. This is the wedge no competitor has nailed.

3. **E2E encryption by default** – table stakes given session files contain code, secrets, and full transcripts.

Risks:

- **Format churn:** Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).
- **Vendor lock-out / ToS:** Storing session data is user-owned data (fine), but *auth tokens* are not transferable (codexSync explicitly refuses; Codex `encrypted\_content` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.
- **Vendors closing the gap:** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This *validates* the demand while *confirming* that only the cross-vendor + cross-OS layer is defensible.
- **Security/privacy expectations:** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).
- **Honest limits:** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

Stage 0 – Fold in DevSync (immediate). Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge). Target the founder's exact case

(Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of
`~/.claude/projects/` and global config to user-owned storage
(S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- **First-class OS-aware path remapping:** on pull, rewrite the
project folder key AND in-file absolute path references (cwd, git
snapshots, file refs) from the source machine's path to the
target machine's path, using the git remote URL as the project
identity (the technique cursaves uses for Cursor). This is the
feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic
writes, conflict preservation.
- **Benchmark to advance:** `claude --resume` on the MacBook
reliably finds and resumes a session created on the Windows
desktop (different absolute paths) in <10s, with no corruption,
across 3 consecutive Claude Code releases.

**Stage 2 – Second and third vendors: Codex + Gemini CLI +
opencode.** Add adapters. For Codex, cooperate with (don't fight)
the App Server – sync what's local and useful, avoid
`encrypted\_content`/token pitfalls. For Gemini, remap
`project\_hash`. For opencode, use safe SQLite snapshotting (the
`sqlite3 .backup` pattern the community scripts use).

- **Benchmark:** each new adapter must pass the same cross-OS
resume test as Claude Code before GA.

Stage 3 – Unified profile + cross-vendor context handoff. One
"profile" = sessions + MCP + skills + agents + instructions,
portable across machines and (for config) across vendors. Add
best-effort context handoff (summarize + re-inject) for cross-
vendor moves – explicitly labeled as context transfer, not
replay.

Monetization & thresholds. Free/open-source core (build
trust, match claude-sync) with a paid tier for managed encrypted
storage, multi-machine dashboards, and team profiles
(DevSynq/OpenSync validate willingness to pay). **Kill/pivot
signal:** if Anthropic ships account-level *local* session sync
with automatic path remapping across OSes (not just history
mirroring, per #71794), the single-vendor wedge collapses – at
that point double down on cross-vendor config/skills/profile
portability, which no vendor will build.

What NOT to do: Don't build on Remote Control/teleport

(gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

Ok, live over looks good is there any way you can check if any domains are available for the name?

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape

TL;DR

- ****The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.**** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows↔macOS paths.
- ****Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;**** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows↔macOS sync.
- ****Recommendation: build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode.**** Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...\` and `/Users/...\` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.
3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.
4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

- Claude Code (Anthropic).** Ships the richest consumer-facing multi-device story:
 - **teleport** (`claude --teleport`): pulls a *cloud* session (started at claude.ai/code or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport`, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires **clean git state** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same claude.ai account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."
 - **Remote Control** (`claude remote-control` / `/remote-control`, `--rc`): pushes a *live local* session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an *undocumented* ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `ANTHROPIC\_BASE\_URL` is redirected; unavailable under Zero Data Retention.
 - **Claude Code on the web** (claude.ai/code): runs on Anthropic cloud infra, no local filesystem.
 - **Residual gap:** No true device-to-device *local* session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an *unexpected*, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.
- Codex CLI (OpenAI).** The most architecturally mature cross-surface model:
 - CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared **App Server** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under `~/.codex/sessions/YYYY/MM/DD/rollout-\*.jsonl`, with a `session_meta` first line.
 - Rich lifecycle: `codex resume` (picker), `--last`, `--resume-session-id`, `/fork`,

archive/unarchive (v0.136), delete (v0.140). Imports histories from other agents via an ``external-agent-sessions`` crate.

- **Known gaps/bugs:** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing ``encrypted_content`` often can't be continued after crossing provider/account (``invalid_encrypted_content``); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI↔app-server live sync.
- **Residual gap:** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

Gemini CLI (Google). Auto-saves sessions to ``~/gemini/tmp/<project_hash>/chats/`` (project-scoped, hash of project root). ``/resume`` session browser, ``--resume``/``-r`` flag, ``/chat save`` checkpoints, and a shadow-git checkpointing system under ``~/gemini/history/<project_hash>``. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. **Residual gap:** everything local + path-hash-keyed; no sync.

opencode (sst). SQLite + file storage under ``~/local/share/opencode/`` (``opencode.db``, ``storage/session|message|part``). Native **session sharing** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). **Residual gap:** sharing ≠ syncing; DB concurrency and cross-OS path issues remain.

Cursor CLI (cursor-agent). Chats live in local SQLite; CLI chats under ``~/cursor/chats`` and resume via ``cursor-agent --resume``. **IDE chats and CLI chats do NOT share a session store** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend ``&`` to push to a Cloud Agent, pick up at cursor.com/agents), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). **Residual gap:** no local cross-device sync, IDE↔CLI split, path issues.

Grok / Grok Build (xAI). Sessions auto-saved to ``~/grok/sessions/``, keyed by working directory. ``/resume``, ``--resume``, ``-c``, ``/fork``; subscription-gated (SuperGrok/X Premium+). Config in ``~/grok/config.toml``. No native cross-device sync. Still beta (v0.2.x as of July 2026). **Residual gap:** no sync at all.

GitHub Copilot CLI. Sessions in ``~/copilot/`` (``session-state/<uuid>/events.jsonl``). Notably, Copilot CLI **syncs sessions to your GitHub account by default** and offers **remote control** from GitHub.com/GitHub Mobile (origin machine must stay online, ``/keep-alive``). But **cross-device resume** is still not supported – manually copying a session dir and running ``--resume`` overwrites ``events.jsonl`` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied ``session_id``. **Residual gap:** the sync is telemetry/history, not resumable working state across machines.

Amp (Sourcegraph). Threads sync to ampcode.com by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). **Residual gap:** vendor-locked, server-stored, no cross-vendor.

Cross-vendor summary: Every vendor is building *within* its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

	Tool	Location	Format	Path keying	Cross-device resume native?
					--- --- --- --- ---
	Claude Code	<code>`~/claude/projects/<encoded-path>/<session>.jsonl`</code>	JSONL, append-only event stream	Absolute path, non-alphanumerics→dashes	Via cloud only (teleport/RC)
	Codex	<code>`~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl`</code>	JSONL + <code>`session_meta`</code> header		

		date-based; cwd in meta	Yes, via App Server
Gemini CLI	~/gemini/tmp/<project_hash>/chats/	JSON	**Hash of project root**
opencode	~/local/share/opencode/	(SQLite + files)	SQLite/JSON
			project-keyed
			Sharing only
Cursor	local SQLite	DB rows	workspace/path metadata
			No (cloud handoff only)
		Grok	~/grok/sessions/
			JSON
			working directory
			No
Copilot CLI	~/copilot/session-state/<uuid>/events.jsonl	JSONL events	uuid
			Account history sync; no resume

****The path-mapping problem (the crux).**** Claude Code derives the project folder name by escaping the absolute path: ``/Users/me/app`` → ``~/claude/projects/-Users-me-app/``, while the same repo at ``C:\Users\me\app`` (or a different macOS username/location) produces a *different* folder. Consequently, after a naive file sync, ``claude --resume`` on the second machine cannot find the first machine's sessions. This is confirmed by the claude-sync author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker." Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via ``project_hash``; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. ****This is the single highest-value, currently-unmet technical feature.****

****Format churn risk.**** Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

****Mid-session cross-device resume feasibility (same vendor):**** Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present – *if* path keys are remapped. Codex's App Server makes it cloud-native already.

****Cross-vendor session replay (e.g., Claude Code session → Codex):**** ****Fundamentally impractical.**** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) ``encrypted_content`` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. ****Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.****

3. Competitor teardown

****Session-continuity tools:****

- ****tawanorg/claude-sync**** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs ``~/claude`` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + ****age E2E encryption**** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.
- ****npow/session-sync**** (~1 star, very new): the only cross-vendor *session* translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- ****dantelex/aisync ("ss")****: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- ****codsync/codsync**** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- ****kroxiksut/codexSync****: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- ****waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)**** (~10 stars): Convex-

- backed cloud sync/search/share; opensync.dev SaaS option.
- **iHildy/opencode-synced** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- **Dinesh3184/claude-session-sync**: iCloud Drive sync GUI for Claude Code (macOS + Windows).
 - **renefichtmueller/claude-sync**: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.
- Config/MCP/skills sync (DevSync's neighborhood)**:
- **Leoyang183/sync-agents-settings** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
 - **nicepkg/vsync** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
 - **baranovxyz/agentsync** and **spxrogers/agentsync** (beta): canonical ``.agents/`` config synced to 9–31 agents.
 - **neon-solutions/add-mcp**: one-command MCP install across ~16 agents (Neon).
 - **ztripez/mcp-sync**: MCP config sync across tools.
 - **AgentSync (Rust)**: symlink-based config sync.
 - **skills-sync**: skills + MCP workspace.
 - **DevSynq** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.
- URL-addressable / mobile-control / viewer approaches**:
- **TokenRip** (tokenrip.com): makes session state a URL-addressable artifact (``/a/<slug>``) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- **Omnara** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was **archived** in February 2026⁶ because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3⁷ mobile-app rating was seen but is unverified.)
- **Syncode – AI Programming Tool**: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- **jazzyalex/agent-sessions** & **kenn-io/agentsview**: local-first session browsers/search across 20+ agents (read/resume-command, not sync).
- **LAN-workspace approaches** and Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

Takeaway: The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- **Claude Code**: #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- **Codex**: #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI ↔ app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
 - **Copilot CLI**: #1947 (cloud-synced sessions), #1635 (cross-environment resume).
- **Cursor forum**: multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the

explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is. Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. **Cross-vendor unified profile** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.
2. **OS-aware path remapping as a first-class feature** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows ↔ macOS. This is the wedge no competitor has nailed.
3. **E2E encryption by default** – table stakes given session files contain code, secrets, and full transcripts.

Risks:

- **Format churn:** Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).
- **Vendor lock-out / ToS:** Storing session data is user-owned data (fine), but *auth tokens* are not transferable (codexSync explicitly refuses; Codex `encrypted\_content` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.
 - **Vendors closing the gap:** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This *validates* the demand while *confirming* that only the cross-vendor + cross-OS layer is defensible.
- **Security/privacy expectations:** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).
- **Honest limits:** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

Stage 0 – Fold in DevSync (immediate). Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge). Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/ .claude/projects/` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- **First-class OS-aware path remapping:** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
- **Benchmark to advance:** `claude --resume` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode. Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid

`encrypted\_content`/token pitfalls. For Gemini, remap `project\_hash`. For opencode, use safe SQLite snapshotting (the `sqlite3 .backup` pattern the community scripts use).
- **Benchmark:** each new adapter must pass the same cross-OS resume test as Claude Code before GA.

Stage 3 – Unified profile + cross-vendor context handoff. One "profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not replay.

Monetization & thresholds. Free/open-source core (build trust, match claude-sync) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSynq/OpenSync validate willingness to pay). **Kill/pivot signal:** if Anthropic ships account-level *local* session sync with automatic path remapping across OSes (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

What NOT to do: Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

Straight answer first: **I can't authoritatively check domain availability.** I have web search, not WHOIS or a registrar API — and search only reveals domains that have *live sites on them*. Plenty of registered domains sit parked with nothing to show, so "I found nothing" is not evidence a domain is free. Anything I told you about [layover.dev](#) being open would be a guess dressed up as a fact.

What search does show — all occupied:

- [layover.studio](#) — the web design shop from earlier
- [layover.so](#) — registered, but the product is dead: the site says Layover is no longer available and pitches a health app instead
- [layover.guide](#) — a crew-only app for aircrews to make the most of layovers
- [thelayoverapp.com](#) — a social app for connecting with fellow travelers during layovers
- [layoverinc.com](#) — the micro-hotel company

No signal either way on [.dev](#), [.io](#), or [.app](#).

The thing worth flagging, though: look at that list. Layover is crowded with consumer travel apps — there's even a United crew app called Layover, plus a novel, a film, a TV series, and a 2023 EP by V. Your *category* is clear (nobody's in dev tooling), so the name is safe to use. But you will never rank for "layover" alone. Your searchable handle becomes "layover sync" or "layover dev" — which is workable, most dev tools live on GitHub and word of mouth anyway, but it's a real cost you should accept knowingly rather than discover later.

The 60-second check to run yourself:

```
whois layover.dev
whois layover.sh
npm view layover
```

Then hit github.com/layover and instantdomainsearch.com/search/layover. For a CLI tool, honestly, **npm and the GitHub org matter more than the domain** — [layover](https://www.npmjs.com/package/layover) on npm is a common enough word that I'd expect it squatted, and that affects your install command directly.

Fallbacks if the good TLDs are gone: layoverhq.com, uselayover.com, or getlayover.com — all standard patterns nobody blinks at. And if npm's [layover](https://www.npmjs.com/package/layover) is taken, scoping it ([@yourhandle/layover](https://www.npmjs.com/package/@yourhandle/layover)) is completely normal. Check npm before the domain. That's the one that shows up in your README.

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

```
# Cross-Device Sync for Terminal AI Coding Agents: Competitive &
Technical Landscape
```

```
## TL;DR
```

```
- **The durable opportunity is a cross-vendor abstraction layer
with OS-aware path remapping as its headline feature – NOT
another "sync my Claude sessions" tool.** Every major vendor is
now shipping single-ecosystem sync natively (Claude Code Remote
Control/teleport, Codex's unified App Server backend, Amp
threads, Copilot CLI cloud sync), so the defensible wedge is the
one thing no vendor will ever build: normalizing sessions + MCP +
skills + instruction files across *competing* tools and remapping
Windows↔macOS paths.
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex)
is effectively impossible and should be dropped from scope;**
config/skills/instruction portability and *same-vendor* cross-
device session continuity are the achievable, valuable core. The
single sharpest, currently-unsolved technical pain – which even
the leading competitor (claude-sync) explicitly punts on – is
```


Claude Code's absolute-path project indexing breaking on Windows
↔ macOS sync.

- **Recommendation:** build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode. Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...\` and `/Users/...\` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.
3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.
4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

Claude Code (Anthropic). Ships the richest consumer-facing multi-device story:

- ***/teleport*** (`claude --teleport`): pulls a *cloud* session (started at `claude.ai/code` or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport`, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires **clean git state** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same `claude.ai` account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."
- **Remote Control** (`claude remote-control` / `/remote-control`, `--rc`): pushes a *live local* session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an *undocumented* ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `ANTHROPIC_BASE_URL` is redirected; unavailable under Zero Data Retention.
- **Claude Code on the web** (`claude.ai/code`): runs on Anthropic cloud infra, no local filesystem.
- **Residual gap**: No true device-to-device *local* session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an *unexpected*, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

Codex CLI (OpenAI). The most architecturally mature cross-surface model:

- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared **App Server** (long-lived

process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under `~/.codex/sessions/YYYY/MM/DD/rollout-*.jsonl``, with a ``session_meta`` first line.

- Rich lifecycle: ``codex resume`` (picker), ``--last``, ``--resume-session-id``, ``/fork``, archive/unarchive (v0.136), delete (v0.140). Imports histories from other agents via an ``external-agent-sessions`` crate.
- **Known gaps/bugs:** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing ``encrypted_content`` often can't be continued after crossing provider/account (``invalid_encrypted_content``); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.
- **Residual gap:** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

Gemini CLI (Google). Auto-saves sessions to `~/.gemini/tmp/<project_hash>/chats/`` (project-scoped, hash of project root). ``/resume`` session browser, ``--resume`/`-r`` flag, ``/chat save`` checkpoints, and a shadow-git checkpointing system under `~/.gemini/history/<project_hash>``. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. **Residual gap:** everything local + path-hash-keyed; no sync.

opencode (sst). SQLite + file storage under `~/.local/share/opencode/`` (``opencode.db``, ``storage/session|message|part``). Native **session sharing** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). **Residual gap:** sharing ≠ syncing; DB concurrency and cross-OS path issues remain.

Cursor CLI (cursor-agent). Chats live in local SQLite; CLI chats under `~/.cursor/chats`` and resume via ``cursor-agent --resume``. **IDE chats and CLI chats do NOT share a session store** (confirmed by Cursor staff in the forum). Cloud handoff exists

(prepend `&` to push to a Cloud Agent, pick up at cursor.com/agents), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). **Residual gap:** no local cross-device sync, IDE↔CLI split, path issues.

Grok / Grok Build (xAI). Sessions auto-saved to `~/.grok/sessions/`, keyed by working directory. `/resume`, `--resume`, `-c`, `/fork`; subscription-gated (SuperGrok/X Premium+). Config in `~/.grok/config.toml`. No native cross-device sync. Still beta (v0.2.x as of July 2026). **Residual gap:** no sync at all.

GitHub Copilot CLI. Sessions in `~/.copilot/` (`session-state/<uuid>/events.jsonl`). Notably, Copilot CLI **syncs** sessions to your GitHub account by default and offers **remote control** from GitHub.com/GitHub Mobile (origin machine must stay online, `/keep-alive`). But **cross-device resume** is still not supported – manually copying a session dir and running `--resume` overwrites `events.jsonl` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session\_id`. **Residual gap:** the sync is telemetry/history, not resumable working state across machines.

Amp (Sourcegraph). Threads sync to ampcode.com by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). **Residual gap:** vendor-locked, server-stored, no cross-vendor.

Cross-vendor summary: Every vendor is building *within* its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

Tool	Location	Format	Path keying	Cross-device resume native?
Claude Code	~/.claude/projects/<encoded-path>/<session>.jsonl	JSONL, append-only event stream		

****Absolute path, non-alphanumerics→dashes**** | Via cloud only (teleport/RC) |
| Codex | `~/ .codex/sessions/YYYY/MM/DD/rollout-\*.jsonl` | JSONL + `session\_meta` header | date-based; cwd in meta | Yes, via App Server |
| Gemini CLI | `~/ .gemini/tmp/<project\_hash>/chats/` | JSON |
****Hash of project root**** | No |
opencode	`~/ .local/share/opencode/` (SQLite + files)	SQLite/JSON	project-keyed	Sharing only
Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)
Grok	`~/ .grok/sessions/`	JSON	working directory	No
Copilot CLI	`~/ .copilot/session-state/<uuid>/events.jsonl`	JSONL events	uuid	Account history sync; no resume

****The path-mapping problem (the crux).**** Claude Code derives the project folder name by escaping the absolute path: `~/Users/me/app` → `~/ .claude/projects/-Users-me-app/`, while the same repo at `C:\Users\me\app` (or a different macOS username/location) produces a *different* folder. Consequently, after a naive file sync, `claude --resume` on the second machine cannot find the first machine's sessions. This is confirmed by the claude-sync author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker." Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via `project\_hash`; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. ****This is the single highest-value, currently-unmet technical feature.****

****Format churn risk.**** Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

****Mid-session cross-device resume feasibility (same vendor):****

Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present – *if* path keys are remapped. Codex's App Server makes it cloud-native already.

Cross-vendor session replay (e.g., Claude Code session → Codex): **Fundamentally impractical.** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) `encrypted_content` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. **Recommendation:** treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.

3. Competitor teardown

Session-continuity tools:

- **tawanorg/claude-sync** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/ .claude` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + **age** E2E encryption ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. **Punts on path remapping** (its explicit, acknowledged weakness). Open-source, no business model.
- **npow/session-sync** (~1 star, very new): the only cross-vendor *session* translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- **dantelex/aisync ("ss")**: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- **codsync/codsync** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- **kroxiksut/codexSync**: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- **waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)** (~10 stars): Convex-backed cloud sync/search/share;

opensync.dev SaaS option.

- ****iHildy/opencode-synced**** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- ****Dinesh3184/claude-session-sync****: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- ****renefichtmueller/claude-sync****: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

****Config/MCP/skills sync (DevSync's neighborhood):****

- ****Leoyangl83/sync-agents-settings**** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- ****nicepkg/vsync**** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- ****baranovxyz/agentsync**** and ****spxrogers/agentsync**** (beta): canonical ``.agents/`` config synced to 9–31 agents.
- ****neon-solutions/add-mcp****: one-command MCP install across ~16 agents (Neon).
- ****ztripez/mcp-sync****: MCP config sync across tools.
- ****AgentSync (Rust)****: symlink-based config sync.
- ****skills-sync****: skills + MCP workspace.
- ****DevSynq**** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

****URL-addressable / mobile-control / viewer approaches:****

- ****TokenRip**** (tokenrip.com): makes session state a URL-addressable artifact (``/a/<slug>``) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- ****Omnara**** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was ****archived in February 2026**** because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)
- ****Syncode – AI Programming Tool****: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- ****jazzyalex/agent-sessions**** & ****kenn-io/agentsview****: local-first session browsers/search across 20+ agents (read/resume-command, not sync).

- **LAN-workspace approaches** and Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

Takeaway: The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- **Claude Code:** #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- **Codex:** #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI ↔ app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
- **Copilot CLI:** #1947 (cloud-synced sessions), #1635 (cross-environment resume).
- **Cursor forum:** multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is. Vendors will keep building **intra**-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. **Cross-vendor unified profile** – one normalized model for

MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.

2. **OS-aware path remapping as a first-class feature** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows↔macOS. This is the wedge no competitor has nailed.

3. **E2E encryption by default** – table stakes given session files contain code, secrets, and full transcripts.

****Risks:****

- **Format churn:** Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).
- **Vendor lock-out / ToS:** Storing session data is user-owned data (fine), but *auth tokens* are not transferable (codexSync explicitly refuses; Codex ``encrypted_content`` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.
- **Vendors closing the gap:** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This *validates* the demand while *confirming* that only the cross-vendor + cross-OS layer is defensible.
- **Security/privacy expectations:** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).
- **Honest limits:** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

Stage 0 – Fold in DevSync (immediate). Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor,

Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

****Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge).**** Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/.claude/projects/` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- ****First-class OS-aware path remapping:**** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
- ****Benchmark to advance:**** `claude --resume` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

****Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode.**** Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid `encrypted\_content`/token pitfalls. For Gemini, remap `project\_hash`. For opencode, use safe SQLite snapshotting (the `sqlite3 .backup` pattern the community scripts use).

- ****Benchmark:**** each new adapter must pass the same cross-OS resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One "profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not replay.

****Monetization & thresholds.**** Free/open-source core (build trust, match claude-sync) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSynq/OpenSync validate willingness to pay). ****Kill/pivot signal:**** if Anthropic ships account-level *local* session sync

with automatic path remapping across OSes (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

Let's just brainstorm for new ideas and concepts for its name

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape

TL;DR

- ****The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.**** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows  macOS paths.
- ****Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;**** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-

unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows↔macOS sync.

- **Recommendation:** build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opcode.** Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...\` and `/Users/...\` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.
3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.
4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

- Claude Code (Anthropic).** Ships the richest consumer-facing multi-device story:
 - **`/teleport`** (`claude --teleport`): pulls a *cloud* session (started at claude.ai/code or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport`, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires **clean git state** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same claude.ai account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."
 - **Remote Control** (`claude remote-control` / `/remote-control`, `--rc`): pushes a *live local* session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an *undocumented* ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `ANTHROPIC\_BASE\_URL` is redirected; unavailable under Zero Data Retention.
 - **Claude Code on the web** (claude.ai/code): runs on Anthropic cloud infra, no local filesystem.
 - **Residual gap:** No true device-to-device *local* session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an *unexpected*, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

****Codex CLI (OpenAI).**** The most architecturally mature cross-surface model:
- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared ****App Server**** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under `~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl`, with a `session_meta` first line.
- Rich lifecycle: `codex resume` (picker), `--last`, `--resume-session-id`, `/fork`, `archive/unarchive` (v0.136), `delete` (v0.140). Imports histories from other agents via an `external-agent-sessions` crate.
- ****Known gaps/bugs:**** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing `encrypted_content` often can't be continued after crossing provider/account (`invalid_encrypted_content`); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI↔app-server live sync.
- ****Residual gap:**** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

****Gemini CLI (Google).**** Auto-saves sessions to `~/gemini/tmp/<project_hash>/chats/` (project-scoped, hash of project root). `/resume` session browser, `--resume`/`-r` flag, `/chat save` checkpoints, and a shadow-git checkpointing system under `~/gemini/history/<project_hash>`. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. ****Residual gap:**** everything local + path-hash-keyed; no sync.

****opencode (sst).**** SQLite + file storage under `~/local/share/opencode/` (`opencode.db`, `storage/session|message|part`). Native ****session sharing**** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). ****Residual gap:**** sharing ≠ syncing; DB concurrency and cross-OS path issues remain.

****Cursor CLI (cursor-agent).**** Chats live in local SQLite; CLI chats under `~/cursor/chats` and resume via `cursor-agent --resume`. ****IDE chats and CLI chats do NOT share a session store**** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend `&` to push to a Cloud Agent, pick up at `cursor.com/agents`), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). ****Residual gap:**** no local cross-device sync, IDE↔CLI split, path issues.

****Grok / Grok Build (xAI).**** Sessions auto-saved to `~/grok/sessions/`, keyed by working directory. `/resume`, `--resume`, `-c`, `/fork`; subscription-gated (SuperGrok/X Premium+). Config in `~/grok/config.toml`. No native cross-device sync. Still beta (v0.2.x as of July 2026). ****Residual gap:**** no sync at all.

****GitHub Copilot CLI.**** Sessions in `~/copilot/` (`session-state/<uuid>/events.jsonl`). Notably, Copilot CLI ****syncs sessions to your GitHub account by default**** and offers ****remote control**** from `GitHub.com/GitHub Mobile` (origin machine must stay online, `/keep-alive`). But ****cross-device *resume* is still not supported**** – manually copying a session dir and running `--resume` overwrites `events.jsonl` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session_id`. ****Residual gap:**** the sync is telemetry/history, not resumable working state across machines.

****Amp (Sourcegraph).**** Threads sync to `ampcode.com` by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). ****Residual gap:**** vendor-locked, server-stored, no cross-vendor.

****Cross-vendor summary:**** Every vendor is building **within** its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

	Tool	Location	Format	Path keying	Cross-device resume native?	
						--- --- --- --- ---
Claude Code		~/ .claude/projects/<encoded-path>/<session>.jsonl`	JSONL, append-only event stream	**Absolute path, non-alphanumerics→dashes**	Via cloud only (teleport/RC)	
Codex		~/ .codex/sessions/YYYY/MM/DD/rollout-*.jsonl`	JSONL + `session_meta` header	date-based; cwd in meta	Yes, via App Server	
Gemini CLI		~/ .gemini/tmp/<project_hash>/chats/`	JSON	**Hash of project root**	No	
opencode		~/ .local/share/opencode/`	(SQLite + files)	SQLite/JSON	project-keyed	Sharing only
Cursor		local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)	
Grok		~/ .grok/sessions/`	JSON	working directory	No	
Copilot CLI		~/ .copilot/session-state/<uuid>/events.jsonl`	JSONL events	uuid	Account history sync; no resume	

****The path-mapping problem (the crux).**** Claude Code derives the project folder name by escaping the absolute path: ``/Users/me/app`` → `~/ .claude/projects/-Users-me-app/``, while the same repo at ``C:\Users\me\app`` (or a different macOS username/location) produces a *\*different\** folder. Consequently, after a naive file sync, ``claude --resume`` on the second machine cannot find the first machine's sessions. This is confirmed by the claude-sync author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker." Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via ``project_hash``; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. ****This is the single highest-value, currently-unmet technical feature.****

****Format churn risk.**** Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

****Mid-session cross-device resume feasibility (same vendor):**** Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present — **if** path keys are remapped. Codex's App Server makes it cloud-native already.

****Cross-vendor session replay (e.g., Claude Code session → Codex):**** ****Fundamentally impractical.**** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) ``encrypted_content`` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. ****Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.****

3. Competitor teardown

****Session-continuity tools:****

- ****tawanorg/claude-sync**** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/ .claude`` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + ****age E2E encryption**** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.
- ****npow/session-sync**** (~1 star, very new): the only cross-vendor **session** translator — Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- ****dantelex/aisync ("ss")****: multi-device continuity for Claude/Codex/Cursor/opencode via a

provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)

- **codsync/codsync** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- **kroxiksut/codexSync**: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- **waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- **iHildy/opencode-synced** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- **Dinesh3184/claude-session-sync**: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- **renefichtmueller/claude-sync**: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

Config/MCP/skills sync (DevSync's neighborhood):

- **Leoyang183/sync-agents-settings** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- **nicepkg/vsync** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- **baranovxyz/agentsync** and **spxrogers/agentsync** (beta): canonical `./agents/` config synced to 9–31 agents.
- **neon-solutions/add-mcp**: one-command MCP install across ~16 agents (Neon).
- **ztripez/mcp-sync**: MCP config sync across tools.
- **AgentSync (Rust)**: symlink-based config sync.
- **skills-sync**: skills + MCP workspace.
- **DevSynq** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

URL-addressable / mobile-control / viewer approaches:

- **TokenRip** (tokenrip.com): makes session state a URL-addressable artifact (`/a/<slug>`) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- **Omnara** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was **archived** in February 2026 because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)
- **Syncode – AI Programming Tool**: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- **jazzyalex/agent-sessions** & **kenn-io/agentsview**: local-first session browsers/search across 20+ agents (read/resume-command, not sync).
- **LAN-workspace approaches** and **Syncthing/NAS/dotfiles/chezmoi** + age: DIY approaches that work for config but hit the path problem for sessions.

Takeaway: The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- **Claude Code**: #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- **Codex**: #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI  app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe

- and codex-workspace-sync), #4514 (forking).
- **Copilot CLI:** #1947 (cloud-synced sessions), #1635 (cross-environment resume).
- **Cursor forum:** multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is. Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. **Cross-vendor unified profile** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.
2. **OS-aware path remapping as a first-class feature** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows↔macOS. This is the wedge no competitor has nailed.
3. **E2E encryption by default** – table stakes given session files contain code, secrets, and full transcripts.

Risks:

- **Format churn:** Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).
- **Vendor lock-out / ToS:** Storing session data is user-owned data (fine), but *auth tokens* are not transferable (codexSync explicitly refuses; Codex `encrypted\_content` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.
 - **Vendors closing the gap:** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This *validates* the demand while *confirming* that only the cross-vendor + cross-OS layer is defensible.
- **Security/privacy expectations:** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).
- **Honest limits:** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

Stage 0 – Fold in DevSync (immediate). Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge). Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/claude/projects/` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- **First-class OS-aware path remapping:** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.

- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
- ****Benchmark to advance:**** `claude --resume` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

****Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode.**** Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid `encrypted\_content`/token pitfalls. For Gemini, remap `project\_hash`. For opencode, use safe SQLite snapshotting (the `sqlite3 .backup` pattern the community scripts use).

- ****Benchmark:**** each new adapter must pass the same cross-OS resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One "profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not replay.

****Monetization & thresholds.**** Free/open-source core (build trust, match claude-sync) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSynq/OpenSync validate willingness to pay). ****Kill/pivot signal:**** if Anthropic ships account-level **local** session sync with automatic path remapping across OSes (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

Rather than another flat list, let me give you territories — the concept first, then names that live in it. Most of these are ground we haven't touched.

1. Sleep / wake (OS vocabulary)

This is the most conceptually exact territory available and we've completely skipped it. *Suspend* and *resume* are literally what operating systems call saving state to disk and restoring it. Hibernation writes memory out so the machine can wake up exactly where it was. That's your product, described by an existing technical vocabulary your users already know.

→ **Hibernate, Reveille** (the bugle call that wakes the camp — beautiful and unclaimed), **Rouse, Torpor, Waking**

2. Holding your place in a book

Not a metaphor for syncing — a metaphor for *resuming*, which is the emotional core of the product.

→ **Dogear** (folding the corner to mark where you stopped — nearly perfect), **Ribbon** (the sewn bookmark in a good hardback), **Marginalia**

3. Banking a fire

To *bank* a fire is to cover the coals at night so they survive till morning and can be revived. Your desktop session isn't dead when you close the laptop — it's banked. Ancient, warm, and totally unmined.

→ **Banked, Hearth, Coalbed**. (Ember and Kindle are gone — Ember.js and Amazon.)

4. Music: continuing the piece

Musical notation has precise words for pausing and picking back up.

→ **Segue** (proceed to the next without pause — semantically ideal), **Reprise** (the return of an earlier theme), **Fermata** (a held pause; the music resumes after)

5. Relay stations on a long road

Before trains, long journeys worked by swapping to a fresh horse at a waystation and continuing the *same* trip. Desktop → laptop is a remount.

→ **Serai** (short for caravanserai, the roadside inn), **Remount, Waystation, Posthouse**

6. Knitting: live stitches

When a knitter puts work down, the unfinished stitches are called *live stitches*, and a **lifeline** is a thread run through them so you can always come back safely. It's the thread metaphor you liked, from an angle nobody uses.

→ **Lifeline, Livestitch, Holder**

7. Sanskrit and Hindi roots

You brought up Chaayos yourself, so this feels invited — and it's the one territory that would make you genuinely un-copyable. Some of these are uncannily on-point:

→ **Sutra** — literally means *thread* in Sanskrit, and is globally recognizable. Probably the single best word in this entire conversation for your product.

→ **Sandhi** — the Sanskrit grammatical rule for how words join at their boundaries. For a tool that joins sessions across machine boundaries, that's exquisite.

→ **Padav** (पद) — a halt or camp on a journey. It *is* "layover," in Hindi.

→ **Dhaga** (धागा) — thread, plainly.

→ **Tantu** — filament, thread.

8. Film continuity

Movies shoot out of order across weeks; an entire crew role exists to ensure the next shot matches the last.

→ **Matchcut, Continuity, Throughline**

Conflicts I already know about: Ember (Ember.js), Flyway (DB migrations), Coda (Coda.io), Setu (Indian fintech, so I left it out), Lifeline (worth checking, feels grabbable).

My three favorites from this batch: Sutra — it means thread, it's short,

it's yours culturally, and it sounds like infrastructure. **Dogear** — the most human description of what your product does. **Reveille** — the wake-up call, which is exactly what you're doing to a sleeping session.

Which territory pulls you? I can go deep on any one of them.

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

```
# Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape
```

```
## TL;DR
```

```
- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows↔macOS paths.
```

```
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows↔macOS sync.
```

```
- **Recommendation: build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode.** Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.
```

```
## Key Findings
```

```
1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a
```

MacBook.

2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...\` and `/Users/...\` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.

3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.

4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

Claude Code (Anthropic). Ships the richest consumer-facing multi-device story:

- **/teleport** (`claude --teleport`): pulls a *cloud* session (started at claude.ai/code or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport`, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires **clean git state** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same claude.ai account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."
- **Remote Control** (`claude remote-control` / `/remote-control`, `--rc`): pushes a *live local* session to phone/web. The session runs locally the whole time; web/mobile are "a window

into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an **undocumented** ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `ANTHROPIC\_BASE\_URL` is redirected; unavailable under Zero Data Retention.

- **Claude Code on the web** (claude.ai/code): runs on Anthropic cloud infra, no local filesystem.
- **Residual gap:** No true device-to-device **local** session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an **unexpected**, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

Codex CLI (OpenAI). The most architecturally mature cross-surface model:

- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared **App Server** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under `~/codex/sessions/YYYY/MM/DD/rollout-\*.jsonl`, with a `session\_meta` first line.
- Rich lifecycle: `codex resume` (picker), `--last`, `--resume-session-id`, `/fork`, archive/unarchive (v0.136), delete (v0.140). Imports histories from other agents via an `external-agent-sessions` crate.
- **Known gaps/bugs:** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing `encrypted\_content` often can't be continued after crossing provider/account (`invalid\_encrypted\_content`); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.
- **Residual gap:** Sync is OpenAI-account-bound and cloud-

mediated; no cross-vendor; local file portability still hits path issues.

****Gemini CLI (Google).**** Auto-saves sessions to `~/.gemini/tmp/<project_hash>/chats/`` (project-scoped, hash of project root). `/resume`` session browser, `--resume`/`-r`` flag, `/chat save`` checkpoints, and a shadow-git checkpointing system under `~/.gemini/history/<project_hash>``. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. ****Residual gap:**** everything local + path-hash-keyed; no sync.

****opencode (sst).**** SQLite + file storage under `~/.local/share/opencode/`` (`opencode.db``, `storage/session|message|part``). Native ****session sharing**** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). ****Residual gap:**** sharing $\neq$ syncing; DB concurrency and cross-OS path issues remain.

****Cursor CLI (cursor-agent).**** Chats live in local SQLite; CLI chats under `~/.cursor/chats`` and resume via `cursor-agent --resume``. ****IDE chats and CLI chats do NOT share a session store**** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend `&`` to push to a Cloud Agent, pick up at `cursor.com/agents`), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). ****Residual gap:**** no local cross-device sync, IDE ↔ CLI split, path issues.

****Grok / Grok Build (xAI).**** Sessions auto-saved to `~/.grok/sessions/``, keyed by working directory. `/resume``, `--resume``, `-c``, `/fork``; subscription-gated (SuperGrok/X Premium+). Config in `~/.grok/config.toml``. No native cross-device sync. Still beta (v0.2.x as of July 2026). ****Residual gap:**** no sync at all.

****GitHub Copilot CLI.**** Sessions in `~/.copilot/`` (`session-state/<uuid>/events.jsonl``). Notably, Copilot CLI ****syncs sessions to your GitHub account by default**** and offers ****remote control**** from GitHub.com/GitHub Mobile (origin machine must stay online, `/keep-alive``). But ****cross-device *resume* is still not**

supported** – manually copying a session dir and running `--resume` overwrites `events.jsonl` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session\_id`. **Residual gap:** the sync is telemetry/history, not resumable working state across machines.

Amp (Sourcegraph). Threads sync to ampcode.com by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). **Residual gap:** vendor-locked, server-stored, no cross-vendor.

Cross-vendor summary: Every vendor is building *within* its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

Tool	Location	Format	Path keying	Cross-device resume native?
Claude Code	~/.claude/projects/<encoded-path>/<session>.jsonl	JSONL, append-only event stream	**Absolute path, non-alphanumerics→dashes**	Via cloud only (teleport/RC)
Codex	~/.codex/sessions/YYYY/MM/DD/rollout-*.jsonl	JSONL + `session_meta` header	date-based; cwd in meta	Yes, via App Server
Gemini CLI	~/.gemini/tmp/<project_hash>/chats/`	JSON	**Hash of project root**	No
opencode	~/.local/share/opencode/`	(SQLite + files)	SQLite/JSON	project-keyed Sharing only
Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)
Grok	~/.grok/sessions/`	JSON	working directory	No
Copilot CLI	~/.copilot/session-state/<uuid>/events.jsonl	JSONL events	uuid	Account history sync; no resume

The path-mapping problem (the crux). Claude Code derives the project folder name by escaping the absolute path: `~/Users/me/app` → `~/.claude/projects/-Users-me-app/`, while the

same repo at `C:\Users\me\app` (or a different macOS username/location) produces a **different** folder. Consequently, after a naive file sync, `claude --resume` on the second machine cannot find the first machine's sessions. This is confirmed by the claude-sync author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker." Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via `project\_hash`; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. ****This is the single highest-value, currently-unmet technical feature.****

****Format churn risk.**** Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

****Mid-session cross-device resume feasibility (same vendor):**** Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present – **if** path keys are remapped. Codex's App Server makes it cloud-native already.

****Cross-vendor session replay (e.g., Claude Code session → Codex):**** ****Fundamentally impractical.**** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) `encrypted\_content` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. ****Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.****

3. Competitor teardown

****Session-continuity tools:****

- ****tawanorg/claude-sync**** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/.claude` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + ****age E2E encryption**** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.
- ****npow/session-sync**** (~1 star, very new): the only cross-vendor *session* translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- ****dantelex/aisync ("ss")****: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- ****codsync/codsync**** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- ****kroxiksut/codexSync****: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- ****waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)**** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- ****iHildy/opencode-synced**** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- ****Dinesh3184/claude-session-sync****: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- ****renefichtmueller/claude-sync****: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

****Config/MCP/skills sync (DevSync's neighborhood):****

- ****Leoyangl83/sync-agents-settings**** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- ****nicepkg/vsync**** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- ****baranovxyz/agentsync**** and ****spxrogers/agentsync**** (beta): canonical `~/.agents/`` config synced to 9–31 agents.

- **neon-solutions/add-mcp**: one-command MCP install across ~16 agents (Neon).
- **ztripez/mcp-sync**: MCP config sync across tools.
- **AgentSync (Rust)**: symlink-based config sync.
- **skills-sync**: skills + MCP workspace.
- **DevSynq** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

URL-addressable / mobile-control / viewer approaches:

- **TokenRip** (tokenrip.com): makes session state a URL-addressable artifact (`/a/<slug>`) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- **Omnara** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was **archived** in February 2026 because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)
- **Syncode – AI Programming Tool**: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- **jazzyalex/agent-sessions** & **kenn-io/agentsview**: local-first session browsers/search across 20+ agents (read/resume-command, not sync).
- **LAN-workspace approaches** and Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

Takeaway: The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- **Claude Code**: #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any

client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.

- **Codex:** #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI ↔ app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
- **Copilot CLI:** #1947 (cloud-synced sessions), #1635 (cross-environment resume).
- **Cursor forum:** multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is. Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. **Cross-vendor unified profile** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.

2. **OS-aware path remapping as a first-class feature** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows ↔ macOS. This is the wedge no competitor has nailed.

3. **E2E encryption by default** – table stakes given session files contain code, secrets, and full transcripts.

Risks:

- **Format churn:** Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).
- **Vendor lock-out / ToS:** Storing session data is user-owned

data (fine), but **auth tokens** are not transferable (codexSync explicitly refuses; Codex ``encrypted_content`` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.

- ****Vendors closing the gap:**** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This **validates** the demand while **confirming** that only the cross-vendor + cross-OS layer is defensible.

- ****Security/privacy expectations:**** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).

- ****Honest limits:**** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

****Stage 0 – Fold in DevSync (immediate).**** Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

****Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge).**** Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/ .claude/projects/`` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- ****First-class OS-aware path remapping:**** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.

- ****Benchmark to advance:**** ``claude --resume`` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

****Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode.**** Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid ``encrypted_content`/token` pitfalls. For Gemini, remap ``project_hash``. For opencode, use safe SQLite snapshotting (the ``sqlite3 .backup`` pattern the community scripts use).

- ****Benchmark:**** each new adapter must pass the same cross-OS resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One "profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not replay.

****Monetization & thresholds.**** Free/open-source core (build trust, match claude-sync) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSynq/OpenSync validate willingness to pay). ****Kill/pivot signal:**** if Anthropic ships account-level **local** session sync with automatic path remapping across OSes (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is

unverified.

- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

how about remount?

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape

TL;DR

- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows↔macOS paths.
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows↔macOS sync.
- **Recommendation:** build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode.** Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...\` and `/Users/...\` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.
3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.
4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No

tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

- Claude Code (Anthropic).** Ships the richest consumer-facing multi-device story:
 - teleport** (`claude --teleport`): pulls a *cloud* session (started at `claude.ai/code` or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport`, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires **clean git state** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same `claude.ai` account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."
 - Remote Control** (`claude remote-control` / `/remote-control`, `--rc`): pushes a *live local* session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an *undocumented* ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `ANTHROPIC_BASE_URL` is redirected; unavailable under Zero Data Retention.
 - Claude Code on the web** (`claude.ai/code`): runs on Anthropic cloud infra, no local filesystem.
 - Residual gap:** No true device-to-device *local* session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an *unexpected*, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.
 - Codex CLI (OpenAI).** The most architecturally mature cross-surface model:
 - CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared **App Server** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under `~/.codex/sessions/YYYY/MM/DD/rollout-*.jsonl`, with a `session_meta` first line.
 - Rich lifecycle: `codex resume` (picker), `--last`, `--resume-session-id`, `/fork`, `archive/unarchive` (v0.136), `delete` (v0.140). Imports histories from other agents via an `external-agent-sessions` crate.
 - Known gaps/bugs:** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing `encrypted_content` often can't be continued after crossing provider/account (`invalid_encrypted_content`); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.
 - Residual gap:** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.
 - Gemini CLI (Google).** Auto-saves sessions to `~/.gemini/tmp/<project_hash>/chats/` (project-scoped, hash of project root). `/resume` session browser, `--resume` `/-r` flag, `/chat save` checkpoints, and a shadow-git checkpointing system under `~/.gemini/history/<project_hash>`. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. **Residual gap:** everything local + path-hash-keyed; no sync.
 - opencode (sst).** SQLite + file storage under `~/.local/share/opencode/` (`opencode.db`, `storage/session|message|part`). Native **session sharing** publishes to Cloudflare Durable

Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). **Residual gap:** sharing ≠ syncing; DB concurrency and cross-OS path issues remain.

Cursor CLI (cursor-agent). Chats live in local SQLite; CLI chats under `~/cursor/chats` and resume via `cursor-agent --resume`. **IDE chats and CLI chats do NOT share a session store** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend `&` to push to a Cloud Agent, pick up at cursor.com/agents), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). **Residual gap:** no local cross-device sync, IDE↔CLI split, path issues.

Grok / Grok Build (xAI). Sessions auto-saved to `~/grok/sessions/`, keyed by working directory. `~/resume`, `--resume`, `-c`, `~/fork`; subscription-gated (SuperGrok/X Premium+). Config in `~/grok/config.toml`. No native cross-device sync. Still beta (v0.2.x as of July 2026). **Residual gap:** no sync at all.

GitHub Copilot CLI. Sessions in `~/copilot/` (`session-state/<uuid>/events.jsonl`). Notably, Copilot CLI **syncs sessions to your GitHub account by default** and offers **remote control** from GitHub.com/GitHub Mobile (origin machine must stay online, `~/keep-alive`). But **cross-device resume** is still not supported – manually copying a session dir and running `--resume` overwrites `events.jsonl` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session\_id`. **Residual gap:** the sync is telemetry/history, not resumable working state across machines.

Amp (Sourcegraph). Threads sync to ampcode.com by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). **Residual gap:** vendor-locked, server-stored, no cross-vendor.

Cross-vendor summary: Every vendor is building *within* its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

	Tool	Location	Format	Path keying	Cross-device resume native?
					--- --- --- --- ---
	Claude Code	~/claude/projects/<encoded-path>/<session>.jsonl`	JSONL, append-only event stream	Absolute path, non-alphanumerics→dashes	Via cloud only (teleport/RC)
	Codex	~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl`	JSONL + `session_meta` header	date-based; cwd in meta	Yes, via App Server
	Gemini CLI	~/gemini/tmp/<project_hash>/chats/`	JSON	Hash of project root	No
	opencode	~/local/share/opencode/`	(SQLite + files)	SQLite/JSON	project-keyed Sharing only
	Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)
	Grok	~/grok/sessions/`	JSON	working directory	No
	Copilot CLI	~/copilot/session-state/<uuid>/events.jsonl`	JSONL events	uuid	Account history sync; no resume

The path-mapping problem (the crux). Claude Code derives the project folder name by escaping the absolute path: `/Users/me/app` → `~/claude/projects/-Users-me-app/`, while the same repo at `C:\Users\me\app` (or a different macOS username/location) produces a *different* folder. Consequently, after a naive file sync, `claude --resume` on the second machine cannot find the first machine's sessions. This is confirmed by the claude-sync author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker." Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via `project\_hash`; Cursor stores path metadata in

chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. ****This is the single highest-value, currently-unmet technical feature.****

****Format churn risk.**** Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

****Mid-session cross-device resume feasibility (same vendor):**** Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present – ***if*** path keys are remapped. Codex's App Server makes it cloud-native already.

****Cross-vendor session replay (e.g., Claude Code session → Codex):**** ****Fundamentally impractical.**** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) ``encrypted_content`` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. ****Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.****

3. Competitor teardown

****Session-continuity tools:****

- ****tawanorg/claude-sync**** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/.claude`` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + ****age E2E encryption**** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.
- ****npow/session-sync**** (~1 star, very new): the only cross-vendor ***session*** translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- ****dantelex/aisync ("ss")****: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- ****codsync/codsync**** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- ****kroxiksut/codexSync****: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- ****waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)**** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- ****iHildy/opencode-synced**** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- ****Dinesh3184/claude-session-sync****: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- ****renefichtmueller/claude-sync****: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

****Config/MCP/skills sync (DevSync's neighborhood):****

- ****Leoyang183/sync-agents-settings**** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- ****nicepkg/vsync**** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- ****baranovxyz/agentsync**** and ****spxrogers/agentsync**** (beta): canonical ``.agents/`` config synced to 9–31 agents.
- ****neon-solutions/add-mcp****: one-command MCP install across ~16 agents (Neon).
- ****ztripez/mcp-sync****: MCP config sync across tools.
- ****AgentSync (Rust)****: symlink-based config sync.
- ****skills-sync****: skills + MCP workspace.
- ****DevSynq**** (devsynq.app): commercial-looking MCP/API-key sync across IDEs +

launcher/marketplace.

****URL-addressable / mobile-control / viewer approaches:****

- ****TokenRip**** (tokenrip.com): makes session state a URL-addressable artifact (`/a/<slug>`) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- ****Omnara**** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was ****archived in February 2026**** because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)
- ****Syncode – AI Programming Tool****: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- ****jazzyalex/agent-sessions**** & ****kenn-io/agentsview****: local-first session browsers/search across 20+ agents (read/resume-command, not sync).
- ****LAN-workspace approaches**** and Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

****Takeaway:**** The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- ****Claude Code:**** #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- ****Codex:**** #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI  app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
- ****Copilot CLI:**** #1947 (cloud-synced sessions), #1635 (cross-environment resume).
- ****Cursor forum:**** multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

****Where the durable moat is.**** Vendors will keep building **intra**-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. ****Cross-vendor unified profile**** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.
2. ****OS-aware path remapping as a first-class feature**** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows  macOS. This is the wedge no competitor has nailed.
3. ****E2E encryption by default**** – table stakes given session files contain code, secrets, and full transcripts.

****Risks:****

- ****Format churn:**** Claude Code's format "changes between versions" by Anthropic's own

admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write — the pattern `codexSync/cursaves` use).

- **Vendor lock-out / ToS:** Storing session data is user-owned data (fine), but *auth tokens* are not transferable (codexSync explicitly refuses; Codex `encrypted_content` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR — don't build on them.
- **Vendors closing the gap:** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This *validates* the demand while *confirming* that only the cross-vendor + cross-OS layer is defensible.
- **Security/privacy expectations:** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote — all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).
- **Honest limits:** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

Stage 0 — Fold in DevSync (immediate). Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

Stage 1 — MVP: Claude Code session sync with path remapping + E2E encryption (the wedge). Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/ .claude/projects/` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- **First-class OS-aware path remapping:** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks — lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.

- **Benchmark to advance:** `claude --resume`` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

Stage 2 — Second and third vendors: Codex + Gemini CLI + opencode. Add adapters. For Codex, cooperate with (don't fight) the App Server — sync what's local and useful, avoid `encrypted_content`/token` pitfalls. For Gemini, remap `project_hash``. For opencode, use safe SQLite snapshotting (the `sqlite3 .backup`` pattern the community scripts use).

- **Benchmark:** each new adapter must pass the same cross-OS resume test as Claude Code before GA.

Stage 3 — Unified profile + cross-vendor context handoff. One "profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves — explicitly labeled as context transfer, not replay.

Monetization & thresholds. Free/open-source core (build trust, match claude-sync) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSynq/OpenSync validate willingness to pay). **Kill/pivot signal:** if Anthropic ships account-level *local* session sync with automatic path remapping across OSes (not just history mirroring, per #71794), the single-vendor wedge collapses — at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

What NOT to do: Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources — treat as ~40-116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats — especially Claude Code's — are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot

sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.

- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

Interesting one, because the concept is **better** than the horse metaphor I originally pitched it with — and the word is more crowded than I realized.

Why it's conceptually excellent. Forget waystations. In Unix, `mount` means *making content available at a path*. And your product's entire moat — the thing the research flagged as the one unsolved problem — is taking the same session content and making it available at a *different* path on a different machine. That's not a metaphor for mounting. That's mounting. The name describes your differentiator more precisely than anything else in this conversation.

Why it's a problem anyway. The word is already doing two jobs in your users' daily vocabulary:

- **Unix:** `mount -o remount,rw /` means re-mounting a filesystem with changed options. Established, specific, different.
- **React:** forcing a component remount via the `key` prop is everyday JS vocabulary — there are whole blog posts on how to remount a component on URL or prop change. Your users are largely JS/TS devs.

And the npm name is gone: `remount` is taken by rstacruz — "Mount React components to the DOM using custom elements," with 3 dependent projects. It's a real, documented package with a GitHub repo. Dormant (v1.0.0, three years quiet), but npm doesn't recycle names, so you'd be shipping `@you/remount` or a variant — friction in the one place it hurts, your install command.

Verdict: usable, not clean. This is the Rebranch critique but softer — with Rebranch the meaning was *inverted*, which was fatal. Here the meaning is compatible, arguably perfect. The problem is purely occupancy: you'd spend forever competing with man pages and React docs, and you'd never own the term.

The upgrade, if you like this direction: look at `mount --bind`. A **bind mount** in Linux takes a directory and makes it appear at a second path — same content, different location. That is *literally* your path-remapping feature, described by an existing kernel primitive.

- **Bindmount** — exact, technical, credible to the audience that matters. Nine characters, a bit dense.
- **Bindpoint** — coined from bind + mountpoint. Cleaner, still legible, likely wide open.

- **Bind** — short and clicky, but too generic and definitely contested (DNS BIND, among others).

If you want my honest ranking right now: **Bindpoint** for technical precision, **Layover** for warmth and memorability, **Reinstate** as the safe strong fallback. Remount sits just below all three — great idea, occupied word.

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

```
# Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape
```

```
## TL;DR
```

```
- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows↔macOS paths.
```

```
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows↔macOS sync.
```

```
- **Recommendation: build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode.** Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.
```

```
## Key Findings
```

```
1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who
```

mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.

2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...\` and `/Users/...\` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.

3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.

4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

Claude Code (Anthropic). Ships the richest consumer-facing multi-device story:

- **`/teleport`** (``claude --teleport``): pulls a *cloud* session (started at claude.ai/code or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport``, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires **clean git state** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same claude.ai account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."
- **Remote Control** (``claude remote-control` / `/remote-control`, --rc`): pushes a live local session to phone/web.`

The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an **undocumented** ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `ANTHROPIC\_BASE\_URL` is redirected; unavailable under Zero Data Retention.

- **Claude Code on the web** (claude.ai/code): runs on Anthropic cloud infra, no local filesystem.

- **Residual gap:** No true device-to-device **local** session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an **unexpected**, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

Codex CLI (OpenAI). The most architecturally mature cross-surface model:

- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared **App Server** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under

`~/.codex/sessions/YYYY/MM/DD/rollout-\*.jsonl`, with a `session\_meta` first line.

- Rich lifecycle: `codex resume` (picker), `--last`, `--resume-session-id`, `/fork`, archive/unarchive (v0.136), delete (v0.140). Imports histories from other agents via an `external-agent-sessions` crate.

- **Known gaps/bugs:** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing `encrypted\_content` often can't be continued after crossing provider/account (`invalid\_encrypted\_content`); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.

- **Residual gap:** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

Gemini CLI (Google). Auto-saves sessions to `~/gemini/tmp/<project_hash>/chats/`` (project-scoped, hash of project root). `/resume`` session browser, `--resume`/`-r`` flag, `/chat save`` checkpoints, and a shadow-git checkpointing system under `~/gemini/history/<project_hash>``. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. **Residual gap:** everything local + path-hash-keyed; no sync.

opencode (sst). SQLite + file storage under `~/local/share/opencode/`` (`opencode.db``, `storage/session|message|part``). Native **session sharing** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). **Residual gap:** sharing $\neq$ syncing; DB concurrency and cross-OS path issues remain.

Cursor CLI (cursor-agent). Chats live in local SQLite; CLI chats under `~/cursor/chats`` and resume via `cursor-agent --resume``. **IDE chats and CLI chats do NOT share a session store** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend `&`` to push to a Cloud Agent, pick up at `cursor.com/agents`), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). **Residual gap:** no local cross-device sync, IDE ↔ CLI split, path issues.

Grok / Grok Build (xAI). Sessions auto-saved to `~/grok/sessions/``, keyed by working directory. `/resume``, `--resume``, `-c``, `/fork``; subscription-gated (SuperGrok/X Premium+). Config in `~/grok/config.toml``. No native cross-device sync. Still beta (v0.2.x as of July 2026). **Residual gap:** no sync at all.

GitHub Copilot CLI. Sessions in `~/copilot/`` (`session-state/<uuid>/events.jsonl``). Notably, Copilot CLI **syncs sessions to your GitHub account by default** and offers **remote control** from `GitHub.com/GitHub Mobile` (origin machine must stay

online, `/keep-alive`). But ****cross-device *resume* is still not supported**** – manually copying a session dir and running `--resume` overwrites `events.jsonl` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session\_id`. ****Residual gap:**** the sync is telemetry/history, not resumable working state across machines.

****Amp (Sourcegraph).**** Threads sync to ampcode.com by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). ****Residual gap:**** vendor-locked, server-stored, no cross-vendor.

****Cross-vendor summary:**** Every vendor is building **within** its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

Tool	Location	Format	Path keying	Cross-device resume native?
Claude Code	~/ .claude/projects/<encoded-path>/<session>.jsonl`	JSONL, append-only event stream	**Absolute path, non-alphanumerics→dashes**	Via cloud only (teleport/RC)
Codex	~/ .codex/sessions/YYYY/MM/DD/rollout-*.jsonl`	JSONL + `session_meta` header	date-based; cwd in meta	Yes, via App Server
Gemini CLI	~/ .gemini/tmp/<project_hash>/chats/`	JSON	**Hash of project root**	No
opencode	~/ .local/share/opencode/`	(SQLite + files)	SQLite/JSON	project-keyed
Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)
Grok	~/ .grok/sessions/`	JSON	working directory	No
Copilot CLI	~/ .copilot/session-state/<uuid>/events.jsonl`	JSONL events	uuid	Account history sync; no resume

****The path-mapping problem (the crux).**** Claude Code derives the project folder name by escaping the absolute path:

`/Users/me/app` → `~/.claude/projects/-Users-me-app/`, while the same repo at `C:\Users\me\app` (or a different macOS username/location) produces a *different* folder. Consequently, after a naive file sync, `claude --resume` on the second machine cannot find the first machine's sessions. This is confirmed by the claude-sync author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker." Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via `project\_hash`; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. ****This is the single highest-value, currently-unmet technical feature.****

****Format churn risk.**** Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

****Mid-session cross-device resume feasibility (same vendor):**** Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present — *if* path keys are remapped. Codex's App Server makes it cloud-native already.

****Cross-vendor session replay (e.g., Claude Code session → Codex):**** ****Fundamentally impractical.**** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) `encrypted\_content` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. ****Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.****

3. Competitor teardown

****Session-continuity tools:****

- ****tawanorg/claude-sync**** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/.claude` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + ****age E2E encryption**** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.
- ****npow/session-sync**** (~1 star, very new): the only cross-vendor *session* translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- ****dantelex/aisync ("ss")****: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- ****codsync/codsync**** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- ****kroxiksut/codexSync****: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- ****waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)**** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- ****iHildy/opencode-synced**** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- ****Dinesh3184/claude-session-sync****: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- ****renefichtmueller/claude-sync****: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

****Config/MCP/skills sync (DevSync's neighborhood):****

- ****Leoyangl83/sync-agents-settings**** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- ****nicepkg/vsync**** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- ****baranovxyz/agentsync**** and ****spxrogers/agentsync**** (beta): canonical `~.agents/`` config synced to 9–31 agents.

- **neon-solutions/add-mcp**: one-command MCP install across ~16 agents (Neon).
- **ztripez/mcp-sync**: MCP config sync across tools.
- **AgentSync (Rust)**: symlink-based config sync.
- **skills-sync**: skills + MCP workspace.
- **DevSynq** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

URL-addressable / mobile-control / viewer approaches:

- **TokenRip** (tokenrip.com): makes session state a URL-addressable artifact (`/a/<slug>`) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- **Omnara** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was **archived** in February 2026 because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)
- **Syncode – AI Programming Tool**: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- **jazzyalex/agent-sessions** & **kenn-io/agentsview**: local-first session browsers/search across 20+ agents (read/resume-command, not sync).
- **LAN-workspace approaches** and Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

Takeaway: The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- **Claude Code**: #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any

client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.

- **Codex:** #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI ↔ app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
- **Copilot CLI:** #1947 (cloud-synced sessions), #1635 (cross-environment resume).
- **Cursor forum:** multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is. Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. **Cross-vendor unified profile** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.

2. **OS-aware path remapping as a first-class feature** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows ↔ macOS. This is the wedge no competitor has nailed.

3. **E2E encryption by default** – table stakes given session files contain code, secrets, and full transcripts.

Risks:

- **Format churn:** Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).
- **Vendor lock-out / ToS:** Storing session data is user-owned

data (fine), but **auth tokens** are not transferable (codexSync explicitly refuses; Codex ``encrypted_content`` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.

- ****Vendors closing the gap:**** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This **validates** the demand while **confirming** that only the cross-vendor + cross-OS layer is defensible.

- ****Security/privacy expectations:**** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).

- ****Honest limits:**** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

****Stage 0 – Fold in DevSync (immediate).**** Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

****Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge).**** Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/\.claude/projects/`` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- ****First-class OS-aware path remapping:**** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.

- ****Benchmark to advance:**** ``claude --resume`` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

****Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode.**** Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid ``encrypted_content`/token` pitfalls. For Gemini, remap ``project_hash``. For opencode, use safe SQLite snapshotting (the ``sqlite3 .backup`` pattern the community scripts use).

- ****Benchmark:**** each new adapter must pass the same cross-OS resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One "profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not replay.

****Monetization & thresholds.**** Free/open-source core (build trust, match claude-sync) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSynq/OpenSync validate willingness to pay). ****Kill/pivot signal:**** if Anthropic ships account-level **local** session sync with automatic path remapping across OSes (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is

unverified.

- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

OpenSync?

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape

TL;DR

- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows↔macOS paths.
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows↔macOS sync.
- **Recommendation:** build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode. Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...\` and `/Users/...\` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.
3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.

4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

Claude Code (Anthropic). Ships the richest consumer-facing multi-device story:

- **teleport** (`claude --teleport`): pulls a *cloud* session (started at `claude.ai/code` or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport`, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires **clean git state** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same `claude.ai` account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."
- **Remote Control** (`claude remote-control` / `/remote-control`, `--rc`): pushes a *live local* session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an *undocumented* ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `ANTHROPIC_BASE_URL` is redirected; unavailable under Zero Data Retention.
- **Claude Code on the web** (`claude.ai/code`): runs on Anthropic cloud infra, no local filesystem.
- **Residual gap:** No true device-to-device *local* session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an *unexpected*, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

Codex CLI (OpenAI). The most architecturally mature cross-surface model:

- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared **App Server** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under `~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl`, with a `session_meta` first line.
- Rich lifecycle: `codex resume` (picker), `--last`, `--resume-session-id`, `/fork`, `archive/unarchive` (v0.136), `delete` (v0.140). Imports histories from other agents via an `external-agent-sessions` crate.
- **Known gaps/bugs:** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing `encrypted_content` often can't be continued after crossing provider/account (`invalid_encrypted_content`); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.
- **Residual gap:** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

Gemini CLI (Google). Auto-saves sessions to `~/gemini/tmp/<project_hash>/chats/` (project-scoped, hash of project root). `/resume` session browser, `--resume` `/-r` flag, `/chat save` checkpoints, and a shadow-git checkpointing system under `~/gemini/history/<project_hash>`. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. **Residual gap:** everything local + path-hash-keyed; no sync.

opencode (sst). SQLite + file storage under `~/local/share/opencode/` (`opencode.db`,

`storage/session|message|part`). Native **session sharing** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). **Residual gap:** sharing ≠ syncing; DB concurrency and cross-OS path issues remain.

Cursor CLI (cursor-agent). Chats live in local SQLite; CLI chats under `~/cursor/chats` and resume via `cursor-agent --resume`. **IDE chats and CLI chats do NOT share a session store** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend `&` to push to a Cloud Agent, pick up at cursor.com/agents), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). **Residual gap:** no local cross-device sync, IDE↔CLI split, path issues.

Grok / Grok Build (xAI). Sessions auto-saved to `~/grok/sessions/`, keyed by working directory. `resume`, `--resume`, `-c`, `/fork`; subscription-gated (SuperGrok/X Premium+). Config in `~/grok/config.toml`. No native cross-device sync. Still beta (v0.2.x as of July 2026). **Residual gap:** no sync at all.

GitHub Copilot CLI. Sessions in `~/copilot/` (`session-state/<uuid>/events.jsonl`). Notably, Copilot CLI **syncs sessions to your GitHub account by default** and offers **remote control** from GitHub.com/GitHub Mobile (origin machine must stay online, `/keep-alive`). But **cross-device resume** is still not supported – manually copying a session dir and running `--resume` overwrites `events.jsonl` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session\_id`. **Residual gap:** the sync is telemetry/history, not resumable working state across machines.

Amp (Sourcegraph). Threads sync to ampcode.com by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). **Residual gap:** vendor-locked, server-stored, no cross-vendor.

Cross-vendor summary: Every vendor is building *within* its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

	Tool	Location	Format	Path keying	Cross-device resume native?
					--- --- --- --- ---
	Claude Code	~/claude/projects/<encoded-path>/<session>.jsonl`	JSONL, append-only event stream	Absolute path, non-alphanumerics→dashes	Via cloud only (teleport/RC)
	Codex	~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl`	JSONL + `session_meta` header	date-based; cwd in meta	Yes, via App Server
	Gemini CLI	~/gemini/tmp/<project_hash>/chats/`	JSON	Hash of project root	No
	opencode	~/local/share/opencode/`	(SQLite + files)	SQLite/JSON	project-keyed
					Sharing only
	Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)
					Grok
		~/grok/sessions/`	JSON	working directory	No
	Copilot CLI	~/copilot/session-state/<uuid>/events.jsonl`	JSONL events	uuid	Account history sync; no resume

The path-mapping problem (the crux). Claude Code derives the project folder name by escaping the absolute path: `/Users/me/app` → `~/claude/projects/-Users-me-app/`, while the same repo at `C:\Users\me\app` (or a different macOS username/location) produces a *different* folder. Consequently, after a naive file sync, `claude --resume` on the second machine cannot find the first machine's sessions. This is confirmed by the claude-sync author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker." Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths

everywhere. Gemini CLI has the same issue via `project\_hash`; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. ****This is the single highest-value, currently-unmet technical feature.****

****Format churn risk.**** Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

****Mid-session cross-device resume feasibility (same vendor):**** Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present – ***if*** path keys are remapped. Codex's App Server makes it cloud-native already.

****Cross-vendor session replay (e.g., Claude Code session → Codex):**** ****Fundamentally impractical.**** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) `encrypted\_content` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. ****Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.****

3. Competitor teardown

****Session-continuity tools:****

- ****tawanorg/claude-sync**** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/.claude` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + ****age E2E encryption**** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.
- ****npow/session-sync**** (~1 star, very new): the only cross-vendor ***session*** translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- ****dantelex/aisync ("ss")****: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- ****codsync/codsync**** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- ****kroxiksut/codexSync****: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- ****waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)**** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- ****iHildy/opencode-synced**** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- ****Dinesh3184/claude-session-sync****: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- ****renefichtmueller/claude-sync****: multi-backend (Git/iCloud/Dropbox/SyncThing/rsync) with snapshots/rollback and encryption.

****Config/MCP/skills sync (DevSync's neighborhood):****

- ****Leoyang183/sync-agents-settings**** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- ****nicepkg/vsync**** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- ****baranovxyz/agentsync**** and ****spxrogers/agentsync**** (beta): canonical `~/.agents/` config synced to 9–31 agents.
- ****neon-solutions/add-mcp****: one-command MCP install across ~16 agents (Neon).
- ****ztripez/mcp-sync****: MCP config sync across tools.
- ****AgentSync (Rust)****: symlink-based config sync.
- ****skills-sync****: skills + MCP workspace.

- **DevSync** (devsync.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

URL-addressable / mobile-control / viewer approaches:

- **TokenRip** (tokenrip.com): makes session state a URL-addressable artifact (`/a/<slug>`) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- **Omnara** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was **archived** in February 2026 because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)
- **Syncode – AI Programming Tool**: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- **jazzya/agent-sessions** & **kenn-io/agentsview**: local-first session browsers/search across 20+ agents (read/resume-command, not sync).
- **LAN-workspace approaches** and **Syncthing/NAS/dotfiles/chezmoi**+age: DIY approaches that work for config but hit the path problem for sessions.

Takeaway: The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- **Claude Code**: #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- **Codex**: #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI ↔ app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
- **Copilot CLI**: #1947 (cloud-synced sessions), #1635 (cross-environment resume).
- **Cursor forum**: multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is. Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. **Cross-vendor unified profile** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.
2. **OS-aware path remapping as a first-class feature** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows  macOS. This is the wedge no competitor has nailed.
3. **E2E encryption by default** – table stakes given session files contain code, secrets, and full transcripts.

Risks:

- **Format churn:** Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern `codexSync/cursaves` use).
- **Vendor lock-out / ToS:** Storing session data is user-owned data (fine), but *auth tokens* are not transferable (`codexSync` explicitly refuses; Codex `encrypted_content` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.
 - **Vendors closing the gap:** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This *validates* the demand while *confirming* that only the cross-vendor + cross-OS layer is defensible.
- **Security/privacy expectations:** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).
- **Honest limits:** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

Stage 0 – Fold in DevSync (immediate). Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like `vsync` and `sync-agents-settings`).

Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge). Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/ .claude/projects/` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching `claude-sync`'s proven approach.
- **First-class OS-aware path remapping:** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique `cursaves` uses for Cursor). This is the feature `claude-sync` explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
- **Benchmark to advance:** `claude --resume` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode. Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid `encrypted_content`/token pitfalls. For Gemini, remap `project_hash`. For opencode, use safe SQLite snapshotting (the `sqlite3 .backup` pattern the community scripts use).

- **Benchmark:** each new adapter must pass the same cross-OS resume test as Claude Code before GA.

Stage 3 – Unified profile + cross-vendor context handoff. One "profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not replay.

Monetization & thresholds. Free/open-source core (build trust, match `claude-sync`) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSync/OpenSync validate willingness to pay). **Kill/pivot signal:** if Anthropic ships account-level *local* session sync with automatic path remapping across OSes (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

What NOT to do: Don't build on Remote Control/teleport (gated, one-way, origin-online,

with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini-Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

Hard no, on three counts — and one of them is in your own research report from earlier in this thread.

1. It's a trademarked major framework. OpenSync™ was launched by Plume and Samsung in 2018 under a BSD-3 license as a cloud-to-hardware abstraction layer for routers, gateways, and Wi-Fi CPEs. It's now backed by more than 30 CPE manufacturers and 400 service providers, managing 43M+ locations and 2B+ client devices. They own opensync.io, the GitHub org, an annual summit, and a certification program. That's not a name you share.

2. A direct competitor already uses it. Scroll back to the landscape report — under the opencode tooling section: waynesutton's [opencode-sync-plugin](#) and [codex-sync-plugin](#), branded **OpenSync**, with a SaaS at opensync.dev. That's session sync for coding agents. Same category, same problem, same name. You'd be launching into a collision with someone doing *your product*.

3. It's the opposite of what you asked for. Six turns ago you rejected CarryOn for being "very generic, like any other tech company." OpenSync is the two most saturated words in this entire category welded together — and we established that the [open-](#) prefix puts you in the commodity-infrastructure lane, not the brand lane you said you wanted.

A gentle observation. You've now circled back to a shape you already ruled out, which usually means naming fatigue rather than a real change of mind. You have three genuinely good, genuinely available options sitting right there: **Layover** (warm, memorable, category clear), **Reinstate** (the pun you loved, wide open, zero competition), and **Bindpoint** (technically precise, points at your moat).

Any of those three is good enough to ship. None of them will determine whether this works — the Windows  macOS path remapping will. If it helps: pick Reinstate today as a placeholder, register it, start building, and rename before launch if something better arrives. Names get much easier to judge once the thing exists.

Cross-Device Sync for Terminal AI Coding Agents: Competitive and

Technical Landscape

Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape

TL;DR

- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows ↔ macOS paths.
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows ↔ macOS sync.
- **Recommendation:** build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode. Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...\` and `/Users/...\` produces orphaned,

unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.

3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.

4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

Claude Code (Anthropic). Ships the richest consumer-facing multi-device story:

- **/teleport** (``claude --teleport``): pulls a *cloud* session (started at `claude.ai/code` or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with ``--teleport``, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires **clean git state** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same `claude.ai` account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."
- **Remote Control** (``claude remote-control`` / ``/remote-control``, ``--rc``): pushes a *live local* session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an

undocumented ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `ANTHROPIC\_BASE\_URL` is redirected; unavailable under Zero Data Retention.

- ****Claude Code on the web**** (claude.ai/code): runs on Anthropic cloud infra, no local filesystem.

- ****Residual gap:**** No true device-to-device ***local*** session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an ***unexpected***, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

****Codex CLI (OpenAI).**** The most architecturally mature cross-surface model:

- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared ****App Server**** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under `~/.codex/sessions/YYYY/MM/DD/rollout-\*.jsonl`, with a `session\_meta` first line.

- Rich lifecycle: `codex resume` (picker), `--last`, `--resume-session-id`, `/fork`, archive/unarchive (v0.136), delete (v0.140). Imports histories from other agents via an `external-agent-sessions` crate.

- ****Known gaps/bugs:**** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing `encrypted\_content` often can't be continued after crossing provider/account (`invalid\_encrypted\_content`); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.

- ****Residual gap:**** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

****Gemini CLI (Google).**** Auto-saves sessions to

`~/.gemini/tmp/<project\_hash>/chats/` (project-scoped, hash of project root). `/resume` session browser, `--resume`/`-r` flag,

`/chat save` checkpoints, and a shadow-git checkpointing system under `~/.gemini/history/<project\_hash>`. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. ****Residual gap:**** everything local + path-hash-keyed; no sync.

****opencode (sst).**** SQLite + file storage under `~/.local/share/opencode/` (`opencode.db`, `storage/session|message|part`). Native ****session sharing**** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). ****Residual gap:**** sharing $\neq$ syncing; DB concurrency and cross-OS path issues remain.

****Cursor CLI (cursor-agent).**** Chats live in local SQLite; CLI chats under `~/.cursor/chats` and resume via `cursor-agent --resume`. ****IDE chats and CLI chats do NOT share a session store**** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend `&` to push to a Cloud Agent, pick up at cursor.com/agents), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). ****Residual gap:**** no local cross-device sync, IDE ↔ CLI split, path issues.

****Grok / Grok Build (xAI).**** Sessions auto-saved to `~/.grok/sessions/`, keyed by working directory. `/resume`, `--resume`, `-c`, `/fork`; subscription-gated (SuperGrok/X Premium+). Config in `~/.grok/config.toml`. No native cross-device sync. Still beta (v0.2.x as of July 2026). ****Residual gap:**** no sync at all.

****GitHub Copilot CLI.**** Sessions in `~/.copilot/` (`session-state/<uuid>/events.jsonl`). Notably, Copilot CLI ****syncs sessions to your GitHub account by default**** and offers ****remote control**** from GitHub.com/GitHub Mobile (origin machine must stay online, `/keep-alive`). But ****cross-device *resume* is still not supported**** – manually copying a session dir and running `--resume` overwrites `events.jsonl` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session\_id`. ****Residual gap:**** the sync is telemetry/history, not resumable working state across machines.

****Amp (Sourcegraph).**** Threads sync to ampcode.com by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). ****Residual gap:**** vendor-locked, server-stored, no cross-vendor.

****Cross-vendor summary:**** Every vendor is building **within** its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

Tool	Location	Format	Path keying	Cross-device resume native?
Claude Code	`\${HOME}/.claude/projects/<encoded-path>/<session>.jsonl`	JSONL, append-only event stream	**Absolute path, non-alphanumerics→dashes**	Via cloud only (teleport/RC)
Codex	`\${HOME}/.codex/sessions/YYYY/MM/DD/rollout-*.jsonl`	JSONL + `session_meta` header	date-based; cwd in meta	Yes, via App Server
Gemini CLI	`\${HOME}/.gemini/tmp/<project_hash>/chats/`	JSON	**Hash of project root**	No
opencode	`\${HOME}/.local/share/opencode/`	(SQLite + files)	SQLite/JSON	project-keyed
Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)
Grok	`\${HOME}/.grok/sessions/`	JSON	working directory	No
Copilot CLI	`\${HOME}/.copilot/session-state/<uuid>/events.jsonl`	JSONL events	uuid	Account history sync; no resume

****The path-mapping problem (the crux).**** Claude Code derives the project folder name by escaping the absolute path: ``/Users/me/app`` → ``~/.claude/projects/-Users-me-app/``, while the same repo at ``C:\Users\me\app`` (or a different macOS username/location) produces a **different** folder. Consequently, after a naive file sync, ``claude --resume`` on the second machine cannot find the first machine's sessions. This is confirmed by the claude-sync author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation?"

Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker." Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via `project\_hash`; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. **This is the single highest-value, currently-unmet technical feature.**

Format churn risk. Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

Mid-session cross-device resume feasibility (same vendor): Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present – *if* path keys are remapped. Codex's App Server makes it cloud-native already.

Cross-vendor session replay (e.g., Claude Code session → Codex): **Fundamentally impractical.** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) `encrypted\_content` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. **Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.**

3. Competitor teardown

Session-continuity tools:

- **tawanorg/claude-sync** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/.claude` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + **age E2E encryption** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-

derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.

- ****npow/session-sync**** (~1 star, very new): the only cross-vendor *session* translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- ****dantelex/aisync ("ss")****: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- ****codsync/codsync**** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- ****kroxiksut/codexSync****: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- ****waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)**** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- ****iHildy/opencode-synced**** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- ****Dinesh3184/claude-session-sync****: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- ****renefichtmueller/claude-sync****: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

****Config/MCP/skills sync (DevSync's neighborhood):****

- ****Leoyangl83/sync-agents-settings**** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- ****nicepkg/vsync**** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- ****baranovxyz/agentsync**** and ****spxrogers/agentsync**** (beta): canonical ``.agents/`` config synced to 9–31 agents.
- ****neon-solutions/add-mcp****: one-command MCP install across ~16 agents (Neon).
- ****ztripez/mcp-sync****: MCP config sync across tools.
- ****AgentSync (Rust)****: symlink-based config sync.
- ****skills-sync****: skills + MCP workspace.
- ****DevSynq**** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

****URL-addressable / mobile-control / viewer approaches:****

- ****TokenRip**** (tokenrip.com): makes session state a URL-addressable artifact (``/a/<slug>``) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- ****Omnara**** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was ****archived in February 2026**** because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)
- ****Syncode – AI Programming Tool****: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- ****jazzyalex/agent-sessions**** & ****kenn-io/agentsview****: local-first session browsers/search across 20+ agents (read/resume-command, not sync).
- ****LAN-workspace approaches**** and Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

****Takeaway:**** The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- ****Claude Code:**** #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- ****Codex:**** #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI  app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
- ****Copilot CLI:**** #1947 (cloud-synced sessions), #1635 (cross-

environment resume).

- **Cursor forum:** multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is. Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. **Cross-vendor unified profile** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.

2. **OS-aware path remapping as a first-class feature** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows↔macOS. This is the wedge no competitor has nailed.

3. **E2E encryption by default** – table stakes given session files contain code, secrets, and full transcripts.

Risks:

- **Format churn:** Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).

- **Vendor lock-out / ToS:** Storing session data is user-owned data (fine), but *auth tokens* are not transferable (codexSync explicitly refuses; Codex `encrypted\_content` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.

- **Vendors closing the gap:** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This *validates* the demand while

confirming that only the cross-vendor + cross-OS layer is defensible.

- ****Security/privacy expectations:**** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).
- ****Honest limits:**** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

****Stage 0 – Fold in DevSync (immediate).**** Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

****Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge).**** Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/.claude/projects/` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- ****First-class OS-aware path remapping:**** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
- ****Benchmark to advance:**** `claude --resume` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

****Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode.**** Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid

`encrypted\_content`/token pitfalls. For Gemini, remap
`project\_hash`. For opencode, use safe SQLite snapshotting (the
`sqlite3.backup` pattern the community scripts use).
- ****Benchmark:**** each new adapter must pass the same cross-OS
resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One
"profile" = sessions + MCP + skills + agents + instructions,
portable across machines and (for config) across vendors. Add
best-effort context handoff (summarize + re-inject) for cross-
vendor moves – explicitly labeled as context transfer, not
replay.

****Monetization & thresholds.**** Free/open-source core (build
trust, match claude-sync) with a paid tier for managed encrypted
storage, multi-machine dashboards, and team profiles
(DevSynq/OpenSync validate willingness to pay). ****Kill/pivot
signal:**** if Anthropic ships account-level **local** session sync
with automatic path remapping across OSes (not just history
mirroring, per #71794), the single-vendor wedge collapses – at
that point double down on cross-vendor config/skills/profile
portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport
(gated, one-way, origin-online, with documented ~10-min network
and undocumented ~15-min idle timeouts). Don't promise cross-
vendor session replay. Don't sync auth tokens. Don't store
plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment

based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

Ok i have changed my mind i kind of wanna name that describes what it does you know like open router or open code they all sort of describe what they do but like you know open resume or open sync makes sense to an extent but doesn't really you know tell the entire product in one word which is sort of the goal but it won't be really possible to fit all of that in one word but we can try our best so let's you know brainstorm for such names that you know just describes our product and conveys our product and communicates our product through the name itself as much as it possibly can feel free to go in any direction it doesn't really need to start with open or doesn't really need to be a company name it can be like a generic any name basically that describes the product as you know as well as it possibly can

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape

TL;DR

- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows↔macOS paths.
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows↔macOS sync.
- **Recommendation:** build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode. Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...` and `/Users/...` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.
3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-

engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.

4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

- Claude Code (Anthropic).** Ships the richest consumer-facing multi-device story:
 - **teleport** (`claude --teleport`): pulls a *cloud* session (started at `claude.ai/code` or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport`, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires **clean git state** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same `claude.ai` account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."
 - **Remote Control** (`claude remote-control` / `/remote-control`, `--rc`): pushes a *live local* session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an *undocumented* ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `ANTHROPIC_BASE_URL` is redirected; unavailable under Zero Data Retention.
 - **Claude Code on the web** (`claude.ai/code`): runs on Anthropic cloud infra, no local filesystem.
 - **Residual gap:** No true device-to-device *local* session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an *unexpected*, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.
 - Codex CLI (OpenAI).** The most architecturally mature cross-surface model:
 - CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared **App Server** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under `~/.codex/sessions/YYYY/MM/DD/rollout-*.jsonl`, with a `session_meta` first line.
 - Rich lifecycle: `codex resume` (picker), `--last`, `--resume-session-id`, `/fork`, `archive/unarchive` (v0.136), `delete` (v0.140). Imports histories from other agents via an `external-agent-sessions` crate.
 - **Known gaps/bugs:** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing `encrypted_content` often can't be continued after crossing provider/account (`invalid_encrypted_content`); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.
 - **Residual gap:** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.
 - Gemini CLI (Google).** Auto-saves sessions to `~/.gemini/tmp/<project_hash>/chats/` (project-scoped, hash of project root). `/resume` session browser, `--resume` `/-r` flag, `/chat save` checkpoints, and a shadow-git checkpointing system under `~/.gemini/history/<project_hash>`. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. **Residual**

gap:** everything local + path-hash-keyed; no sync.

opencode (sst). SQLite + file storage under `~/local/share/opencode/`(`opencode.db`,
`storage/session|message|part`)`. Native **session sharing** publishes to Cloudflare Durable
Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-
party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync,
opencode-session-backup). **Residual gap:** sharing ≠ syncing; DB concurrency and cross-OS
path issues remain.

Cursor CLI (cursor-agent). Chats live in local SQLite; CLI chats under `~/cursor/chats``
and resume via `cursor-agent --resume``. **IDE chats and CLI chats do NOT share a session
store** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend `&`` to push to
a Cloud Agent, pick up at `cursor.com/agents`), and "My Machines" lets cloud agents run on your
registered machine. Strong user demand for account-level chat sync (forum). **Residual gap:**
no local cross-device sync, IDE↔CLI split, path issues.

Grok / Grok Build (xAI). Sessions auto-saved to `~/grok/sessions/``, keyed by working
directory. `/resume``, `--resume``, `-c``, `/fork``; subscription-gated (SuperGrok/X Premium+).
Config in `~/grok/config.toml``. No native cross-device sync. Still beta (v0.2.x as of July
2026). **Residual gap:** no sync at all.

GitHub Copilot CLI. Sessions in `~/copilot/`(`session-state/<uuid>/events.jsonl`)`.
Notably, Copilot CLI **syncs sessions to your GitHub account by default** and offers **remote
control** from `GitHub.com/GitHub Mobile` (origin machine must stay online, `/keep-alive``). But
cross-device resume is still not supported – manually copying a session dir and running
`--resume`` overwrites `events.jsonl`` and destroys history (issue #1635); cloud-synced cross-
device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via
a user-supplied `session_id``. **Residual gap:** the sync is telemetry/history, not resumable
working state across machines.

Amp (Sourcegraph). Threads sync to `ampcode.com` by default and can be continued across
devices; "orbs" (remote machines) keep running after you close your laptop. This is the
closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph
servers (a privacy consideration; MCP config is GUI-global, not file-based). **Residual gap:**
vendor-locked, server-stored, no cross-vendor.

Cross-vendor summary: Every vendor is building *within* its ecosystem. The white space is
precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

	Tool	Location	Format	Path keying	Cross-device resume native?
					--- --- --- --- ---
	Claude Code	<code>~/local/share/claude/projects/<encoded-path>/<session>.jsonl`</code>	JSONL, append-only event stream	Absolute path, non-alphanumerics→dashes	Via cloud only (teleport/RC)
	Codex	<code>~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl`</code>	JSONL + <code>session_meta`</code> header	date-based; cwd in meta	Yes, via App Server
	Gemini CLI	<code>~/gemini/tmp/<project_hash>/chats/`</code>	JSON	Hash of project root	No
	opencode	<code>~/local/share/opencode/`</code> (SQLite + files)	SQLite/JSON	project-keyed	Sharing only
	Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)
	Grok	<code>~/grok/sessions/`</code>	JSON	working directory	No
	Copilot CLI	<code>~/copilot/session-state/<uuid>/events.jsonl`</code>	JSONL events	uuid	Account history sync; no resume

The path-mapping problem (the crux). Claude Code derives the project folder name by
escaping the absolute path: `/Users/me/app`` → `~/local/share/claude/projects/-Users-me-app/``, while the
same repo at `C:\Users\me\app`` (or a different macOS username/location) produces a *different*
folder. Consequently, after a naive file sync, `claude --resume`` on the second machine cannot
find the first machine's sessions. This is confirmed by the `claude-sync` author's own
documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool

handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker."

Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via `project\_hash`; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. ****This is the single highest-value, currently-unmet technical feature.****

****Format churn risk.**** Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

****Mid-session cross-device resume feasibility (same vendor):**** Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present — ***if*** path keys are remapped. Codex's App Server makes it cloud-native already.

****Cross-vendor session replay (e.g., Claude Code session → Codex):**** ****Fundamentally impractical.**** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) `encrypted\_content` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. ****Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.****

3. Competitor teardown

****Session-continuity tools:****

- ****tawanorg/claude-sync**** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/.claude` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + ****age E2E encryption**** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.
- ****npow/session-sync**** (~1 star, very new): the only cross-vendor ***session*** translator — Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- ****dantelex/aisync ("ss")****: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing — treat traction as unverified.)
- ****codsync/codsync**** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- ****kroxiksut/codexSync****: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- ****waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)**** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- ****iHildy/opencode-synced**** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- ****Dinesh3184/claude-session-sync****: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- ****renefichtmueller/claude-sync****: multi-backend (Git/iCloud/Dropbox/SyncThing/rsync) with snapshots/rollback and encryption.

****Config/MCP/skills sync (DevSync's neighborhood):****

- ****Leoyang183/sync-agents-settings**** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- ****nicepkg/vsync**** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- ****baranovxyz/agentsync**** and ****spxrogers/agentsync**** (beta): canonical `~.agents/` config synced to 9–31 agents.
- ****neon-solutions/add-mcp****: one-command MCP install across ~16 agents (Neon).

- **ztripez/mcp-sync**: MCP config sync across tools.
- **AgentSync (Rust)**: symlink-based config sync.
- **skills-sync**: skills + MCP workspace.
- **DevSync** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

URL-addressable / mobile-control / viewer approaches:

- **TokenRip** (tokenrip.com): makes session state a URL-addressable artifact (``/a/<slug>``) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- **Omnara** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was **archived** in February 2026 because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)
- **Syncode – AI Programming Tool**: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- **jazzyalex/agent-sessions** & **kenn-io/agentsview**: local-first session browsers/search across 20+ agents (read/resume-command, not sync).
- **LAN-workspace approaches** and **Synthing/NAS/dotfiles/chezmoi+age**: DIY approaches that work for config but hit the path problem for sessions.

Takeaway: The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- **Claude Code**: #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- **Codex**: #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI ↔ app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
- **Copilot CLI**: #1947 (cloud-synced sessions), #1635 (cross-environment resume).
- **Cursor forum**: multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is: Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. **Cross-vendor unified profile** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.
2. **OS-aware path remapping as a first-class feature** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows ↔ macOS. This is the wedge no competitor has nailed.
3. **E2E encryption by default** – table stakes given session files contain code, secrets, and

full transcripts.

****Risks:****

- ****Format churn:**** Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern `codexSync/cursaves` use).
- ****Vendor lock-out / ToS:**** Storing session data is user-owned data (fine), but **auth tokens** are not transferable (`codexSync` explicitly refuses; `Codex `encrypted_content`` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.
 - ****Vendors closing the gap:**** Anthropic's account sync (#71794), `Codex's App Server`, `Copilot's GitHub sync` are all encroaching on single-vendor sync. This **validates** the demand while **confirming** that only the cross-vendor + cross-OS layer is defensible.
- ****Security/privacy expectations:**** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. `Amp` (server-stored) and `OpenSync` (Convex).
- ****Honest limits:**** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

****Stage 0 – Fold in DevSync (immediate).**** Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (`CLAUDE.md/AGENTS.md/GEMINI.md`), and settings across Claude Code, `Codex`, `Gemini CLI`, `opencode`, `Cursor`, `Copilot`, `Grok`, `Amp`. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like `vsync` and `sync-agents-settings`).

****Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge).****
Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/ .claude/projects/`` and global config to user-owned storage (`S3/R2/GCS/WebDAV`), matching `claude-sync`'s proven approach.
- ****First-class OS-aware path remapping:**** on pull, rewrite the project folder key AND in-file absolute path references (`cwd`, `git snapshots`, `file refs`) from the source machine's path to the target machine's path, using the `git remote URL` as the project identity (the technique `cursaves` uses for `Cursor`). This is the feature `claude-sync` explicitly lacks – lead marketing with it.
 - Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
 - ****Benchmark to advance:**** ``claude --resume`` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

****Stage 2 – Second and third vendors: `Codex` + `Gemini CLI` + `opencode`.** Add adapters. For `Codex`, cooperate with (don't fight) the App Server – sync what's local and useful, avoid ``encrypted_content`/token` pitfalls. For `Gemini`, remap ``project_hash``. For `opencode`, use safe SQLite snapshotting (the ``sqlite3 .backup`` pattern the community scripts use).

- ****Benchmark:**** each new adapter must pass the same cross-OS resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One "profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not replay.

****Monetization & thresholds.**** Free/open-source core (build trust, match `claude-sync`) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (`DevSync/OpenSync` validate willingness to pay). ****Kill/pivot signal:**** if Anthropic ships account-level **local** session sync with automatic path remapping across OSe (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on

cross-vendor config/skills/profile portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini-Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

Worth noticing why your two examples actually work: **openrouter** and **opencode** don't describe the whole product. They pick the single most load-bearing noun and let it carry everything. "Router" implies routing between many providers. "Code" implies the domain. One noun, maximum inference. So the real question is: what's *your* load-bearing noun? I'd argue it's **agent** — because cross-vendor is your actual differentiator — paired with whatever you're moving.

Tier 1 — "agent + what's synced" (most descriptive per syllable)

- **AgentState** — the literal, complete answer. Sessions, config, skills, MCP are all "state." Says what's synced and implies where.
- **AgentHome** — you are literally syncing home directories: `~/.claude`, `~/.codex`, `~/.gemini`. Devs get this instantly.
- **AgentPort** — "port" carries double duty: portability, and porting something between systems.
- **AgentProfile** — matches the unified-profile framing from the research: sessions + MCP + skills + instructions as one object.
- **AgentBridge** — between tools and between machines.
- `~~AgentSync~~` — **taken twice** (baranovxyz and spxrogers both have agentsync repos).

Tier 2 — the dotfiles framing (highest instant legibility)

Every dev already understands "dotfiles sync." You're dotfiles-sync for AI agents, plus sessions.

- **dotagent** — short, lowercase, reads native in a terminal. Probably my favorite in this whole tier.
- **agentdots**
- **dotsync**

Tier 3 — continuity-first

- **CarryState** — from that earlier ranked list; plain and accurate.
- **StatePort**
- **SessionPort**
- **Continuum** — one word meaning unbroken continuity. Elegant, but abstract enough that it under-describes.

Tier 4 — the "everywhere" framing

- **AnyAgent** — implies both any tool and any machine.
- **AgentAnywhere** — long, but it's the pitch verbatim.
- **Roaming / AgentRoam** — telecom roaming is exactly the concept: your setup follows you. (**OpenRoaming** is a Wi-Fi standard, so avoid the **open-** form.)

Tier 5 — maximally descriptive, no cleverness

Sometimes the boring name wins because it needs zero explanation:

AgentSessionSync, CLIAgentSync, AgentStateSync. Ugly, but a stranger reading your README knows exactly what it is in one second.

My three picks:

dotagent — best legibility-to-length ratio, sounds like something that already ships with your shell.

AgentState — most complete description in two words; "state" honestly covers sessions *and* config, which "session" doesn't.

AgentHome — the warmest of the accurate ones, and technically precise since home directories are literally what you're syncing.

Known-taken in this space, so skip: **agentsync, opensync, add-mcp, vsync, plexus, aisync.**

Want me to check **dotagent, agentstate, and agenthome** against **npm, GitHub, and domains** before you get attached to one?

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

```
# Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape
```

```
## TL;DR
```

```
- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the
```

one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows↔macOS paths.

- **Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;** config/skills/instruction portability and ***same-vendor*** cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows↔macOS sync.

- **Recommendation:** build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode.** Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.

2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...\` and `/Users/...\` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.

3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.

4. ****The competitor field is fragmented, early, and mostly config-only or single-vendor.**** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

****Claude Code (Anthropic).**** Ships the richest consumer-facing multi-device story:

- ****`/teleport`**** (``claude --teleport``): pulls a **cloud** session (started at `claude.ai/code` or the mobile app) down into the local terminal. It is ****one-way**** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport``, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires ****clean git state**** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same `claude.ai` account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."
- ****Remote Control**** (``claude remote-control`` / ``/remote-control``, `--rc``): pushes a **live local** session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an **undocumented** ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when ``ANTHROPIC_BASE_URL`` is redirected; unavailable under Zero Data Retention.
- ****Claude Code on the web**** (`claude.ai/code`): runs on Anthropic cloud infra, no local filesystem.
- ****Residual gap:**** No true device-to-device **local** session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun

syncing conversation history across devices on the same account (reported in issue #71794 as an **unexpected**, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

Codex CLI (OpenAI). The most architecturally mature cross-surface model:

- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared **App Server** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under `~/.codex/sessions/YYYY/MM/DD/rollout-*.jsonl``, with a ``session_meta`` first line.
- Rich lifecycle: ``codex resume`` (picker), ``--last``, ``--resume-session-id``, ``/fork``, archive/unarchive (v0.136), delete (v0.140). Imports histories from other agents via an ``external-agent-sessions`` crate.
- **Known gaps/bugs:** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing ``encrypted_content`` often can't be continued after crossing provider/account (``invalid_encrypted_content``); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.
- **Residual gap:** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

Gemini CLI (Google). Auto-saves sessions to `~/.gemini/tmp/<project_hash>/chats/`` (project-scoped, hash of project root). ``/resume`` session browser, ``--resume`/`-r`` flag, ``/chat save`` checkpoints, and a shadow-git checkpointing system under `~/.gemini/history/<project_hash>``. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. **Residual gap:** everything local + path-hash-keyed; no sync.

opencode (sst). SQLite + file storage under `~/.local/share/opencode/`` (``opencode.db``, ``storage/session|message|part``). Native **session sharing** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party

ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup).
Residual gap: sharing $\neq$ syncing; DB concurrency and cross-OS path issues remain.

Cursor CLI (cursor-agent). Chats live in local SQLite; CLI chats under `~/cursor/chats`` and resume via `cursor-agent --resume``. **IDE chats and CLI chats do NOT share a session store** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend `&`` to push to a Cloud Agent, pick up at cursor.com/agents), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). **Residual gap:** no local cross-device sync, IDE ↔ CLI split, path issues.

Grok / Grok Build (xAI). Sessions auto-saved to `~/grok/sessions/``, keyed by working directory. `/resume``, `--resume``, `-c``, `/fork``; subscription-gated (SuperGrok/X Premium+). Config in `~/grok/config.toml``. No native cross-device sync. Still beta (v0.2.x as of July 2026). **Residual gap:** no sync at all.

GitHub Copilot CLI. Sessions in `~/copilot/`` (`session-state/<uuid>/events.jsonl``). Notably, Copilot CLI **syncs sessions to your GitHub account by default** and offers **remote control** from [GitHub.com/GitHub Mobile](https://github.com/GitHubMobile) (origin machine must stay online, `/keep-alive``). But \*\*cross-device \*resume\* is still not supported\*\* – manually copying a session dir and running `--resume`` overwrites `events.jsonl`` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session_id``. **Residual gap:** the sync is telemetry/history, not resumable working state across machines.

Amp (Sourcegraph). Threads sync to ampcode.com by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). **Residual gap:** vendor-locked, server-stored, no cross-vendor.

Cross-vendor summary: Every vendor is building *within* its ecosystem. The white space is precisely between ecosystems and

across OSes.

2. Session storage formats & technical feasibility

Tool	Location	Format	Path keying	Cross-device resume native?
Claude Code	`\${HOME}/.claude/projects/<encoded-path>/<session>.jsonl`	JSONL, append-only event stream	**Absolute path, non-alphanumerics→dashes**	Via cloud only (teleport/RC)
Codex	`\${HOME}/.codex/sessions/YYYY/MM/DD/rollout-*.jsonl`	JSONL + `session_meta` header	date-based; cwd in meta	Yes, via App Server
Gemini CLI	`\${HOME}/.gemini/tmp/<project_hash>/chats/`	JSON	**Hash of project root**	No
opencode	`\${HOME}/.local/share/opencode/`	(SQLite + files)	SQLite/JSON	project-keyed
Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)
Grok	`\${HOME}/.grok/sessions/`	JSON	working directory	No
Copilot CLI	`\${HOME}/.copilot/session-state/<uuid>/events.jsonl`	JSONL events	uuid	Account history sync; no resume

****The path-mapping problem (the crux).**** Claude Code derives the project folder name by escaping the absolute path: ``/Users/me/app`` → ``~/.claude/projects/-Users-me-app/``, while the same repo at ``C:\Users\me\app`` (or a different macOS username/location) produces a *different* folder. Consequently, after a naive file sync, ``claude --resume`` on the second machine cannot find the first machine's sessions. This is confirmed by the `claude-sync` author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker." Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via ``project_hash``; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. ****This is the single highest-value, currently-unmet technical feature.****

****Format churn risk.**** Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

****Mid-session cross-device resume feasibility (same vendor):**** Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present – **if** path keys are remapped. Codex's App Server makes it cloud-native already.

****Cross-vendor session replay (e.g., Claude Code session → Codex):**** ****Fundamentally impractical.**** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) ``encrypted_content`` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. ****Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.****

3. Competitor teardown

****Session-continuity tools:****

- ****tawanorg/claude-sync**** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/ .claude`` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + ****age E2E encryption**** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.
- ****npow/session-sync**** (~1 star, very new): the only cross-vendor **session** translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- ****dantelex/aisync ("ss")****: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)

- ****codsync/codsync**** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- ****kroxiksut/codexSync****: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- ****waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)**** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- ****iHildy/opencode-synced**** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- ****Dinesh3184/claude-session-sync****: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- ****renefichtmueller/claude-sync****: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

****Config/MCP/skills sync (DevSync's neighborhood):****

- ****Leoyangl83/sync-agents-settings**** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- ****nicepkg/vsync**** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- ****baranovxyz/agentsync**** and ****spxrogers/agentsync**** (beta): canonical ``.agents/`` config synced to 9–31 agents.
- ****neon-solutions/add-mcp****: one-command MCP install across ~16 agents (Neon).
- ****ztripez/mcp-sync****: MCP config sync across tools.
- ****AgentSync (Rust)****: symlink-based config sync.
- ****skills-sync****: skills + MCP workspace.
- ****DevSynq**** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

****URL-addressable / mobile-control / viewer approaches:****

- ****TokenRip**** (tokenrip.com): makes session state a URL-addressable artifact (``/a/<slug>``) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- ****Omnara**** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was ****archived in February 2026**** because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the

company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)

- ****Syncode – AI Programming Tool****: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).

- ****jazzyalex/agent-sessions**** & ****kenn-io/agentsview****: local-first session browsers/search across 20+ agents (read/resume-command, not sync).

- ****LAN-workspace approaches**** and

Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

****Takeaway:**** The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- ****Claude Code:**** #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.

- ****Codex:**** #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI  app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).

- ****Copilot CLI:**** #1947 (cloud-synced sessions), #1635 (cross-environment resume).

- ****Cursor forum:**** multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

****Where the durable moat is.**** Vendors will keep building **intra**-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. ****Cross-vendor unified profile**** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.

2. ****OS-aware path remapping as a first-class feature**** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows↔macOS. This is the wedge no competitor has nailed.

3. ****E2E encryption by default**** – table stakes given session files contain code, secrets, and full transcripts.

****Risks:****

- ****Format churn:**** Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).

- ****Vendor lock-out / ToS:**** Storing session data is user-owned data (fine), but **auth tokens** are not transferable (codexSync explicitly refuses; Codex `encrypted\_content` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.

- ****Vendors closing the gap:**** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This **validates** the demand while **confirming** that only the cross-vendor + cross-OS layer is defensible.

- ****Security/privacy expectations:**** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).

- ****Honest limits:**** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

****Stage 0 – Fold in DevSync (immediate).**** Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

****Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge).**** Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/.claude/projects/` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- ****First-class OS-aware path remapping:**** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
- ****Benchmark to advance:**** `claude --resume` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

****Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode.**** Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid `encrypted\_content`/token pitfalls. For Gemini, remap `project\_hash`. For opencode, use safe SQLite snapshotting (the `sqlite3 .backup` pattern the community scripts use).

- ****Benchmark:**** each new adapter must pass the same cross-OS resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One "profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not

replay.

****Monetization & thresholds.**** Free/open-source core (build trust, match claude-sync) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSynq/OpenSync validate willingness to pay). ****Kill/pivot signal:**** if Anthropic ships account-level **local** session sync with automatic path remapping across OSes (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

Okay, sorry I forgot to mention but I don't like the names where there are two words combined into a single word and the initials of both those words are capitalized. Like I really hate those names and I find them weird. So can we get like one word names?

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape

TL;DR

- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows↔macOS paths.
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows↔macOS sync.
- **Recommendation:** build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode. Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...` and `/Users/...` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.
3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.
4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

- **Claude Code (Anthropic).** Ships the richest consumer-facing multi-device story:
 - **`/teleport`** (`claude --teleport`): pulls a *cloud* session (started at claude.ai/code or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport`, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires **clean git state** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same claude.ai account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."
 - **Remote Control** (`claude remote-control` / `/remote-control`, `--rc`): pushes a *live local* session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970

reports an **undocumented** ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when ``ANTHROPIC_BASE_URL`` is redirected; unavailable under Zero Data Retention.

- **Claude Code on the web** (claude.ai/code): runs on Anthropic cloud infra, no local filesystem.

- **Residual gap:** No true device-to-device **local** session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an **unexpected**, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

Codex CLI (OpenAI). The most architecturally mature cross-surface model:

- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared **App Server** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under ``~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl``, with a ``session_meta`` first line.

- Rich lifecycle: ``codex resume`` (picker), ``--last``, ``--resume-session-id``, ``/fork``, `archive/unarchive` (v0.136), `delete` (v0.140). Imports histories from other agents via an ``external-agent-sessions`` crate.

- **Known gaps/bugs:** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing ``encrypted_content`` often can't be continued after crossing provider/account (``invalid_encrypted_content``); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.

- **Residual gap:** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

Gemini CLI (Google). Auto-saves sessions to ``~/gemini/tmp/<project_hash>/chats/`` (project-scoped, hash of project root). ``/resume`` session browser, ``--resume`` ``-r`` flag, ``/chat save`` checkpoints, and a shadow-git checkpointing system under ``~/gemini/history/<project_hash>``. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. **Residual gap:** everything local + path-hash-keyed; no sync.

opencode (sst). SQLite + file storage under ``~/local/share/opencode/`` (``opencode.db``, ``storage/session|message|part``). Native **session sharing** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). **Residual gap:** sharing ≠ syncing; DB concurrency and cross-OS path issues remain.

Cursor CLI (cursor-agent). Chats live in local SQLite; CLI chats under ``~/cursor/chats`` and resume via ``cursor-agent --resume``. **IDE chats and CLI chats do NOT share a session store** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend ``&`` to push to a Cloud Agent, pick up at cursor.com/agents), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). **Residual gap:** no local cross-device sync, IDE ↔ CLI split, path issues.

Grok / Grok Build (xAI). Sessions auto-saved to ``~/grok/sessions/``, keyed by working directory. ``/resume``, ``--resume``, ``-c``, ``/fork``; subscription-gated (SuperGrok/X Premium+). Config in ``~/grok/config.toml``. No native cross-device sync. Still beta (v0.2.x as of July 2026). **Residual gap:** no sync at all.

GitHub Copilot CLI. Sessions in ``~/copilot/`` (``session-state/<uuid>/events.jsonl``). Notably, Copilot CLI **syncs** sessions to your GitHub account by default and offers **remote control** from GitHub.com/GitHub Mobile (origin machine must stay online, ``/keep-alive``). But **cross-device *resume*** is still not supported – manually copying a session dir and running ``--resume`` overwrites ``events.jsonl`` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via

a user-supplied `session\_id`. **Residual gap:** the sync is telemetry/history, not resumable working state across machines.

Amp (Sourcegraph). Threads sync to ampcode.com by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). **Residual gap:** vendor-locked, server-stored, no cross-vendor.

Cross-vendor summary: Every vendor is building *within* its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

Tool	Location	Format	Path keying	Cross-device resume native?
				--- --- --- --- ---
Claude Code	~/ .claude/projects/<encoded-path>/<session>.jsonl`	JSONL, append-only event stream	Absolute path, non-alphanumerics→dashes	Via cloud only (teleport/RC)
Codex	~/ .codex/sessions/YYYY/MM/DD/rollout-*.jsonl`	JSONL + `session_meta` header	date-based; cwd in meta	Yes, via App Server
Gemini CLI	~/ .gemini/tmp/<project_hash>/chats/`	JSON	Hash of project root	No
opencode	~/ .local/share/opencode/`	(SQLite + files)	SQLite/JSON	project-keyed
				Sharing only
Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)
Grok	~/ .grok/sessions/`	JSON	working directory	No
Copilot CLI	~/ .copilot/session-state/<uuid>/events.jsonl`	JSONL events	uuid	Account history sync; no resume

The path-mapping problem (the crux). Claude Code derives the project folder name by escaping the absolute path: `/Users/me/app` → `~/ .claude/projects/-Users-me-app/`, while the same repo at `C:\Users\me\app` (or a different macOS username/location) produces a *different* folder. Consequently, after a naive file sync, `claude --resume` on the second machine cannot find the first machine's sessions. This is confirmed by the claude-sync author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker." Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via `project\_hash`; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. **This is the single highest-value, currently-unmet technical feature.**

Format churn risk. Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

Mid-session cross-device resume feasibility (same vendor): Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present — *if* path keys are remapped. Codex's App Server makes it cloud-native already.

Cross-vendor session replay (e.g., Claude Code session → Codex): **Fundamentally impractical.** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) `encrypted\_content` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. **Recommendation:** treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.

3. Competitor teardown

****Session-continuity tools:****

- ****tawanorg/claude-sync**** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/.claude` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + ****age E2E encryption**** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.
 - ****npow/session-sync**** (~1 star, very new): the only cross-vendor ***session*** translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
 - ****dantelex/aisync ("ss")****: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
 - ****codsync/codsync**** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
 - ****kroxiksut/codexSync****: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
 - ****waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)**** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- ****iHildy/opencode-synced**** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- ****Dinesh3184/claude-session-sync****: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- ****renefichtmueller/claude-sync****: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

****Config/MCP/skills sync (DevSync's neighborhood):****

- ****Leoyang183/sync-agents-settings**** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
 - ****nicepkg/vsync**** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
 - ****baranovxyz/agentsync**** and ****spxrogers/agentsync**** (beta): canonical `~/.agents/`` config synced to 9–31 agents.
 - ****neon-solutions/add-mcp****: one-command MCP install across ~16 agents (Neon).
 - ****ztripez/mcp-sync****: MCP config sync across tools.
 - ****AgentSync (Rust)****: symlink-based config sync.
 - ****skills-sync****: skills + MCP workspace.
 - ****DevSynq**** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

****URL-addressable / mobile-control / viewer approaches:****

- ****TokenRip**** (tokenrip.com): makes session state a URL-addressable artifact (``/a/<slug>``) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- ****Omnara**** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was ****archived in February 2026**** because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)
- ****Syncode – AI Programming Tool****: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- ****jazzyalex/agent-sessions**** & ****kenn-io/agentsview****: local-first session browsers/search across 20+ agents (read/resume-command, not sync).
- ****LAN-workspace approaches**** and Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

****Takeaway:**** The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- **Claude Code:** #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- **Codex:** #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI ↔ app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
 - **Copilot CLI:** #1947 (cloud-synced sessions), #1635 (cross-environment resume).
- **Cursor forum:** multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is. Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. **Cross-vendor unified profile** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.
2. **OS-aware path remapping as a first-class feature** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows↔macOS. This is the wedge no competitor has nailed.
3. **E2E encryption by default** – table stakes given session files contain code, secrets, and full transcripts.

Risks:

- **Format churn:** Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).
- **Vendor lock-out / ToS:** Storing session data is user-owned data (fine), but *auth tokens* are not transferable (codexSync explicitly refuses; Codex `encrypted\_content` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.
 - **Vendors closing the gap:** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This *validates* the demand while *confirming* that only the cross-vendor + cross-OS layer is defensible.
- **Security/privacy expectations:** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).
- **Honest limits:** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

Stage 0 – Fold in DevSync (immediate). Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync

and sync-agents-settings).

****Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge).****
Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:
- Encrypted (age/passphrase-derived) sync of `~/ .claude/projects/` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- ****First-class OS-aware path remapping:**** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
- ****Benchmark to advance:**** `claude --resume` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

****Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode.**** Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid `encrypted\_content`/token pitfalls. For Gemini, remap `project\_hash`. For opencode, use safe SQLite snapshotting (the `sqlite3 .backup` pattern the community scripts use).
- ****Benchmark:**** each new adapter must pass the same cross-OS resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One "profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not replay.

****Monetization & thresholds.**** Free/open-source core (build trust, match claude-sync) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSynq/OpenSync validate willingness to pay). ****Kill/pivot signal:**** if Anthropic ships account-level *local* session sync with automatic path remapping across OSe (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

dotagent is dead, and how it's dead matters more than the name.

johnlindquist/dotagent is a "Universal AI agent configuration parser and converter" on npm as `dotagent` (v2.10.0). There's also sgmonda/dotagent, Koryto/dotagent, and dotagent.ai. But the one you should actually look at: **Sentry shipped dotagents** — declare skills, MCP servers, hooks, and subagents once in `agents.toml`, and it generates native runtime files for

Claude, Cursor, Codex, and OpenCode. That's DevSync's config half, built by Sentry. Worth reading before you finalize scope — it strengthens the case that your differentiator has to be *sessions plus path remapping*, not config sync alone.

Single real words, no compounds:

What gets carried forward

- **carryover** — a real single word meaning something carried from one period into the next. Closest one-word description of your product, and it can't be misread as two words the way carry-on could.
- **holdover** — something retained from before.
- **rollover** — carried forward rather than lost. Familiar from telecom minutes.
- **persistence** — the technical term for state surviving past a session. Accurate, slightly long.

The action

- **reinstate** — your earlier favorite, still wide open, still good.
- **instate** — the shorter cut.
- **reattach** — `tmux attach` muscle memory.
- **respool** — winding thread back onto the reel.
- **rejoin** — plainest possible verb for what the user does.

Cross-tool unification

- **federation** — separate systems under one umbrella. Describes the cross-vendor half well.
- **convergence** — separate things becoming one.
- **concord** — agreement between parties.

Skip these despite fitting: unison (Unison is a well-known bidirectional file synchronizer — direct category collision), **continuity** (Apple's feature name for exactly this behavior across devices), **parity** (Parity Technologies), **portage** (taken).

My three: carryover for description, **reinstate** for the pun plus cleanliness, **respool** for the terminal feel.

One honest note: a single word can't encode "AI agents" *and* "sessions" *and* "cross-device." The ceiling here is one strong noun plus a tagline doing the rest — same as openrouter, which is just "router" trusting the docs to explain the rest. Carryover with "your coding agent sessions, on every machine" underneath does the job.

Cross-Device Sync for Terminal AI Coding Agents: Competitive and

Technical Landscape

Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape

TL;DR

- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows ↔ macOS paths.
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows ↔ macOS sync.
- **Recommendation:** build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode. Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...\` and `/Users/...\` produces orphaned,

unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.

3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.

4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

Claude Code (Anthropic). Ships the richest consumer-facing multi-device story:

- **/teleport** (``claude --teleport``): pulls a *cloud* session (started at `claude.ai/code` or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with ``--teleport``, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires **clean git state** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same `claude.ai` account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."
- **Remote Control** (``claude remote-control`` / ``/remote-control``, ``--rc``): pushes a *live local* session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an

undocumented ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `ANTHROPIC\_BASE\_URL` is redirected; unavailable under Zero Data Retention.

- ****Claude Code on the web**** (claude.ai/code): runs on Anthropic cloud infra, no local filesystem.

- ****Residual gap:**** No true device-to-device ***local*** session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an ***unexpected***, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

****Codex CLI (OpenAI).**** The most architecturally mature cross-surface model:

- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared ****App Server**** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under `~/.codex/sessions/YYYY/MM/DD/rollout-\*.jsonl`, with a `session\_meta` first line.

- Rich lifecycle: `codex resume` (picker), `--last`, `--resume-session-id`, `/fork`, archive/unarchive (v0.136), delete (v0.140). Imports histories from other agents via an `external-agent-sessions` crate.

- ****Known gaps/bugs:**** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing `encrypted\_content` often can't be continued after crossing provider/account (`invalid\_encrypted\_content`); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.

- ****Residual gap:**** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

****Gemini CLI (Google).**** Auto-saves sessions to

`~/.gemini/tmp/<project\_hash>/chats/` (project-scoped, hash of project root). `/resume` session browser, `--resume`/`-r` flag,

`/chat save` checkpoints, and a shadow-git checkpointing system under `~/.gemini/history/<project\_hash>`. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. ****Residual gap:**** everything local + path-hash-keyed; no sync.

****opencode (sst).**** SQLite + file storage under `~/.local/share/opencode/` (`opencode.db`, `storage/session|message|part`). Native ****session sharing**** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). ****Residual gap:**** sharing $\neq$ syncing; DB concurrency and cross-OS path issues remain.

****Cursor CLI (cursor-agent).**** Chats live in local SQLite; CLI chats under `~/.cursor/chats` and resume via `cursor-agent --resume`. ****IDE chats and CLI chats do NOT share a session store**** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend `&` to push to a Cloud Agent, pick up at cursor.com/agents), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). ****Residual gap:**** no local cross-device sync, IDE ↔ CLI split, path issues.

****Grok / Grok Build (xAI).**** Sessions auto-saved to `~/.grok/sessions/`, keyed by working directory. `/resume`, `--resume`, `-c`, `/fork`; subscription-gated (SuperGrok/X Premium+). Config in `~/.grok/config.toml`. No native cross-device sync. Still beta (v0.2.x as of July 2026). ****Residual gap:**** no sync at all.

****GitHub Copilot CLI.**** Sessions in `~/.copilot/` (`session-state/<uuid>/events.jsonl`). Notably, Copilot CLI ****syncs sessions to your GitHub account by default**** and offers ****remote control**** from GitHub.com/GitHub Mobile (origin machine must stay online, `/keep-alive`). But ****cross-device *resume* is still not supported**** – manually copying a session dir and running `--resume` overwrites `events.jsonl` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session\_id`. ****Residual gap:**** the sync is telemetry/history, not resumable working state across machines.

****Amp (Sourcegraph).**** Threads sync to ampcode.com by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). ****Residual gap:**** vendor-locked, server-stored, no cross-vendor.

****Cross-vendor summary:**** Every vendor is building **within** its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

Tool	Location	Format	Path keying	Cross-device resume native?
Claude Code	`\${HOME}/.claude/projects/<encoded-path>/<session>.jsonl`	JSONL, append-only event stream	**Absolute path, non-alphanumerics→dashes**	Via cloud only (teleport/RC)
Codex	`\${HOME}/.codex/sessions/YYYY/MM/DD/rollout-*.jsonl`	JSONL + `session_meta` header	date-based; cwd in meta	Yes, via App Server
Gemini CLI	`\${HOME}/.gemini/tmp/<project_hash>/chats/`	JSON	**Hash of project root**	No
opencode	`\${HOME}/.local/share/opencode/`	(SQLite + files)	SQLite/JSON	project-keyed
Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)
Grok	`\${HOME}/.grok/sessions/`	JSON	working directory	No
Copilot CLI	`\${HOME}/.copilot/session-state/<uuid>/events.jsonl`	JSONL events	uuid	Account history sync; no resume

****The path-mapping problem (the crux).**** Claude Code derives the project folder name by escaping the absolute path: ``/Users/me/app`` → ``~/.claude/projects/-Users-me-app/``, while the same repo at ``C:\Users\me\app`` (or a different macOS username/location) produces a **different** folder. Consequently, after a naive file sync, ``claude --resume`` on the second machine cannot find the first machine's sessions. This is confirmed by the claude-sync author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation?"

Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker." Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via `project\_hash`; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. **This is the single highest-value, currently-unmet technical feature.**

Format churn risk. Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

Mid-session cross-device resume feasibility (same vendor): Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present — *if* path keys are remapped. Codex's App Server makes it cloud-native already.

Cross-vendor session replay (e.g., Claude Code session → Codex): **Fundamentally impractical.** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) `encrypted\_content` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. **Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.**

3. Competitor teardown

Session-continuity tools:

- **tawanorg/claude-sync** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/.claude` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + **age E2E encryption** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-

derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.

- ****npow/session-sync**** (~1 star, very new): the only cross-vendor *session* translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- ****dantelex/aisync ("ss")****: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- ****codsync/codsync**** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- ****kroxiksut/codexSync****: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- ****waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)**** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- ****iHildy/opencode-synced**** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- ****Dinesh3184/claude-session-sync****: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- ****renefichtmueller/claude-sync****: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

****Config/MCP/skills sync (DevSync's neighborhood):****

- ****Leoyangl83/sync-agents-settings**** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- ****nicepkg/vsync**** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- ****baranovxyz/agentsync**** and ****spxrogers/agentsync**** (beta): canonical ``.agents/`` config synced to 9–31 agents.
- ****neon-solutions/add-mcp****: one-command MCP install across ~16 agents (Neon).
- ****ztripez/mcp-sync****: MCP config sync across tools.
- ****AgentSync (Rust)****: symlink-based config sync.
- ****skills-sync****: skills + MCP workspace.
- ****DevSynq**** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

****URL-addressable / mobile-control / viewer approaches:****

- ****TokenRip**** (tokenrip.com): makes session state a URL-addressable artifact (``/a/<slug>``) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- ****Omnara**** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was ****archived in February 2026**** because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)
- ****Syncode – AI Programming Tool****: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- ****jazzyalex/agent-sessions**** & ****kenn-io/agentsview****: local-first session browsers/search across 20+ agents (read/resume-command, not sync).
- ****LAN-workspace approaches**** and Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

****Takeaway:**** The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- ****Claude Code:**** #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- ****Codex:**** #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI  app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
- ****Copilot CLI:**** #1947 (cloud-synced sessions), #1635 (cross-

environment resume).

- **Cursor forum:** multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is. Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. **Cross-vendor unified profile** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.

2. **OS-aware path remapping as a first-class feature** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows↔macOS. This is the wedge no competitor has nailed.

3. **E2E encryption by default** – table stakes given session files contain code, secrets, and full transcripts.

Risks:

- **Format churn:** Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).

- **Vendor lock-out / ToS:** Storing session data is user-owned data (fine), but *auth tokens* are not transferable (codexSync explicitly refuses; Codex `encrypted\_content` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.

- **Vendors closing the gap:** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This *validates* the demand while

confirming that only the cross-vendor + cross-OS layer is defensible.

- ****Security/privacy expectations:**** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).
- ****Honest limits:**** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

****Stage 0 – Fold in DevSync (immediate).**** Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

****Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge).**** Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/.claude/projects/` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- ****First-class OS-aware path remapping:**** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
- ****Benchmark to advance:**** `claude --resume` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

****Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode.**** Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid

`encrypted\_content`/token pitfalls. For Gemini, remap
`project\_hash`. For opencode, use safe SQLite snapshotting (the
`sqlite3.backup` pattern the community scripts use).
- ****Benchmark:**** each new adapter must pass the same cross-OS
resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One
"profile" = sessions + MCP + skills + agents + instructions,
portable across machines and (for config) across vendors. Add
best-effort context handoff (summarize + re-inject) for cross-
vendor moves – explicitly labeled as context transfer, not
replay.

****Monetization & thresholds.**** Free/open-source core (build
trust, match claude-sync) with a paid tier for managed encrypted
storage, multi-machine dashboards, and team profiles
(DevSynq/OpenSync validate willingness to pay). ****Kill/pivot
signal:**** if Anthropic ships account-level **local** session sync
with automatic path remapping across OSes (not just history
mirroring, per #71794), the single-vendor wedge collapses – at
that point double down on cross-vendor config/skills/profile
portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport
(gated, one-way, origin-online, with documented ~10-min network
and undocumented ~15-min idle timeouts). Don't promise cross-
vendor session replay. Don't sync auth tokens. Don't store
plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment

based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

Okay, I think it's between carryover and reinstate. Which one do you think we should go for?

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape

TL;DR

- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows↔macOS paths.
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows↔macOS sync.
- **Recommendation:** build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode. Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...` and `/Users/...` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.
3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.
4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

- Claude Code (Anthropic).** Ships the richest consumer-facing multi-device story:
 - `/teleport` (`claude --teleport`): pulls a *cloud* session (started at `claude.ai/code` or

the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport`, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires **clean git state** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same claude.ai account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."

- **Remote Control** (`claude remote-control` / `remote-control`, --rc`): pushes a **live local** session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an **undocumented** ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `ANTHROPIC_BASE_URL` is redirected; unavailable under Zero Data Retention.

- **Claude Code on the web** (claude.ai/code): runs on Anthropic cloud infra, no local filesystem.

- **Residual gap:** No true device-to-device **local** session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an **unexpected**, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

Codex CLI (OpenAI). The most architecturally mature cross-surface model:

- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared **App Server** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under `~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl`, with a `session_meta` first line.

- Rich lifecycle: `codex resume` (picker), --last`, --resume-session-id`, /fork`, archive/unarchive` (v0.136), delete` (v0.140). Imports histories from other agents via an external-agent-sessions` crate.`

- **Known gaps/bugs:** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing `encrypted_content` often can't be continued after crossing provider/account (`invalid_encrypted_content`); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.

- **Residual gap:** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

Gemini CLI (Google). Auto-saves sessions to `~/gemini/tmp/<project_hash>/chats/` (project-scoped, hash of project root). /resume` session browser, --resume`/`-r` flag, /chat save` checkpoints, and a shadow-git checkpointing system under ~/gemini/history/<project_hash>`. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. Residual gap: everything local + path-hash-keyed; no sync.`

opencode (sst). SQLite + file storage under `~/local/share/opencode/` (opencode.db`, storage/session|message|part`). Native session sharing publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). Residual gap: sharing ≠ syncing; DB concurrency and cross-OS path issues remain.`

Cursor CLI (cursor-agent). Chats live in local SQLite; CLI chats under `~/cursor/chats` and resume via cursor-agent --resume`. IDE chats and CLI chats do NOT share a session store (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend &` to push to a Cloud Agent, pick up at cursor.com/agents), and "My Machines" lets cloud agents run on your`

registered machine. Strong user demand for account-level chat sync (forum). ****Residual gap:****
no local cross-device sync, IDE↔CLI split, path issues.

****Grok / Grok Build (xAI).**** Sessions auto-saved to `~/grok/sessions/`, keyed by working directory. `~/resume`, `--resume`, `-c`, `~/fork`; subscription-gated (SuperGrok/X Premium+). Config in `~/grok/config.toml`. No native cross-device sync. Still beta (v0.2.x as of July 2026). ****Residual gap:**** no sync at all.

****GitHub Copilot CLI.**** Sessions in `~/copilot/` (`session-state/<uuid>/events.jsonl`). Notably, Copilot CLI ****syncs sessions to your GitHub account by default**** and offers ****remote control**** from GitHub.com/GitHub Mobile (origin machine must stay online, `~/keep-alive`). But ****cross-device *resume* is still not supported**** – manually copying a session dir and running `--resume` overwrites `events.jsonl` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session\_id`. ****Residual gap:**** the sync is telemetry/history, not resumable working state across machines.

****Amp (Sourcegraph).**** Threads sync to ampcode.com by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). ****Residual gap:**** vendor-locked, server-stored, no cross-vendor.

****Cross-vendor summary:**** Every vendor is building **within** its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

	Tool	Location	Format	Path keying	Cross-device resume native?
					--- --- --- --- ---
	Claude Code	~/claude/projects/<encoded-path>/<session>.jsonl`	JSONL, append-only event stream	**Absolute path, non-alphanumerics→dashes**	Via cloud only (teleport/RC)
	Codex	~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl`	JSONL + `session_meta` header	date-based; cwd in meta	Yes, via App Server
	Gemini CLI	~/gemini/tmp/<project_hash>/chats/`	JSON	**Hash of project root**	No
	opencode	~/local/share/opencode/`	(SQLite + files)	SQLite/JSON	project-keyed
					Sharing only
	Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)
				Grok	~/grok/sessions/`
					JSON
	Copilot CLI	~/copilot/session-state/<uuid>/events.jsonl`	JSONL events	uuid	Account history sync; no resume

****The path-mapping problem (the crux).**** Claude Code derives the project folder name by escaping the absolute path: `/Users/me/app` → `~/claude/projects/-Users-me-app/`, while the same repo at `C:\Users\me\app` (or a different macOS username/location) produces a **different** folder. Consequently, after a naive file sync, `claude --resume` on the second machine cannot find the first machine's sessions. This is confirmed by the claude-sync author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker."

Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via `project\_hash`; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. ****This is the single highest-value, currently-unmet technical feature.****

****Format churn risk.**** Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

****Mid-session cross-device resume feasibility (same vendor):**** Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present – ***if*** path keys are remapped. Codex's App Server makes it cloud-native already.

****Cross-vendor session replay (e.g., Claude Code session → Codex):**** ****Fundamentally impractical.**** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) ``encrypted_content`` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. ****Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.****

3. Competitor teardown

****Session-continuity tools:****

- ****tawanorg/claude-sync**** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/.claude`` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + ****age E2E encryption**** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.
- ****npow/session-sync**** (~1 star, very new): the only cross-vendor ***session*** translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- ****dantelex/aisync ("ss")****: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- ****codsync/codsync**** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- ****kroxiksut/codexSync****: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- ****waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)**** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- ****iHildy/opencode-synced**** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- ****Dinesh3184/claude-session-sync****: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- ****renefichtmueller/claude-sync****: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

****Config/MCP/skills sync (DevSync's neighborhood):****

- ****Leoyang183/sync-agents-settings**** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- ****nicepkg/vsync**** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- ****baranovxyz/agentsync**** and ****spxrogers/agentsync**** (beta): canonical ``.agents/`` config synced to 9–31 agents.
- ****neon-solutions/add-mcp****: one-command MCP install across ~16 agents (Neon).
- ****ztripez/mcp-sync****: MCP config sync across tools.
- ****AgentSync (Rust)****: symlink-based config sync.
- ****skills-sync****: skills + MCP workspace.
- ****DevSynq**** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

****URL-addressable / mobile-control / viewer approaches:****

- ****TokenRip**** (tokenrip.com): makes session state a URL-addressable artifact (`/a/<slug>`) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- ****Omnara**** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was ****archived in February 2026**** because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent

SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)

- **Syncode – AI Programming Tool**: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- **jazzyalex/agent-sessions** & **kenn-io/agentsview**: local-first session browsers/search across 20+ agents (read/resume-command, not sync).
- **LAN-workspace approaches** and Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

Takeaway: The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- **Claude Code**: #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- **Codex**: #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI ↔ app-server sessions), discussion #14067 (cross-device thread/contextsync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
- **Copilot CLI**: #1947 (cloud-synced sessions), #1635 (cross-environment resume).
- **Cursor forum**: multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is. Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. **Cross-vendor unified profile** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.
2. **OS-aware path remapping as a first-class feature** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows ↔ macOS. This is the wedge no competitor has nailed.
3. **E2E encryption by default** – table stakes given session files contain code, secrets, and full transcripts.

Risks:

- **Format churn**: Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).
- **Vendor lock-out / ToS**: Storing session data is user-owned data (fine), but *auth tokens* are not transferable (codexSync explicitly refuses; Codex `encrypted\_content` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.
- **Vendors closing the gap**: Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This *validates* the demand while *confirming* that only the cross-vendor + cross-OS layer is defensible.

- ****Security/privacy expectations:**** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).
- ****Honest limits:**** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

****Stage 0 – Fold in DevSync (immediate).**** Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

****Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge).****
 Target the founder's exact case (Windows desktop 🖥️ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/ .claude/projects/` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- ****First-class OS-aware path remapping:**** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
- ****Benchmark to advance:**** `claude --resume` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

****Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode.**** Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid `encrypted\_content`/token pitfalls. For Gemini, remap `project\_hash`. For opencode, use safe SQLite snapshotting (the `sqlite3 .backup` pattern the community scripts use).

- ****Benchmark:**** each new adapter must pass the same cross-OS resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One "profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not replay.

****Monetization & thresholds.**** Free/open-source core (build trust, match claude-sync) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSync/OpenSync validate willingness to pay). ****Kill/pivot signal:**** if Anthropic ships account-level *local* session sync with automatic path remapping across OSes (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync,

Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.

- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

Reinstate. Not close, on reflection.

1. It's verified clean, and that's worth more than anything else here.

You googled it yourself and found only dictionary entries. Look at what happened to everything else in this conversation — Rethread, Duffel, dotagent, OpenSync, Filum all died on collisions. Carryover is unverified, and I'd expect it to be crowded: it's a heavily-used generic term in accounting (carryover losses, carryover basis), HR (PTO carryover), and statistics (carryover effects in crossover trials). A clean namespace is a real asset; a generic one is a permanent tax.

2. It contains the right noun. "State" is precisely what you sync — sessions, config, skills, MCP servers are all state. "Carry" describes motion but names nothing. Reinstate encodes the object; carryover only encodes the verb.

3. It works as a bare command. `reinstate` on the laptop is a complete instruction. `carryover` is meaningless alone and always needs a subcommand.

4. Your stickiness objection doesn't actually favor carryover. Both are nine characters and three syllables. Carryover isn't clickier — it's flatter, and its everyday associations (tax forms, leftover vacation days) are arguably more bureaucratic than `reinstate`'s. If anything, `reinstate` has a bit of drama to it: something lost, then restored.

5. The pun rewards without requiring. People who notice "state" sitting in the middle feel clever. People who don't still understand the word means *restore to how it was*. Carryover has no second layer to discover.

Carryover's only edge is being marginally more literal to someone who's never heard of your product. That's a small gain against a large availability risk.

One caveat: my npm check came back inconclusive — the query returned registry docs rather than package pages. Run `npm view reinstate` and `whois reinstate.dev` before you commit. If npm is taken, `@yourhandle/reinstate` is completely normal and doesn't change the call. Then go build the path remapping. That's the part that decides whether this works.

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

```
# Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape
```

TL;DR

- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows ↔ macOS paths.
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows ↔ macOS sync.
- **Recommendation:** build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode. Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...\` and `/Users/...\` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.

3. ****Demand is loud and repeatedly expressed**** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.

4. ****The competitor field is fragmented, early, and mostly config-only or single-vendor.**** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

****Claude Code (Anthropic).**** Ships the richest consumer-facing multi-device story:

- ****`/teleport`**** (``claude --teleport``): pulls a **cloud** session (started at `claude.ai/code` or the mobile app) down into the local terminal. It is ****one-way**** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with ``--teleport``, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires ****clean git state**** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same `claude.ai` account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."
- ****Remote Control**** (``claude remote-control`` / ``/remote-control``, ``--rc``): pushes a **live local** session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an **undocumented** ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when ``ANTHROPIC_BASE_URL`` is

redirected; unavailable under Zero Data Retention.

- **Claude Code on the web** (claude.ai/code): runs on Anthropic cloud infra, no local filesystem.

- **Residual gap:** No true device-to-device *local* session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an *unexpected*, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

Codex CLI (OpenAI). The most architecturally mature cross-surface model:

- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared **App Server** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under

`~/codex/sessions/YYYY/MM/DD/rollout-\*.jsonl`, with a `session\_meta` first line.

- Rich lifecycle: `codex resume` (picker), `--last`, `--resume-session-id`, `/fork`, archive/unarchive (v0.136), delete (v0.140). Imports histories from other agents via an `external-agent-sessions` crate.

- **Known gaps/bugs:** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing `encrypted\_content` often can't be continued after crossing provider/account (`invalid\_encrypted\_content`); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.

- **Residual gap:** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

Gemini CLI (Google). Auto-saves sessions to

`~/gemini/tmp/<project\_hash>/chats/` (project-scoped, hash of project root). `/resume` session browser, `--resume`/`-r` flag, `/chat save` checkpoints, and a shadow-git checkpointing system under `~/gemini/history/<project\_hash>`. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. **Residual gap:** everything local + path-hash-keyed; no sync.

opencode (sst). SQLite + file storage under `~/.local/share/opencode/`(`opencode.db`,
`storage/session|message|part`)`. Native **session sharing** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). **Residual gap:** sharing $\neq$ syncing; DB concurrency and cross-OS path issues remain.

Cursor CLI (cursor-agent). Chats live in local SQLite; CLI chats under `~/.cursor/chats`` and resume via `cursor-agent --resume``. **IDE chats and CLI chats do NOT share a session store** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend `&`` to push to a Cloud Agent, pick up at cursor.com/agents), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). **Residual gap:** no local cross-device sync, IDE ↔ CLI split, path issues.

Grok / Grok Build (xAI). Sessions auto-saved to `~/.grok/sessions/``, keyed by working directory. `/resume``, `--resume``, `-c``, `/fork``; subscription-gated (SuperGrok/X Premium+). Config in `~/.grok/config.toml``. No native cross-device sync. Still beta (v0.2.x as of July 2026). **Residual gap:** no sync at all.

GitHub Copilot CLI. Sessions in `~/.copilot/`(`session-state/<uuid>/events.jsonl`)`. Notably, Copilot CLI **syncs sessions to your GitHub account by default** and offers **remote control** from [GitHub.com/GitHub Mobile](https://github.com/GitHubMobile) (origin machine must stay online, `/keep-alive``). But **cross-device \*resume\*** is still not supported – manually copying a session dir and running `--resume`` overwrites `events.jsonl`` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session_id``. **Residual gap:** the sync is telemetry/history, not resumable working state across machines.

Amp (Sourcegraph). Threads sync to ampcode.com by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is

GUI-global, not file-based). ****Residual gap:**** vendor-locked, server-stored, no cross-vendor.

****Cross-vendor summary:**** Every vendor is building **within** its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

Tool	Location	Format	Path keying	Cross-device resume native?
Claude Code	`\${~}/.claude/projects/<encoded-path>/<session>.jsonl`	JSONL, append-only event stream	**Absolute path, non-alphanumerics→dashes**	Via cloud only (teleport/RC)
Codex	`\${~}/.codex/sessions/YYYY/MM/DD/rollout-*.jsonl`	JSONL + `session_meta` header	date-based; cwd in meta	Yes, via App Server
Gemini CLI	`\${~}/.gemini/tmp/<project_hash>/chats/`	JSON	**Hash of project root**	No
opencode	`\${~}/.local/share/opencode/`	(SQLite + files)	SQLite/JSON	project-keyed
Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)
Grok	`\${~}/.grok/sessions/`	JSON	working directory	No
Copilot CLI	`\${~}/.copilot/session-state/<uuid>/events.jsonl`	JSONL events	uuid	Account history sync; no resume

****The path-mapping problem (the crux).**** Claude Code derives the project folder name by escaping the absolute path: ``/Users/me/app`` → ``~/.claude/projects/-Users-me-app/``, while the same repo at ``C:\Users\me\app`` (or a different macOS username/location) produces a **different** folder. Consequently, after a naive file sync, ``claude --resume`` on the second machine cannot find the first machine's sessions. This is confirmed by the `claude-sync` author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker." Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via ``project_hash``; Cursor stores path metadata in chat rows

(cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. ****This is the single highest-value, currently-unmet technical feature.****

****Format churn risk.**** Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

****Mid-session cross-device resume feasibility (same vendor):**** Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present – *if* path keys are remapped. Codex's App Server makes it cloud-native already.

****Cross-vendor session replay (e.g., Claude Code session → Codex):**** ****Fundamentally impractical.**** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) `encrypted\_content` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. ****Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.****

3. Competitor teardown

****Session-continuity tools:****

- ****tawanorg/claude-sync**** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/.claude` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + ****age E2E encryption**** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.
- ****npow/session-sync**** (~1 star, very new): the only cross-vendor *session* translator – Claude Code → Codex/Amp/Gemini via

PostToolUse/Stop hooks. Proof the idea is wanted; nascent.

- ****dantelex/aisync ("ss")****: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- ****codsync/codsync**** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- ****kroxiksut/codexSync****: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- ****waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)**** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- ****iHildy/opencode-synced**** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- ****Dinesh3184/claude-session-sync****: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- ****renefichtmueller/claude-sync****: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

****Config/MCP/skills sync (DevSync's neighborhood):****

- ****Leoyangl83/sync-agents-settings**** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- ****nicepkg/vsync**** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- ****baranovxyz/agentsync**** and ****spxrogers/agentsync**** (beta): canonical ``.agents/`` config synced to 9–31 agents.
- ****neon-solutions/add-mcp****: one-command MCP install across ~16 agents (Neon).
- ****ztripez/mcp-sync****: MCP config sync across tools.
- ****AgentSync (Rust)****: symlink-based config sync.
- ****skills-sync****: skills + MCP workspace.
- ****DevSynq**** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

****URL-addressable / mobile-control / viewer approaches:****

- ****TokenRip**** (tokenrip.com): makes session state a URL-addressable artifact (``/a/<slug>``) any agent can fetch – a genuinely different architectural bet ("don't sync files; give

state a permanent URL").

- **Omnara** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was **archived in February 2026** because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)
- **Syncode – AI Programming Tool**: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- **jazzyalex/agent-sessions** & **kenn-io/agentsview**: local-first session browsers/search across 20+ agents (read/resume-command, not sync).
- **LAN-workspace approaches** and Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

Takeaway: The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- **Claude Code:** #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- **Codex:** #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI  app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
- **Copilot CLI:** #1947 (cloud-synced sessions), #1635 (cross-environment resume).
- **Cursor forum:** multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

****Where the durable moat is.**** Vendors will keep building ***intra*-ecosystem sync** (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. ****Cross-vendor unified profile**** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.

2. ****OS-aware path remapping as a first-class feature**** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows↔macOS. This is the wedge no competitor has nailed.

3. ****E2E encryption by default**** – table stakes given session files contain code, secrets, and full transcripts.

****Risks:****

- ****Format churn:**** Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).

- ****Vendor lock-out / ToS:**** Storing session data is user-owned data (fine), but ***auth tokens*** are not transferable (codexSync explicitly refuses; Codex `encrypted\_content` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.

- ****Vendors closing the gap:**** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This ***validates*** the demand while ***confirming*** that only the cross-vendor + cross-OS layer is defensible.

- ****Security/privacy expectations:**** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no

plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).

- **Honest limits:** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

Stage 0 – Fold in DevSync (immediate). Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge). Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/.claude/projects/` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- **First-class OS-aware path remapping:** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
- **Benchmark to advance:** `claude --resume` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode. Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid `encrypted\_content`/token pitfalls. For Gemini, remap `project\_hash`. For opencode, use safe SQLite snapshotting (the `sqlite3 .backup` pattern the community scripts use).

- **Benchmark:** each new adapter must pass the same cross-OS

resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One "profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not replay.

****Monetization & thresholds.**** Free/open-source core (build trust, match claude-sync) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSynq/OpenSync validate willingness to pay). ****Kill/pivot signal:**** if Anthropic ships account-level **local** session sync with automatic path remapping across OSes (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

Okay nice so reinstate it is so reinstate.dev is available should I go for it
should I purchase it?

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape

TL;DR

- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows↔macOS paths.
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows↔macOS sync.
- **Recommendation: build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode.** Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...\` and `/Users/...\` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.
3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.
4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

- **Claude Code (Anthropic).** Ships the richest consumer-facing multi-device story:
 - **`/teleport`** (`claude --teleport``): pulls a *cloud* session (started at claude.ai/code or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport``, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires **clean git state** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo

checkout, the branch pushed to remote, and the same claude.ai account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."

- **Remote Control** (``claude remote-control` / `/remote-control`, `--rc`): pushes a *live local* session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an *undocumented* ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `ANTHROPIC_BASE_URL` is redirected; unavailable under Zero Data Retention.`
- **Claude Code on the web** (claude.ai/code): runs on Anthropic cloud infra, no local filesystem.

- **Residual gap:** No true device-to-device **local** session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an **unexpected**, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

Codex CLI (OpenAI). The most architecturally mature cross-surface model:

- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared **App Server** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under ``~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl``, with a ``session_meta`` first line.

- Rich lifecycle: ``codex resume`` (picker), ``--last``, ``--resume-session-id``, ``/fork``, ``archive/unarchive`` (v0.136), ``delete`` (v0.140). Imports histories from other agents via an ``external-agent-sessions`` crate.

- **Known gaps/bugs:** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing ``encrypted_content`` often can't be continued after crossing provider/account (``invalid_encrypted_content``); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.

- **Residual gap:** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

Gemini CLI (Google). Auto-saves sessions to ``~/gemini/tmp/<project_hash>/chats/`` (project-scoped, hash of project root). ``/resume`` session browser, ``--resume``/``-r`` flag, ``/chat save`` checkpoints, and a shadow-git checkpointing system under ``~/gemini/history/<project_hash>``. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. **Residual gap:** everything local + path-hash-keyed; no sync.

opencode (sst). SQLite + file storage under ``~/local/share/opencode/`` (``opencode.db``, ``storage/session|message|part``). Native **session sharing** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). **Residual gap:** sharing ≠ syncing; DB concurrency and cross-OS path issues remain.

Cursor CLI (cursor-agent). Chats live in local SQLite; CLI chats under ``~/cursor/chats`` and resume via ``cursor-agent --resume``. **IDE chats and CLI chats do NOT share a session store** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend ``&`` to push to a Cloud Agent, pick up at cursor.com/agents), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). **Residual gap:** no local cross-device sync, IDE ↔ CLI split, path issues.

Grok / Grok Build (xAI). Sessions auto-saved to ``~/grok/sessions/``, keyed by working directory. ``/resume``, ``--resume``, ``-c``, ``/fork``; subscription-gated (SuperGrok/X Premium+). Config in ``~/grok/config.toml``. No native cross-device sync. Still beta (v0.2.x as of July

2026). ****Residual gap:**** no sync at all.

****GitHub Copilot CLI.**** Sessions in `~/copilot/` (`session-state/<uuid>/events.jsonl`). Notably, Copilot CLI ****syncs sessions to your GitHub account by default**** and offers ****remote control**** from GitHub.com/GitHub Mobile (origin machine must stay online, `/keep-alive`). But ****cross-device *resume* is still not supported**** – manually copying a session dir and running `--resume` overwrites `events.jsonl` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session\_id`. ****Residual gap:**** the sync is telemetry/history, not resumable working state across machines.

****Amp (Sourcegraph).**** Threads sync to ampcode.com by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). ****Residual gap:**** vendor-locked, server-stored, no cross-vendor.

****Cross-vendor summary:**** Every vendor is building *within* its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

Tool	Location	Format	Path keying	Cross-device resume native?
				--- --- --- --- ---
Claude Code	~/claude/projects/<encoded-path>/<session>.jsonl	JSONL, append-only event stream	**Absolute path, non-alphanumerics→dashes**	Via cloud only (teleport/RC)
Codex	~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl	JSONL + `session_meta` header	date-based; cwd in meta	Yes, via App Server
Gemini CLI	~/gemini/tmp/<project_hash>/chats/	JSON	**Hash of project root**	No
opencode	~/local/share/opencode/	(SQLite + files)	SQLite/JSON	project-keyed Sharing only
Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)
Grok	~/grok/sessions/	JSON	working directory	No
Copilot CLI	~/copilot/session-state/<uuid>/events.jsonl	JSONL events	uuid	Account history sync; no resume

****The path-mapping problem (the crux).**** Claude Code derives the project folder name by escaping the absolute path: `/Users/me/app` → `~/claude/projects/-Users-me-app/`, while the same repo at `C:\Users\me\app` (or a different macOS username/location) produces a *different* folder. Consequently, after a naive file sync, `claude --resume` on the second machine cannot find the first machine's sessions. This is confirmed by the claude-sync author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker."

Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via `project\_hash`; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. ****This is the single highest-value, currently-unmet technical feature.****

****Format churn risk.**** Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

****Mid-session cross-device resume feasibility (same vendor):**** Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present – *if* path keys are remapped. Codex's App Server makes it cloud-native already.

****Cross-vendor session replay (e.g., Claude Code session → Codex):**** ****Fundamentally**

impractical.** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) `encrypted\_content` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. **Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.**

3. Competitor teardown

Session-continuity tools:

- **tawanorg/claude-sync** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/.claude` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + **age** E2E encryption ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. **Punts on path remapping** (its explicit, acknowledged weakness). Open-source, no business model.
 - **npow/session-sync** (~1 star, very new): the only cross-vendor **session** translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
 - **dantelex/aisync** ("ss"): multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
 - **codsync/codsync** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
 - **kroxiksut/codexSync**: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
 - **waynesutton/opencode-sync-plugin** & **codex-sync-plugin** (OpenSync) (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- **iHildy/opencode-synced** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- **Dinesh3184/claude-session-sync**: iCloud Drive sync GUI for Claude Code (macOS + Windows).
 - **renefichtmueller/claude-sync**: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

Config/MCP/skills sync (DevSync's neighborhood):

- **Leoyang183/sync-agents-settings** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
 - **nicepkg/vsync** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- **baranovxyz/agentsync** and **spxrogers/agentsync** (beta): canonical `~/.agents/` config synced to 9–31 agents.
 - **neon-solutions/add-mcp**: one-command MCP install across ~16 agents (Neon).
 - **ztripez/mcp-sync**: MCP config sync across tools.
 - **AgentSync** (Rust): symlink-based config sync.
 - **skills-sync**: skills + MCP workspace.
 - **DevSynq** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

URL-addressable / mobile-control / viewer approaches:

- **TokenRip** (tokenrip.com): makes session state a URL-addressable artifact (`/a/<slug>`) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- **Omnara** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was **archived** in February 2026 because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)
- **Syncode – AI Programming Tool**: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- **jazzyalex/agent-sessions** & **kenn-io/agentsview**: local-first session browsers/search

across 20+ agents (read/resume-command, not sync).

- **LAN-workspace approaches** and Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

Takeaway: The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- **Claude Code:** #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- **Codex:** #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI ↔ app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
- **Copilot CLI:** #1947 (cloud-synced sessions), #1635 (cross-environment resume).
- **Cursor forum:** multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is. Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. **Cross-vendor unified profile** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.
2. **OS-aware path remapping as a first-class feature** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows ↔ macOS. This is the wedge no competitor has nailed.
3. **E2E encryption by default** – table stakes given session files contain code, secrets, and full transcripts.

Risks:

- **Format churn:** Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).
- **Vendor lock-out / ToS:** Storing session data is user-owned data (fine), but *auth tokens* are not transferable (codexSync explicitly refuses; Codex `encrypted\_content` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.
- **Vendors closing the gap:** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This *validates* the demand while *confirming* that only the cross-vendor + cross-OS layer is defensible.
- **Security/privacy expectations:** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).
- **Honest limits:** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort

context handoff.

Recommendations

****Stage 0 – Fold in DevSync (immediate).**** Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

****Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge).****
Target the founder's exact case (Windows desktop 🖥️ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/ .claude/projects/` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- ****First-class OS-aware path remapping:**** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
- ****Benchmark to advance:**** `claude --resume` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

****Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode.**** Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid `encrypted\_content`/token pitfalls. For Gemini, remap `project\_hash`. For opencode, use safe SQLite snapshotting (the `sqlite3 .backup` pattern the community scripts use).

- ****Benchmark:**** each new adapter must pass the same cross-OS resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One "profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not replay.

****Monetization & thresholds.**** Free/open-source core (build trust, match claude-sync) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSync/OpenSync validate willingness to pay). ****Kill/pivot signal:**** if Anthropic ships account-level *local* session sync with automatic path remapping across OSES (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

Yes. Buy it today.

The math is trivial: a .dev runs roughly \$12–15/year. If you abandon the project or rename in three months, you're out fifteen dollars. If you deliberate for two weeks and someone else registers it, you've lost a name you spent this entire conversation earning. There's no version of this where waiting is the smart play.

But do the free stuff in the same sitting, because it matters more.

For a CLI tool, the domain is the *least* important namespace:

- **npm** — `npm view reinstate`. If it's free, publish a stub package immediately to claim it. This is the name in your install command.
- **GitHub org** — `github.com/reinstate`. Free, thirty seconds, and it's where your users actually go.
- **Domain** — `reinstate.dev`.

If npm turns out taken, ship as `@yourhandle/reinstate` and keep everything else. Doesn't change the decision.

Registrar notes: Cloudflare Registrar sells domains at wholesale with no markup and no cheap-first-year-then-triple-renewal games. Porkbun and Namecheap are also fine. Skip GoDaddy. Turn on auto-renew and WHOIS privacy (free at all three).

One thing to know about .dev: the whole TLD is on the HSTS preload list, so it's HTTPS-only — browsers won't load it over plain HTTP at all. Completely fine for a dev tool, just don't be confused when a local HTTP test fails.

Don't over-buy. You don't need .com, .io, or .sh right now. One domain, one npm name, one GitHub org. If the project takes off you can grab defensives later; buying them now is money spent on a hypothetical.

Two things worth knowing, neither a blocker: "reinstate" is a common English word, so you'd have weak trademark protection if you ever wanted it — but that cuts both ways, since nobody can credibly come after you either. And you'll rank for "reinstate cli" or "reinstate dev tool" rather than "reinstate" alone, which is normal and fine.

Go register it, then get to the path remapping.

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

```
# Cross-Device Sync for Terminal AI Coding Agents: Competitive &
Technical Landscape
```

```
## TL;DR
```

```
- **The durable opportunity is a cross-vendor abstraction layer
with OS-aware path remapping as its headline feature – NOT
another "sync my Claude sessions" tool.** Every major vendor is
```

now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across **competing** tools and remapping Windows ↔ macOS paths.

- ****Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;**** config/skills/instruction portability and **same-vendor** cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows ↔ macOS sync.

- ****Recommendation: build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opcode.**** Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. ****Native vendor sync is real but ecosystem-locked and gap-riddled.**** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.

2. ****The path-mapping problem is the crux and is largely unsolved.**** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...\` and `/Users/...\` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.

3. ****Demand is loud and repeatedly expressed**** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask

for exactly this. Vendors are responding, but only within their own walls.

4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

Claude Code (Anthropic). Ships the richest consumer-facing multi-device story:

- **`/teleport`** (`claude --teleport`): pulls a *cloud* session (started at `claude.ai/code` or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport`, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires **clean git state** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same `claude.ai` account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."
- **Remote Control** (`claude remote-control` / `/remote-control`, `--rc`): pushes a *live local* session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an *undocumented* ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `ANTHROPIC_BASE_URL` is redirected; unavailable under Zero Data Retention.
- **Claude Code on the web** (`claude.ai/code`): runs on Anthropic cloud infra, no local filesystem.

- ****Residual gap:**** No true device-to-device ***local*** session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an ***unexpected***, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

****Codex CLI (OpenAI).**** The most architecturally mature cross-surface model:

- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared ****App Server**** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under

`~/.codex/sessions/YYYY/MM/DD/rollout-\*.jsonl`, with a `session\_meta` first line.

- Rich lifecycle: `codex resume` (picker), `--last`, `--resume-session-id`, `/fork`, archive/unarchive (v0.136), delete (v0.140). Imports histories from other agents via an `external-agent-sessions` crate.

- ****Known gaps/bugs:**** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing `encrypted\_content` often can't be continued after crossing provider/account (`invalid\_encrypted\_content`); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.

- ****Residual gap:**** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

****Gemini CLI (Google).**** Auto-saves sessions to

`~/.gemini/tmp/<project\_hash>/chats/` (project-scoped, hash of project root). `/resume` session browser, `--resume`/`-r` flag, `/chat save` checkpoints, and a shadow-git checkpointing system under `~/.gemini/history/<project\_hash>`. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. ****Residual gap:**** everything local + path-hash-keyed; no sync.

****opencode (sst).**** SQLite + file storage under

`~/.local/share/opencode/` (`opencode.db`,

`storage/session|message|part`). Native ****session sharing**** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). ****Residual gap:**** sharing $\neq$ syncing; DB concurrency and cross-OS path issues remain.

****Cursor CLI (cursor-agent).**** Chats live in local SQLite; CLI chats under `~/.cursor/chats` and resume via `cursor-agent --resume`. ****IDE chats and CLI chats do NOT share a session store**** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend `&` to push to a Cloud Agent, pick up at cursor.com/agents), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). ****Residual gap:**** no local cross-device sync, IDE ↔ CLI split, path issues.

****Grok / Grok Build (xAI).**** Sessions auto-saved to `~/.grok/sessions/`, keyed by working directory. `/resume`, `--resume`, `-c`, `/fork`; subscription-gated (SuperGrok/X Premium+). Config in `~/.grok/config.toml`. No native cross-device sync. Still beta (v0.2.x as of July 2026). ****Residual gap:**** no sync at all.

****GitHub Copilot CLI.**** Sessions in `~/.copilot/` (`session-state/<uuid>/events.jsonl`). Notably, Copilot CLI ****syncs sessions to your GitHub account by default**** and offers ****remote control**** from GitHub.com/GitHub Mobile (origin machine must stay online, `/keep-alive`). But ****cross-device *resume* is still not supported**** – manually copying a session dir and running `--resume` overwrites `events.jsonl` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session\_id`. ****Residual gap:**** the sync is telemetry/history, not resumable working state across machines.

****Amp (Sourcegraph).**** Threads sync to ampcode.com by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). ****Residual gap:**** vendor-locked, server-stored, no cross-vendor.

****Cross-vendor summary:**** Every vendor is building **within** its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

Tool	Location	Format	Path keying	Cross-device resume native?
Claude Code	`\${HOME}/.claude/projects/<encoded-path>/<session>.jsonl`	JSONL, append-only event stream	**Absolute path, non-alphanumerics→dashes**	Via cloud only (teleport/RC)
Codex	`\${HOME}/.codex/sessions/YYYY/MM/DD/rollout-*.jsonl`	JSONL + `session_meta` header	date-based; cwd in meta	Yes, via App Server
Gemini CLI	`\${HOME}/.gemini/tmp/<project_hash>/chats/`	JSON	**Hash of project root**	No
opencode	`\${HOME}/.local/share/opencode/`	(SQLite + files)	SQLite/JSON	project-keyed Sharing only
Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)
Grok	`\${HOME}/.grok/sessions/`	JSON	working directory	No
Copilot CLI	`\${HOME}/.copilot/session-state/<uuid>/events.jsonl`	JSONL events	uuid	Account history sync; no resume

****The path-mapping problem (the crux).**** Claude Code derives the project folder name by escaping the absolute path: ``/Users/me/app`` → ``~/.claude/projects/-Users-me-app/``, while the same repo at ``C:\Users\me\app`` (or a different macOS username/location) produces a **different** folder. Consequently, after a naive file sync, ``claude --resume`` on the second machine cannot find the first machine's sessions. This is confirmed by the `claude-sync` author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker." Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via ``project_hash``; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path

references on pull. ****This is the single highest-value, currently-unmet technical feature.****

****Format churn risk.**** Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

****Mid-session cross-device resume feasibility (same vendor):**** Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present – *if* path keys are remapped. Codex's App Server makes it cloud-native already.

****Cross-vendor session replay (e.g., Claude Code session → Codex):**** ****Fundamentally impractical.**** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) `encrypted_content` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. ****Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.****

3. Competitor teardown

****Session-continuity tools:****

- ****tawanorg/claude-sync**** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/ .claude` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + ****age E2E encryption**** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.
- ****npow/session-sync**** (~1 star, very new): the only cross-vendor *session* translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- ****dantelex/aisync ("ss")****: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder

(iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)

- ****codsync/codsync**** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- ****kroxiksut/codexSync****: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- ****waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)**** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- ****iHildy/opencode-synced**** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- ****Dinesh3184/claude-session-sync****: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- ****renefichtmueller/claude-sync****: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

****Config/MCP/skills sync (DevSync's neighborhood):****

- ****Leoyang183/sync-agents-settings**** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- ****nicepkg/vsync**** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- ****baranovxyz/agentsync**** and ****spxrogers/agentsync**** (beta): canonical ``.agents/`` config synced to 9–31 agents.
- ****neon-solutions/add-mcp****: one-command MCP install across ~16 agents (Neon).
- ****ztripez/mcp-sync****: MCP config sync across tools.
- ****AgentSync (Rust)****: symlink-based config sync.
- ****skills-sync****: skills + MCP workspace.
- ****DevSynq**** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

****URL-addressable / mobile-control / viewer approaches:****

- ****TokenRip**** (tokenrip.com): makes session state a URL-addressable artifact (``/a/<slug>``) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- ****Omnara**** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo

(~2.6k stars) was **archived** in February 2026 because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)

- **Syncode – AI Programming Tool**: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).

- **jazzyalex/agent-sessions** & **kenn-io/agentsview**: local-first session browsers/search across 20+ agents (read/resume-command, not sync).

- **LAN-workspace approaches** and

Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

Takeaway: The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- **Claude Code**: #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.

- **Codex**: #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI  app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).

- **Copilot CLI**: #1947 (cloud-synced sessions), #1635 (cross-environment resume).

- **Cursor forum**: multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature

requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

****Where the durable moat is.**** Vendors will keep building ***intra*-ecosystem sync** (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. ****Cross-vendor unified profile**** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.

2. ****OS-aware path remapping as a first-class feature**** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows↔macOS. This is the wedge no competitor has nailed.

3. ****E2E encryption by default**** – table stakes given session files contain code, secrets, and full transcripts.

****Risks:****

- ****Format churn:**** Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).

- ****Vendor lock-out / ToS:**** Storing session data is user-owned data (fine), but ***auth tokens*** are not transferable (codexSync explicitly refuses; Codex `encrypted\_content` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.

- ****Vendors closing the gap:**** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This ***validates*** the demand while ***confirming*** that only the cross-vendor + cross-OS layer is defensible.

- ****Security/privacy expectations:**** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).

- ****Honest limits:**** Cross-vendor mid-session replay is

impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

****Stage 0 – Fold in DevSync (immediate).**** Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

****Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge).**** Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/.claude/projects/` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- ****First-class OS-aware path remapping:**** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
- ****Benchmark to advance:**** `claude --resume` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

****Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode.**** Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid `encrypted\_content`/token pitfalls. For Gemini, remap `project\_hash`. For opencode, use safe SQLite snapshotting (the `sqlite3 .backup` pattern the community scripts use).

- ****Benchmark:**** each new adapter must pass the same cross-OS resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One

"profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not replay.

****Monetization & thresholds.**** Free/open-source core (build trust, match claude-sync) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSynq/OpenSync validate willingness to pay). ****Kill/pivot signal:**** if Anthropic ships account-level **local** session sync with automatic path remapping across OSes (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.

acquired. Now a very important decision or a very important choice I guess or like a strategic decision Should we open source it or should we build it as an open source product from day one from scratch? or should we go like closed

source route first and then eventually open source it? Which route should we take? So essentially the question is do we want it to be a tool or do we want it to be a product?

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape

TL;DR

- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows↔macOS paths.
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;** config/skills/instruction portability and *same-vendor* cross-device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows↔macOS sync.
- **Recommendation:** build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode. Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...` and `/Users/...` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.
3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.
4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

- **Claude Code (Anthropic).** Ships the richest consumer-facing multi-device story: `/teleport`` (``claude --teleport``): pulls a *cloud* session (started at claude.ai/code or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with ``--teleport``, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only

exception. Teleport requires ****clean git state**** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same claude.ai account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."

- ****Remote Control**** (``claude remote-control` / `/remote-control`, `--rc`): pushes a *live local* session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an *undocumented* ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `ANTHROPIC_BASE_URL` is redirected; unavailable under Zero Data Retention.`

- ****Claude Code on the web**** (claude.ai/code): runs on Anthropic cloud infra, no local filesystem.

- ****Residual gap:**** No true device-to-device **local** session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an **unexpected**, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

****Codex CLI (OpenAI).**** The most architecturally mature cross-surface model:

- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared ****App Server**** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under ``~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl``, with a ``session_meta`` first line.

- Rich lifecycle: ``codex resume`` (picker), ``--last``, ``--resume-session-id``, ``/fork``, ``archive/unarchive`` (v0.136), ``delete`` (v0.140). Imports histories from other agents via an ``external-agent-sessions`` crate.

- ****Known gaps/bugs:**** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing ``encrypted_content`` often can't be continued after crossing provider/account (``invalid_encrypted_content``); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.

- ****Residual gap:**** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

****Gemini CLI (Google).**** Auto-saves sessions to ``~/gemini/tmp/<project_hash>/chats/`` (project-scoped, hash of project root). ``/resume`` session browser, ``--resume``/``-r`` flag, ``/chat save`` checkpoints, and a shadow-git checkpointing system under ``~/gemini/history/<project_hash>``. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. ****Residual gap:**** everything local + path-hash-keyed; no sync.

****opencode (sst).**** SQLite + file storage under ``~/local/share/opencode/`` (``opencode.db``, ``storage/session|message|part``). Native ****session sharing**** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). ****Residual gap:**** sharing ≠ syncing; DB concurrency and cross-OS path issues remain.

****Cursor CLI (cursor-agent).**** Chats live in local SQLite; CLI chats under ``~/cursor/chats`` and resume via ``cursor-agent --resume``. ****IDE chats and CLI chats do NOT share a session store**** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend ``&`` to push to a Cloud Agent, pick up at cursor.com/agents), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). ****Residual gap:**** no local cross-device sync, IDE ↔ CLI split, path issues.

****Grok / Grok Build (xAI).**** Sessions auto-saved to ``~/grok/sessions/``, keyed by working

directory. ``~/resume`, --resume`, -c`, `~/fork`; subscription-gated (SuperGrok/X Premium+). Config in ~/grok/config.toml`. No native cross-device sync. Still beta (v0.2.x as of July 2026). Residual gap: no sync at all.`

GitHub Copilot CLI. Sessions in `~/copilot/` (session-state/<uuid>/events.jsonl`)`. Notably, Copilot CLI **syncs sessions to your GitHub account by default** and offers **remote control** from GitHub.com/GitHub Mobile (origin machine must stay online, `/keep-alive`)`. But **cross-device resume** is still not supported – manually copying a session dir and running `--resume`` overwrites `events.jsonl`` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session_id``. **Residual gap:** the sync is telemetry/history, not resumable working state across machines.

Amp (Sourcegraph). Threads sync to `ampcode.com` by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). **Residual gap:** vendor-locked, server-stored, no cross-vendor.

Cross-vendor summary: Every vendor is building *within* its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

	Tool	Location	Format	Path keying	Cross-device resume native?
					--- --- --- --- ---
	Claude Code	<code>~/claude/projects/<encoded-path>/<session>.jsonl`</code>	JSONL, append-only event stream	Absolute path, non-alphanumerics→dashes	Via cloud only (teleport/RC)
	Codex	<code>~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl`</code>	JSONL + <code>session_meta`</code> header	date-based; cwd in meta	Yes, via App Server
	Gemini CLI	<code>~/gemini/tmp/<project_hash>/chats/`</code>	JSON	Hash of project root	No
	opencode	<code>~/local/share/opencode/`</code>	(SQLite + files)	SQLite/JSON	project-keyed
					Sharing only
	Cursor	local SQLite	DB rows	workspace/path metadata	No (cloud handoff only)
				Grok	<code>~/grok/sessions/`</code>
	Copilot CLI	<code>~/copilot/session-state/<uuid>/events.jsonl`</code>	JSONL events	uuid	Account history sync; no resume

The path-mapping problem (the crux). Claude Code derives the project folder name by escaping the absolute path: `/Users/me/app`` → `~/claude/projects/-Users-me-app/``, while the same repo at `C:\Users\me\app`` (or a different macOS username/location) produces a *different* folder. Consequently, after a naive file sync, `claude --resume`` on the second machine cannot find the first machine's sessions. This is confirmed by the `claude-sync` author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker."

Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via `project_hash``; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. **This is the single highest-value, currently-unmet technical feature.**

Format churn risk. Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

Mid-session cross-device resume feasibility (same vendor): Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present – *if* path keys are remapped. Codex's App Server makes it cloud-native already.

****Cross-vendor session replay (e.g., Claude Code session → Codex):**** ****Fundamentally impractical.**** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) `encrypted\_content` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. ****Recommendation: treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.****

3. Competitor teardown

****Session-continuity tools:****

- ****tawanorg/claude-sync**** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/.claude` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + ****age E2E encryption**** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.
- ****npow/session-sync**** (~1 star, very new): the only cross-vendor *session* translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- ****dantelex/aisync ("ss")****: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- ****codsync/codsync**** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- ****kroxiksut/codexSync****: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- ****waynesutton/opencode-sync-plugin & codex-sync-plugin (OpenSync)**** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.
- ****iHildy/opencode-synced**** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- ****Dinesh3184/claude-session-sync****: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- ****renefichtmueller/claude-sync****: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

****Config/MCP/skills sync (DevSync's neighborhood):****

- ****Leoyang183/sync-agents-settings**** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- ****nicepkg/vsync**** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- ****baranovxyz/agentsync**** and ****spxrogers/agentsync**** (beta): canonical `~/.agents/` config synced to 9–31 agents.
- ****neon-solutions/add-mcp****: one-command MCP install across ~16 agents (Neon).
- ****ztripez/mcp-sync****: MCP config sync across tools.
- ****AgentSync (Rust)****: symlink-based config sync.
- ****skills-sync****: skills + MCP workspace.
- ****DevSynq**** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

****URL-addressable / mobile-control / viewer approaches:****

- ****TokenRip**** (tokenrip.com): makes session state a URL-addressable artifact (`/a/<slug>`) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- ****Omnara**** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was ****archived in February 2026**** because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)
- ****Syncode – AI Programming Tool****: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).

- **jazzyalex/agent-sessions** & **kenn-io/agentsview**: local-first session browsers/search across 20+ agents (read/resume-command, not sync).
- **LAN-workspace approaches** and Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

Takeaway: The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- **Claude Code:** #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- **Codex:** #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
- **Copilot CLI:** #1947 (cloud-synced sessions), #1635 (cross-environment resume).
- **Cursor forum:** multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is. Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. **Cross-vendor unified profile** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.
2. **OS-aware path remapping as a first-class feature** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows/macOS. This is the wedge no competitor has nailed.
3. **E2E encryption by default** – table stakes given session files contain code, secrets, and full transcripts.

Risks:

- **Format churn:** Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).
- **Vendor lock-out / ToS:** Storing session data is user-owned data (fine), but *auth tokens* are not transferable (codexSync explicitly refuses; Codex `encrypted\_content` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.
- **Vendors closing the gap:** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This *validates* the demand while *confirming* that only the cross-vendor + cross-OS layer is defensible.
- **Security/privacy expectations:** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).
- **Honest limits:** Cross-vendor mid-session replay is impossible; over-promising it will

burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

****Stage 0 – Fold in DevSync (immediate).**** Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

****Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge).****
Target the founder's exact case (Windows desktop 🖥️ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/ .claude/projects/` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- ****First-class OS-aware path remapping:**** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
- ****Benchmark to advance:**** `claude --resume` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

****Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode.**** Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid `encrypted\_content`/token pitfalls. For Gemini, remap `project\_hash`. For opencode, use safe SQLite snapshotting (the `sqlite3 .backup` pattern the community scripts use).

- ****Benchmark:**** each new adapter must pass the same cross-OS resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One "profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not replay.

****Monetization & thresholds.**** Free/open-source core (build trust, match claude-sync) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSynq/OpenSynq validate willingness to pay). ****Kill/pivot signal:**** if Anthropic ships account-level *local* session sync with automatic path remapping across OSeS (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show

Open source from day one. And the reason is specific to your product, not a general preference.

The decisive argument: you're asking for extraordinary trust.

Your tool reads `~/.claude/projects/`. That directory contains source code, architecture discussions, API keys that leaked into transcripts, and client work under NDA. Then it uploads all of it somewhere. A closed-source binary doing that is a very hard ask — and correctly so. The entire credibility of claude-sync, the leading tool in your space, rests on being open source *plus* age-encrypted, so anyone can verify the encryption happens before anything leaves the machine.

Closed source can't tell that story. Open source plus E2E encryption can: "*we can't read your data, and here's the code that proves it.*" That's not a nice-to-have for you, it's the precondition for anyone installing this.

Closed-then-open is the worst of the three options. It's tempting because it feels reversible, but here it isn't:

- You burn the trust window exactly when you need it — during early adoption, when nobody knows you.
- Your entire competitive set (claude-sync, codsync, opencode-synced, Sentry's dotagents) is already open. Being the one closed tool that reads people's session files makes you the suspicious one by default.
- Opening later doesn't retroactively earn goodwill. People remember the posture you launched with.
- You'll accumulate architectural decisions that assume closed — hardcoded backends, telemetry, license headers — that are painful to unwind.

There's also a maintenance argument that's unusually strong here.

The research flagged that Claude Code's session format "changes between versions," and you'll need a new adapter for every agent — Codex, Gemini, opencode, Grok, Amp, plus whatever ships next quarter. Adapters are exactly the contribution open source gets well: someone who uses Amp writes the Amp adapter because they want it to work. Closed source means you personally maintain every adapter against every upstream release, forever. That's the thing that kills solo projects.

And there's no secret to protect. Path remapping is engineering effort, not IP. Anyone can watch your binary's file writes and reimplement it in a weekend.

Now the sharper question: tool or product?

These are sequential, not exclusive, and the sequencing is the actual decision:

- **Tool:** open-source CLI, local-first, bring-your-own-storage (S3, R2,

WebDAV, a private git repo). Free. Grows via GitHub, HN, r/ClaudeAI.

- **Product:** hosted backend for people who don't want to configure R2 buckets, plus team profiles, a dashboard, multi-machine management.

The tool is the distribution channel for the product. Sentry, Supabase, PostHog, Cal.com all ran this play. **The one architectural decision that keeps both doors open: define a storage backend interface in your first commit.** Local folder, S3-compatible, WebDAV — and later, "Reinstate Cloud" is just another implementation of that interface. Cost today: near zero. Cost of retrofitting it later: substantial.

License: Apache-2.0. MIT is fine too. Skip AGPL and BSL — for a CLI at your stage, restrictive licenses cost you adoption and protect against a threat (someone hosting a competing Reinstate Cloud) that isn't real until you have traction. You can always relicense a hosted *service* differently later; the CLI staying permissive is what gets it installed.

The honest caveat. Open source is a distribution strategy, not a business model. Syncthing is beloved and earns nothing. The realistic median outcome for a project like this is a well-liked repo with a few hundred stars and no revenue — which, given you opened this conversation saying you'd build it anyway to scratch your own itch, may be exactly the right outcome. Just know which one you're aiming at, because it changes whether you build the hosted layer in month two or month twenty.

My call: open source, Apache-2.0, tool-first, pluggable storage from commit one. Decide about the product after you find out whether anyone but you wants it.

Cross-Device Sync for Terminal AI Coding Agents: Competitive and Technical Landscape

```
# Cross-Device Sync for Terminal AI Coding Agents: Competitive & Technical Landscape
```

```
## TL;DR
```

```
- **The durable opportunity is a cross-vendor abstraction layer with OS-aware path remapping as its headline feature – NOT another "sync my Claude sessions" tool.** Every major vendor is now shipping single-ecosystem sync natively (Claude Code Remote Control/teleport, Codex's unified App Server backend, Amp threads, Copilot CLI cloud sync), so the defensible wedge is the one thing no vendor will ever build: normalizing sessions + MCP + skills + instruction files across *competing* tools and remapping Windows↔macOS paths.
```

```
- **Mid-session cross-vendor replay (e.g., Claude Code → Codex) is effectively impossible and should be dropped from scope;** config/skills/instruction portability and *same-vendor* cross-
```

device session continuity are the achievable, valuable core. The single sharpest, currently-unsolved technical pain – which even the leading competitor (claude-sync) explicitly punts on – is Claude Code's absolute-path project indexing breaking on Windows  macOS sync.

- **Recommendation:** build a "universal profile" that folds DevSync's MCP-sync foundation in, adds E2E-encrypted session sync with first-class path remapping for Claude Code first, then Codex/Gemini/opencode. Ship path remapping as the MVP differentiator; treat E2E encryption as table stakes; do not try to out-run vendors on same-vendor live handoff.

Key Findings

1. **Native vendor sync is real but ecosystem-locked and gap-riddled.** Claude Code teleport is one-way (cloud→local) and requires a clean git tree; Remote Control needs the origin machine to stay awake and online. Codex has the most mature multi-surface story (shared App Server backend) but it's OpenAI-only and cloud-account-bound. None of these help a developer who mixes Claude Code on a Windows desktop and Codex or Claude on a MacBook.
2. **The path-mapping problem is the crux and is largely unsolved.** Claude Code, Gemini CLI, and Cursor all key sessions to the absolute project path (or a hash of it). Syncing raw files between `C:\Users\...\` and `/Users/...\` produces orphaned, unresumable sessions. The leading sync tool (claude-sync) openly admits "there's no path remapping logic." Only a couple of niche tools (cursaves for Cursor, claudepath for local moves) attempt rewriting.
3. **Demand is loud and repeatedly expressed** in vendor issue trackers – multiple high-engagement Claude Code and Codex feature requests, Copilot CLI requests, and Cursor forum threads all ask for exactly this. Vendors are responding, but only within their own walls.
4. **The competitor field is fragmented, early, and mostly config-only or single-vendor.** No tool today delivers a polished, cross-vendor, path-aware, E2E-encrypted unified profile. That is the open lane.

Details

1. Native vendor capability matrix (as of July 2026)

Claude Code (Anthropic). Ships the richest consumer-facing multi-device story:

- **`/teleport`** (`claude --teleport`): pulls a *cloud* session (started at `claude.ai/code` or the mobile app) down into the local terminal. It is **one-way** – you cannot push a terminal session to the web via teleport. Anthropic's docs state: "From the CLI, session handoff is one-way: you can pull cloud sessions into your terminal with `--teleport`, but you can't push an existing terminal session to the web." The Desktop app's "Continue in" menu is the only exception. Teleport requires **clean git state** ("Your working directory must have no uncommitted changes. Teleport prompts you to stash changes if needed"), the same repo checkout, the branch pushed to remote, and the same `claude.ai` account. Teleport also gives the terminal its own copy – "new work there stays local and doesn't appear in the cloud session."
- **Remote Control** (`claude remote-control` / `/remote-control`, `--rc`): pushes a *live local* session to phone/web. The session runs locally the whole time; web/mobile are "a window into that local session." Limitations, verbatim from the docs: "Remote Control runs as a local process. If you close the terminal, quit VS Code, or otherwise stop the claude process, the session ends," and "if your machine is awake but unable to reach the network for more than roughly 10 minutes, the session times out and the process exits." (Separately, issue #39970 reports an *undocumented* ~15-minute idle timeout on the Desktop/Code surface that "is not documented and the timeout is not communicated to the user before it happens.") Research preview; Pro/Max/Team/Enterprise only (no API keys); disabled on Bedrock/Vertex/Foundry and when `ANTHROPIC_BASE_URL` is redirected; unavailable under Zero Data Retention.
- **Claude Code on the web** (`claude.ai/code`): runs on Anthropic cloud infra, no local filesystem.
- **Residual gap**: No true device-to-device *local* session migration (Windows desktop ↔ MacBook) without the origin machine staying online. No cross-vendor anything. Anthropic has begun syncing conversation history across devices on the same account (reported in issue #71794 as an *unexpected*, un-opted-in behavior), suggesting account-level sync is coming – but it will be Claude-only.

Codex CLI (OpenAI). The most architecturally mature cross-surface model:

- CLI, Desktop app (macOS + Windows), IDE/VS Code extension, and Cloud runtime all connect to a shared ****App Server**** (long-lived process exposing a JSON-RPC 2.0 API over stdio or WebSocket). A thread started in the CLI can be resumed in Desktop "without any export/import dance." Sessions are JSONL under `~/codex/sessions/YYYY/MM/DD/rollout-*.jsonl``, with a ``session_meta`` first line.
- Rich lifecycle: ``codex resume`` (picker), ``--last``, ``--resume-session-id``, ``/fork``, `archive/unarchive (v0.136)`, `delete (v0.140)`. Imports histories from other agents via an ``external-agent-sessions`` crate.
- ****Known gaps/bugs:**** Desktop may not list CLI sessions if the app-server wasn't running at creation; ephemeral sessions aren't persisted; sessions containing ``encrypted_content`` often can't be continued after crossing provider/account (``invalid_encrypted_content``); Desktop's session list only pulls the recent ~50. Open requests (#12593, #14722, discussion #14067) ask for account-linked cross-device sync and CLI ↔ app-server live sync.
- ****Residual gap:**** Sync is OpenAI-account-bound and cloud-mediated; no cross-vendor; local file portability still hits path issues.

****Gemini CLI (Google).**** Auto-saves sessions to `~/gemini/tmp/<project_hash>/chats/`` (project-scoped, hash of project root). ``/resume`` session browser, ``--resume`/`-r`` flag, ``/chat save`` checkpoints, and a shadow-git checkpointing system under `~/gemini/history/<project_hash>``. No native cross-device cloud sync. Note: Google announced Gemini CLI is being replaced by "Antigravity CLI" for unpaid/Google One tiers. ****Residual gap:**** everything local + path-hash-keyed; no sync.

****opencode (sst).**** SQLite + file storage under `~/local/share/opencode/`` (``opencode.db``, ``storage/session|message|part``). Native ****session sharing**** publishes to Cloudflare Durable Objects for a public web viewer (read/playback, not cross-device resume). A healthy third-party ecosystem already exists (opencode-synced, opencode-sync, opencode-sync-plugin/OpenSync, opencode-session-backup). ****Residual gap:**** sharing ≠ syncing; DB concurrency and cross-OS path issues remain.

****Cursor CLI (cursor-agent).**** Chats live in local SQLite; CLI chats under `~/cursor/chats`` and resume via ``cursor-agent --`

resume`. **\*\*IDE chats and CLI chats do NOT share a session store\*\*** (confirmed by Cursor staff in the forum). Cloud handoff exists (prepend `&` to push to a Cloud Agent, pick up at cursor.com/agents), and "My Machines" lets cloud agents run on your registered machine. Strong user demand for account-level chat sync (forum). ****Residual gap:**** no local cross-device sync, IDE↔CLI split, path issues.

****Grok / Grok Build (xAI).**** Sessions auto-saved to `~/.grok/sessions/`, keyed by working directory. `~/resume`, `--resume`, `-c`, `~/fork`; subscription-gated (SuperGrok/X Premium+). Config in `~/.grok/config.toml`. No native cross-device sync. Still beta (v0.2.x as of July 2026). ****Residual gap:**** no sync at all.

****GitHub Copilot CLI.**** Sessions in `~/.copilot/` (`session-state/<uuid>/events.jsonl`). Notably, Copilot CLI ****syncs sessions to your GitHub account by default**** and offers ****remote control**** from [GitHub.com/GitHub Mobile](https://github.com/GitHubMobile) (origin machine must stay online, `~/keep-alive`). But ****cross-device *resume* is still not supported**** – manually copying a session dir and running `--resume` overwrites `events.jsonl` and destroys history (issue #1635); cloud-synced cross-device continuity is an open request (#1947). The Copilot SDK supports resumable sessions via a user-supplied `session\_id`. ****Residual gap:**** the sync is telemetry/history, not resumable working state across machines.

****Amp (Sourcegraph).**** Threads sync to ampcode.com by default and can be continued across devices; "orbs" (remote machines) keep running after you close your laptop. This is the closest to "cloud-native by default," but it's Amp-only and stores threads on Sourcegraph servers (a privacy consideration; MCP config is GUI-global, not file-based). ****Residual gap:**** vendor-locked, server-stored, no cross-vendor.

****Cross-vendor summary:**** Every vendor is building **within** its ecosystem. The white space is precisely between ecosystems and across OSes.

2. Session storage formats & technical feasibility

Tool	Location	Format	Path keying	Cross-device resume native?
---	---	---	---	---

```

| Claude Code | `~/.claude/projects/<encoded-
path>/<session>.jsonl` | JSONL, append-only event stream |
**Absolute path, non-alphanumerics→dashes** | Via cloud only
(teleport/RC) |
| Codex | `~/.codex/sessions/YYYY/MM/DD/rollout-*.jsonl` | JSONL
+ `session_meta` header | date-based; cwd in meta | Yes, via App
Server |
| Gemini CLI | `~/.gemini/tmp/<project_hash>/chats/` | JSON |
**Hash of project root** | No |
| opencode | `~/.local/share/opencode/` (SQLite + files) |
SQLite/JSON | project-keyed | Sharing only |
| Cursor | local SQLite | DB rows | workspace/path metadata | No
(cloud handoff only) |
| Grok | `~/.grok/sessions/` | JSON | working directory | No |
| Copilot CLI | `~/.copilot/session-state/<uuid>/events.jsonl` |
JSONL events | uuid | Account history sync; no resume |

```

****The path-mapping problem (the crux).**** Claude Code derives the project folder name by escaping the absolute path: ``~/Users/me/app`` → ``~/.claude/projects/-Users-me-app/``, while the same repo at ``C:\Users\me\app`` (or a different macOS username/location) produces a *different* folder. Consequently, after a naive file sync, ``claude --resume`` on the second machine cannot find the first machine's sessions. This is confirmed by the `claude-sync` author's own documentation and raised pointedly by users in the tool's DEV Community thread: "does the tool handle path remapping somehow, or is this a known limitation? Because if project session continuity doesn't work across different paths, the core use case feels significantly weaker." Community writeups (Rockford Lhotka, Nick Ang) "sidestep" it by keeping identical paths everywhere. Gemini CLI has the same issue via ``project_hash``; Cursor stores path metadata in chat rows (cursaves rewrites these; it also matches projects by git remote URL). Tools that solve it either (a) require identical paths on all machines, or (b) rewrite the path key + in-file path references on pull. ****This is the single highest-value, currently-unmet technical feature.****

****Format churn risk.**** Anthropic explicitly warns: "The entry format is internal to Claude Code and changes between versions, so scripts that parse these files directly can break on any release." This is a real, ongoing maintenance tax for any file-parsing approach and must be designed for (adapters, schema versioning, graceful degradation).

****Mid-session cross-device resume feasibility (same vendor):****
Achievable for Claude Code, Codex, Gemini, Grok, opencode by copying/rewriting the session file + ensuring the repo/branch is present – **if** path keys are remapped. Codex's App Server makes it cloud-native already.

****Cross-vendor session replay (e.g., Claude Code session → Codex):**** ****Fundamentally impractical.**** Each tool encodes tool-call schemas, thinking blocks, provider-specific message formats, and (in Codex's case) ``encrypted_content`` that can't even survive an account change. npow/session-sync attempts this by translating Claude Code tool calls into other agents' native session dirs via hooks, but this is best-effort context transfer, not faithful state replay. ****Recommendation:** treat cross-vendor as config/skills/instruction portability + context handoff, never as live session replay.**

3. Competitor teardown

****Session-continuity tools:****

- ****tawanorg/claude-sync**** (~100 stars, actively maintained, 26 releases, latest ~April 2026): Go binary; syncs `~/ .claude`` (sessions, agents, skills, plugins, rules, settings) to R2/S3/GCS/S3-compatible/WebDAV; gzip + ****age E2E encryption**** ("Every file is encrypted before it leaves your machine using age, a modern encryption tool by Filippo Valsorda"), passphrase-derived keys via Argon2 (a memory-hard KDF). The reference implementation for this space. ****Punts on path remapping**** (its explicit, acknowledged weakness). Open-source, no business model.
- ****npow/session-sync**** (~1 star, very new): the only cross-vendor **session** translator – Claude Code → Codex/Amp/Gemini via PostToolUse/Stop hooks. Proof the idea is wanted; nascent.
- ****dantelex/aisync ("ss")****: multi-device continuity for Claude/Codex/Cursor/opencode via a provider-backed sync folder (iCloud/Dropbox/OneDrive/etc.); claims auto path detection and handoff blocks. (Repo was not retrievable at time of writing – treat traction as unverified.)
- ****codsync/codsync**** (~2 stars, alpha): encrypted Git-backed Codex session + config sync; Windows/macOS; conservative conflict handling; self-describes as alpha.
- ****kroxiksut/codexSync****: local-first "cold-sync" Codex state handoff, Windows-first; explicitly does NOT transfer auth tokens (OpenAI licensing).
- ****waynesutton/opencode-sync-plugin & codex-sync-plugin**

(OpenSync)** (~10 stars): Convex-backed cloud sync/search/share; opensync.dev SaaS option.

- ****iHildy/opencode-synced**** (~40 stars, active): opencode config + optional session sync via Git or Turso (concurrency-safe); opt-in sessions.
- ****Dinesh3184/claude-session-sync****: iCloud Drive sync GUI for Claude Code (macOS + Windows).
- ****renefichtmueller/claude-sync****: multi-backend (Git/iCloud/Dropbox/Syncthing/rsync) with snapshots/rollback and encryption.

****Config/MCP/skills sync (DevSync's neighborhood):****

- ****Leoyang183/sync-agents-settings**** (~8 stars): MCP + CLAUDE.md sync from Claude Code to 12+ agents; drift detection.
- ****nicepkg/vsync**** (~46 stars, active): MCP/Skills/Agents/Commands across Claude Code/Cursor/opencode/Codex, one source of truth.
- ****baranovxyz/agentsync**** and ****spxrogers/agentsync**** (beta): canonical ``.agents/`` config synced to 9–31 agents.
- ****neon-solutions/add-mcp****: one-command MCP install across ~16 agents (Neon).
- ****ztripez/mcp-sync****: MCP config sync across tools.
- ****AgentSync (Rust)****: symlink-based config sync.
- ****skills-sync****: skills + MCP workspace.
- ****DevSynq**** (devsynq.app): commercial-looking MCP/API-key sync across IDEs + launcher/marketplace.

****URL-addressable / mobile-control / viewer approaches:****

- ****TokenRip**** (tokenrip.com): makes session state a URL-addressable artifact (``/a/<slug>``) any agent can fetch – a genuinely different architectural bet ("don't sync files; give state a permanent URL").
- ****Omnara**** (YC S25): mobile control of Claude Code/Codex with cloud sync + local handoff; the original open-source wrapper repo (~2.6k stars) was ****archived in February 2026**** because "it was built as a wrapper around the Claude Code CLI, which became unfeasible to maintain with Claude Code's constant updates" – the company pivoted to a proprietary Claude Agent SDK-based voice-first platform at omnara.com. (A ~4.3  mobile-app rating was seen but is unverified.)
- ****Syncode – AI Programming Tool****: iOS + macOS companion, LAN pairing, cross-device (Mac execute → iPhone follow-up).
- ****jazzyalex/agent-sessions**** & ****kenn-io/agentsview****: local-first session browsers/search across 20+ agents (read/resume-

command, not sync).

- **LAN-workspace approaches** and

Syncthing/NAS/dotfiles/chezmoi+age: DIY approaches that work for config but hit the path problem for sessions.

Takeaway: The config-sync lane (where DevSync sits) is crowded but shallow; the session-sync lane has a clear leader (claude-sync) that explicitly punts on the hardest problem (path remapping) and is single-vendor. No one owns "unified, cross-vendor, path-aware, encrypted profile."

4. Market demand signals

Vendor issue trackers show sustained, high-engagement demand:

- **Claude Code:** #47926 ("Allow resuming Claude Code sessions across devices"), #52052 (opt-in cloud sync of transcripts/plan/memory), #26045 (cross-device memory sync), #45358 (cross-machine session sync, Mac+Windows+Linux), #51816 (Mac+Windows instance-to-instance), #60058 (connect from any client), #22648 (account-level settings sync, with a long list of duplicates: #6037, #19634, #13461, #12119, #17682). #71794 shows Anthropic quietly began syncing history across devices.
- **Codex:** #12593 (VS Code account-linked cross-device sync), #14722 (sync CLI ↔ app-server sessions), discussion #14067 (cross-device thread/context sync; spawned third-party CodeVibe and codex-workspace-sync), #4514 (forking).
- **Copilot CLI:** #1947 (cloud-synced sessions), #1635 (cross-environment resume).
- **Cursor forum:** multiple threads demanding account-level chat/agent sync, especially for Remote SSH users.

Recurring workarounds users cite (scp, symlinks to iCloud/Dropbox, git repos, identical paths everywhere) all "require user setup and maintenance." The volume of feature requests, the explosion of DIY tools, and paid mobile apps (Omnara IAP, Syncode IAP) are evidence of willingness to pay for a frictionless solution.

5. Strategic assessment

Where the durable moat is. Vendors will keep building *intra*-ecosystem sync (and doing it better than any third party, because they control the backend and auth). Competing there is a losing race. The durable, vendor-proof value is:

1. ****Cross-vendor unified profile**** – one normalized model for MCP servers + skills + custom agents + instruction files (CLAUDE.md/AGENTS.md/GEMINI.md) + settings, mapped into each tool's native format. This is DevSync's existing strength, extended.
2. ****OS-aware path remapping as a first-class feature**** – the concrete, unsolved pain that makes same-vendor session sync actually work across Windows↔macOS. This is the wedge no competitor has nailed.
3. ****E2E encryption by default**** – table stakes given session files contain code, secrets, and full transcripts.

****Risks:****

- ****Format churn:**** Claude Code's format "changes between versions" by Anthropic's own admission. Mitigate with per-tool adapters, schema versioning, and conservative fail-safe behavior (never corrupt local state; always back up before write – the pattern codexSync/cursaves use).
- ****Vendor lock-out / ToS:**** Storing session data is user-owned data (fine), but **auth tokens** are not transferable (codexSync explicitly refuses; Codex ``encrypted_content`` breaks across accounts). Never sync credentials; never depend on undocumented cloud endpoints. Remote Control/teleport are gated to subscriptions and blocked under ZDR – don't build on them.
- ****Vendors closing the gap:**** Anthropic's account sync (#71794), Codex's App Server, Copilot's GitHub sync are all encroaching on single-vendor sync. This **validates** the demand while **confirming** that only the cross-vendor + cross-OS layer is defensible.
- ****Security/privacy expectations:**** E2E encryption (age or equivalent), local-first defaults, explicit opt-in per repo, no plaintext in any remote – all mandatory. This is also a differentiator vs. Amp (server-stored) and OpenSync (Convex).
- ****Honest limits:**** Cross-vendor mid-session replay is impossible; over-promising it will burn trust. Sell config/skills portability + same-vendor session continuity + best-effort context handoff.

Recommendations

****Stage 0 – Fold in DevSync (immediate).**** Keep and harden the existing MCP-config sync as the "config" half of the unified profile. Extend its normalized model to cover skills, custom agents, instruction files (CLAUDE.md/AGENTS.md/GEMINI.md), and

settings across Claude Code, Codex, Gemini CLI, opencode, Cursor, Copilot, Grok, Amp. This is shippable now and already differentiated from single-vendor sync tools (and from crowded but shallow rivals like vsync and sync-agents-settings).

****Stage 1 – MVP: Claude Code session sync with path remapping + E2E encryption (the wedge).**** Target the founder's exact case (Windows desktop ↔ macOS MacBook, both daily). Deliver:

- Encrypted (age/passphrase-derived) sync of `~/.claude/projects/` and global config to user-owned storage (S3/R2/GCS/WebDAV), matching claude-sync's proven approach.
- ****First-class OS-aware path remapping:**** on pull, rewrite the project folder key AND in-file absolute path references (cwd, git snapshots, file refs) from the source machine's path to the target machine's path, using the git remote URL as the project identity (the technique cursaves uses for Cursor). This is the feature claude-sync explicitly lacks – lead marketing with it.
- Safe-by-default writes: back up before overwrite, atomic writes, conflict preservation.
- ****Benchmark to advance:**** `claude --resume` on the MacBook reliably finds and resumes a session created on the Windows desktop (different absolute paths) in <10s, with no corruption, across 3 consecutive Claude Code releases.

****Stage 2 – Second and third vendors: Codex + Gemini CLI + opencode.**** Add adapters. For Codex, cooperate with (don't fight) the App Server – sync what's local and useful, avoid `encrypted\_content`/token pitfalls. For Gemini, remap `project\_hash`. For opencode, use safe SQLite snapshotting (the `sqlite3 .backup` pattern the community scripts use).

- ****Benchmark:**** each new adapter must pass the same cross-OS resume test as Claude Code before GA.

****Stage 3 – Unified profile + cross-vendor context handoff.**** One "profile" = sessions + MCP + skills + agents + instructions, portable across machines and (for config) across vendors. Add best-effort context handoff (summarize + re-inject) for cross-vendor moves – explicitly labeled as context transfer, not replay.

****Monetization & thresholds.**** Free/open-source core (build trust, match claude-sync) with a paid tier for managed encrypted storage, multi-machine dashboards, and team profiles (DevSynq/OpenSync validate willingness to pay). ****Kill/pivot**

signal:** if Anthropic ships account-level *local* session sync with automatic path remapping across OSes (not just history mirroring, per #71794), the single-vendor wedge collapses – at that point double down on cross-vendor config/skills/profile portability, which no vendor will build.

****What NOT to do:**** Don't build on Remote Control/teleport (gated, one-way, origin-online, with documented ~10-min network and undocumented ~15-min idle timeouts). Don't promise cross-vendor session replay. Don't sync auth tokens. Don't store plaintext anywhere remote.

Caveats

- Star counts and release dates are point-in-time (mid-2026) and volatile; several competitors are days-old and could stall or surge. iHildy/opencode-synced was reported at both ~40 (GitHub profile listing) and ~116 stars across sources – treat as ~40–116. dantelex/aisync traction is unverified (repo not retrievable at writing). Omnara's ~4.3  mobile rating is unverified.
- Session file formats – especially Claude Code's – are explicitly documented as unstable across versions; any parsing approach carries ongoing maintenance risk.
- Vendor capabilities (teleport, Remote Control, Codex App Server, Copilot sync, Gemini→Antigravity migration) are evolving fast; specifics may have changed since the cited docs.
- "Impossible" cross-vendor replay is an engineering judgment based on schema/encryption divergence, not a vendor statement; niche context-transfer tools (npow/session-sync) show partial approaches exist.